VVroom

Angular Framework Development Guide

A Comprehensive Textbook for Building

Domain-Driven Angular Applications

URL-First State Management • Generic Components • Domain Configuration

Table of Contents

Phase 0: API Contract

051. API Overview
052. API Endpoints
053. Naming Conventions

Phase 1: Foundation

101. Project Setup
102. App Shell
103. Routing
104. Environment Configuration

Interlude A: TypeScript Generics

150. TypeScript Generics Primer

Phase 2: Framework Models

202. Domain Config Interface
203. Filter Definition Interface
204. Table Config Interface
205. Chart Config Interface
206. Picker Config Interface
207. Query Control Config Interface
208. Statistics Model Interface
209. Error Notification Interface

Interlude B: RxJS Patterns

250. RxJS Patterns Primer

Phase 3: Framework Services

301. URL State Service
302. API Service
303. Request Coordinator Service
304. Domain Config Registry
305. Domain Config Validator
306. Resource Management Service
307. Highlight State Service
308. Statistics Processor Service

309. Export Service
310. Keyboard Shortcuts Service
311. Theme Service
312. Notification Service
313. Window Communication Service
314. Popout Manager Service
315. Analytics Service

Phase 4: Domain Models

401. Automobile Filter Model
402. Automobile Data Model
403. Automobile Statistics Model

Phase 5: Domain Adapters

501. Automobile API Adapter
502. Automobile URL Mapper
503. Automobile Cache Key Builder

Phase 6: Domain Configs

601. Filter Definitions Config
602. Table Config
603. Picker Configs
604. Query Control Filters Config
605. Highlight Filters Config
606. Chart Configs
607. Domain Config Assembly
608. Domain Provider Registration

Phase 6B: Chart Data Sources

651. Manufacturer Chart Source
652. Year Chart Source
653. Body Class Chart Source
654. Top Models Chart Source

Phase 8: Framework Components

801. Base Chart Component
802. Base Picker Component
803. Results Table Component
804. Query Panel Component
805. Query Control Component
806. Statistics Panel Component

807. Search Component
808. Pagination Component
809. Export Button Component

Phase 9: Feature Components

901. Home Component
902. Automobile Component
903. Automobile Discover Component
904. Popout Component
905. Route Configuration
906. Module Assembly
907. Final Integration

Phase 9B: Testing

951. Unit Testing Setup
952. Integration Testing
953. E2E Testing
954. Performance Testing

Appendices

A01. Appendix: Angular CLI Reference
A02. Appendix: TypeScript Tips

Phase 0: API Contract

Chapter 051: API Overview

Chapter 052: API Endpoints

Chapter 053: Naming Conventions

Chapter 051

API Overview

051: API Contract Overview

Status: Complete

Depends On: None

Blocks: 052, 053, all implementation phases

Objective

Document the API contract that the vvroom application will consume. This includes base URL configuration, authentication requirements, pagination conventions, error formats, and general request/response patterns.

Why

Before writing any code — models, services, or components — you must understand the shape of the data you're working with. The API contract is an **input** to the entire system:

- **Models** are defined by what the API returns
- **Adapters** translate between API responses and models
- **Services** call endpoints defined by the API
- **Components** display data shaped by the API

Without this document, developers would be guessing at data shapes, leading to runtime errors and constant refactoring.

API Overview

Base URL

Environment	Base URL
Development	`http://generic-prime.minilab/api/specs/v1`
Production	`http://generic-prime.minilab/api/specs/v1`

The API is accessed via Traefik ingress on the Kubernetes cluster. The hostname `generic-prime.minilab` resolves to the cluster ingress controller.

Authentication

None required. The API is accessible without authentication for this internal application.

Content Type

All requests and responses use JSON:

```
Content-Type: application/json
Accept: application/json
```

Pagination Convention

All list endpoints support pagination with these query parameters:

Parameter	Type	Default	Description
`page`	number	1	Page number (1-indexed)
`size`	number	20	Items per page

Paginated Response Shape

```
{
  "results": [...],      // Array of items for current page
  "total": 1234,         // Total items across all pages
  "page": 1,             // Current page number
  "size": 20,            // Items per page
  "totalPages": 62       // Total number of pages
}
```

Sorting Convention

Sorting is controlled by these query parameters:

Parameter	Type	Values	Description
`sortBy`	string	field name	Field to sort by
`sortOrder`	string	`asc`, `desc`	Sort direction

Example: `?sortBy=manufacturer&sortOrder=asc`

Error Response Format

When an error occurs, the API returns:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",     // Error type identifier
    "message": "Invalid year range", // Human-readable message
    "details": {                     // Optional additional context
      "field": "yearMin",
      "reason": "must be less than yearMax"
    }
  }
}
```

Common Error Codes

Code	HTTP Status	Description
`VALIDATION_ERROR`	400	Invalid request parameters
`NOT_FOUND`	404	Resource not found
`INTERNAL_ERROR`	500	Server error

Highlight Parameters (h_* prefix)

The API supports "highlight" parameters for segmented statistics. These are query parameters with an `h_` prefix that tell the backend to compute statistics with a highlighted subset.

Purpose: Enable stacked bar charts showing "highlighted vs total" data.

Example Request:

```
GET /vehicles/details?manufacturer=Toyota&h_yearMin=2020&h_yearMax=2024
```

Effect on Response:

Statistics include `{total, highlighted}` objects instead of simple counts:

```
{
  "statistics": {
    "byManufacturer": {
      "Toyota": { "total": 665, "highlighted": 234 },
      "Honda": { "total": 849, "highlighted": 0 }
    }
  }
}
```

Vehicles from Toyota are highlighted (234 of 665 match the year range 2020-2024), while Honda vehicles are not highlighted (0 of 849 match).

Request/Response Flow

```
■■■
```

```
■    "total": 234,                                                    ■
■    "page": 1,                                                       ■
■    "size": 20,                                                      ■
■    "totalPages": 12,                                                ■
■    "statistics": { byManufacturer, byBodyClass, byYearRange, ... }  ■
■  }                                                                  ■
■■■
```

```
    /** Human-readable error message */
    message: string;

    /** Optional additional error details */
    details?: Record<string, any>;
  };
}

/**
 * Generic success response wrapper
 */
export interface ApiSuccessResponse<TData> {
  /** Success flag (always true for successful responses) */
  success: true;

  /** Response data */
  data: TData;
}

/**
 * Standard API response type (success or error)
 */
export type StandardApiResponse<TData> = ApiSuccessResponse<TData> | ApiErrorResponse;
```

2. Environment Configuration

File: `src/environments/environment.ts` (development)

```
export const environment = {
  production: false,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1',
  includeTestIds: true
};
```

File: `src/environments/environment.prod.ts` (production)

```
export const environment = {
  production: true,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1',
  includeTestIds: false
};
```

Verification

After implementing this section:

1. **Check file exists:**

```bash
```

ls -la src/app/framework/models/api-response.interface.ts
```

2. **Check environment files:**

```bash
grep apiBaseUrl src/environments/environment*.ts
```

3. **Verify TypeScript compiles:**

```bash
npx tsc --noEmit
```

Acceptance Criteria

- [x] `ApiResponse<TData>` interface created in `src/app/framework/models/`
- [x] `ApiErrorResponse` interface created
- [x] `ApiSuccessResponse<TData>` interface created
- [x] `StandardApiResponse<TData>` type alias created
- [x] `environment.ts` includes `apiBaseUrl` property
- [x] `environment.prod.ts` includes `apiBaseUrl` property

Next Step

Proceed to `052-automobile-endpoints.md` for detailed documentation of each automobile-specific endpoint with request/response examples.
```

Chapter 052

API Endpoints

052: Automobile Endpoints

Status: Reference

Depends On: 051-api-contract-overview

Blocks: 401-403 (Domain Models), 501-503 (Domain Adapters)

Objective

Document every API endpoint used by the automobile domain, including:

- Request URL and method
- Query parameters
- Complete response JSON shapes
- Example requests and responses

This document is the source of truth for building TypeScript models and adapters.

Endpoints Summary

Endpoint	Method	Purpose
`/vehicles/details`	GET	Search vehicles with pagination and statistics
`/statistics`	GET	Fetch statistics only (without vehicle data)

Endpoint: GET /vehicles/details

The primary endpoint for vehicle discovery. Returns paginated vehicle results with aggregated statistics.

Request

URL: `{baseUrl}/vehicles/details`

Query Parameters:

Parameter	Type	Required	Description
`manufacturer`	string	No	Filter by manufacturer name (partial match)
`model`	string	No	Filter by model name (partial match)
`yearMin`	number	No	Minimum year (inclusive)
`yearMax`	number	No	Maximum year (inclusive)
`bodyClass`	string	No	Filter by body class (can be comma-separated for multiple)
`instanceCountMin`	number	No	Minimum VIN instance count
`instanceCountMax`	number	No	Maximum VIN instance count
`search`	string	No	Global search across all fields
`models`	string	No	Model combinations: `Manufacturer:Model,Manufacturer:Model`
`page`	number	No	Page number (default: 1)
`size`	number	No	Items per page (default: 20)
`sortBy`	string	No	Sort field
`sortOrder`	string	No	Sort direction: `asc` or `desc`
`h_yearMin`	number	No	Highlight: minimum year
`h_yearMax`	number	No	Highlight: maximum year

`h_manufacturer`	string	No	Highlight: manufacturer
`h_modelCombos`	string	No	Highlight: model combinations
`h_bodyClass`	string	No	Highlight: body class

Example Request

```
GET /vehicles/details?manufacturer=Toyota&yearMin=2020&yearMax=2024&page=1&size=20&sortBy=year&sortOrder=desc
```

Response Shape

```
{
  "results": VehicleResult[],
  "total": number,
  "page": number,
  "size": number,
  "totalPages": number,
  "statistics": VehicleStatistics
}
```

Example Response


```
{
  "results": [
    {
      "vehicle_id": "TOY-CAM-2024-SED",
      "manufacturer": "Toyota",
      "model": "Camry",
      "year": 2024,
      "body_class": "Sedan",
      "instance_count": 156,
      "first_seen": "2024-01-15T10:30:00Z",
      "last_seen": "2024-11-20T14:22:00Z",
      "drive_type": "FWD",
      "engine": "I4",
      "transmission": "Automatic",
      "fuel_type": "Gasoline"
    },
    {
      "vehicle_id": "TOY-RAV-2024-SUV",
      "manufacturer": "Toyota",
      "model": "RAV4",
      "year": 2024,
      "body_class": "SUV",
      "instance_count": 234,
      "first_seen": "2024-02-10T08:15:00Z",
      "last_seen": "2024-11-19T16:45:00Z",
      "drive_type": "AWD",
      "engine": "I4",
      "transmission": "Automatic",
```

```
      "fuel_type": "Hybrid"
    }
  ],
  "total": 234,
  "page": 1,
  "size": 20,
  "totalPages": 12,
  "statistics": {
    "totalCount": 234,
    "byManufacturer": {
      "Toyota": { "total": 234, "highlighted": 234 }
    },
    "byBodyClass": {
      "Sedan": { "total": 89, "highlighted": 89 },
      "SUV": { "total": 78, "highlighted": 78 },
      "Truck": { "total": 45, "highlighted": 45 },
      "Coupe": { "total": 22, "highlighted": 22 }
    },
    "byYear": {
      "2024": { "total": 67, "highlighted": 67 },
      "2023": { "total": 58, "highlighted": 58 },
      "2022": { "total": 52, "highlighted": 52 },
      "2021": { "total": 34, "highlighted": 34 },
      "2020": { "total": 23, "highlighted": 23 }
    },
    "modelsByManufacturer": {
      "Toyota": {
        "Camry": { "total": 45, "highlighted": 45 },
        "RAV4": { "total": 52, "highlighted": 52 },
        "Corolla": { "total": 38, "highlighted": 38 },
        "Highlander": { "total": 29, "highlighted": 29 }
      }
    }
  }
}
```

Live Data Sample: Filter by Manufacturer

The following is actual data from the vvroom API when filtering by `manufacturer=Ford`:

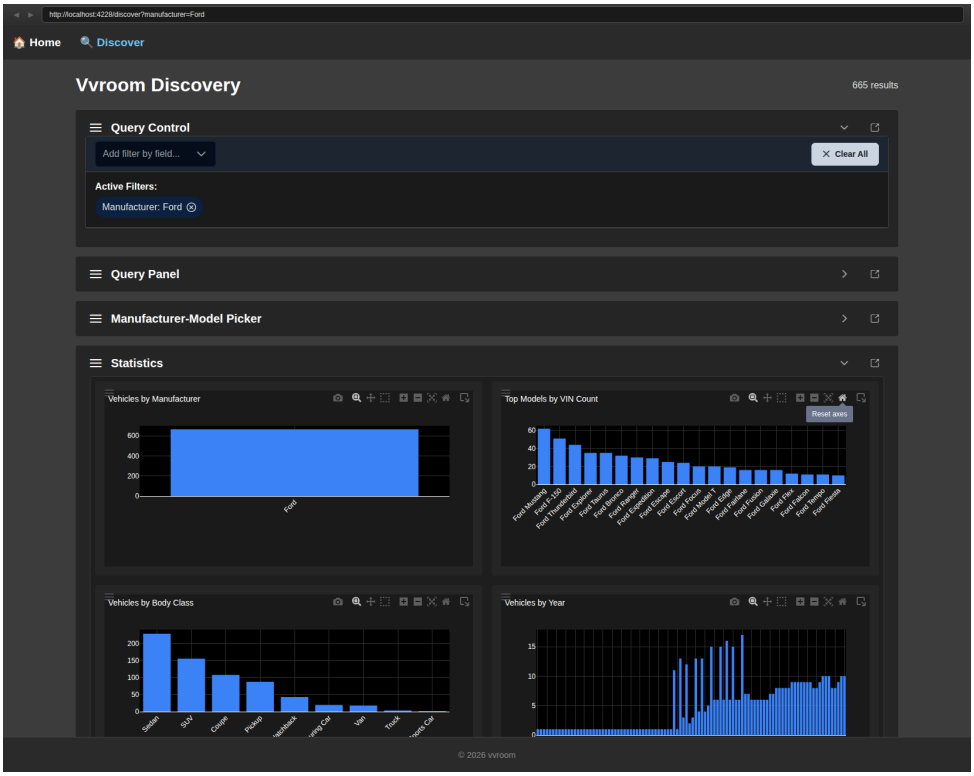
Request: `GET /vehicles/details?manufacturer=Ford&size=5`

Response (abbreviated):


```
{
  "total": 665,
  "page": 1,
  "size": 5,
  "totalPages": 133,
  "results": [
    {
      "vehicle_id": "nhtsa-ford-mustang-2020",
      "manufacturer": "Ford",
```

```
      "model": "Mustang",
      "year": 2020,
      "body_class": "Coupe",
      "instance_count": 12
    },
    {
      "vehicle_id": "nhtsa-ford-f-150-2018",
      "manufacturer": "Ford",
      "model": "F-150",
      "year": 2018,
      "body_class": "Pickup",
      "instance_count": 12
    }
  ],
  "statistics": {
    "byManufacturer": { "Ford": 665 },
    "byBodyClass": {
      "Sedan": 229,
      "SUV": 156,
      "Coupe": 108,
      "Pickup": 88
    },
    "modelsByManufacturer": {
      "Ford": {
        "Mustang": 62,
        "F-150": 51,
        "Thunderbird": 44,
        "Explorer": 35,
        "Taurus": 35
      }
    }
  }
}
```


Figure 052.1 shows what this data looks like rendered in the application's results table:



Filtered results showing Ford vehicles

Figure 052.1: Results table filtered by manufacturer=Ford, showing 665 total vehicles with the top models being Mustang (62), F-150 (51), and Thunderbird (44).

VehicleResult Object

Each item in the `results` array has this shape:

```
interface VehicleResult {
  // Required fields
  vehicle_id: string;      // Unique ID: "MANUFACTURER-MODEL-YEAR-BODYCLASS"
  manufacturer: string;    // e.g., "Toyota", "Honda", "Ford"
  model: string;           // e.g., "Camry", "Accord", "F-150"
  year: number;            // e.g., 2024
  body_class: string;      // e.g., "Sedan", "SUV", "Truck"
  instance_count: number;  // Number of VINs for this vehicle config

  // Optional fields
  first_seen?: string;     // ISO 8601 datetime
  last_seen?: string;      // ISO 8601 datetime
  drive_type?: string;     // e.g., "FWD", "RWD", "AWD", "4WD"
  engine?: string;         // e.g., "V6", "I4", "V8", "Electric"
  transmission?: string;   // e.g., "Automatic", "Manual", "CVT"
  fuel_type?: string;      // e.g., "Gasoline", "Diesel", "Electric", "Hybrid"
  vehicle_class?: string;  // e.g., "Passenger Car", "Light Truck"
```

```
}
```

Field Notes

Field	Format	Example
`vehicle_id`	`{MFR}-{MODEL}-{YEAR}-{BODY}`	`TOY-CAM-2024-SED`
`year`	4-digit integer	`2024`
`instance_count`	Non-negative integer	`156`
`first_seen`	ISO 8601	`2024-01-15T10:30:00Z`
`body_class`	Title case	`Sedan`, `SUV`, `Truck`

Statistics Object

The `statistics` field in the response contains aggregated data for charts and summaries.

Structure

```
interface VehicleStatistics {
  // Total count of vehicles matching filters
  totalCount: number;

  // Segmented by manufacturer: { "Toyota": { total, highlighted }, ... }
  byManufacturer: Record<string, { total: number; highlighted: number }>;

  // Segmented by body class: { "Sedan": { total, highlighted }, ... }
  byBodyClass: Record<string, { total: number; highlighted: number }>;

  // Segmented by year: { "2024": { total, highlighted }, ... }
  byYear: Record<string, { total: number; highlighted: number }>;

  // Models nested under manufacturers
  modelsByManufacturer: Record<string, Record<string, { total: number; highlighted: number }>>;
}
```

Segmented Statistics Explained

Each statistic entry contains:

- `total`: Count of all items in this category
- `highlighted`: Count of items matching highlight filters (h_* parameters)

Example without highlights:

```
{
  "byManufacturer": {
    "Toyota": { "total": 234, "highlighted": 234 },
    "Honda": { "total": 187, "highlighted": 187 }
  }
}
```

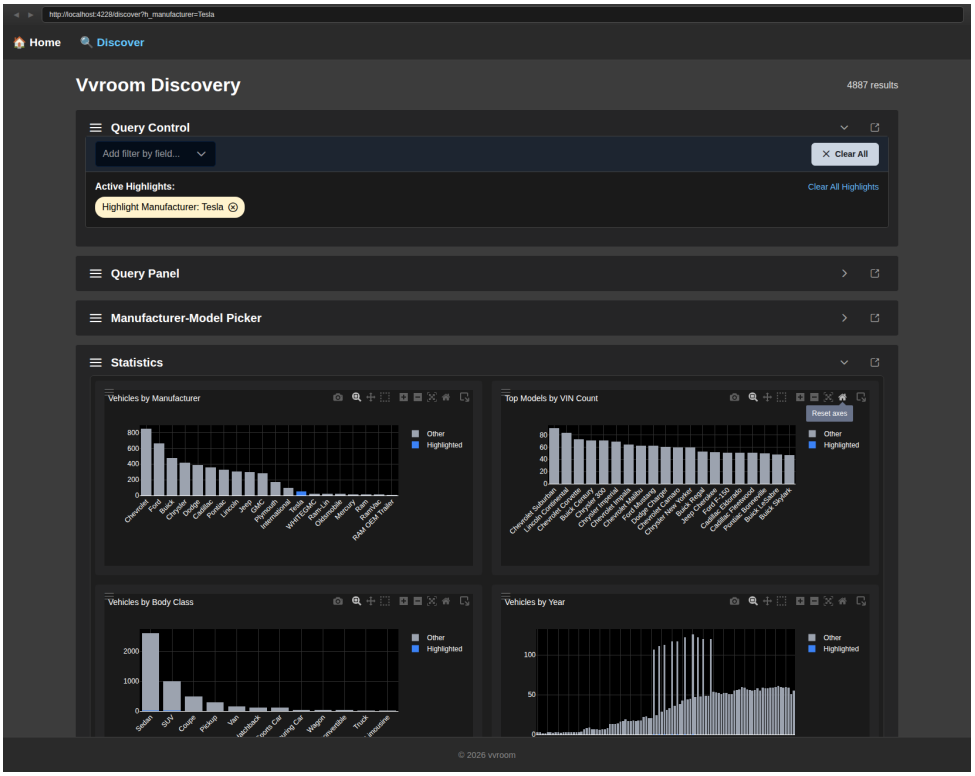
When no highlight filters are applied, `highlighted` equals `total`.

Example with highlights (`h_yearMin=2022&h_yearMax=2024`):

```
{
  "byManufacturer": {
    "Toyota": { "total": 234, "highlighted": 156 },
    "Honda": { "total": 187, "highlighted": 98 }
  }
}
```

Only 156 of Toyota's 234 vehicles fall within 2022-2024.

Figure 052.2 shows how highlights appear in the statistics panel — Tesla is highlighted while other manufacturers show their total counts:



Statistics panel with Tesla highlighted

Figure 052.2: Statistics panel showing highlighted manufacturer (Tesla) with segmented bar visualization. The highlighted portion represents vehicles matching the h_manufacturer=Tesla filter.

Endpoint: GET /statistics

Fetches statistics only, without vehicle data. Useful for refreshing charts without reloading the table.

Request

URL: `{baseUrl}/statistics`

Query Parameters: Same filter parameters as `/vehicles/details` (excluding `page`, `size`, `sortBy`, `sortOrder`)

Example Request

```
GET /statistics?manufacturer=Toyota&yearMin=2020
```

Response Shape

Same as the `statistics` object from `/vehicles/details`.

URL Parameter Mapping

This table maps URL query parameters to filter object properties:

URL Parameter	Filter Property	Type
`manufacturer`	`manufacturer`	string
`model`	`model`	string
`yearMin`	`yearMin`	number
`yearMax`	`yearMax`	number
`bodyClass`	`bodyClass`	string

`instanceCountMin`	`instanceCountMin`	number
`instanceCountMax`	`instanceCountMax`	number
`search`	`search`	string
`models`	`modelCombos`	string
`page`	`page`	number
`size`	`size`	number
`sortBy`	`sort`	string
`sortOrder`	`sortDirection`	string

Note: The URL uses `models` but the filter object uses `modelCombos`. The adapter handles this translation.

Model Combinations Format

The `models` parameter (and `h_modelCombos` highlight parameter) uses a specific format:

```
Manufacturer:Model,Manufacturer:Model,...
```

Examples:

- Single model: `Ford:F-150`
- Multiple models: `Ford:F-150,Toyota:Camry,Honda:Accord`
- Same manufacturer, different models: `Toyota:Camry,Toyota:Corolla`

Field Name Conventions

The API uses **snake_case** for field names:

- `vehicle_id`, `body_class`, `instance_count`, `first_seen`, `last_seen`
- `fuel_type`, `drive_type`, `vehicle_class`

The frontend models use **camelCase** internally, with adapters handling the translation.

Next Step

Proceed to `053-naming-conventions.md` to understand which code is framework (reusable across domains) vs domain-specific (automobile only).

Chapter 053

Naming Conventions

053: Naming Conventions — Framework vs Domain

Status: Reference

Depends On: 051, 052

Blocks: All implementation phases

Objective

Establish a clear distinction between **framework code** (domain-agnostic, reusable) and **domain code** (automobile-specific). This separation enables:

1. A smaller companion book for adding new domains (e.g., "Adding Agriculture to Vroom")
2. Clear understanding of which code changes when adding a new domain
3. Proper placement of new code during development

Why This Matters

The vroom application is built on a **configuration-driven architecture**. The framework provides generic capabilities, and domain configuration tells it how to behave for a specific data domain.

To add a new domain (e.g., agriculture), you only implement the right column.

The framework code (left column) remains untouched.

Framework vs Domain: Complete Reference

Services

Framework (Never Changes)	Domain-Specific (Per Domain)
`ResourceManagementService`	—
`UrlStateService`	—
`ApiService`	—
`RequestCoordinatorService`	—
`DomainConfigRegistry`	—
`DomainConfigValidator`	—
`PopOutContextService`	—
`PopOutManagerService`	—
`UserPreferencesService`	—
`FilterOptionsService`	—
`PickerConfigRegistry`	—
`ErrorNotificationService`	—
`HttpErrorInterceptor`	—
`GlobalErrorHandler`	—

Note: Services are 100% framework. Domains don't create new services.

Interfaces

Framework (Never Changes)	Domain-Specific (Per Domain)
`DomainConfig<TFilters, TData, TStats>`	—
`IApiAdapter<TFilters, TData, TStats>`	`AutomobileApiAdapter` (implements)
`IFilterUrlMapper<TFilters>`	`AutomobileUrlMapper` (implements)

`ICacheKeyBuilder<TFilters>`	`AutomobileCacheKeyBuilder` (implements)
`ResourceState<TFilters, TData, TStats>`	—
`ApiResponse<TData>`	—
`FilterDefinition`	—
`TableConfig`	—
`PickerConfig`	—
`ChartConfig`	—

Models

Framework (Never Changes)	Domain-Specific (Per Domain)
—	`AutoSearchFilters`
—	`HighlightFilters`
—	`VehicleResult`
—	`VinInstance`
—	`VehicleStatistics`
—	`ManufacturerStat`
—	`ModelStat`
—	`BodyClassStat`
—	`YearStat`

Note: Models are 100% domain-specific. Each domain defines its own data shapes.

Adapters

Framework (Never Changes)	Domain-Specific (Per Domain)
Interface: `IApiAdapter`	`AutomobileApiAdapter`
Interface: `IFilterUrlMapper`	`AutomobileUrlMapper`

Interface: `ICacheKeyBuilder`	`AutomobileCacheKeyBuilder`

Pattern: Framework defines interfaces. Domains implement them.

Configurations

Framework (Never Changes)	Domain-Specific (Per Domain)
—	`automobile.filter-definitions.ts`
—	`automobile.table-config.ts`
—	`automobile.picker-configs.ts`
—	`automobile.query-control-filters.ts`
—	`automobile.highlight-filters.ts`
—	`automobile.chart-configs.ts`
—	`automobile.domain-config.ts`

Chart Data Sources

Framework (Never Changes)	Domain-Specific (Per Domain)
Interface: `ChartDataSource`	`ManufacturerChartSource`
—	`YearChartSource`
—	`BodyClassChartSource`
—	`TopModelsChartSource`

Components

Framework (Never Changes)	Domain-Specific (Per Domain)
`BaseChartComponent`	—

`BasePickerComponent`	—
`BasicResultsTableComponent`	—
`ResultsTableComponent`	—
`DynamicResultsTableComponent`	—
`QueryPanelComponent`	—
`QueryControlComponent`	—
`StatisticsPanel2Component`	—
`DockviewStatisticsPanelComponent`	—

Note: UI components are 100% framework. They render based on configuration.

Feature Components

Framework (Never Changes)	Domain-Specific (Per Domain)
`HomeComponent`	—
`PopoutComponent`	—
—	`AutomobileComponent` (landing)
—	`AutomobileDiscoverComponent` (main page)

Pattern: The discover page is domain-specific because it wires up domain configuration. Popout is framework because it renders any domain's panels.

Routes

Framework (Never Changes)	Domain-Specific (Per Domain)
`/` (redirect)	—
`/home`	—
`/popout/:gridId/:componentId`	—
—	`/automobiles`

—	/automobiles/discover

Directory Structure

Figure 053.1 provides a visual overview of the directory structure, color-coded to distinguish framework code from domain-specific code:

```
src/app/
├── app.component.ts                # Root component (navigation, AI chat toggle)
├── app.module.ts                   # Root NgModule (providers, declarations)
├── app-routing.module.ts           # Route definitions
├── features/                       # Domain-specific feature components
│   ├── home/                       # Landing page
│   ├── automobile/                 # Automobile domain
│   ├── discover/                   # Generic discovery component
│   ├── panel-popout/               # Pop-out window component
│   ├── popout/                     # Popout feature
│   └── ...
│
├── framework/                      # Domain-agnostic framework
│   ├── components/                 # Reusable UI components
│   │   ├── ai-chat/                # LLM chat interface
│   │   ├── base-chart/             # Plotly chart wrapper
│   │   ├── base-picker/            # Multi-select picker
│   │   ├── basic-results-table/    # Simple data table
│   │   ├── dynamic-results-table/  # Table with column manipulation
│   │   ├── query-control/          # Advanced filter dialogs
│   │   ├── query-panel/            # Inline filter panel
│   │   ├── results-table/          # Full-featured results table
│   │   ├── statistics-panel/       # Statistics panel
│   │   ├── statistics-panel-2/     # Chart grid with drag-drop
│   │   ├── column-manager/         # Column management
│   │   └── inline-filters/         # Inline filter components
│
│   ├── services/                   # Application services
│   │   ├── api.service.ts          # HTTP client wrapper
│   │   ├── resource-management.service.ts  # State orchestrator
│   │   ├── url-state.service.ts    # URL synchronization
│   │   ├── popout-context.service.ts       # Pop-out communication
│   │   ├── ai.service.ts           # LLM integration
│   │   ├── user-preferences.service.ts     # Preference persistence
│   │   ├── request-coordinator.service.ts  # Cache + dedup + retry
│   │   ├── domain-config-registry.service.ts  # Contains DOMAIN_CONFIG token
│   │   ├── domain-config-validator.service.ts
│   │   ├── error-notification.service.ts
│   │   ├── global-error.handler.ts
│   │   └── http-error.interceptor.ts
│
│   ├── models/                     # TypeScript interfaces
│   │   ├── domain-config.interface.ts   # DomainConfig interface
│   │   ├── filter-definition.interface.ts  # Filter configuration
│   │   ├── table-config.interface.ts   # Table column config
│   │   ├── picker-config.interface.ts  # Picker definitions
│   │   ├── popout.interface.ts     # Pop-out types and utilities
│   │   ├── resource-management.interface.ts  # State management types
│   │   ├── ai.models.ts            # AI/chat models
│   │   └── ...
│
│   └── tokens/                     # DI injection tokens
│       └── popout.token.ts         # IS_POPOUT_TOKEN
│
├── domain-config/                  # Domain-specific configurations
│   ├── domain-providers.ts         # Central domain registration
│   └── automobile/                 # Automobile domain config
│       ├── automobile.domain-config.ts
│       ├── models/
│       ├── adapters/
│       ├── configs/
│       └── chart-sources/
│
├── environments/                   # Environment configuration
│   ├── environment.ts              # Development
│   └── environment.prod.ts         # Production
│
├── assets/                         # Static assets
└── styles.scss                     # Global styles
```

Directory structure showing framework vs domain separation

Figure 053.1: Directory structure of the vvroom application. Framework code (blue) is reusable across all domains. Domain-specific code (green) contains automobile-specific models, adapters, and configurations.

The text representation below provides the same information for reference:

```
src/
■■■ app/
■   ■■■ app.component.ts            # Framework
■   ■■■ app.config.ts               # Framework
■   ■■■ app.routes.ts               # Framework + Domain routes
■   ■■■ features/
■       ■■■ home/                   # Framework
■       ■■■ popout/                 # Framework
■       ■■■ automobile/             # DOMAIN-SPECIFIC
```

```
■           ■■■ automobile.component.ts
■           ■■■ automobile-discover/
■
■■■ framework/                    # ALL FRAMEWORK
■   ■■■ components/
■   ■■■ models/
■   ■■■ services/
■   ■■■ tokens/
■
■■■ domain-config/                # ALL DOMAIN-SPECIFIC
■   ■■■ domain-providers.ts        # Registers all domains
■   ■■■ automobile/
■       ■■■ adapters/
■       ■■■ chart-sources/
■       ■■■ configs/
■       ■■■ models/
■       ■■■ automobile.domain-config.ts
■       ■■■ index.ts
■
■■■ environments/                 # Framework
```

Observable Streams (Framework)

These observable names are **framework conventions** — they don't change per domain:

Observable	Type	Description	
`state$`	`Observable<ResourceState>`	Complete state object	
`filters$`	`Observable<TFilters>`	Current filter values	
`results$`	`Observable<TData[]>`	Current page results	
`totalResults$`	`Observable<number>`	Total count	
`loading$`	`Observable<boolean>`	Loading state	
`error$`	`Observable<Error \	null>`	Error state	
`statistics$`	`Observable<TStats>`	Statistics data	
`highlights$`	`Observable<any>`	Highlight filters	

Methods (Framework)

These method names are **framework conventions**:

Method	Signature	Description
`updateFilters()`	`(partial: Partial<TFilters>) => void`	Update filters via URL
`clearFilters()`	`() => void`	Reset to defaults
`refresh()`	`() => void`	Re-fetch with current filters
`getCurrentState()`	`() => ResourceState`	Get state snapshot
`getCurrentFilters()`	`() => TFilters`	Get filters snapshot

Injection Tokens (Framework)

Token	Type	Description
`DOMAIN_CONFIG`	`InjectionToken<DomainConfig>`	Provides domain configuration
`IS_POPOUT_TOKEN`	`InjectionToken<boolean>`	Indicates pop-out context

What Changes When Adding a New Domain?

To add agriculture support to vroom, you would create:

```
src/domain-config/agriculture/
■■■ adapters/
■   ■■■ agriculture-url-mapper.ts      # Implements IFilterUrlMapper
■   ■■■ agriculture-api.adapter.ts     # Implements IApiAdapter
■   ■■■ agriculture-cache-key-builder.ts
■■■ chart-sources/
■   ■■■ region-chart-source.ts
■   ■■■ crop-chart-source.ts
■■■ configs/
■   ■■■ agriculture.filter-definitions.ts
■   ■■■ agriculture.table-config.ts
■   ■■■ agriculture.chart-configs.ts
■■■ models/
■   ■■■ agriculture.filters.ts
■   ■■■ agriculture.data.ts
■   ■■■ agriculture.statistics.ts
```

```
■■■ agriculture.domain-config.ts
■■■ index.ts

src/app/features/agriculture/
■■■ agriculture.component.ts
■■■ agriculture-discover/
    ■■■ agriculture-discover.component.ts
```

And update:

- `src/domain-config/domain-providers.ts` (register the new domain)
- `src/app/app.routes.ts` (add agriculture routes)

No changes to framework code.

Naming Patterns

Domain Config Files

Pattern: `{domain}.{type}.ts`

Examples:

- `automobile.filter-definitions.ts`
- `automobile.table-config.ts`
- `agriculture.filter-definitions.ts`

Adapter Classes

Pattern: `{Domain}{Type}` (PascalCase)

Examples:

- `AutomobileApiAdapter`
- `AutomobileUrlMapper`
- `AgricultureApiAdapter`

Model Classes

Pattern: `{DomainEntity}` (domain-meaningful names)

Examples:

- `VehicleResult` (not `AutomobileResult`)
- `AutoSearchFilters` (not `AutomobileFilters`)

- `CropYield` (for agriculture)

Feature Components

Pattern: `{Domain}DiscoverComponent`

Examples:

- `AutomobileDiscoverComponent`
- `AgricultureDiscoverComponent`

Companion Book Scope

A companion book "Adding Agriculture to Vroom" would cover only:

Phase	Documents	Content
API Contract	1	Agriculture endpoints
Domain Models	3	Filters, Data, Statistics
Domain Adapters	3	URL Mapper, API Adapter, Cache Key Builder
Domain Configs	6	Filter definitions, table, charts, etc.
Chart Sources	2-4	Domain-specific chart transformations
Feature Components	2	Landing page, Discover page
Routes	1	Add routes to app.routes.ts
Total	~18	~150 pages

Compare to the full vroom book: **65 documents, ~525 pages**.

The companion book is **~70% smaller** because all framework code is reused.

Summary

Category	Framework	Domain
Services	14	0
Interfaces	10	0 (implement them)
Models	0	9
Adapters	0	3
Configs	0	7
Chart Sources	0	4
UI Components	9	0
Feature Components	2	2
Effort to add domain	0%	100%

This document is the key to understanding vroom's architecture. Consult it whenever uncertain about where new code belongs.

Phase 1: Foundation

Chapter 101: Project Setup

Chapter 102: App Shell

Chapter 103: Routing

Chapter 104: Environment Configuration

Chapter 101

Project Setup

101: Project Cleanup

Status: Planning

Depends On: 000-book-conventions, 051-api-contract-overview

Blocks: 102-app-shell

Learning Objectives

After completing this section, you will:

- Understand why removing boilerplate establishes ownership of your codebase
- Know how Angular environment files enable configuration without code changes
- Recognize the directory structure that separates framework code from domain code

---

Objective

Remove Angular CLI boilerplate and establish the directory structure that will support our URL-First architecture. After this section, you will have a clean foundation ready for building the vvroom application.

Why

When you generate a new Angular project with `ng new`, the CLI creates placeholder content designed to help newcomers verify their setup works. This placeholder content includes:

- A welcome message with the Angular logo

- Links to Angular documentation
- Sample text that has nothing to do with your application

This is noise. Every line of code in your project should serve a purpose. Removing boilerplate immediately establishes a professional mindset: you own every line of code in this project, and you understand why it's there.

Additionally, we need to:

1. **Configure the environment** — Set the API base URL so our services know where to fetch data
2. **Establish the directory structure** — Create folders that match our architecture before we need them
3. **Update the HTML title** — Small detail, but professionalism shows in the details

Angular Style Guide References

- Style 04-06: Create sub-folders for feature areas
- Style 04-07: Create a folder for each feature module

What

Step 101.1: Understand the Current State

Before making changes, examine what the Angular CLI generated. Open a terminal and navigate to your project:

```
$ cd ~/projects/vvroom
$ ls -la src/app/
```

You should see:

```
total 16
drwxr-xr-x 2 user user 4096 Feb  9 10:00 .
drwxr-xr-x 5 user user 4096 Feb  9 10:00 ..
-rw-r--r-- 1 user user 1234 Feb  9 10:00 app.component.ts
-rw-r--r-- 1 user user  345 Feb  9 10:00 app.module.ts
```

The current `src/app/app.component.ts` contains placeholder content with an inline template showing a welcome message, the Angular logo, and documentation links. This is what we'll clean up.

Step 101.2: Clean Up AppComponent

Open `src/app/app.component.ts` and replace its entire contents with:

```
// src/app/app.component.ts
// VERSION 1 (Section 101) - Minimal placeholder
// This will be replaced with the full shell in Section 102

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h1>vvroom</h1>
    <p>Automobile Discovery Platform</p>
  `,
  styles: []
})
export class AppComponent {}
```

What changed:

Before	After
32 lines with placeholder content	13 lines of clean code
`title` property (unused baggage)	No unnecessary properties
Inline Angular logo (base64 SVG)	Removed
Documentation links	Removed

Why this template?

The simple heading and tagline serve as a "smoke test" — when you run the app, you'll immediately see whether your changes worked. We're not adding navigation or routing yet; that comes in document 102.

Step 101.3: Configure Environment for API Access

Our application will fetch data from the automobile API. Configure the base URL in the environment file.

Open `src/environments/environment.ts` and replace its contents with:

```
// src/environments/environment.ts

export const environment = {
  production: false,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
};
```

Now create the production environment file. Open `src/environments/environment.prod.ts` and replace its contents with:

```
// src/environments/environment.prod.ts

export const environment = {
  production: true,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
};
```

Why the same URL for both?

In this application, development and production use the same API server. In a typical enterprise environment, you might have:

- Development: `http://localhost:3000/api`
- Production: `https://api.yourcompany.com/v1`

The environment file pattern allows this flexibility without changing application code.

Step 101.4: Create Directory Structure

Create the directories that will hold our framework and domain-specific code:

```
$ cd ~/projects/vvroom

# Framework directories (domain-agnostic, reusable)
$ mkdir -p src/app/framework/services
$ mkdir -p src/app/framework/models
$ mkdir -p src/app/framework/components
$ mkdir -p src/app/framework/tokens

# Feature directories (page-level components)
$ mkdir -p src/app/features/home
$ mkdir -p src/app/features/discover
$ mkdir -p src/app/features/popout

# Domain configuration (automobile-specific)
$ mkdir -p src/app/domain-config/automobile/models
$ mkdir -p src/app/domain-config/automobile/adapters
$ mkdir -p src/app/domain-config/automobile/configs
$ mkdir -p src/app/domain-config/automobile/chart-sources
```

Verify the structure:

```
$ find src/app -type d | sort
```

Expected output:

```
src/app
src/app/domain-config
src/app/domain-config/automobile
src/app/domain-config/automobile/adapters
src/app/domain-config/automobile/chart-sources
src/app/domain-config/automobile/configs
src/app/domain-config/automobile/models
src/app/features
src/app/features/discover
src/app/features/home
src/app/features/popout
src/app/framework
src/app/framework/components
src/app/framework/models
src/app/framework/services
src/app/framework/tokens
```

What do these directories mean?

Directory	Purpose	Examples
`framework/services`	Domain-agnostic services	`UrlStateService`, `ApiService`
`framework/models`	TypeScript interfaces for framework	`DomainConfig`, `ApiResponse`
`framework/components`	Reusable UI components	`BaseChartComponent`, `ResultsTableComponent`
`framework/tokens`	Angular injection tokens	`DOMAIN_CONFIG`, `IS_POPOUT_TOKEN`
`features/*`	Page-level components	`HomeComponent`, `DiscoverComponent`
`domain-config/automobile/*`	Automobile-specific configuration	Filters, adapters, chart sources

This structure directly reflects the architecture described in `053-naming-conventions.md`. Framework code never changes when you add a new domain; only `domain-config/` grows.

Step 101.5: Add Placeholder Files

Empty directories are ignored by git. Add `.gitkeep` files to preserve the structure:

```
$ cd ~/projects/vvroom

# Add .gitkeep to each empty directory
$ touch src/app/framework/services/.gitkeep
$ touch src/app/framework/models/.gitkeep
$ touch src/app/framework/components/.gitkeep
$ touch src/app/framework/tokens/.gitkeep
```

```
$ touch src/app/features/home/.gitkeep
$ touch src/app/features/discover/.gitkeep
$ touch src/app/features/popout/.gitkeep
$ touch src/app/domain-config/automobile/models/.gitkeep
$ touch src/app/domain-config/automobile/adapters/.gitkeep
$ touch src/app/domain-config/automobile/configs/.gitkeep
$ touch src/app/domain-config/automobile/chart-sources/.gitkeep
```

Note: These `.gitkeep` files are a convention, not a git feature. They're empty files that exist solely to make git track otherwise-empty directories. We'll delete them as we add real files to each directory.

Step 101.6: Update Page Title

Open `src/index.html` and verify it has a meaningful title:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Vvroom - Automobile Discovery</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

The only change is the `<title>` tag: from "Vvroom" to "Vvroom - Automobile Discovery".

Step 101.7: Add Base Styles

Open `src/styles.css` and add minimal global styles:

```
/* src/styles.css */

/* Reset and base styles */
*, *::before, *::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, sans-serif;
  font-size: 16px;
  line-height: 1.5;
  color: #333;
```

```
  background-color: #f5f5f5;
}

h1, h2, h3, h4, h5, h6 {
  margin-top: 0;
  margin-bottom: 0.5rem;
  font-weight: 500;
  line-height: 1.2;
}

a {
  color: #1976d2;
  text-decoration: none;
}

a:hover {
  text-decoration: underline;
}
```

Why these styles?

- `box-sizing: border-box` — Makes width/height calculations predictable (padding included)
- System font stack — Uses the operating system's native font for fast loading and native feel
- Reset margins — Browsers have inconsistent defaults; we normalize them
- Link color — A professional blue that's accessible and familiar

These styles provide a clean foundation. We'll add component-specific styles as we build the UI.

Verification

1. Build the Application

```
$ cd ~/projects/vvroom
$ ng build
```

Expected output (no errors):

```
✔ Browser application bundle generation complete.
✔ Copying assets complete.
✔ Index html generation complete.

Initial Chunk Files           | Names         |  Raw Size
main.js                       | main          |  47.42 kB |
polyfills.js                  | polyfills     | 339.16 kB |
runtime.js                    | runtime       |   6.54 kB |
styles.css                    | styles        |   1.23 kB |

Build at: 2026-02-09T17:00:00.000Z - Hash: abc123 - Time: 5000ms
```

2. Serve the Application

```
$ ng serve --open
```

Your browser should open to `http://localhost:4200` showing:

```
vvroom
Automobile Discovery Platform
```

If you see the Angular logo and "Welcome to vvroom!" links, you missed Step 101.2. Go back and update `app.component.ts`.

3. Check the Browser Tab

The browser tab should show "Vvroom - Automobile Discovery" (not just "Vvroom").

4. Verify Directory Structure

```
$ find src/app -type d | wc -l
```

Expected: `16` (the app directory plus 15 subdirectories we created)

5. Check Environment Configuration

```
$ grep -r "apiBaseUrl" src/environments/
```

Expected output:

```
src/environments/environment.ts:  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
src/environments/environment.prod.ts:  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
```

Common Problems

Symptom	Cause	Solution
`ng build` fails with "Cannot find module '@angular/core'"	Node modules not installed	Run `npm install` in the project root
Browser shows blank page	Template syntax error in `app.component.ts`	Check for missing backticks or quotes in the template

"Vvroom" title instead of "Vvroom - Automobile Discovery"	Didn't save `index.html`	Save the file and refresh the browser
Directories not created	Typo in `mkdir` command	Re-run the commands carefully; use tab completion
`ng serve` shows "Port 4200 is already in use"	Another process using the port	Kill the other process or use `ng serve --port 4201`

Key Takeaways

1. **Own your code** — Remove boilerplate so every line serves a purpose you understand
2. **Environment files separate configuration from code** — Change URLs without changing TypeScript
3. **Directory structure reflects architecture** — Framework code in `framework/`, domain code in `domain-config/`

Acceptance Criteria

- [ ] `src/app/app.component.ts` contains only the clean template (no Angular logo, no links)
- [ ] Application displays "vvroom" heading and "Automobile Discovery Platform" tagline
- [ ] `src/environments/environment.ts` includes `apiBaseUrl` property
- [ ] `src/environments/environment.prod.ts` includes `apiBaseUrl` property
- [ ] Directory structure created: `framework/`, `features/`, `domain-config/`
- [ ] All directories contain `.gitkeep` placeholder files
- [ ] Browser tab shows "Vvroom - Automobile Discovery"
- [ ] `src/styles.css` contains base reset styles
- [ ] `ng build` completes with no errors
- [ ] `ng serve` shows the clean application

What We Accomplished

Item	Before	After

AppComponent	32 lines of boilerplate	13 lines of clean code
Environment config	No API URL	API URL configured
Directory structure	Flat `src/app/`	Organized by architecture
Page title	Generic "Vvroom"	Descriptive "Vvroom - Automobile Discovery"
Global styles	Empty	Professional base styles

Next Step

Proceed to `102-app-shell.md` to build the application shell with navigation layout.

Chapter 102

App Shell

102: App Shell

Status: Planning

Depends On: 101-project-cleanup

Blocks: 103-routing

Learning Objectives

After completing this section, you will:

- Understand the container/presentational component pattern for application layout
- Know how to use the `:host` CSS selector to style Angular components
- Be able to create a flexbox-based full-viewport layout

Objective

Build the application shell — the outermost structural component that provides consistent navigation and layout across all pages. After this section, you'll have a header with navigation links and a content area where routed components will render.

Why

Every web application needs a shell: a consistent frame that surrounds page content. The shell typically includes:

- **Header** — Application name/logo and primary navigation

- **Content area** — Where page-specific content renders
- **Optional footer** — Copyright, links, version info

Building the shell before routing has practical benefits:

1. **Visual confirmation** — You see the navigation structure before wiring it up
2. **Router outlet placement** — You know exactly where routed content will appear
3. **Separation of concerns** — Layout logic stays in AppComponent; page logic stays in feature components

Angular Style Guide References

- Style 02-01: Use consistent naming for components
- Style 05-03: Put presentation logic in the component class

URL-First Architecture Reference

The shell is framework code — it doesn't change when you add new domains. The navigation links will eventually include domain-specific routes, but the shell structure remains constant.

What

Step 102.1: Design the Shell Layout

Before writing code, understand what we're building:

```
■■■
```

2. **Main content** — Takes remaining vertical space, contains `<router-outlet>`

Step 102.2: Update AppComponent with Shell Structure

Open `src/app/app.component.ts` and replace its contents with:

```
// src/app/app.component.ts
// VERSION 2 (Section 102) - Shell with navigation
// Replaces VERSION 1 from Section 101

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <header class="app-header">
      <div class="app-header-brand">
        <span class="app-header-logo">■</span>
        <span class="app-header-title">vvroom</span>
      </div>
      <nav class="app-header-nav">
        <a class="nav-link" href="/home">Home</a>
        <a class="nav-link" href="/discover">Discover</a>
      </nav>
    </header>
    <main class="app-content">
      <p>Content will appear here once routing is configured.</p>
    </main>
  `,
  styles: [`
    :host {
      display: flex;
      flex-direction: column;
      min-height: 100vh;
    }

    .app-header {
      display: flex;
      align-items: center;
      justify-content: space-between;
      padding: 0 1.5rem;
      height: 56px;
      background-color: #1976d2;
      color: white;
      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
    }

    .app-header-brand {
      display: flex;
      align-items: center;
      gap: 0.5rem;
    }

    .app-header-logo {
```

```
      font-size: 1.5rem;
    }

    .app-header-title {
      font-size: 1.25rem;
      font-weight: 600;
      letter-spacing: -0.5px;
    }

    .app-header-nav {
      display: flex;
      gap: 0.5rem;
    }

    .nav-link {
      padding: 0.5rem 1rem;
      color: rgba(255, 255, 255, 0.9);
      text-decoration: none;
      border-radius: 4px;
      transition: background-color 0.2s;
    }

    .nav-link:hover {
      background-color: rgba(255, 255, 255, 0.1);
      text-decoration: none;
    }

    .app-content {
      flex: 1;
      padding: 1.5rem;
    }
  `]
})
export class AppComponent {}
```

What this code does:

Element	Purpose
`:host`	Styles the component itself as a flex column taking full viewport height
`.app-header`	Blue header bar with flexbox layout
`.app-header-brand`	Logo and title grouped together
`.app-header-nav`	Navigation links aligned to the right
`.nav-link`	Styled anchor tags with hover effect
`.app-content`	Main content area with `flex: 1` to fill remaining space

Why inline styles?

For small components, inline styles in the `styles` array keep everything in one file. This follows Angular's recommendation for simple components. We'll extract styles to separate files for larger components later.

Note on the emoji: We're using ■ as a temporary logo. In a production application, you'd use an SVG or image file. The emoji works fine for learning.

Step 102.3: Understand the Navigation Links (Temporary)

Notice the navigation links use plain `href` attributes:

```
<a class="nav-link" href="/home">Home</a>
<a class="nav-link" href="/discover">Discover</a>
```

This is intentional but temporary. These are standard HTML links that cause a full page reload. In the next document (103-routing), we'll:

1. Import `RouterModule`
2. Replace `href` with `routerLink`
3. Add `routerLinkActive` for highlighting the current route

For now, clicking these links will show a 404 or reload the app — that's expected.

Step 102.4: Update AppModule to Import Required Modules

The current `AppModule` is minimal. We don't need any additional imports yet, but let's verify it's correct.

Open `src/app/app.module.ts` and confirm it contains:

```
// src/app/app.module.ts

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
```

```
export class AppModule {}
```

No changes needed yet. We'll add `RouterModule` in document 103.

Step 102.5: Understanding the Flex Layout

The shell uses flexbox for layout. Here's how it works:

```
:host {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
```

This makes the component a vertical flex container that's at least viewport height.

```
.app-header {
  height: 56px;
  /* ... other styles ... */
}
```

The header has a fixed height of 56px (a common Material Design height).

```
.app-content {
  flex: 1;
  /* ... other styles ... */
}
```

The content area uses `flex: 1` to expand and fill all remaining vertical space.

Visual result:

```
┌──────────────────────────────────────┐
│ Header (fixed 56px)                  │
├──────────────────────────────────────┤
│                                      │
│ Content (flexible, fills rest)       │
│                                      │
│                                      │
│                                      │
└──────────────────────────────────────┘
```

This ensures the app always fills the viewport, even with little content.

Step 102.6: The Host Element Pattern

Notice we style `:host` rather than adding a wrapper div:

```
:host {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
```

`:host` is a CSS pseudo-class that targets the component's host element — in this case, `<app-root>`. This is an Angular best practice:

Without :host (anti-pattern):

```
<app-root>
  <div class="wrapper">  <!-- Unnecessary wrapper -->
    <header>...</header>
    <main>...</main>
  </div>
</app-root>
```

With :host (correct):

```
<app-root>  <!-- app-root IS the flex container -->
  <header>...</header>
  <main>...</main>
</app-root>
```

One less DOM element, cleaner structure, same result.

Verification

1. Build the Application

```
$ cd ~/projects/vvroom
$ ng build
```

Expected: Build succeeds with no errors.

2. Serve the Application

```
$ ng serve --open
```

Expected: Browser opens to `http://localhost:4200`

3. Visual Check

You should see:

```
■■■
```

Common Problems

Symptom	Cause	Solution
Header not full width	Missing `:host` styles	Ensure `:host { display: flex; }` is present
Content area doesn't fill viewport	Missing `min-height: 100vh` on `:host`	Add the missing style
Emoji not displaying	System font doesn't support emoji	This is rare; try a different browser
Styles not applying	Syntax error in `styles` array	Check for missing backticks or brackets
White gap at bottom of page	`flex: 1` not on `.app-content`	Add `flex: 1` to the content area

---

Key Takeaways

1. **The shell is the application frame** — It provides consistent structure across all pages
2. `:host` **styles the component element** — No wrapper divs needed
3. `flex: 1` **fills remaining space** — Combined with `min-height: 100vh`, creates full-viewport layouts

---

Acceptance Criteria

- [ ] `src/app/app.component.ts` contains the shell template with header and content area
- [ ] Header displays logo (■), title (vvroom), and navigation links (Home, Discover)
- [ ] Header has blue background (#1976d2) with white text
- [ ] Navigation links have hover effect
- [ ] Content area fills remaining viewport height
- [ ] `:host` selector is used for component-level layout
- [ ] `ng build` completes with no errors
- [ ] `ng serve` shows the shell correctly in the browser

What We Accomplished

Item	Before	After
AppComponent template	Simple heading	Full shell with header and content
Layout	None	Flexbox-based full-height layout
Navigation	None	Placeholder links (non-functional)
Styling	None	Professional header with hover effects
Component pattern	Basic	Uses `:host` for layout

Architecture Note

The app shell follows the **container/presentational pattern**:

- **AppComponent (container)** — Provides structure and layout
- **Feature components (presentational)** — Render inside the content area

This separation means:

- Layout changes happen in one place (AppComponent)
- Feature components focus on their specific functionality
- The shell is framework code — it works for any domain

Next Step

Proceed to `103-routing.md` to configure Angular Router and make the navigation links functional.

Chapter 103

Routing

103: Routing

Status: Planning

Depends On: 102-app-shell

Blocks: 104-environment-config

Learning Objectives

After completing this section, you will:

- Understand how Angular Router maps URLs to components
- Know how to use `routerLink` for navigation without full page reloads
- Be able to highlight the current route using `routerLinkActive`

Objective

Configure Angular Router with routes for Home, Discover, and Popout pages. Replace the placeholder `href` links with Angular's `routerLink` directive so navigation updates the URL without reloading the page.

Why

The URL is the foundation of our application's state management. Before we build any features, we need the routing skeleton in place. Here's why routing comes early:

URLs Are State

In traditional web applications, the server generates each page. In a single-page application (SPA), the client handles navigation — but the URL remains the source of truth. When a user bookmarks `/discover?make=Toyota`, they expect to return to that exact view.

This is the core insight of **URL-First State Management**: the URL isn't just an address; it's a serialized representation of application state.

Why Not Just Use `href`?

Standard HTML links (`<a href="/home">`) work, but they cause problems:

`href` **Behavior**	**Problem**
Full page reload	Loses JavaScript state
Server request	Slower navigation
Flash of white	Poor user experience

Angular's `routerLink` directive intercepts clicks and updates the URL without reloading. The Router then renders the appropriate component. This is faster, smoother, and preserves application state.

Angular Style Guide References

- Style 04-10: Use redirects for default routes
- Style 02-05: Suffix routing modules with `-routing`

URL-First Architecture Reference

See `docs/README.md` for the full URL-First pattern. This section establishes the routing foundation that later sections will build upon.

What

Step 103.1: Create the Home Component

The Home component is a simple landing page. Create the file `src/app/features/home/home.component.ts`:

```
// src/app/features/home/home.component.ts
// VERSION 1 (Section 103) - Placeholder component
```

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  template: `
    <div class="home-container">
      <h1>Welcome to Vvroom</h1>
      <p>Your automobile discovery platform.</p>
      <p>Use the navigation above to explore vehicles.</p>
    </div>
  `,
  styles: [`
    .home-container {
      max-width: 600px;
      margin: 2rem auto;
      text-align: center;
    }

    h1 {
      color: #1976d2;
      margin-bottom: 1rem;
    }

    p {
      color: #666;
      margin-bottom: 0.5rem;
    }
  `]
})
export class HomeComponent {}
```

Delete the `.gitkeep` file since the directory now has real content:

```
$ rm src/app/features/home/.gitkeep
```

Step 103.2: Create the Discover Component

The Discover component will eventually display vehicle search results. For now, it's a placeholder. Create `src/app/features/discover/discover.component.ts`:

```
// src/app/features/discover/discover.component.ts
// VERSION 1 (Section 103) - Placeholder component
// This will be replaced with the full discover page in Phase 9

import { Component } from '@angular/core';

@Component({
  selector: 'app-discover',
  template: `
    <div class="discover-container">
      <h1>Discover Vehicles</h1>
      <p>Vehicle search and filtering will appear here.</p>
      <p class="hint">Watch the URL bar as you navigate — this is where state will live.</p>
```

```
    </div>
  `,
  styles: [`
    .discover-container {
      max-width: 800px;
      margin: 2rem auto;
    }

    h1 {
      color: #1976d2;
      margin-bottom: 1rem;
    }

    p {
      color: #666;
      margin-bottom: 0.5rem;
    }

    .hint {
      margin-top: 2rem;
      padding: 1rem;
      background-color: #e3f2fd;
      border-left: 4px solid #1976d2;
      color: #1565c0;
    }
  `]
})
export class DiscoverComponent {}
```

Delete the `.gitkeep` file:

```
$ rm src/app/features/discover/.gitkeep
```

Step 103.3: Create the Popout Component

The Popout component renders in a separate browser window. It will display panels that users can pop out from the main interface. Create `src/app/features/popout/popout.component.ts`:

```
// src/app/features/popout/popout.component.ts
// VERSION 1 (Section 103) - Placeholder component
// This will be replaced with the full popout implementation in Phase 3B

import { Component } from '@angular/core';

@Component({
  selector: 'app-popout',
  template: `
    <div class="popout-container">
      <h1>Popout Window</h1>
      <p>This component renders in a separate browser window.</p>
      <p>It will display charts and panels that communicate with the main window.</p>
    </div>
  `,
```

```
  styles: [`
    .popout-container {
      padding: 1rem;
    }

    h1 {
      color: #1976d2;
      margin-bottom: 1rem;
    }

    p {
      color: #666;
      margin-bottom: 0.5rem;
    }
  `]
})
export class PopoutComponent {}
```

Delete the `.gitkeep` file:

```
$ rm src/app/features/popout/.gitkeep
```

Step 103.4: Create the Routing Module

Angular 13 uses a routing module to configure routes. Create `src/app/app-routing.module.ts`:

```
// src/app/app-routing.module.ts
// VERSION 1 (Section 103) - Basic routing configuration

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PopoutComponent } from './features/popout/popout.component';

const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'discover', component: DiscoverComponent },
  { path: 'popout', component: PopoutComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

What each route does:

Path	Component	Purpose
`''`	(redirect)	Redirects root URL to `/home`
`home`	HomeComponent	Landing page
`discover`	DiscoverComponent	Vehicle search (placeholder)
`popout`	PopoutComponent	Pop-out window content

Why `pathMatch: 'full'`?

Without `pathMatch: 'full'`, the empty path `''` would match every URL (since every URL starts with nothing). The `pathMatch: 'full'` option ensures the redirect only triggers when the entire URL path is empty.

Step 103.5: Update AppModule

Import the routing module and declare the new components. Open `src/app/app.module.ts` and replace its contents with:

```
// src/app/app.module.ts
// VERSION 2 (Section 103) - With routing and feature components
// Replaces VERSION 1 from Section 101

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PopoutComponent } from './features/popout/popout.component';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    DiscoverComponent,
    PopoutComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

What changed:

Before	After
Only `AppComponent` declared	Four components declared
Only `BrowserModule` imported	`AppRoutingModule` also imported

Why declare components in AppModule?

In Angular 13 with NgModules, every component must be declared in exactly one module. Since we don't have feature modules yet, all components go in `AppModule`. Later, as the application grows, we might create `HomeModule`, `DiscoverModule`, etc.

Step 103.6: Add Router Outlet to AppComponent

The Router needs a place to render components. Update `src/app/app.component.ts` to add `<router-outlet>` and replace `href` with `routerLink`:

```
// src/app/app.component.ts
// VERSION 3 (Section 103) - With router outlet and routerLink
// Replaces VERSION 2 from Section 102

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <header class="app-header">
      <div class="app-header-brand">
        <span class="app-header-logo">■</span>
        <span class="app-header-title">vvroom</span>
      </div>
      <nav class="app-header-nav">
        <a class="nav-link" routerLink="/home" routerLinkActive="nav-link-active">Home</a>
        <a class="nav-link" routerLink="/discover" routerLinkActive="nav-link-active">Discover</a>
      </nav>
    </header>
    <main class="app-content">
      <router-outlet></router-outlet>
    </main>
  `,
  styles: [`
    :host {
      display: flex;
      flex-direction: column;
      min-height: 100vh;
    }

    .app-header {
      display: flex;
      align-items: center;
      justify-content: space-between;
```

```
    padding: 0 1.5rem;
    height: 56px;
    background-color: #1976d2;
    color: white;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  }

  .app-header-brand {
    display: flex;
    align-items: center;
    gap: 0.5rem;
  }

  .app-header-logo {
    font-size: 1.5rem;
  }

  .app-header-title {
    font-size: 1.25rem;
    font-weight: 600;
    letter-spacing: -0.5px;
  }

  .app-header-nav {
    display: flex;
    gap: 0.5rem;
  }

  .nav-link {
    padding: 0.5rem 1rem;
    color: rgba(255, 255, 255, 0.9);
    text-decoration: none;
    border-radius: 4px;
    transition: background-color 0.2s;
  }

  .nav-link:hover {
    background-color: rgba(255, 255, 255, 0.1);
    text-decoration: none;
  }

  .nav-link-active {
    background-color: rgba(255, 255, 255, 0.2);
    color: white;
  }

  .app-content {
    flex: 1;
    padding: 1.5rem;
  }
`]
})
export class AppComponent {}
```

What changed from VERSION 2:

Before	After

`href="/home"`	`routerLink="/home"`
`href="/discover"`	`routerLink="/discover"`
Static placeholder text	`<router-outlet></router-outlet>`
No active link style	`.nav-link-active` class with `routerLinkActive`

Understanding the new directives:

Directive	Purpose
`routerLink="/home"`	Navigate to `/home` without page reload
`routerLinkActive="nav-link-active"`	Add CSS class when this route is active
`<router-outlet>`	Placeholder where routed components render

Why `routerLinkActive`?

Users need visual feedback about their current location. The `routerLinkActive` directive automatically adds a CSS class when the link's route matches the current URL. When you navigate to `/home`, the Home link gets the `nav-link-active` class, giving it a slightly different background.

Step 103.7: Understanding Router Outlet

The `<router-outlet>` is a placeholder directive. When the URL changes, Angular:

1. Matches the URL against the routes array
2. Creates an instance of the matching component
3. Inserts the component's view after `<router-outlet>`

Visual representation:

When URL is `/home`:

```
<main class="app-content">
  <router-outlet></router-outlet>
  <app-home>
    <div class="home-container">
      <h1>Welcome to Vvroom</h1>
      ...
    </div>
  </app-home>
</main>
```

When URL is `/discover`:

```
<main class="app-content">
  <router-outlet></router-outlet>
  <app-discover>
    <div class="discover-container">
      <h1>Discover Vehicles</h1>
      ...
    </div>
  </app-discover>
</main>
```

The component renders as a sibling *after* the outlet, not inside it. This is why we styled `.app-content` rather than `router-outlet` — the outlet itself has no dimensions.

The Aha Moment

Routes are the skeleton. The URL is where state lives.

Right now, our routes are simple: `/home`, `/discover`, `/popout`. But watch what happens when you click the navigation links:

1. The URL in your browser's address bar changes
2. The page does *not* reload
3. The content area updates instantly

This is the foundation of URL-First architecture. In later sections, we'll add query parameters to the URL:

```
/discover?make=Toyota&year=2023&page=2
```

Every piece of state — the selected make, the year filter, the current page — will be encoded in the URL. Users can:

- Bookmark their exact search
- Share the URL with colleagues
- Use the browser's back button to undo filter changes
- Refresh without losing their place

The URL becomes a serialized snapshot of application state. This section establishes the routing skeleton that makes all of this possible.

Verification

1. Build the Application

```
$ cd ~/projects/vvroom
$ ng build
```

Expected output (no errors):

```
✔ Browser application bundle generation complete.
✔ Copying assets complete.
✔ Index html generation complete.

Initial Chunk Files           | Names         |  Raw Size
main.js                       | main          |  52.18 kB |
polyfills.js                  | polyfills     | 339.16 kB |
runtime.js                    | runtime       |   6.54 kB |
styles.css                    | styles        |   1.23 kB |

Build at: 2026-02-09T17:00:00.000Z - Hash: abc123 - Time: 5000ms
```

Note: `main.js` is slightly larger now because it includes the Router and three new components.

2. Serve the Application

```
$ ng serve --open
```

Browser opens to `http://localhost:4200`

3. Verify Redirect

When you first load `http://localhost:4200`, you should be redirected to `http://localhost:4200/home`. Check the URL bar — it should show `/home`, not just `/`.

4. Test Navigation

Click "Discover" in the navigation:

- URL changes to `http://localhost:4200/discover`
- Page does NOT reload (no white flash)
- Content area shows "Discover Vehicles" heading
- "Discover" link is highlighted (slightly brighter background)

Click "Home":

- URL changes to `http://localhost:4200/home`
- Content shows "Welcome to Vvroom"
- "Home" link is highlighted

5. Test Browser History

1. Navigate to Home
2. Navigate to Discover
3. Click the browser's back button

Expected: You return to Home without a page reload. The URL updates and the content changes instantly.

6. Test Direct URL Access

Open a new browser tab and navigate directly to:

- `http://localhost:4200/discover`

Expected: The Discover page loads directly, with the navigation showing Discover as active.

7. Test Popout Route

Navigate to `http://localhost:4200/popout` by typing it in the address bar:

Expected: The Popout placeholder component renders.

8. Inspect Active Link Styling

Open browser developer tools (F12), navigate to Home, and inspect the Home link:

```
<a class="nav-link nav-link-active" ...>Home</a>
```

The `nav-link-active` class should be present.

Common Problems

Symptom	Cause	Solution
`ng build` fails with "Cannot find module './features/home/home.component'"	Component file not created or wrong path	Verify file exists at `src/app/features/home/home.component.ts`
"Error: No component factory found for HomeComponent"	Component not declared in AppModule	Add `HomeComponent` to the `declarations` array

Links still cause full page reload	Still using `href` instead of `routerLink`	Replace `href="/home"` with `routerLink="/home"`
"Error: Cannot find primary outlet to load 'HomeComponent'"	Missing `<router-outlet>` in template	Add `<router-outlet></router-outlet>` to AppComponent template
Active link style not showing	Typo in `routerLinkActive` class name	Ensure the class name matches: `routerLinkActive="nav-link-active"` and `.nav-link-active` in styles
Navigating to `/` shows blank content	Redirect not working	Ensure route has `pathMatch: 'full'` on the redirect
"NullInjectorError: No provider for Router"	`AppRoutingModule` not imported	Add `AppRoutingModule` to the `imports` array in `AppModule`

Key Takeaways

1. `routerLink` **replaces** `href` — For SPA navigation without page reloads
2. `<router-outlet>` **is a placeholder** — Routed components render after it, not inside it
3. `routerLinkActive` **provides visual feedback** — Automatically adds a CSS class when the route matches

---

Acceptance Criteria

- [ ] `src/app/features/home/home.component.ts` exists with placeholder content
- [ ] `src/app/features/discover/discover.component.ts` exists with placeholder content
- [ ] `src/app/features/popout/popout.component.ts` exists with placeholder content
- [ ] `src/app/app-routing.module.ts` configures routes for home, discover, and popout
- [ ] Root path (`/`) redirects to `/home`
- [ ] `src/app/app.module.ts` declares all components and imports `AppRoutingModule`
- [ ] `AppComponent` uses `routerLink` instead of `href`
- [ ] `AppComponent` uses `routerLinkActive` for current route highlighting
- [ ] `AppComponent` includes `<router-outlet>` in the template
- [ ] Navigation works without page reload
- [ ] Browser back/forward buttons work correctly
- [ ] Active link has visual distinction (`.nav-link-active` style applied)

- [ ] `ng build` completes with no errors
- [ ] `ng serve` shows working navigation
- [ ] Pop-out button on Discover page opens `/popout` in a new window

---

What We Accomplished

Item	Before	After
Routes	None	Home, Discover, Popout configured
Navigation	`href` (full reload)	`routerLink` (SPA navigation)
Active state	None	Visual highlight on current route
Content area	Static text	Dynamic `<router-outlet>`
Components	1 (AppComponent)	4 (App, Home, Discover, Popout)
URL behavior	Ignored	Foundation for state management

Architecture Note

All three feature components (Home, Discover, Popout) are **framework components** — they exist regardless of the domain. When we add automobile-specific functionality later:

- `HomeComponent` will remain a generic landing page
- `DiscoverComponent` will become `AutomobileDiscoverComponent` (domain-specific)
- `PopoutComponent` will remain generic (renders any domain's panels)

This separation follows the naming conventions in `053-naming-conventions.md`.

Bonus: Pop-Outs Are Just Routes in New Windows

Before moving on, let's demonstrate something important: a pop-out window is nothing more than a route opened in a separate browser window. No special framework features are required — just `window.open()`.

Step 103.8: Add a Pop-Out Button to Discover

Update `src/app/features/discover/discover.component.ts` to add a button that opens the popout route in a new window:

```
// src/app/features/discover/discover.component.ts
// VERSION 2 (Section 103) - With pop-out button
// Replaces VERSION 1 from earlier in this section

import { Component } from '@angular/core';

@Component({
  selector: 'app-discover',
  template: `
    <div class="discover-container">
      <h1>Discover Vehicles</h1>
      <p>Vehicle search and filtering will appear here.</p>
      <p class="hint">Watch the URL bar as you navigate — this is where state will live.</p>

      <div class="popout-demo">
        <h2>Pop-Out Demo</h2>
        <p>Click the button below to open the popout route in a new window:</p>
        <button class="popout-button" (click)="openPopout()">
          Open Pop-Out Window
        </button>
        <p class="note">
          Notice: the pop-out is just <code>/popout</code> opened in a new window.
          Same Angular app, same routes, different window.
        </p>
      </div>
    </div>
  `,
  styles: [`
    .discover-container {
      max-width: 800px;
      margin: 2rem auto;
    }

    h1 {
      color: #1976d2;
      margin-bottom: 1rem;
    }

    h2 {
      color: #1976d2;
      margin-bottom: 0.5rem;
      font-size: 1.25rem;
    }

    p {
      color: #666;
      margin-bottom: 0.5rem;
    }
```

```
    .hint {
      margin-top: 2rem;
      padding: 1rem;
      background-color: #e3f2fd;
      border-left: 4px solid #1976d2;
      color: #1565c0;
    }

    .popout-demo {
      margin-top: 2rem;
      padding: 1.5rem;
      background-color: #f5f5f5;
      border-radius: 8px;
    }

    .popout-button {
      padding: 0.75rem 1.5rem;
      font-size: 1rem;
      background-color: #1976d2;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
      transition: background-color 0.2s;
    }

    .popout-button:hover {
      background-color: #1565c0;
    }

    .note {
      margin-top: 1rem;
      font-size: 0.875rem;
      color: #888;
    }

    code {
      background-color: #e8e8e8;
      padding: 0.125rem 0.375rem;
      border-radius: 3px;
      font-family: monospace;
    }
  `]
})
export class DiscoverComponent {
  openPopout(): void {
    const popoutUrl = `${window.location.origin}/popout`;
    window.open(
      popoutUrl,
      'vvroomPopout',
      'width=600,height=400,menubar=no,toolbar=no,location=no,status=no'
    );
  }
}
```

What the `openPopout()` method does:

Line	Purpose
`window.location.origin`	Gets the base URL (e.g., `http://localhost:4200`)
`'/popout'`	The route we defined earlier
`'vvroomPopout'`	Window name (reuses same window if already open)
`'width=600,height=400,...'`	Window features (size, no browser chrome)

Verification: Test the Pop-Out

1. Navigate to `http://localhost:4200/discover`
2. Click the "Open Pop-Out Window" button
3. A new browser window opens showing the Popout component
4. The URL in the new window is `http://localhost:4200/popout`

Key observation: The pop-out window runs the same Angular application. It just happens to be displaying a different route in a separate window.

Why This Matters

This simple demonstration reveals the core insight behind pop-out windows:

> ***Pop-outs are not special. They're just routes rendered in separate windows.***

In Phase 3B (documents 307-308), we'll add services that enable the main window and pop-out windows to communicate. But the foundation is what you see here: `window.open()` pointing to a route in your application.

When you understand this, the complexity of pop-out panels dissolves. A "pop-out chart" is just:

1. A route that renders a chart component
2. Opened in a new window
3. With a service that syncs state between windows

Steps 1 and 2 are already working. Step 3 comes later.

Next Step

Proceed to `104-environment-config.md` to configure environment settings and verify API connectivity.

Chapter 104

Environment Configuration

104: Environment Config Verification

Status: Planning

Depends On: 103-routing

Blocks: 150-typescript-generics-primer

Learning Objectives

After completing this section, you will:

- Understand how Angular's environment files enable build-time configuration
- Know how to verify your project compiles and runs correctly
- Recognize that Phase 1 establishes the foundation for all subsequent work

---

Objective

Verify the environment configuration from Section 101 and confirm the Phase 1 foundation is complete. This section is a consolidation checkpoint — ensuring everything from Sections 101-103 works together before proceeding to framework development.

Why

Checkpoints Prevent Cascading Errors

Before building framework services and components, we must confirm the foundation is solid. A misconfigured environment or broken route will cause confusing errors later. It's easier to diagnose problems in isolation than to debug a broken service that depends on three other broken things.

The Phase 1 Checkpoint

Document 002 (Dissection Rubric) defines this checkpoint:

> ***Phase 1 Checkpoint:*** *Navigation between empty pages works. Browser history works. Readers observe URL changes in dev tools.*

This section verifies we've achieved that checkpoint.

What

Step 104.1: Verify Environment Files

The environment files should already exist from Section 101. Confirm their contents.

Open `src/environments/environment.ts` and verify it contains:

```
// src/environments/environment.ts

export const environment = {
  production: false,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
};
```

Open `src/environments/environment.prod.ts` and verify it contains:

```
// src/environments/environment.prod.ts

export const environment = {
  production: true,
  apiBaseUrl: 'http://generic-prime.minilab/api/specs/v1'
};
```

If these files don't match, update them now. The `apiBaseUrl` property will be used by services in Phase 3.

Step 104.2: Understand Build-Time File Replacement

Angular's build system replaces environment files at build time. This is configured in `angular.json`:

```
"configurations": {
  "production": {
    "fileReplacements": [
      {
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.prod.ts"
      }
    ]
  }
}
```

When you run `ng build --configuration=production`, Angular swaps `environment.ts` for `environment.prod.ts`. Your code always imports from `environment.ts`, but the actual values come from the appropriate file.

This is not runtime configuration. The values are baked into the compiled JavaScript bundle. To change the API URL, you must rebuild.

Step 104.3: Verify Project Structure

Run the following command to confirm the directory structure from Section 101:

```
$ cd ~/projects/vvroom
$ find src/app -type d | sort
```

Expected output:

```
src/app
src/app/domain-config
src/app/domain-config/automobile
src/app/domain-config/automobile/adapters
src/app/domain-config/automobile/chart-sources
src/app/domain-config/automobile/configs
src/app/domain-config/automobile/models
src/app/features
src/app/features/discover
src/app/features/home
src/app/features/popout
src/app/framework
src/app/framework/components
src/app/framework/models
src/app/framework/services
src/app/framework/tokens
```

If directories are missing, create them using the commands from Section 101.

Step 104.4: Verify Component Files

Confirm the feature components from Section 103 exist:

```
$ ls -la src/app/features/*/
```

Expected output:

```
src/app/features/discover/:
discover.component.ts

src/app/features/home/:
home.component.ts

src/app/features/popout/:
popout.component.ts
```

Step 104.5: Build and Serve

Run the build to confirm everything compiles:

```
$ cd ~/projects/vvroom
$ ng build
```

Expected: Build succeeds with no errors.

Start the development server:

```
$ ng serve
```

Open your browser to `http://localhost:4200`.

Step 104.6: Execute Phase 1 Checkpoint Tests

Perform each test manually:

Test 1: Root Redirect

1. Navigate to `http://localhost:4200`
2. Observe: URL changes to `http://localhost:4200/home`
3. Observe: Home component content displays

Test 2: Navigation Without Reload

1. Click "Discover" in the navigation
2. Observe: URL changes to `/discover`
3. Observe: Page does NOT reload (no white flash)
4. Observe: Discover component content displays

Test 3: Active Link Highlighting

1. While on `/discover`, observe: "Discover" link has brighter background
2. Click "Home"
3. Observe: "Home" link now has brighter background

Test 4: Browser History

1. Navigate: Home → Discover → Home → Discover
2. Click browser back button
3. Observe: Returns to Home without reload
4. Click browser forward button
5. Observe: Returns to Discover without reload

Test 5: Direct URL Access

1. Open new tab
2. Navigate directly to `http://localhost:4200/discover`
3. Observe: Discover page loads correctly
4. Observe: "Discover" link is highlighted

Test 6: Popout Route

1. Navigate to `http://localhost:4200/popout`
2. Observe: Popout component content displays

Phase 1 Complete

You have successfully completed Phase 1: Foundation. Your application now has:

Capability	Status
Clean project structure	Ready for framework and domain code
Environment configuration	API URL configured for both dev and prod
App shell with navigation	Header, content area, flexbox layout

Angular Router	Routes for Home, Discover, Popout
SPA navigation	No page reloads when clicking links
Browser history integration	Back/forward buttons work
Active route highlighting	Visual feedback for current location

The Phase 1 Aha Moment

Routes are the skeleton. The URL is where state lives.

Right now, your URLs are simple paths: `/home`, `/discover`, `/popout`. In Phase 3, we'll add services that read and write query parameters:

```
/discover?make=Toyota&year=2023&bodyClass=SUV&page=2
```

Every filter selection, every page navigation, every user choice will be encoded in the URL. This is URL-First State Management — and Phase 1 established the routing skeleton that makes it possible.

Verification

All verification is covered in Step 104.6. If any test fails, review the relevant section:

Failed Test	Review Section
Root redirect	103, Step 103.4 (routes)
Navigation reloads page	103, Step 103.6 (routerLink)
Active link not highlighted	103, Step 103.6 (routerLinkActive)
Browser history broken	103, Step 103.5 (AppRoutingModule)
Direct URL access fails	103, Step 103.4 (routes)
Build errors	101, 102, 103 (various)

Common Problems

Symptom	Cause	Solution
Build fails with environment error	`apiBaseUrl` not added to environment files	Update both environment files per Step 104.1
Missing directories	Section 101 steps skipped	Run mkdir commands from Section 101
Component files missing	Section 103 steps skipped	Create components per Section 103
Tests pass locally but not described here	You're ahead of the manuscript	Great! Continue to the next section

---

Key Takeaways

1. **Environment files are build-time configuration** — Values are baked into the bundle, not loaded at runtime
2. **Checkpoints prevent cascading errors** — Verify the foundation before building on it
3. **Phase 1 establishes the routing skeleton** — All URL-First state management builds on this foundation

---

Acceptance Criteria

- [ ] `src/environments/environment.ts` contains `apiBaseUrl`
- [ ] `src/environments/environment.prod.ts` contains `apiBaseUrl`
- [ ] Directory structure matches expected layout
- [ ] All three feature components exist (Home, Discover, Popout)
- [ ] `ng build` succeeds with no errors
- [ ] `ng serve` starts without errors
- [ ] Root URL redirects to `/home`
- [ ] Navigation works without page reload
- [ ] Active links are visually highlighted
- [ ] Browser back/forward buttons work
- [ ] Direct URL access works for all routes

What We Accomplished

Phase 1 Document	Content
101: Project Cleanup	Removed boilerplate, created directories, configured environment
102: App Shell	Built header, navigation, content area with flexbox layout
103: Routing	Configured Router, created components, added routerLink
104: Verification	Confirmed all pieces work together

Next Step

Proceed to `150-typescript-generics-primer.md` (Interlude A) to build foundational understanding of TypeScript generics before tackling the framework models in Phase 2.

Interlude A: TypeScript Generics

Chapter 150: TypeScript Generics Primer

Chapter 150

TypeScript Generics Primer

150: TypeScript Generics Primer

Status: Planning

Depends On: 104-environment-config

Blocks: 201-domain-config-interface

Learning Objectives

After completing this section, you will:

- Understand why generics exist and what problem they solve
- Be able to read and interpret generic type signatures
- Know the difference between concrete types and type parameters

---

Objective

Build foundational understanding of TypeScript generics before tackling the framework models in Phase 2. This is a teaching interlude, not a code implementation section — you won't modify the vvroom project here.

Why This Interlude Exists

Phase 2 introduces interfaces like this:

```
export interface DomainConfig<TFilters, TData, TStatistics> {
  urlMapper: IFilterUrlMapper<TFilters>;
  apiAdapter: IApiAdapter<TFilters, TData, TStatistics>;
```

```
  tableConfig: TableConfig<TData>;
  // ...
}
```

If you've never worked with generics, this looks intimidating. What are `TFilters`, `TData`, and `TStatistics`? Why are there angle brackets everywhere?

This primer answers those questions. By the end, you'll read generic signatures as fluently as regular TypeScript.

What Are Generics?

The Problem: Type Safety vs Code Reuse

Consider a function that returns the first element of an array:

```
// Without generics - loses type information
function getFirst(arr: any[]): any {
  return arr[0];
}

const numbers = [1, 2, 3];
const first = getFirst(numbers);  // first is 'any', not 'number'
```

The `any` type works, but we've lost type safety. TypeScript can't help us if we try to call `first.toUpperCase()` on a number.

We could write separate functions for each type:

```
// Type-safe but repetitive
function getFirstNumber(arr: number[]): number {
  return arr[0];
}

function getFirstString(arr: string[]): string {
  return arr[0];
}

function getFirstVehicle(arr: Vehicle[]): Vehicle {
  return arr[0];
}
```

This is type-safe, but we've duplicated the same logic three times. What if we need 50 types?

The Solution: Generics

Generics let us write one function that works with any type while preserving type information:

```
// With generics - type-safe AND reusable
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

const numbers = [1, 2, 3];
const first = getFirst(numbers);  // first is 'number'

const names = ['Alice', 'Bob'];
const name = getFirst(names);     // name is 'string'

const vehicles: Vehicle[] = [/* ... */];
const vehicle = getFirst(vehicles); // vehicle is 'Vehicle'
```

The `<T>` is a **type parameter**. When you call `getFirst(numbers)`, TypeScript infers that `T` is `number`. The function returns `T`, so the return type is also `number`.

This is the Aha Moment: Generics give us type safety without code duplication.

```
■■■
```

```
interface Container<T> {
  value: T;
  getValue(): T;
}
```

Part	Meaning
`Container`	The interface name
`<T>`	Type parameter declaration
`value: T`	Property of type T
`getValue(): T`	Method returning type T

The `T` is a placeholder. When you use `Container<string>`, every `T` becomes `string`:

```
const box: Container<string> = {
  value: 'hello',
  getValue() { return this.value; }
};

box.value;       // type is 'string'
box.getValue();  // returns 'string'
```

Multiple Type Parameters

Generics can have multiple parameters:

```
interface Pair<K, V> {
  key: K;
  value: V;
}

const entry: Pair<string, number> = {
  key: 'age',
  value: 30
};
```

Here `K` becomes `string` and `V` becomes `number`.

Conventional Type Parameter Names

Name	Convention
T	General "Type"
K	Key type (in maps/objects)

V	Value type (in maps/objects)
E	Element type (in collections)
R	Return type

These are conventions, not requirements. In vvroom, we use descriptive names:

Name	Meaning in vvroom
`TFilters`	Filter model type (e.g., `AutomobileFilters`)
`TData`	Data model type (e.g., `VehicleResult`)
`TStats`	Statistics model type (e.g., `VehicleStatistics`)

The `T` prefix indicates "this is a Type parameter, not a concrete type."

Concrete Types vs Type Parameters

Concrete Types

A **concrete type** is a specific, known type:

```
// Concrete types - we know exactly what these are
let name: string;
let count: number;
let vehicle: Vehicle;
let filters: AutomobileFilters;
```

Type Parameters

A **type parameter** is a placeholder filled in later:

```
// Type parameters - placeholders
interface Container<T> {     // T is a parameter
  value: T;
}

// Now we fill in the parameter with a concrete type
const box: Container<string> = { value: 'hello' };
```

Think of type parameters like function parameters:

```
// Function parameter: x is filled in when called
function double(x: number): number {
  return x * 2;
}
double(5);  // x = 5

// Type parameter: T is filled in when used
interface Container<T> {
  value: T;
}
const box: Container<string> = { value: 'hello' };  // T = string
```

Built-in Generic Types

You've already used generics — TypeScript's built-in types use them extensively.

Array<T>

```
// These are equivalent
const numbers: number[] = [1, 2, 3];
const numbers: Array<number> = [1, 2, 3];
```

`Array<number>` means "an array where every element is a number."

Promise<T>

```
// A promise that resolves to a string
const greeting: Promise<string> = Promise.resolve('hello');

// A promise that resolves to a Vehicle
const vehiclePromise: Promise<Vehicle> = fetchVehicle(id);
```

`Promise<Vehicle>` means "a promise that, when resolved, gives you a Vehicle."

Map<K, V>

```
// A map from string keys to number values
const ages: Map<string, number> = new Map();
ages.set('Alice', 30);
ages.set('Bob', 25);

ages.get('Alice');  // returns number | undefined
```

Partial<T>

```
interface Vehicle {
  make: string;
  model: string;
  year: number;
}

// All properties become optional
const partial: Partial<Vehicle> = {
  make: 'Toyota'
  // model and year are optional
};
```

`Partial<Vehicle>` creates a new type where all properties of `Vehicle` are optional. This is useful for update operations where you only change some fields.

Generic Constraints

Sometimes you need to limit what types can be used as a parameter.

The Problem

```
function getLength<T>(item: T): number {
  return item.length;  // Error: Property 'length' does not exist on type 'T'
}
```

TypeScript doesn't know that `T` has a `length` property. What if someone passes a number?

The Solution: extends

```
interface HasLength {
  length: number;
}

function getLength<T extends HasLength>(item: T): number {
  return item.length;  // Now TypeScript knows T has 'length'
}

getLength('hello');     // OK - strings have length
getLength([1, 2, 3]);   // OK - arrays have length
getLength(42);          // Error - numbers don't have length
```

The `extends` keyword constrains `T` to types that have a `length` property.

Constraints in vvroom

In Phase 2, you'll see constraints like:

```
interface IApiAdapter<TFilters, TData, TStatistics = any> {
  fetchData(filters: TFilters): Observable<ApiResponse<TData>>;
  // ...
}
```

The `= any` provides a default type. If you don't specify `TStatistics`, it defaults to `any`.

Applying Generics: A Preview of Phase 2

Here's how vvroom uses generics. Don't memorize this — just recognize the patterns.

The DomainConfig Interface

```
export interface DomainConfig<TFilters, TData, TStatistics> {
  // Identity
  domainKey: string;
  displayName: string;

  // Type references (so the framework knows what types to expect)
  filterModel: Type<TFilters>;
  dataModel: Type<TData>;

  // Adapters (these use the same type parameters)
  apiAdapter: IApiAdapter<TFilters, TData, TStatistics>;
  urlMapper: IFilterUrlMapper<TFilters>;

  // UI configuration
  tableConfig: TableConfig<TData>;
}
```

Reading this signature:

1. `DomainConfig` takes three type parameters
2. `TFilters` flows through to `urlMapper` and `apiAdapter`
3. `TData` flows through to `apiAdapter` and `tableConfig`
4. `TStatistics` flows through to `apiAdapter`

Using the Interface

When we create the automobile domain config:

```
const AUTOMOBILE_CONFIG: DomainConfig<
  AutomobileFilters,    // TFilters = AutomobileFilters
  VehicleResult,        // TData = VehicleResult
  VehicleStatistics     // TStatistics = VehicleStatistics
```

```
> = {
  domainKey: 'automobile',
  displayName: 'Automobiles',
  filterModel: AutomobileFilters,
  dataModel: VehicleResult,
  apiAdapter: new AutomobileApiAdapter(),
  urlMapper: new AutomobileUrlMapper(),
  tableConfig: automobileTableConfig,
};
```

Now TypeScript knows:

- `urlMapper` must accept `AutomobileFilters`
- `tableConfig` must work with `VehicleResult`
- `apiAdapter` must return `VehicleStatistics`

If any of these are wrong, TypeScript catches the error at compile time.

Practice Exercises

Try these in your editor or the TypeScript Playground (https://www.typescriptlang.org/play).

Exercise 1: Read the Signature

What type is `result` in each case?

```
function identity<T>(value: T): T {
  return value;
}

const result1 = identity('hello');
const result2 = identity(42);
const result3 = identity({ name: 'Alice' });
```

<details>

<summary>Answer</summary>

- `result1` is `string`
- `result2` is `number`
- `result3` is `{ name: string }`

TypeScript infers `T` from the argument type.

</details>

Exercise 2: Write a Generic Function

Write a function `wrapInArray` that takes any value and returns it wrapped in an array.

```
// Your code here

wrapInArray('hello');  // should be ['hello'] with type string[]
wrapInArray(42);       // should be [42] with type number[]
```

<details>

<summary>Answer</summary>

```
function wrapInArray<T>(value: T): T[] {
  return [value];
}
```

</details>

Exercise 3: Multiple Type Parameters

What are the types of `key` and `value`?

```
interface Entry<K, V> {
  key: K;
  value: V;
}

const entry: Entry<string, number> = {
  key: 'count',
  value: 42
};

const { key, value } = entry;
```

<details>

<summary>Answer</summary>

- `key` is `string` (K = string)
- `value` is `number` (V = number)

</details>

Common Mistakes

Mistake 1: Forgetting Type Parameters

```
// Wrong - DomainConfig needs type parameters
const config: DomainConfig = { /* ... */ };

// Right
const config: DomainConfig<AutomobileFilters, VehicleResult, VehicleStatistics> = { /* ... */ };
```

Mistake 2: Mismatched Types

```
interface Container<T> {
  value: T;
}

// Wrong - value type doesn't match
const box: Container<string> = {
  value: 42  // Error: number is not assignable to string
};
```

Mistake 3: Confusing Type Parameters with Values

```
// Wrong - TFilters is a type, not a value
function processFilters<TFilters>(filters: TFilters) {
  console.log(TFilters);  // Error: 'TFilters' only refers to a type
}

// Right - use the parameter, not the type
function processFilters<TFilters>(filters: TFilters) {
  console.log(filters);  // OK
}
```

Key Takeaways

1. **Generics provide type safety without code duplication** — Write once, use with any type
2. **Type parameters are placeholders** — They're filled in when you use the generic type
3. **T-prefixed names indicate type parameters** — `TFilters`, `TData`, `TStats` are conventions, not requirements

---

The Aha Moment

Generics give us type safety without code duplication.

In Phase 2, you'll create interfaces with generic parameters. These interfaces define contracts that work with *any* domain:

- `DomainConfig<TFilters, TData, TStats>` works for automobiles, real estate, or any future domain
- The framework code uses these interfaces without knowing the specific types
- Domain-specific code provides concrete types (`AutomobileFilters`, `VehicleResult`)
- TypeScript ensures everything matches up at compile time

Without generics, we'd either:

- Lose type safety (use `any` everywhere)
- Duplicate code for each domain

Generics give us both type safety and reusability. That's why they're foundational to vvroom's architecture.

Acceptance Criteria

This is a teaching section with no code changes. Criteria are conceptual:

- [ ] You can explain what problem generics solve
- [ ] You can read `Interface<T, U>` and understand T and U are type parameters
- [ ] You can identify the difference between `string` (concrete type) and `T` (type parameter)
- [ ] You can explain what `DomainConfig<TFilters, TData, TStats>` means conceptually

Next Step

Proceed to `201-domain-config-interface.md` to create the central configuration interface using the generic patterns you just learned.

Phase 2: Framework Models

Chapter 202: Domain Config Interface

Chapter 203: Filter Definition Interface

Chapter 204: Table Config Interface

Chapter 205: Chart Config Interface

Chapter 206: Picker Config Interface

Chapter 207: Query Control Config Interface

Chapter 208: Statistics Model Interface

Chapter 209: Error Notification Interface

Chapter 202

Domain Config Interface

202: Resource Management Interface

Status: Planning

Depends On: 201-domain-config-interface

Blocks: 301-url-state-service, 306-resource-management-service

---

Learning Objectives

After completing this section, you will:

- Understand the adapter pattern and why it isolates domain-specific logic from framework code
- Know how to define interfaces that different domains can implement
- Recognize the relationship between URL state, API requests, and caching

---

Objective

Create the resource management interfaces that define how domains interact with URLs, APIs, and caching. These interfaces establish the contract that domain-specific adapters must fulfill for the framework's URL-First architecture to function.

Why

The URL-First architecture requires three key operations:

1. **URL <-> Filters**: Convert between URL query parameters and domain filter objects

2. **Filters -> API**: Fetch data from the API based on current filters
3. **Filters -> Cache Key**: Generate cache keys to prevent duplicate requests

Each domain (automobiles, real estate, inventory) has different filter structures and API endpoints. The resource management interfaces define the contract that each domain's adapters must implement.

This is the Adapter Pattern in action:

```
╔═══════════════════════════════════════════════════════════════════════╗
║                     Framework Services                                ║
║  (UrlStateService, ResourceManagementService, RequestCoordinator)     ║
╚═══════════════════════════════════════════════════════════════════════╝
                              ║
                              ║ Uses interfaces
                              ▼
╔═══════════════════════════════════════════════════════════════════════╗
║             Adapter Interfaces (This Document)                        ║
║        IFilterUrlMapper, IApiAdapter, ICacheKeyBuilder                ║
╚═══════════════════════════════════════════════════════════════════════╝
                              ║
                              ║ Implemented by
                              ▼
╔═══════════════════════════════════════════════════════════════════════╗
║                  Domain-Specific Adapters                             ║
║   AutomobileUrlMapper, AutomobileApiAdapter, etc.                     ║
╚═══════════════════════════════════════════════════════════════════════╝
```

Framework services depend on *interfaces*, not concrete implementations. This means:

- Framework code never changes when adding new domains
- Each domain provides its own adapter implementations
- Type safety is maintained throughout

URL-First Architecture Reference

The resource management interfaces directly support the URL-First pattern (see `docs/README.md`):

- `IFilterUrlMapper` enables the URL to be the single source of truth
- `IApiAdapter` transforms URL-derived filters into API requests
- `ICacheKeyBuilder` prevents duplicate API calls when URL doesn't change

What

Step 202.1: Create the Resource Management Interface

Create the file `src/app/framework/models/resource-management.interface.ts`:

```
// src/app/framework/models/resource-management.interface.ts
// VERSION 1 (Section 202) - Adapter interfaces for URL-First architecture

import { Observable } from 'rxjs';
import { Params } from '@angular/router';

/**
 * Adapter for mapping filters to/from URL parameters
 *
 * This interface defines how a domain converts between its filter model
 * and URL query parameters. Implementing this interface enables the
 * URL-First architecture where the URL is the single source of truth.
 *
 * @template TFilters - The shape of the filter object
 *
 * @example
 * ```typescript
 * class AutomobileUrlMapper implements IFilterUrlMapper<AutomobileFilters> {
 *   toUrlParams(filters: AutomobileFilters): Params {
 *     const params: Params = {};
 *     if (filters.manufacturer?.length) {
 *       params['manufacturer'] = filters.manufacturer.join(',');
 *     }
 *     if (filters.yearMin) {
 *       params['yearMin'] = filters.yearMin.toString();
 *     }
 *     return params;
 *   }
 *
 *   fromUrlParams(params: Params): AutomobileFilters {
 *     return {
 *       manufacturer: params['manufacturer']?.split(',') || [],
 *       yearMin: params['yearMin'] ? parseInt(params['yearMin'], 10) : null,
 *       yearMax: params['yearMax'] ? parseInt(params['yearMax'], 10) : null
 *     };
 *   }
 * }
 * ```
 */
export interface IFilterUrlMapper<TFilters> {
  /**
   * Convert filters to URL query parameters
   *
   * Takes a domain filter object and converts it to URL-safe query parameters.
   * Empty or null values should be omitted from the result.
   *
   * @param filters - Filter object
   * @returns URL query parameters
   */
  toUrlParams(filters: TFilters): Params;

  /**
   * Convert URL query parameters to filters
   *
   * Takes URL query parameters and constructs a domain filter object.
   * Missing parameters should result in null/empty values in the filter object.
   *
   * @param params - URL query parameters
```

```
   * @returns Filter object
   */
  fromUrlParams(params: Params): TFilters;

  /**
   * Extract highlight filters from URL parameters (optional)
   *
   * Domain-specific strategy for identifying "highlight" parameters.
   * Highlight filters use h_* prefix and provide segmented statistics.
   * If not implemented, the framework assumes no highlights are present.
   *
   * @param params - URL query parameters
   * @returns Highlight filters object (domain-specific structure)
   *
   * @example
   * ```typescript
   * extractHighlights(params: Params): AutomobileHighlights {
   *   return {
   *     manufacturer: params['h_manufacturer']?.split(',') || [],
   *     yearMin: params['h_yearMin'] ? parseInt(params['h_yearMin'], 10) : null
   *   };
   * }
   * ```
   */
  extractHighlights?(params: Params): any;
}

/**
 * Response from API adapter
 *
 * Standardized response format that all API adapters must return.
 * This allows framework services to work with any domain's API.
 *
 * @template TData - The type of individual data items
 * @template TStatistics - The type of statistics object (optional)
 */
export interface ApiAdapterResponse<TData, TStatistics = any> {
  /**
   * Array of data results
   */
  results: TData[];

  /**
   * Total count of results (unpaginated)
   * Used for pagination display ("Showing 1-20 of 1,234 results")
   */
  total: number;

  /**
   * Optional statistics/aggregations
   * Domain-specific summary data (e.g., manufacturer counts, year distribution)
   */
  statistics?: TStatistics;
}

/**
 * Adapter for fetching data from API
 *
 * This interface defines how a domain fetches data from its API.
```

```
 * The adapter is responsible for:
 * - Constructing API requests from filters
 * - Transforming API responses to the standardized format
 * - Handling domain-specific API quirks
 *
 * @template TFilters - The shape of the filter object
 * @template TData - The shape of individual data items
 * @template TStatistics - The shape of statistics object (optional)
 *
 * @example
 * ```typescript
 * class AutomobileApiAdapter implements IApiAdapter<AutomobileFilters, Vehicle, VehicleStats> {
 *   constructor(private http: HttpClient, private baseUrl: string) {}
 *
 *   fetchData(
 *     filters: AutomobileFilters,
 *     highlights?: AutomobileHighlights
 *   ): Observable<ApiAdapterResponse<Vehicle, VehicleStats>> {
 *     const params = this.buildParams(filters, highlights);
 *     return this.http.get<ApiResponse>(`${this.baseUrl}/vehicles`, { params }).pipe(
 *       map(response => ({
 *         results: response.data,
 *         total: response.meta.total,
 *         statistics: response.stats
 *       }))
 *     );
 *   }
 * }
 * ```
 */
export interface IApiAdapter<TFilters, TData, TStatistics = any> {
  /**
   * Fetch data from API based on filters
   *
   * @param filters - Filter object derived from URL parameters
   * @param highlights - Optional highlight filters (h_* parameters for segmented statistics)
   * @returns Observable of API response with results, total count, and optional statistics
   */
  fetchData(
    filters: TFilters,
    highlights?: any
  ): Observable<ApiAdapterResponse<TData, TStatistics>>;
}

/**
 * Adapter for building cache keys from filters
 *
 * This interface defines how a domain generates cache keys.
 * Cache keys are used by the RequestCoordinator to:
 * - Prevent duplicate in-flight requests
 * - Enable response caching for identical filter combinations
 *
 * @template TFilters - The shape of the filter object
 *
 * @example
 * ```typescript
 * class AutomobileCacheKeyBuilder implements ICacheKeyBuilder<AutomobileFilters> {
 *   buildKey(filters: AutomobileFilters, highlights?: AutomobileHighlights): string {
 *     const filterParts = [
```

```
 *       filters.manufacturer?.join(',') || '',
 *       filters.yearMin?.toString() || '',
 *       filters.yearMax?.toString() || ''
 *     ];
 *
 *     const highlightParts = highlights ? [
 *       highlights.manufacturer?.join(',') || ''
 *     ] : [];
 *
 *     return [...filterParts, ...highlightParts].join('|');
 *   }
 * }
 * ```
 */
export interface ICacheKeyBuilder<TFilters> {
  /**
   * Build a unique cache key from filters
   *
   * The key must be unique for each distinct filter combination.
   * Highlight filters must be included to ensure segmented statistics
   * are cached separately from non-highlighted requests.
   *
   * @param filters - Filter object
   * @param highlights - Optional highlight filters (must be included in cache key)
   * @returns Cache key string
   */
  buildKey(filters: TFilters, highlights?: any): string;
}

/**
 * Configuration for ResourceManagementService
 *
 * This interface bundles all the adapters and settings needed
 * to initialize the ResourceManagementService for a specific domain.
 *
 * @template TFilters - The shape of the filter object
 * @template TData - The shape of individual data items
 * @template TStatistics - The shape of statistics object (optional)
 */
export interface ResourceManagementConfig<TFilters, TData, TStatistics = any> {
  /**
   * Filter to URL parameter mapper
   */
  filterMapper: IFilterUrlMapper<TFilters>;

  /**
   * API data fetcher
   */
  apiAdapter: IApiAdapter<TFilters, TData, TStatistics>;

  /**
   * Cache key builder
   */
  cacheKeyBuilder: ICacheKeyBuilder<TFilters>;

  /**
   * Default filter values
   * Applied when application first loads or filters are cleared
   */
```

```
  defaultFilters: TFilters;

  /**
   * Whether to enable automatic data fetching on filter changes
   * When true, changing the URL automatically triggers API requests
   * @default true
   */
  autoFetch?: boolean;

  /**
   * Cache TTL in milliseconds
   * How long cached responses are considered valid
   * @default 30000 (30 seconds)
   */
  cacheTTL?: number;
}

/**
 * State managed by ResourceManagementService
 *
 * This interface represents the complete state of a domain's data
 * as managed by the ResourceManagementService.
 *
 * @template TFilters - The shape of the filter object
 * @template TData - The shape of individual data items
 * @template TStatistics - The shape of statistics object (optional)
 */
export interface ResourceState<TFilters, TData, TStatistics = any> {
  /**
   * Current filter values
   * Derived from URL query parameters
   */
  filters: TFilters;

  /**
   * Data results from the API
   */
  results: TData[];

  /**
   * Total count of results (unpaginated)
   */
  totalResults: number;

  /**
   * Loading state
   * True while API request is in progress
   */
  loading: boolean;

  /**
   * Error state
   * Contains error object if last request failed, null otherwise
   */
  error: Error | null;

  /**
   * Optional statistics/aggregations from API
   */
```

```
  statistics?: TStatistics;

  /**
   * Optional highlight filters (h_* parameters)
   * Used for data segmentation in charts
   */
  highlights?: any;
}

/**
 * Default values for ResourceManagementConfig optional fields
 */
export const RESOURCE_MANAGEMENT_DEFAULTS = {
  autoFetch: true,
  cacheTTL: 30000 // 30 seconds
};
```

Step 202.2: Update the Barrel Export

Update `src/app/framework/models/index.ts` to include the new interface:

```
// src/app/framework/models/index.ts
// VERSION 2 (Section 202) - Added resource management exports
// Replaces VERSION 1 from Section 201

// This barrel file exports all framework model interfaces.
// Import from '@app/framework/models' instead of individual files.

export * from './domain-config.interface';
export * from './resource-management.interface';
```

Step 202.3: Understand the Adapter Pattern

The three adapter interfaces form a cohesive system:

```
═══════════════════════════════════════════════════════════════════════
│                       URL Changes                                 │
│                  ?manufacturer=Ford&yearMin=2020                  │
═══════════════════════════════════════════════════════════════════════
                              │
                              │ (1) Parse URL
                              ▼
═══════════════════════════════════════════════════════════════════════
│                     IFilterUrlMapper                              │
│              fromUrlParams(params) → TFilters                     │
═══════════════════════════════════════════════════════════════════════
                              │
                              │ (2) Build cache key
                              ▼
```

```
■■■
```

2. TypeScript Compilation Check

```
$ cd ~/projects/vvroom
$ npx tsc --noEmit src/app/framework/models/resource-management.interface.ts
```

Expected: No output (no compilation errors).

3. Verify Exports

```
$ grep "^export" src/app/framework/models/resource-management.interface.ts
```

Expected output shows interface exports:

```
export interface IFilterUrlMapper<TFilters> {
export interface ApiAdapterResponse<TData, TStatistics = any> {
export interface IApiAdapter<TFilters, TData, TStatistics = any> {
export interface ICacheKeyBuilder<TFilters> {
export interface ResourceManagementConfig<TFilters, TData, TStatistics = any> {
export interface ResourceState<TFilters, TData, TStatistics = any> {
export const RESOURCE_MANAGEMENT_DEFAULTS = {
```

4. Verify Barrel Export

```
$ grep "resource-management" src/app/framework/models/index.ts
```

Expected output:

```
export * from './resource-management.interface';
```

Common Problems

Symptom	Cause	Solution
`Cannot find module 'rxjs'`	RxJS not installed	Run `npm install rxjs`
`Cannot find module '@angular/router'`	Angular Router not installed	Included with Angular core
`Observable is not generic`	Wrong RxJS import	Use `import { Observable } from 'rxjs';`
`Params is not defined`	Missing router import	Add `import { Params } from '@angular/router';`

Interface properties showing as errors	Domain config using wrong types	Update domain-config.interface.ts to import from here

---

Key Takeaways

1. **The adapter pattern isolates domain-specific logic** — Framework services depend on interfaces, not implementations
2. **Three adapters form a cohesive system** — URL mapping, API fetching, and cache key building work together
3. **ResourceState represents the complete data state** — Loading, results, errors, and filters in one place

---

Acceptance Criteria

- [ ] `src/app/framework/models/resource-management.interface.ts` exists with all interfaces
- [ ] `IFilterUrlMapper<TFilters>` defines `toUrlParams`, `fromUrlParams`, and optional `extractHighlights`
- [ ] `IApiAdapter<TFilters, TData, TStatistics>` defines `fetchData` returning Observable
- [ ] `ICacheKeyBuilder<TFilters>` defines `buildKey` method
- [ ] `ApiAdapterResponse<TData, TStatistics>` defines standard response format
- [ ] `ResourceManagementConfig` bundles all adapters with configuration
- [ ] `ResourceState` captures complete domain data state
- [ ] Barrel file (`index.ts`) exports all resource management interfaces
- [ ] TypeScript compilation succeeds with no errors
- [ ] All interfaces have JSDoc documentation with examples

Next Step

Proceed to `203-filter-definition-interface.md` to define the filter configuration interfaces used in Query Control components.

Chapter 203

Filter Definition Interface

```
// src/app/framework/models/filter-definition.interface.ts
// VERSION 1 (Section 203) - Filter definition interfaces for Query Control

/**
 * Filter Definition Interface
 *
 * Defines the structure for filterable fields in the Query Control component.
 * This is domain-agnostic and works with any domain configuration.
 */

/**
 * Configuration for range-type filters
 *
 * Supports integer, decimal, and datetime ranges with flexible formatting.
 *
 * @example
 * ```typescript
 * // Year range (integer, no grouping)
 * rangeConfig: {
 *   valueType: 'integer',
 *   minLabel: 'Start Year',
 *   maxLabel: 'End Year',
 *   minPlaceholder: 'e.g., 1980',
 *   maxPlaceholder: 'e.g., 2023',
 *   step: 1,
 *   useGrouping: false,
 *   defaultRange: { min: 1900, max: new Date().getFullYear() }
 * }
 *
 * // Price range (decimal, with grouping)
 * rangeConfig: {
 *   valueType: 'decimal',
 *   minLabel: 'Min Price',
 *   maxLabel: 'Max Price',
 *   step: 0.01,
 *   decimalPlaces: 2,
 *   useGrouping: true,
 *   defaultRange: { min: 0, max: 1000000 }
 * }
 * ```
 */
export interface RangeConfig {
  /**
   * The type of values in the range
   * - 'integer': Whole numbers (years, counts)
   * - 'decimal': Floating point numbers (prices, measurements)
   * - 'datetime': Date/time values (ISO 8601 format)
   */
  valueType: 'integer' | 'decimal' | 'datetime';

  /**
```

```
   * Label for the minimum value input
   * @example "Start Year", "Min Price"
   */
  minLabel: string;

  /**
   * Label for the maximum value input
   * @example "End Year", "Max Price"
   */
  maxLabel: string;

  /**
   * Placeholder text for min input
   * @example "e.g., 1980"
   */
  minPlaceholder?: string;

  /**
   * Placeholder text for max input
   * @example "e.g., 2023"
   */
  maxPlaceholder?: string;

  /**
   * Step increment for numeric inputs
   * @default 1 for integer, 0.01 for decimal
   */
  step?: number;

  /**
   * Number of decimal places for 'decimal' type
   * @default 2
   */
  decimalPlaces?: number;

  /**
   * Whether to use thousand separators in display
   * Set to false for years to avoid "2,024"
   * @default false
   */
  useGrouping?: boolean;

  /**
   * Default min/max values when API doesn't provide them
   */
  defaultRange?: { min: number; max: number };
}

/**
 * Defines a filterable field in the domain
 *
 * This interface is used in DomainConfig.queryControlFilters array
 * to define all available filters for the Query Control component.
 *
 * @template T - The filter model type that contains these fields
 *
 * @example
 * ```typescript
 * // Multiselect filter for manufacturer
```

```
 * const manufacturerFilter: FilterDefinition<AutomobileFilters> = {
 *   field: 'manufacturer',
 *   label: 'Manufacturer',
 *   type: 'multiselect',
 *   optionsEndpoint: '/agg/manufacturers',
 *   urlParams: 'manufacturer',
 *   searchPlaceholder: 'Search manufacturers...',
 *   dialogTitle: 'Select Manufacturers'
 * };
 *
 * // Range filter for year
 * const yearFilter: FilterDefinition<AutomobileFilters> = {
 *   field: 'year',
 *   label: 'Model Year',
 *   type: 'range',
 *   urlParams: { min: 'yearMin', max: 'yearMax' },
 *   rangeConfig: {
 *     valueType: 'integer',
 *     minLabel: 'Start Year',
 *     maxLabel: 'End Year',
 *     useGrouping: false,
 *     defaultRange: { min: 1900, max: 2025 }
 *   }
 * };
 * ```
 */
export interface FilterDefinition<T = any> {
  /**
   * Unique field identifier matching a property in the filter model
   * Uses keyof T for type safety
   */
  field: keyof T;

  /**
   * Display label for the filter shown in Query Control component
   */
  label: string;

  /**
   * Filter type determines the UI component rendered
   * - 'multiselect': Dropdown with checkboxes
   * - 'range': Two inputs for min/max values
   * - 'text': Single text input
   * - 'date': Date picker
   */
  type: 'multiselect' | 'range' | 'text' | 'date';

  /**
   * API endpoint for fetching multiselect filter options
   * Relative to the domain's apiBaseUrl
   *
   * @example 'agg/manufacturers'
   * Fetches from: {apiBaseUrl}/agg/manufacturers
   */
  optionsEndpoint?: string;

  /**
   * Transformer function to convert API response to FilterOption[]
   * Required when API response doesn't match expected format
```

```
   *
   * @example
   * optionsTransformer: (response) => response.values.map(v => ({
   *   value: v.name,
   *   label: v.name,
   *   count: v.count
   * }))
   */
  optionsTransformer?: (response: any) => FilterOption[];

  /**
   * URL parameter name(s) for state synchronization
   *
   * For simple filters (text, multiselect): string
   * For range filters: object with min and max keys
   *
   * @example
   * urlParams: 'manufacturer'           // ?manufacturer=Ford,Toyota
   * urlParams: { min: 'yearMin', max: 'yearMax' }  // ?yearMin=2020&yearMax=2024
   */
  urlParams: string | { min: string; max: string };

  /**
   * Placeholder text for search box in multiselect dialogs
   */
  searchPlaceholder?: string;

  /**
   * Subtitle text shown in filter dialog/modal
   */
  dialogSubtitle?: string;

  /**
   * Title for the filter dialog
   * @default "Select {label}"
   */
  dialogTitle?: string;

  /**
   * Configuration for type='range' filters
   * Required when type is 'range'
   */
  rangeConfig?: RangeConfig;
}

/**
 * Option for multiselect filters
 *
 * Represents a single selectable option in a multiselect dropdown.
 *
 * @example
 * ```typescript
 * // API response format (from /agg/manufacturers endpoint)
 * {
 *   field: "manufacturer",
 *   values: [
 *     { value: "Toyota", label: "Toyota", count: 1543 },
 *     { value: "Honda", label: "Honda", count: 1234 },
 *     { value: "Ford", label: "Ford", count: 987 }
```

```
 *   ]
 * }
 * ```
 */
export interface FilterOption {
  /**
   * The actual value used when filter is selected
   * Sent to API and stored in URL
   */
  value: string | number;

  /**
   * Display text shown to user in dropdown
   */
  label: string;

  /**
   * Optional count of items matching this option
   * Displayed as badge: "Toyota (1,543)"
   */
  count?: number;
}
```

Chapter 204

Table Config Interface

```
// src/app/framework/models/table-config.interface.ts
// VERSION 1 (Section 204) - Table configuration for PrimeNG Table

/**
 * Table Configuration Interfaces
 *
 * Configuration-driven approach for PrimeNG Table.
 * These interfaces provide type-safe table configuration without
 * requiring custom table wrapper components.
 *
 * @example
 * ```typescript
 * const vehicleTableConfig: TableConfig<Vehicle> = {
 *   tableId: 'vehicle-table',
 *   stateKey: 'vehicle-table-state',
 *   dataKey: 'id',
 *   columns: [
 *     { field: 'manufacturer', header: 'Manufacturer', sortable: true },
 *     { field: 'model', header: 'Model', sortable: true, filterable: true }
 *   ],
 *   expandable: true,
 *   selectable: false,
 *   paginator: true,
 *   rows: 20,
 *   lazy: true,
 *   stateStorage: 'local'
 * };
 * ```
 */

/**
 * PrimeNG column configuration
 *
 * Defines a single column in a PrimeNG Table with type-safe field reference.
 *
 * @template T - The data model type
 */
export interface PrimeNGColumn<T> {
  /**
   * Field name from data model (type-safe via keyof T)
   */
  field: keyof T;

  /**
   * Display header text
   */
  header: string;

  /**
   * Enable column sorting
   * @default false
```

```
 */
sortable?: boolean;

/**
 * Enable column filtering
 * @default false
 */
filterable?: boolean;

/**
 * Filter match mode for PrimeNG
 * @default 'contains'
 */
filterMatchMode?:
  | 'startsWith'
  | 'contains'
  | 'notContains'
  | 'endsWith'
  | 'equals'
  | 'notEquals'
  | 'in'
  | 'lt'
  | 'lte'
  | 'gt'
  | 'gte'
  | 'between'
  | 'dateIs'
  | 'dateIsNot'
  | 'dateBefore'
  | 'dateAfter';

/**
 * Enable column reordering
 * @default true
 */
reorderable?: boolean;

/**
 * Column width (CSS value: '100px', '20%', etc.)
 */
width?: string;

/**
 * Frozen column (stick to left/right during horizontal scroll)
 * @default false
 */
frozen?: boolean;

/**
 * Alignment of column content
 * @default 'left'
 */
align?: 'left' | 'center' | 'right';

/**
 * Custom CSS classes for column
 */
styleClass?: string;
```

```
  /**
   * Data type for better filtering/sorting
   */
  dataType?: 'text' | 'numeric' | 'date' | 'boolean';
}

/**
 * Table configuration for PrimeNG Table
 *
 * Comprehensive configuration object that drives PrimeNG Table behavior.
 * Use this instead of creating custom table wrapper components.
 *
 * @template T - The data model type
 */
export interface TableConfig<T> {
  /**
   * Unique identifier for this table instance
   * Used for debugging and state management
   */
  tableId: string;

  /**
   * State storage key for PrimeNG's stateStorage feature
   * Used to persist column order, filters, etc. in local/session storage
   */
  stateKey: string;

  /**
   * Column definitions
   */
  columns: PrimeNGColumn<T>[];

  /**
   * Unique key field for row tracking (required for expandable/selectable)
   * Should be a unique identifier field like 'id' or 'vin'
   */
  dataKey: keyof T;

  /**
   * Enable row expansion
   * When true, rows can be expanded to show additional details
   * @default false
   */
  expandable?: boolean;

  /**
   * Enable row selection
   * @default false
   */
  selectable?: boolean;

  /**
   * Selection mode if selectable is true
   * @default 'multiple'
   */
  selectionMode?: 'single' | 'multiple';

  /**
   * Enable paginator
```

```
 * @default true
 */
paginator?: boolean;

/**
 * Number of rows per page
 * @default 20
 */
rows?: number;

/**
 * Rows per page options for paginator dropdown
 * @default [10, 20, 50, 100]
 */
rowsPerPageOptions?: number[];

/**
 * Enable lazy loading mode (server-side pagination/sorting/filtering)
 * When true, table emits events instead of handling data locally
 * @default true
 */
lazy?: boolean;

/**
 * State storage mode for persisting table state
 * - 'local': localStorage (persists across sessions)
 * - 'session': sessionStorage (cleared on browser close)
 * - null: no persistence
 * @default 'local'
 */
stateStorage?: 'local' | 'session' | null;

/**
 * Enable responsive mode
 * @default true
 */
responsive?: boolean;

/**
 * Responsive layout mode
 * - 'scroll': Horizontal scrollbar on small screens
 * - 'stack': Stack columns vertically on small screens
 * @default 'scroll'
 */
responsiveLayout?: 'scroll' | 'stack';

/**
 * Enable column resizing
 * @default false
 */
resizableColumns?: boolean;

/**
 * Column resize mode
 * - 'fit': Resizing a column adjusts adjacent columns
 * - 'expand': Resizing a column expands table width
 * @default 'fit'
 */
columnResizeMode?: 'fit' | 'expand';
```

```
  /**
   * Enable column reordering via drag-and-drop
   * @default true
   */
  reorderableColumns?: boolean;

  /**
   * CSS style classes for table element
   */
  styleClass?: string;

  /**
   * Show gridlines between cells
   * @default true
   */
  gridlines?: boolean;

  /**
   * Alternate row colors (zebra striping)
   * @default true
   */
  stripedRows?: boolean;

  /**
   * Show table caption/header
   */
  showCaption?: boolean;

  /**
   * Caption text
   */
  caption?: string;

  /**
   * Enable virtual scrolling for large datasets
   * When true, only visible rows are rendered
   * @default false
   */
  virtualScroll?: boolean;

  /**
   * Virtual scroll item size (height in pixels)
   * Required if virtualScroll is true
   */
  virtualScrollItemSize?: number;

  /**
   * Show loading indicator
   * Typically bound to loading state from service
   * @default false
   */
  loading?: boolean;
}

/**
 * Table state for managing table UI state
 *
 * Represents the runtime state of a table instance.
 * Used by components to track selection, expansion, pagination, etc.
```

```
 *
 * @template T - The data model type
 */
export interface TableState<T> {
  /**
   * Currently selected rows
   */
  selection: T[];

  /**
   * Expanded row keys
   * Key is the value of dataKey field, value is boolean
   */
  expandedRowKeys: { [key: string]: boolean };

  /**
   * First row index (for pagination, 0-indexed)
   */
  first: number;

  /**
   * Rows per page
   */
  rows: number;

  /**
   * Total records (for pagination display)
   */
  totalRecords: number;

  /**
   * Current sort field
   */
  sortField?: keyof T;

  /**
   * Sort order (1 = ascending, -1 = descending)
   */
  sortOrder?: 1 | -1;

  /**
   * Active filters (PrimeNG filter format)
   */
  filters?: { [key: string]: any };
}

/**
 * Create default table configuration
 *
 * Utility function that provides sensible defaults for common table setups.
 * Override specific properties as needed.
 *
 * @template T - The data model type
 * @param tableId - Unique table identifier
 * @param dataKey - Unique key field for row tracking
 * @param columns - Column definitions
 * @returns Complete TableConfig with defaults applied
 *
 * @example
```

```
 * ```typescript
 * const config = getDefaultTableConfig<Vehicle>(
 *   'vehicle-table',
 *   'vin',
 *   [
 *     { field: 'manufacturer', header: 'Manufacturer', sortable: true },
 *     { field: 'model', header: 'Model', sortable: true }
 *   ]
 * );
 * ```
 */
export function getDefaultTableConfig<T>(
  tableId: string,
  dataKey: keyof T,
  columns: PrimeNGColumn<T>[]
): TableConfig<T> {
  return {
    tableId,
    stateKey: `${tableId}-state`,
    columns,
    dataKey,
    expandable: false,
    selectable: false,
    selectionMode: 'multiple',
    paginator: true,
    rows: 20,
    rowsPerPageOptions: [10, 20, 50, 100],
    lazy: true,
    stateStorage: 'local',
    responsive: true,
    responsiveLayout: 'scroll',
    resizableColumns: false,
    columnResizeMode: 'fit',
    reorderableColumns: true,
    styleClass: 'p-datatable-striped p-datatable-gridlines',
    gridlines: true,
    stripedRows: true,
    showCaption: false,
    virtualScroll: false,
    loading: false
  };
}

/**
 * Get visible columns from configuration
 *
 * Filters out hidden columns and returns array suitable for template iteration.
 * Currently returns all columns; extend to support column visibility.
 *
 * @template T - The data model type
 * @param config - Table configuration
 * @returns Array of visible columns
 */
export function getVisibleColumns<T>(
  config: TableConfig<T>
): PrimeNGColumn<T>[] {
  return config.columns;
}
```

```
/**
 * Extract PrimeNG Table bindings from config
 *
 * Converts TableConfig to object suitable for PrimeNG Table attribute binding.
 * Use with spread operator: `<p-table ...getTableBindings(config)>`
 *
 * @template T - The data model type
 * @param config - Table configuration
 * @returns Object with PrimeNG Table bindings
 */
export function getTableBindings<T>(config: TableConfig<T>): {
  dataKey: keyof T;
  stateStorage: 'local' | 'session' | null;
  stateKey: string;
  paginator: boolean;
  rows: number;
  rowsPerPageOptions: number[];
  lazy: boolean;
  reorderableColumns: boolean;
  resizableColumns: boolean;
  columnResizeMode: 'fit' | 'expand';
  responsive: boolean;
  responsiveLayout: 'scroll' | 'stack';
  styleClass: string;
} {
  return {
    dataKey: config.dataKey,
    stateStorage: config.stateStorage ?? 'local',
    stateKey: config.stateKey,
    paginator: config.paginator ?? true,
    rows: config.rows ?? 20,
    rowsPerPageOptions: config.rowsPerPageOptions ?? [10, 20, 50, 100],
    lazy: config.lazy ?? true,
    reorderableColumns: config.reorderableColumns ?? true,
    resizableColumns: config.resizableColumns ?? false,
    columnResizeMode: config.columnResizeMode ?? 'fit',
    responsive: config.responsive ?? true,
    responsiveLayout: config.responsiveLayout ?? 'scroll',
    styleClass: config.styleClass ?? 'p-datatable-striped p-datatable-gridlines'
  };
}
```

Chapter 205

Chart Config Interface

```
// src/app/framework/models/picker-config.interface.ts
// VERSION 1 (Section 205) - Picker configuration for selection tables

import { Observable } from 'rxjs';
import { PrimeNGColumn } from './table-config.interface';

/**
 * Picker Configuration Interfaces
 *
 * Configuration-driven approach for selection pickers (multi-select tables).
 * Pickers use PrimeNG Table with checkboxes for row selection and
 * synchronize selections with URL query parameters.
 *
 * @example
 * ```typescript
 * const manufacturerPickerConfig: PickerConfig<Manufacturer> = {
 *   id: 'manufacturer-picker',
 *   displayName: 'Select Manufacturers',
 *   columns: [
 *     { field: 'name', header: 'Manufacturer', sortable: true },
 *     { field: 'country', header: 'Country', sortable: true }
 *   ],
 *   api: {
 *     fetchData: (params) => http.get('/api/manufacturers', { params }),
 *     responseTransformer: (r) => ({ results: r.data, total: r.total })
 *   },
 *   row: {
 *     keyGenerator: (row) => row.id,
 *     keyParser: (key) => ({ id: key })
 *   },
 *   selection: {
 *     mode: 'multiple',
 *     urlParam: 'manufacturers',
 *     serializer: (items) => items.map(i => i.id).join(','),
 *     deserializer: (url) => url.split(',').map(id => ({ id }))
 *   },
 *   pagination: {
 *     mode: 'server',
 *     defaultPageSize: 20
 *   }
 * };
 * ```
 */

/**
 * Picker API configuration
 *
 * Defines how the picker fetches data from the API.
 *
 * @template T - The data model type
 */
```

```
export interface PickerApiConfig<T> {
  /**
   * Function to fetch data from API
   * Receives pagination/filter params, returns Observable of raw response
   */
  fetchData: (params: PickerApiParams) => Observable<any>;

  /**
   * Transform API response to standard format
   * Required because different APIs have different response shapes
   */
  responseTransformer: (response: any) => PickerApiResponse<T>;

  /**
   * Optional parameter mapper
   * Maps internal picker params to API-specific format
   *
   * @example
   * paramMapper: (params) => ({
   *   offset: params.page * params.size,
   *   limit: params.size,
   *   sort: params.sortField
   * })
   */
  paramMapper?: (params: PickerApiParams) => any;
}

/**
 * Picker API request parameters
 *
 * Standard parameters sent to the API fetch function.
 */
export interface PickerApiParams {
  /**
   * Current page (0-indexed)
   */
  page: number;

  /**
   * Page size (rows per page)
   */
  size: number;

  /**
   * Sort field name
   */
  sortField?: string;

  /**
   * Sort order (1 = ascending, -1 = descending)
   */
  sortOrder?: 1 | -1;

  /**
   * Search/filter term from search box
   */
  search?: string;

  /**
```

```
   * Additional filters (domain-specific)
   */
  filters?: { [key: string]: any };
}

/**
 * Picker API response format
 *
 * Standard response format that all pickers expect.
 *
 * @template T - The data model type
 */
export interface PickerApiResponse<T> {
  /**
   * Array of result items for current page
   */
  results: T[];

  /**
   * Total number of items (for pagination)
   */
  total: number;

  /**
   * Current page number (optional, for verification)
   */
  page?: number;

  /**
   * Page size (optional, for verification)
   */
  size?: number;

  /**
   * Total pages (optional, computed from total/size)
   */
  totalPages?: number;
}

/**
 * Picker row configuration
 *
 * Defines how to generate unique keys for rows and parse them back.
 * Keys are used for selection tracking and URL serialization.
 *
 * @template T - The data model type
 */
export interface PickerRowConfig<T> {
  /**
   * Generate unique key from row data
   * Used for selection tracking and URL serialization
   *
   * @example
   * // Simple ID-based key
   * keyGenerator: (row) => row.id
   *
   * // Composite key
   * keyGenerator: (row) => `${row.manufacturer}|${row.model}`
   */
```

```
  keyGenerator: (row: T) => string;

  /**
   * Parse key back to partial row data
   * Used for URL hydration before data loads
   *
   * @example
   * // Simple ID-based parse
   * keyParser: (key) => ({ id: key })
   *
   * // Composite key parse
   * keyParser: (key) => {
   *   const [manufacturer, model] = key.split('|');
   *   return { manufacturer, model };
   * }
   */
  keyParser: (key: string) => Partial<T>;
}

/**
 * Picker selection configuration
 *
 * Defines selection behavior and URL synchronization.
 *
 * @template T - The data model type
 */
export interface PickerSelectionConfig<T> {
  /**
   * Selection mode
   * - 'single': Only one item can be selected
   * - 'multiple': Multiple items can be selected
   * @default 'multiple'
   */
  mode: 'single' | 'multiple';

  /**
   * URL query parameter name for storing selections
   *
   * @example
   * urlParam: 'selectedManufacturers'
   * // URL becomes: ?selectedManufacturers=Ford,Toyota,Honda
   */
  urlParam: string;

  /**
   * Serialize selected items to URL string
   *
   * @example
   * // Comma-separated IDs
   * serializer: (items) => items.map(i => i.id).join(',')
   *
   * // Composite keys
   * serializer: (items) => items.map(i => `${i.make}:${i.model}`).join(',')
   */
  serializer: (items: T[]) => string;

  /**
   * Deserialize URL string to partial item data
   * Returns partial objects that will be matched against loaded data
```

```
   *
   * @example
   * // Comma-separated IDs
   * deserializer: (url) => url.split(',').map(id => ({ id }))
   *
   * // Composite keys
   * deserializer: (url) => url.split(',').map(pair => {
   *   const [make, model] = pair.split(':');
   *   return { make, model };
   * })
   */
  deserializer: (urlString: string) => Partial<T>[];

  /**
   * Generate key from partial item (from deserializer)
   * If not provided, uses row.keyGenerator
   */
  keyGenerator?: (item: Partial<T>) => string;
}

/**
 * Picker pagination configuration
 */
export interface PickerPaginationConfig {
  /**
   * Pagination mode
   * - 'server': Server-side pagination (API handles paging)
   * - 'client': Client-side pagination (load all, page locally)
   * @default 'server'
   */
  mode: 'server' | 'client';

  /**
   * Default page size
   * @default 20
   */
  defaultPageSize?: number;

  /**
   * Page size options for dropdown
   * @default [10, 20, 50, 100]
   */
  pageSizeOptions?: number[];
}

/**
 * Picker caching configuration
 */
export interface PickerCachingConfig {
  /**
   * Enable caching of picker data
   * @default false
   */
  enabled: boolean;

  /**
   * Cache TTL in milliseconds
   * @default 300000 (5 minutes)
   */
```

```
  ttl?: number;
}

/**
 * Complete picker configuration
 *
 * @template T - The data model type
 */
export interface PickerConfig<T> {
  /**
   * Unique identifier for this picker
   * Used for registry lookup and state management
   */
  id: string;

  /**
   * Display name for picker (shown in UI headers)
   */
  displayName: string;

  /**
   * Column definitions
   * Reuses PrimeNGColumn from table config for consistency
   */
  columns: PrimeNGColumn<T>[];

  /**
   * API configuration
   */
  api: PickerApiConfig<T>;

  /**
   * Row key configuration
   */
  row: PickerRowConfig<T>;

  /**
   * Selection configuration
   */
  selection: PickerSelectionConfig<T>;

  /**
   * Pagination configuration
   */
  pagination: PickerPaginationConfig;

  /**
   * Optional caching configuration
   */
  caching?: PickerCachingConfig;

  /**
   * Optional description/help text
   */
  description?: string;

  /**
   * Show search box
   * @default true
```

```
   */
  showSearch?: boolean;

  /**
   * Search placeholder text
   * @default 'Search...'
   */
  searchPlaceholder?: string;
}

/**
 * Picker selection event
 *
 * Emitted when user changes selection.
 *
 * @template T - The data model type
 */
export interface PickerSelectionEvent<T> {
  /**
   * Picker ID that emitted the event
   */
  pickerId: string;

  /**
   * Selected item objects
   */
  selections: T[];

  /**
   * Selected item keys (from keyGenerator)
   */
  selectedKeys: string[];

  /**
   * Serialized URL parameter value
   */
  urlValue: string;
}

/**
 * Picker state
 *
 * Internal state tracking for picker component.
 *
 * @template T - The data model type
 */
export interface PickerState<T> {
  /**
   * All loaded data for current page
   */
  data: T[];

  /**
   * Total count (for pagination)
   */
  totalCount: number;

  /**
   * Selected row keys (as Set for O(1) lookup)
```

```
   */
  selectedKeys: Set<string>;

  /**
   * Selected row objects
   */
  selectedItems: T[];

  /**
   * Loading state
   */
  loading: boolean;

  /**
   * Error state
   */
  error: Error | null;

  /**
   * Current page (0-indexed)
   */
  currentPage: number;

  /**
   * Page size
   */
  pageSize: number;

  /**
   * Search term from search box
   */
  searchTerm: string;

  /**
   * Current sort field
   */
  sortField?: string;

  /**
   * Current sort order
   */
  sortOrder?: 1 | -1;

  /**
   * Pending hydration keys (from URL, before data loads)
   * These are keys that should be selected once data loads
   */
  pendingHydration: string[];

  /**
   * Whether data has been loaded at least once
   */
  dataLoaded: boolean;
}

/**
 * Create default picker state
 *
 * @template T - The data model type
```

```
 * @param pageSize - Initial page size
 * @returns Default PickerState
 */
export function getDefaultPickerState<T>(pageSize: number = 20): PickerState<T> {
  return {
    data: [],
    totalCount: 0,
    selectedKeys: new Set<string>(),
    selectedItems: [],
    loading: false,
    error: null,
    currentPage: 0,
    pageSize,
    searchTerm: '',
    sortField: undefined,
    sortOrder: undefined,
    pendingHydration: [],
    dataLoaded: false
  };
}
```

Chapter 206

Picker Config Interface

```
// src/app/framework/models/api-response.interface.ts
// VERSION 1 (Section 206) - Standardized API response format

/**
 * Generic API response interface for paginated endpoints
 *
 * This interface defines the standard response format for all paginated
 * API endpoints in the vvroom application. Both the frontend and backend
 * agree on this format, enabling type-safe data handling.
 *
 * @template TData - The type of data items in the results array
 *
 * @example
 * ```typescript
 * interface Vehicle {
 *   vin: string;
 *   manufacturer: string;
 *   model: string;
 *   year: number;
 * }
 *
 * // Typed HTTP request
 * this.http.get<ApiResponse<Vehicle>>('/api/vehicles', { params })
 *   .subscribe(response => {
 *     console.log(response.results);     // Vehicle[]
 *     console.log(response.total);       // number
 *     console.log(response.page);        // number
 *     console.log(response.totalPages);  // number
 *   });
 * ```
 */
export interface ApiResponse<TData> {
  /**
   * Array of result items for the current page
   */
  results: TData[];

  /**
   * Total number of items across all pages
   * Used for pagination display: "Showing 1-20 of 1,234 results"
   */
  total: number;

  /**
   * Current page number (1-indexed)
   * Page 1 is the first page
   */
  page: number;

  /**
   * Number of items per page
```

```
   * Matches the `size` query parameter
   */
  size: number;

  /**
   * Total number of pages
   * Computed as: Math.ceil(total / size)
   */
  totalPages: number;

  /**
   * Optional statistics data (domain-specific)
   * Contains aggregations, counts, and other computed data
   *
   * @example
   * statistics: {
   *   manufacturerCounts: { Ford: 150, Toyota: 120, Honda: 89 },
   *   yearRange: { min: 1990, max: 2024 },
   *   totalVins: 1234
   * }
   */
  statistics?: any;
}

/**
 * Generic error response from API
 *
 * Defines the standard error format returned by the API server.
 * All API errors follow this structure for consistent error handling.
 *
 * @example
 * ```json
 * {
 *   "success": false,
 *   "error": {
 *     "code": "VALIDATION_ERROR",
 *     "message": "Invalid year range: min cannot exceed max",
 *     "details": {
 *       "field": "yearMin",
 *       "value": 2025,
 *       "constraint": "must be less than or equal to yearMax"
 *     }
 *   }
 * }
 * ```
 */
export interface ApiErrorResponse {
  /**
   * Success flag (always false for errors)
   */
  success: false;

  /**
   * Error details
   */
  error: {
    /**
     * Error code for programmatic handling
     * @example 'VALIDATION_ERROR', 'NOT_FOUND', 'UNAUTHORIZED'
```

```
     */
    code: string;

    /**
     * Human-readable error message
     * Suitable for display to users
     */
    message: string;

    /**
     * Optional additional error details
     * Contains field-specific validation info, etc.
     */
    details?: Record<string, any>;
  };
}

/**
 * Generic success response wrapper
 *
 * For non-paginated endpoints that return a single object or operation result.
 *
 * @template TData - The type of the response data
 *
 * @example
 * ```typescript
 * // Create vehicle response
 * interface CreateVehicleResult {
 *   vin: string;
 *   created: boolean;
 * }
 *
 * this.http.post<ApiSuccessResponse<CreateVehicleResult>>('/api/vehicles', vehicle)
 *   .subscribe(response => {
 *     if (response.success) {
 *       console.log('Created:', response.data.vin);
 *     }
 *   });
 * ```
 */
export interface ApiSuccessResponse<TData> {
  /**
   * Success flag (always true for successful responses)
   */
  success: true;

  /**
   * Response data
   */
  data: TData;
}

/**
 * Standard API response type (success or error)
 *
 * Union type for endpoints that may return either success or error.
 * Use type guards to narrow the type:
 *
 * @example
```

```
 * ```typescript
 * function handleResponse(response: StandardApiResponse<Vehicle>) {
 *   if (response.success) {
 *     // TypeScript knows response.data exists here
 *     console.log(response.data);
 *   } else {
 *     // TypeScript knows response.error exists here
 *     console.error(response.error.message);
 *   }
 * }
 * ```
 */
export type StandardApiResponse<TData> = ApiSuccessResponse<TData> | ApiErrorResponse;
```

Chapter 207

Query Control Config Interface

```
// src/app/framework/models/pagination.interface.ts
// VERSION 1 (Section 207) - Pagination and sorting interfaces

/**
 * Pagination Interfaces
 *
 * These interfaces define pagination and sorting parameters for API requests
 * and responses. They complement the ApiResponse interface by defining
 * the input/output structures for paginated queries.
 */

/**
 * Pagination parameters for API requests
 *
 * Represents the pagination portion of an API request.
 * These values are derived from URL query parameters.
 *
 * @example
 * ```typescript
 * // From URL: ?page=2&size=50
 * const params: PaginationParams = {
 *   page: 2,
 *   size: 50
 * };
 *
 * // Convert to HTTP params
 * const httpParams = new HttpParams()
 *   .set('page', params.page.toString())
 *   .set('size', params.size.toString());
 * ```
 */
export interface PaginationParams {
  /**
   * Page number (1-indexed)
   * Page 1 is the first page of results
   */
  page: number;

  /**
   * Number of items per page
   * Common values: 10, 20, 50, 100
   */
  size: number;
}

/**
 * Pagination metadata from API responses
 *
 * Contains complete pagination information returned by the API.
 * Use this to display pagination controls and navigation.
 *
```

```
 * @example
 * ```typescript
 * // Display pagination info
 * const metadata: PaginationMetadata = response.pagination;
 * console.log(`Page ${metadata.page} of ${metadata.totalPages}`);
 * console.log(`Showing ${metadata.size} of ${metadata.total} results`);
 *
 * if (metadata.hasMore) {
 *   showNextButton();
 * }
 * ```
 */
export interface PaginationMetadata {
  /**
   * Current page number (1-indexed)
   */
  page: number;

  /**
   * Number of items per page
   */
  size: number;

  /**
   * Total number of items across all pages
   */
  total: number;

  /**
   * Total number of pages
   * Computed as: Math.ceil(total / size)
   */
  totalPages: number;

  /**
   * Whether there are more pages after current page
   * Equivalent to: page < totalPages
   */
  hasMore: boolean;
}

/**
 * Sorting parameters for API requests
 *
 * Represents the sorting portion of an API request.
 * These values are derived from URL query parameters.
 *
 * @example
 * ```typescript
 * // From URL: ?sortBy=year&sortOrder=desc
 * const params: SortParams = {
 *   sortBy: 'year',
 *   sortOrder: 'desc'
 * };
 *
 * // Convert to HTTP params (API-specific format)
 * const sortParam = `${params.sortBy}:${params.sortOrder}`;
 * // Results in: sort=year:desc
 * ```
```

```
 */
export interface SortParams {
  /**
   * Field to sort by
   * Must match a field name in the data model
   */
  sortBy?: string;

  /**
   * Sort direction
   * - 'asc': Ascending (A-Z, 1-9, oldest first)
   * - 'desc': Descending (Z-A, 9-1, newest first)
   */
  sortOrder?: 'asc' | 'desc';
}
```

Chapter 208

Statistics Model Interface

```
// src/app/framework/models/popout.interface.ts
// VERSION 1 (Section 208) - Pop-out window communication interfaces

/**
 * Popout Interfaces
 *
 * Defines the communication protocol between the parent application
 * and pop-out windows. Uses BroadcastChannel API for cross-window messaging.
 */

/**
 * Pop-out message structure
 *
 * All messages between parent and pop-out windows follow this format.
 *
 * @template T - The payload type for this message
 */
export interface PopOutMessage<T = any> {
  /**
   * Message type identifier
   */
  type: PopOutMessageType;

  /**
   * Message payload (type depends on message type)
   */
  payload?: T;

  /**
   * Timestamp when message was created
   * Used for debugging and ordering
   */
  timestamp?: number;
}

/**
 * Pop-out message types
 *
 * Enumeration of all possible message types in the pop-out protocol.
 */
export enum PopOutMessageType {
  /**
   * State update from parent to pop-out
   * Sent when URL/filter state changes
   */
  STATE_UPDATE = 'STATE_UPDATE',

  /**
   * Request to close a pop-out window
   * Sent from parent when pop-out should be closed
   */
```

```
CLOSE_POPOUT = 'CLOSE_POPOUT',

/**
 * Pop-out panel is ready to receive messages
 * Sent from pop-out after initialization
 */
PANEL_READY = 'PANEL_READY',

/**
 * Picker selection changed in pop-out
 * Sent from pop-out picker to update parent URL
 */
PICKER_SELECTION_CHANGE = 'PICKER_SELECTION_CHANGE',

/**
 * Filter added in pop-out
 * Sent from pop-out filter panel to update parent URL
 */
FILTER_ADD = 'FILTER_ADD',

/**
 * Filter removed in pop-out
 * Sent from pop-out filter panel to update parent URL
 */
FILTER_REMOVE = 'FILTER_REMOVE',

/**
 * Highlight removed in pop-out
 * Sent from pop-out to remove h_* parameter
 */
HIGHLIGHT_REMOVE = 'HIGHLIGHT_REMOVE',

/**
 * Clear all highlights
 * Sent from pop-out to remove all h_* parameters
 */
CLEAR_HIGHLIGHTS = 'CLEAR_HIGHLIGHTS',

/**
 * Clear all filters
 * Sent from pop-out to reset all filters
 */
CLEAR_ALL_FILTERS = 'CLEAR_ALL_FILTERS',

/**
 * URL parameters changed in parent
 * Sent to pop-outs when URL updates
 */
URL_PARAMS_CHANGED = 'URL_PARAMS_CHANGED',

/**
 * Request URL parameter sync
 * Pop-out requests current URL state from parent
 */
URL_PARAMS_SYNC = 'URL_PARAMS_SYNC',

/**
 * Chart element clicked in pop-out
 * Sent to parent to apply filter from chart click
```

```
   */
  CHART_CLICK = 'CHART_CLICK'
}

/**
 * Payload for picker selection change messages
 */
export interface PickerSelectionPayload {
  /**
   * Picker configuration ID
   */
  configId: string;

  /**
   * URL parameter name for this picker's selections
   */
  urlParam: string;

  /**
   * Serialized selection value for URL
   */
  urlValue: string;
}

/**
 * Reference to an open pop-out window
 *
 * Used by PopOutManagerService to track and manage pop-out windows.
 */
export interface PopOutWindowRef {
  /**
   * Reference to the Window object
   */
  window: Window;

  /**
   * BroadcastChannel for communication
   */
  channel: BroadcastChannel;

  /**
   * Interval ID for checking if window is still open
   */
  checkInterval: number;

  /**
   * Panel ID within the grid
   */
  panelId: string;

  /**
   * Panel type (e.g., 'chart', 'picker', 'filter')
   */
  panelType: string;
}

/**
 * Window features for pop-out creation
 *
```

```
 * Configures the appearance and behavior of pop-out windows.
 */
export interface PopOutWindowFeatures {
  /**
   * Window width in pixels
   * @default 1200
   */
  width?: number;

  /**
   * Window height in pixels
   * @default 800
   */
  height?: number;

  /**
   * Left position in pixels
   * @default 100
   */
  left?: number;

  /**
   * Top position in pixels
   * @default 100
   */
  top?: number;

  /**
   * Show menu bar
   * @default false
   */
  menubar?: boolean;

  /**
   * Show toolbar
   * @default false
   */
  toolbar?: boolean;

  /**
   * Show location/address bar
   * @default false
   */
  location?: boolean;

  /**
   * Show status bar
   * @default false
   */
  status?: boolean;

  /**
   * Allow window resizing
   * @default true
   */
  resizable?: boolean;

  /**
   * Show scrollbars when needed
```

```
   * @default true
   */
  scrollbars?: boolean;
}

/**
 * Route parameters for pop-out URLs
 *
 * Pop-out windows use a special route format:
 * /panel/:gridId/:panelId/:type
 */
export interface PopOutRouteParams {
  /**
   * Grid container ID (e.g., 'automobile-discover')
   */
  gridId: string;

  /**
   * Panel ID within the grid (e.g., 'chart-year')
   */
  panelId: string;

  /**
   * Panel type (e.g., 'chart', 'picker', 'table')
   */
  type: string;
}

/**
 * Pop-out context information
 *
 * Provides context about whether the current window is a pop-out
 * and what panel it represents.
 */
export interface PopOutContext {
  /**
   * Whether this window is a pop-out (vs main application)
   */
  isPopOut: boolean;

  /**
   * Panel ID if this is a pop-out
   */
  panelId?: string;

  /**
   * Grid ID if this is a pop-out
   */
  gridId?: string;

  /**
   * Panel type if this is a pop-out
   */
  panelType?: string;
}

/**
 * Build window features string for window.open()
 *
```

```
 * Converts PopOutWindowFeatures to the comma-separated string
 * required by window.open().
 *
 * @param features - Window feature configuration
 * @returns Feature string for window.open()
 *
 * @example
 * ```typescript
 * const features = buildWindowFeatures({ width: 800, height: 600 });
 * // "width=800,height=600,left=100,top=100,menubar=no,..."
 *
 * window.open('/panel/grid/panel/chart', 'popout-panel', features);
 * ```
 */
export function buildWindowFeatures(features: PopOutWindowFeatures): string {
  const {
    width = 1200,
    height = 800,
    left = 100,
    top = 100,
    menubar = false,
    toolbar = false,
    location = false,
    status = false,
    resizable = true,
    scrollbars = true
  } = features;

  const boolToYesNo = (val: boolean) => (val ? 'yes' : 'no');

  return [
    `width=${width}`,
    `height=${height}`,
    `left=${left}`,
    `top=${top}`,
    `menubar=${boolToYesNo(menubar)}`,
    `toolbar=${boolToYesNo(toolbar)}`,
    `location=${boolToYesNo(location)}`,
    `status=${boolToYesNo(status)}`,
    `resizable=${boolToYesNo(resizable)}`,
    `scrollbars=${boolToYesNo(scrollbars)}`
  ].join(',');
}

/**
 * Parse pop-out route to extract context
 *
 * Parses a pop-out URL path to extract grid ID, panel ID, and type.
 *
 * @param url - URL path to parse
 * @returns PopOutContext if URL matches pop-out pattern, null otherwise
 *
 * @example
 * ```typescript
 * const context = parsePopOutRoute('/panel/automobile-discover/chart-year/chart');
 * // {
 * //   isPopOut: true,
 * //   gridId: 'automobile-discover',
 * //   panelId: 'chart-year',
```

```
 * //   panelType: 'chart'
 * // }
 *
 * const notPopout = parsePopOutRoute('/discover');
 * // null
 * ```
 */
export function parsePopOutRoute(url: string): PopOutContext | null {
  // Match /panel/:gridId/:panelId/:type
  const match = url.match(/^\/panel\/([^/]+)\/([^/]+)\/([^/]+)/);

  if (!match) {
    return null;
  }

  return {
    isPopOut: true,
    gridId: match[1],
    panelId: match[2],
    panelType: match[3]
  };
}
```

Chapter 209

Error Notification Interface

```
// src/app/framework/models/error-notification.interface.ts
// VERSION 1 (Section 209) - Error notification interfaces

/**
 * Error Notification Interfaces
 *
 * Provides comprehensive error categorization and notification configuration
 * for user-facing error messages using PrimeNG Toast.
 */

/**
 * Error category enumeration
 *
 * Categorizes errors by their source and nature for appropriate handling
 * and user messaging.
 */
export enum ErrorCategory {
  /**
   * Network-related errors (connection issues, timeouts, etc.)
   * HTTP status 0 or network failures
   */
  NETWORK = 'NETWORK',

  /**
   * Validation errors (invalid input, business rule violations)
   * HTTP status 400, 422
   */
  VALIDATION = 'VALIDATION',

  /**
   * Authorization/authentication errors
   * HTTP status 401 (unauthorized), 403 (forbidden)
   */
  AUTHORIZATION = 'AUTHORIZATION',

  /**
   * Server-side errors
   * HTTP status 5xx
   */
  SERVER = 'SERVER',

  /**
   * Client-side errors (JavaScript errors, component errors)
   * Runtime errors in the application
   */
  CLIENT = 'CLIENT',

  /**
   * Application-level errors (business logic, state errors)
   * Custom application errors
   */
```

```
  APPLICATION = 'APPLICATION',

  /**
   * Unknown or uncategorized errors
   * Fallback for unrecognized error types
   */
  UNKNOWN = 'UNKNOWN'
}

/**
 * Error severity levels
 *
 * Maps to PrimeNG Toast severity levels for visual feedback.
 * - 'success': Green (not typically used for errors)
 * - 'info': Blue (informational messages)
 * - 'warn': Yellow/Orange (warnings, validation errors)
 * - 'error': Red (critical errors)
 */
export type ErrorSeverity = 'success' | 'info' | 'warn' | 'error';

/**
 * Error notification data structure
 *
 * Contains all information needed to display an error to the user.
 *
 * @example
 * ```typescript
 * const notification: ErrorNotification = {
 *   category: ErrorCategory.NETWORK,
 *   severity: 'error',
 *   summary: 'Connection Error',
 *   detail: 'Unable to reach the server. Please check your network connection.',
 *   timestamp: new Date().toISOString(),
 *   status: 0
 * };
 * ```
 */
export interface ErrorNotification {
  /**
   * Error category for categorization and routing
   */
  category: ErrorCategory;

  /**
   * Display severity (maps to PrimeNG Toast severity)
   */
  severity: ErrorSeverity;

  /**
   * Brief error summary (shown as toast title)
   * Should be short and actionable
   */
  summary: string;

  /**
   * Detailed error message (shown as toast body)
   * Provides more context about the error
   */
  detail: string;
```

```
  /**
   * Optional error code for debugging
   * From API or generated locally
   */
  code?: string;

  /**
   * Timestamp of when error occurred
   * ISO 8601 format
   */
  timestamp?: string;

  /**
   * URL where error occurred (for HTTP errors)
   */
  url?: string;

  /**
   * HTTP status code (for network errors)
   */
  status?: number;

  /**
   * Original error object (for logging/debugging)
   * Not displayed to user, but useful for console/reporting
   */
  originalError?: any;
}

/**
 * Error display options
 *
 * Configuration for how errors should be displayed to users.
 * Passed to PrimeNG Toast component.
 */
export interface ErrorDisplayOptions {
  /**
   * Auto-hide duration in milliseconds
   * Set to 0 or null to prevent auto-hide
   * @default 5000 (5 seconds)
   */
  life?: number;

  /**
   * Whether to show close button
   * @default true
   */
  closable?: boolean;

  /**
   * Whether to show in sticky mode (no auto-hide)
   * @default false
   */
  sticky?: boolean;

  /**
   * Custom CSS class for the toast
   */
  styleClass?: string;
```

```
  /**
   * Toast position key (used with multiple toast containers)
   * @default 'app-toast'
   */
  key?: string;
}

/**
 * Default error display options
 */
export const DEFAULT_ERROR_DISPLAY_OPTIONS: ErrorDisplayOptions = {
  life: 5000,
  closable: true,
  sticky: false,
  key: 'app-toast'
};

/**
 * Error severity mapping configuration
 *
 * Maps error categories to default severity levels.
 * - NETWORK, SERVER, CLIENT, APPLICATION, UNKNOWN -> 'error' (red)
 * - VALIDATION, AUTHORIZATION -> 'warn' (yellow)
 */
export const ERROR_CATEGORY_SEVERITY_MAP: Record<ErrorCategory, ErrorSeverity> = {
  [ErrorCategory.NETWORK]: 'error',
  [ErrorCategory.VALIDATION]: 'warn',
  [ErrorCategory.AUTHORIZATION]: 'warn',
  [ErrorCategory.SERVER]: 'error',
  [ErrorCategory.CLIENT]: 'error',
  [ErrorCategory.APPLICATION]: 'error',
  [ErrorCategory.UNKNOWN]: 'error'
};

/**
 * Determine error category from HTTP status code
 *
 * @param status - HTTP status code
 * @returns Appropriate error category
 *
 * @example
 * ```typescript
 * getErrorCategoryFromStatus(401); // ErrorCategory.AUTHORIZATION
 * getErrorCategoryFromStatus(500); // ErrorCategory.SERVER
 * getErrorCategoryFromStatus(0);   // ErrorCategory.NETWORK
 * ```
 */
export function getErrorCategoryFromStatus(status: number): ErrorCategory {
  // Status 0 typically means network failure
  if (status === 0) {
    return ErrorCategory.NETWORK;
  }

  // Authentication/authorization errors
  if (status === 401 || status === 403) {
    return ErrorCategory.AUTHORIZATION;
  }

  // Validation errors
```

```
  if (status === 400 || status === 422) {
    return ErrorCategory.VALIDATION;
  }

  // Server errors
  if (status >= 500 && status < 600) {
    return ErrorCategory.SERVER;
  }

  // Other client errors
  if (status >= 400 && status < 500) {
    return ErrorCategory.CLIENT;
  }

  return ErrorCategory.UNKNOWN;
}

/**
 * Determine error category from error code string
 *
 * @param code - Error code string
 * @returns Appropriate error category
 *
 * @example
 * ```typescript
 * getErrorCategoryFromCode('NETWORK_TIMEOUT');    // ErrorCategory.NETWORK
 * getErrorCategoryFromCode('VALIDATION_FAILED'); // ErrorCategory.VALIDATION
 * getErrorCategoryFromCode('UNAUTHORIZED');      // ErrorCategory.AUTHORIZATION
 * ```
 */
export function getErrorCategoryFromCode(code: string): ErrorCategory {
  const upperCode = code.toUpperCase();

  if (upperCode.includes('NETWORK') || upperCode.includes('TIMEOUT')) {
    return ErrorCategory.NETWORK;
  }

  if (upperCode.includes('VALIDATION') || upperCode.includes('INVALID')) {
    return ErrorCategory.VALIDATION;
  }

  if (
    upperCode.includes('UNAUTHORIZED') ||
    upperCode.includes('FORBIDDEN') ||
    upperCode.includes('AUTH')
  ) {
    return ErrorCategory.AUTHORIZATION;
  }

  if (upperCode.includes('SERVER') || upperCode.includes('INTERNAL')) {
    return ErrorCategory.SERVER;
  }

  if (upperCode.includes('CLIENT')) {
    return ErrorCategory.CLIENT;
  }

  return ErrorCategory.UNKNOWN;
}
```

```
/**
 * Get user-friendly summary for error category
 *
 * Internal helper function to generate summary text.
 *
 * @param category - Error category
 * @returns Summary text suitable for toast title
 */
function getSummaryForCategory(category: ErrorCategory): string {
  switch (category) {
    case ErrorCategory.NETWORK:
      return 'Connection Error';
    case ErrorCategory.VALIDATION:
      return 'Validation Error';
    case ErrorCategory.AUTHORIZATION:
      return 'Access Denied';
    case ErrorCategory.SERVER:
      return 'Server Error';
    case ErrorCategory.CLIENT:
      return 'Application Error';
    case ErrorCategory.APPLICATION:
      return 'Operation Failed';
    case ErrorCategory.UNKNOWN:
    default:
      return 'Error';
  }
}

/**
 * Create error notification from HTTP error
 *
 * Converts an HTTP error response to an ErrorNotification.
 * Typically called from HTTP error interceptor.
 *
 * @param error - HTTP error object (from HttpErrorResponse or interceptor)
 * @returns ErrorNotification object ready for display
 *
 * @example
 * ```typescript
 * // In HTTP interceptor
 * catchError((error: HttpErrorResponse) => {
 *   const notification = createErrorNotificationFromHttpError({
 *     status: error.status,
 *     message: error.message,
 *     url: error.url,
 *     code: error.error?.code
 *   });
 *   this.errorService.show(notification);
 *   return throwError(() => error);
 * })
 * ```
 */
export function createErrorNotificationFromHttpError(error: {
  status?: number;
  message?: string;
  url?: string;
  code?: string;
  timestamp?: string;
}): ErrorNotification {
```

```
  const status = error.status || 0;
  const code = error.code || 'UNKNOWN_ERROR';
  const category = getErrorCategoryFromStatus(status);
  const severity = ERROR_CATEGORY_SEVERITY_MAP[category];

  return {
    category,
    severity,
    summary: getSummaryForCategory(category),
    detail: error.message || 'An unexpected error occurred',
    code,
    timestamp: error.timestamp || new Date().toISOString(),
    url: error.url,
    status,
    originalError: error
  };
}

/**
 * Create error notification from generic Error
 *
 * Converts a JavaScript Error to an ErrorNotification.
 * Typically called from global error handler.
 *
 * @param error - JavaScript Error object
 * @returns ErrorNotification object ready for display
 *
 * @example
 * ```typescript
 * // In global error handler
 * handleError(error: Error): void {
 *   const notification = createErrorNotificationFromError(error);
 *   this.errorService.show(notification);
 * }
 * ```
 */
export function createErrorNotificationFromError(error: Error): ErrorNotification {
  const code = (error as any).code;
  const category = code
    ? getErrorCategoryFromCode(code)
    : ErrorCategory.CLIENT;
  const severity = ERROR_CATEGORY_SEVERITY_MAP[category];

  return {
    category,
    severity,
    summary: getSummaryForCategory(category),
    detail: error.message || 'An unexpected error occurred',
    code,
    timestamp: new Date().toISOString(),
    originalError: error
  };
}
```

Interlude B: RxJS Patterns

Chapter 250: RxJS Patterns Primer

Chapter 250

RxJS Patterns Primer

250: RxJS Patterns Primer

Status: Planning

Depends On: 209-error-notification-interface

Blocks: 301-url-state-service

Learning Objectives

After completing this section, you will:

- Understand why Observables are well-suited for modeling state that changes over time
- Know how to use the key RxJS operators used in vvroom: `switchMap`, `combineLatest`, `distinctUntilChanged`, `shareReplay`
- Be able to apply proper cleanup patterns to prevent memory leaks

---

Objective

Build practical RxJS knowledge before tackling the framework services in Phase 3. This is a teaching interlude focused on the specific RxJS patterns used in vvroom — it's practical, not comprehensive.

Why This Interlude Exists

Phase 3 introduces services like `UrlStateService` and `ResourceManagementService` that use RxJS extensively:

```
// From UrlStateService (Section 301)
this.filters$ = this.route.queryParams.pipe(
  map(params => this.urlMapper.fromUrlParams(params)),
  distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b)),
  shareReplay(1)
);

// From ResourceManagementService (Section 306)
this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters)),
  map(response => response.results),
  shareReplay(1)
);
```

If you've never used RxJS, this code is confusing. What does `switchMap` do? Why `shareReplay(1)`? This primer answers those questions with practical examples.

What Are Observables?

The Problem: Modeling Change Over Time

Traditional variables represent a value at a single point in time:

```
let count = 0;
count = 1;
count = 2;
// Each assignment replaces the previous value
```

But in UI applications, we often need to react to changes:

- When the URL changes, update the display
- When a filter changes, fetch new data
- When data arrives, update the table

Observables model **sequences of values over time**:

```
// Observable of values: 0, then 1, then 2, over time
const count$ = interval(1000); // Emits 0, 1, 2, ... every second

count$.subscribe(value => {
  console.log(value); // Logs each value as it arrives
});
```

The `$` suffix is a naming convention indicating "this is an Observable."

```
■■■
```

```
■                                                              ■
■  Time ■■■
```

```
// Without switchMap: every filter change triggers a request
// Results may arrive out of order, showing stale data

// With switchMap: previous request is cancelled when a new one starts
this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters))
);
```

How it works:

```
■■■
```

```
filters$:      --A----B---------->
highlights$:   ----1----2-------->
combined$:     ----[A,1][B,1][B,2]>

Emits when either changes, always with latest values.
```

Important: `combineLatest` doesn't emit until *all* source Observables have emitted at least once. If one never emits, combined never emits.

When to use: Combining multiple independent state sources.

3. `distinctUntilChanged` — Skip Duplicates

Problem: The URL might "change" to the same value (user clicks same link twice). You don't want to re-fetch identical data.

```
import { distinctUntilChanged } from 'rxjs/operators';

this.filters$ = this.route.queryParams.pipe(
  map(params => this.urlMapper.fromUrlParams(params)),
  distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b))
);
```

How it works:

```
input:    --A--A--B--B--A-->
output:   --A-----B-----A-->

Consecutive duplicates are skipped.
```

The comparator function: For objects, you need a custom comparator because `{a:1} !== {a:1}` (different object references). The example uses `JSON.stringify` for deep comparison. For large objects, consider a more efficient comparison.

When to use: Preventing unnecessary work when values haven't actually changed.

4. `shareReplay(1)` — Share and Cache

Problem: Multiple components subscribe to `filters$`. Without sharing, each subscription triggers a new execution of the Observable chain.

```
import { shareReplay } from 'rxjs/operators';

// Without shareReplay: each subscriber gets a separate stream
// URL parsing happens multiple times

// With shareReplay(1): all subscribers share one stream
this.filters$ = this.route.queryParams.pipe(
  map(params => this.urlMapper.fromUrlParams(params)),
  distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b)),
  shareReplay(1)  // Share with all subscribers, replay last value
```

```
);
```

How it works:

```
Without shareReplay:
  Subscriber A: queryParams → map → distinctUntilChanged
  Subscriber B: queryParams → map → distinctUntilChanged  (separate execution)

With shareReplay(1):
  Subscriber A ■■
                ■■ queryParams → map → distinctUntilChanged (shared)
  Subscriber B ■■
```

The `1` parameter: Replay the last 1 value to new subscribers. This means late subscribers immediately get the current value without waiting for the next emission.

When to use: Any Observable that multiple components will subscribe to.

Combining Operators: A Real Example

Here's how these operators work together in `ResourceManagementService`:

```
// Setup: combine filters and highlights
private readonly combined$ = combineLatest([
  this.filters$,
  this.highlights$
]).pipe(
  // Skip if neither actually changed
  distinctUntilChanged((a, b) =>
    JSON.stringify(a) === JSON.stringify(b)
  )
);

// Fetch data when combined state changes
this.results$ = this.combined$.pipe(
  // Cancel previous request, start new one
  switchMap(([filters, highlights]) =>
    this.apiAdapter.fetchData(filters, highlights)
  ),
  // Extract just the results
  map(response => response.results),
  // Share with all subscribers, cache last value
  shareReplay(1)
);
```

The flow:

1. `combineLatest` — Whenever filters OR highlights change, emit both
2. `distinctUntilChanged` — Skip if the combined value is the same

3. `switchMap` — Cancel any in-flight request, start a new one
4. `map` — Extract the results from the response
5. `shareReplay(1)` — Share with all components, cache the latest

Error Handling

Observables have built-in error handling with `catchError`:

```
import { catchError, of } from 'rxjs';

this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters).pipe(
    catchError(error => {
      // Log the error
      console.error('API Error:', error);

      // Notify the user
      this.errorService.notify(error);

      // Return empty results (don't break the stream)
      return of({ results: [], total: 0 });
    })
  )),
  map(response => response.results),
  shareReplay(1)
);
```

Important: `catchError` must return an Observable. Use `of()` to create an Observable from a static value.

Where to put catchError: Inside the `switchMap` callback, not after it. This way, errors in one request don't kill the entire stream — future filter changes will still trigger new requests.

Cleanup Patterns

Observables can cause memory leaks if not properly cleaned up. Every `subscribe()` must eventually `unsubscribe()`.

Pattern 1: Store and Unsubscribe

```
import { Subscription } from 'rxjs';

@Component({ /* ... */ })
export class MyComponent implements OnDestroy {
```

```
  private subscription: Subscription;

  ngOnInit() {
    this.subscription = this.filters$.subscribe(filters => {
      // Handle filters
    });
  }

  ngOnDestroy() {
    this.subscription.unsubscribe();
  }
}
```

Pattern 2: Use `takeUntil` (Preferred)

```
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

@Component({ /* ... */ })
export class MyComponent implements OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    this.filters$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(filters => {
      // Handle filters
    });

    // Multiple subscriptions can share the same destroy$
    this.results$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(results => {
      // Handle results
    });
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Pattern 3: Use the Async Pipe (Best)

The async pipe in templates automatically subscribes and unsubscribes:

```
@Component({
  template: `
    <div *ngIf="filters$ | async as filters">
      Current manufacturer: {{ filters.manufacturer }}
    </div>
  `
})
```

```
export class MyComponent {
  filters$ = this.urlStateService.filters$;
}
```

Why this is best:

- No manual subscription management
- No memory leak risk
- Angular handles everything

Use the async pipe whenever possible. Fall back to `takeUntil` when you need the value in component code.

The Aha Moment

Observables model change over time. That's why they fit state management.

In vvroom's URL-First architecture:

- The URL is state
- The URL changes over time (user navigates, clicks filters)
- Components need to react to those changes

Observables are the perfect fit:

```
// URL changes over time
this.route.queryParams  // Observable<Params>

// Filters derived from URL, changing over time
this.filters$  // Observable<TFilters>

// Results derived from filters, changing over time
this.results$  // Observable<TData[]>

// Components react to changes
this.results$.subscribe(results => this.updateTable(results));
```

The entire data flow is modeled as Observables:

```
URL → queryParams → filters → API request → results → display
     (Observable)  (Observable)            (Observable)
```

When the URL changes, the entire chain reacts automatically. You don't manually trigger updates — you describe how data flows, and RxJS handles the rest.

Quick Reference

Operator	Purpose	Use When
`switchMap`	Cancel previous, use latest	API calls from user input
`combineLatest`	Combine multiple streams	Need all current values together
`distinctUntilChanged`	Skip duplicates	Preventing unnecessary work
`shareReplay(1)`	Share and cache	Multiple subscribers
`catchError`	Handle errors	API calls that might fail
`takeUntil`	Auto-unsubscribe	Component cleanup
`map`	Transform values	Converting response formats
`filter`	Skip values	Conditional processing
`tap`	Side effects	Logging, debugging

Practice Exercises

Try these in your editor or the TypeScript Playground.

Exercise 1: Basic Observable

What does this log?

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

of(1, 2, 3).pipe(
  map(x => x * 2)
).subscribe(console.log);
```

<details>

<summary>Answer</summary>

```
2
4
6
```

`of(1, 2, 3)` emits three values. `map(x => x * 2)` doubles each.

</details>

Exercise 2: switchMap

If `fetchData(id)` returns a Promise, what's the output?

```
import { of } from 'rxjs';
import { switchMap, delay } from 'rxjs/operators';

of('A', 'B', 'C').pipe(
  switchMap(id => of(`Result: ${id}`).pipe(delay(100)))
).subscribe(console.log);
```

<details>

<summary>Answer</summary>

```
Result: C
```

Only the last value (C) produces output because `switchMap` cancels previous inner Observables when a new value arrives.

</details>

Exercise 3: combineLatest

When does this emit?

```
import { combineLatest, Subject } from 'rxjs';

const a$ = new Subject<string>();
const b$ = new Subject<number>();

combineLatest([a$, b$]).subscribe(console.log);

a$.next('X');
b$.next(1);
a$.next('Y');
```

<details>

<summary>Answer</summary>

```
['X', 1]
['Y', 1]
```

Nothing emits after `a$.next('X')` because `b$` hasn't emitted yet.

After `b$.next(1)`, both have values, so it emits `['X', 1]`.

After `a$.next('Y')`, it emits `['Y', 1]` (latest values).

</details>

Common Mistakes

Mistake 1: Forgetting to Unsubscribe

```
// Wrong: memory leak
ngOnInit() {
  this.filters$.subscribe(filters => { /* ... */ });
}

// Right: cleanup on destroy
ngOnDestroy() {
  this.subscription.unsubscribe();
}
```

Mistake 2: Nested Subscribes

```
// Wrong: callback hell, hard to manage
this.filters$.subscribe(filters => {
  this.apiAdapter.fetchData(filters).subscribe(response => {
    this.results = response.results;
  });
});

// Right: use operators to flatten
this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters)),
  map(response => response.results)
);
```

Mistake 3: Missing shareReplay

```
// Wrong: each subscriber triggers separate API calls
this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters))
);

// Right: share the result
this.results$ = this.filters$.pipe(
  switchMap(filters => this.apiAdapter.fetchData(filters)),
  shareReplay(1)
);
```

Key Takeaways

1. **Observables model change over time** — Perfect for UI state that updates
2. `switchMap` **cancels previous operations** — Use for API calls from user input
3. `shareReplay(1)` **prevents duplicate work** — Always use when multiple components subscribe

---

Acceptance Criteria

This is a teaching section with no code changes. Criteria are conceptual:

- [ ] You can explain why Observables fit state management
- [ ] You know when to use `switchMap` vs other flattening operators
- [ ] You understand what `shareReplay(1)` does and why it matters
- [ ] You can implement proper cleanup with `takeUntil` or async pipe

---

Next Step

Proceed to `301-url-state-service.md` to create the first framework service using the RxJS patterns you just learned.

Phase 3: Framework Services

Chapter 301: URL State Service

Chapter 302: API Service

Chapter 303: Request Coordinator Service

Chapter 304: Domain Config Registry

Chapter 305: Domain Config Validator

Chapter 306: Resource Management Service

Chapter 307: Highlight State Service

Chapter 308: Statistics Processor Service

Chapter 309: Export Service

Chapter 310: Keyboard Shortcuts Service

Chapter 311: Theme Service

Chapter 312: Notification Service

Chapter 313: Window Communication Service

Chapter 314: Popout Manager Service

Chapter 315: Analytics Service

Chapter 301

URL State Service

301: URL State Service

Status: Complete

Depends On: 201-209 (Framework Models), 250-rxjs-patterns-primer

Blocks: 303-request-coordinator, 306-resource-management-service

---

Learning Objectives

After completing this section, you will:

- Understand why the URL is the single source of truth for application state
- Know how to read and write URL query parameters in Angular
- Recognize the role of BehaviorSubject in providing synchronous access to asynchronous state
- Be able to implement bidirectional synchronization between application state and the URL

Objective

Create the `UrlStateService` that provides bidirectional synchronization between application state and URL query parameters. This service is the foundation of the URL-First State Management architecture.

Why

Every web application manages state. The question is: *where does that state live?*

Traditional approaches store state in:

- Component properties (lost on navigation)
- Services (lost on page refresh)
- LocalStorage (requires manual sync)

The URL-First approach stores state in the URL itself. This provides several benefits:

Feature	URL-First	Traditional State
Shareable	Copy URL to share exact application state	Users see different views
Bookmarkable	Browser bookmarks preserve state	Bookmarks lose context
Deep-linkable	External links open specific views	Links open default views
Back/Forward	Browser history "just works"	Requires manual history management
Refresh-safe	State survives page refresh	State is lost
Debuggable	State visible in browser URL bar	State hidden in memory

The Aha Moment

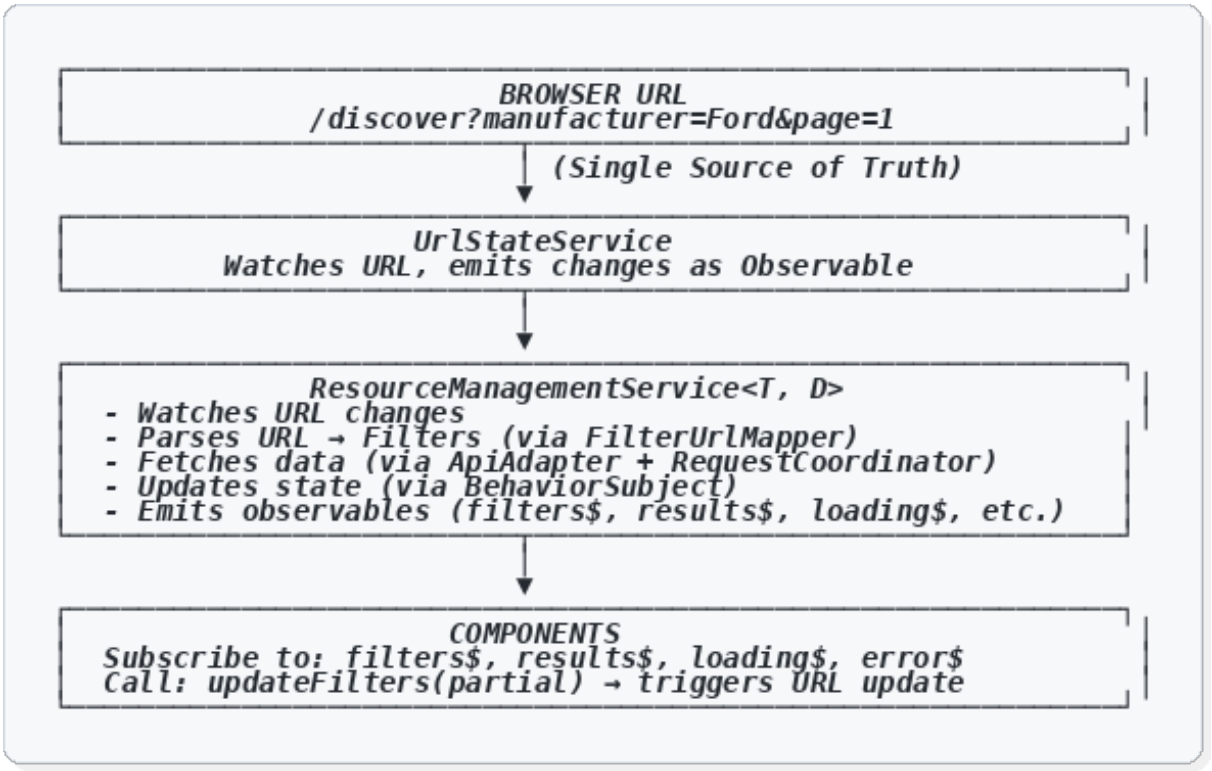
The URL is the single source of truth. Components react to URL changes, they don't control them.

When a user clicks a filter, the component doesn't update its own state. Instead, it updates the URL. The URL change triggers an observable emission, which the component subscribes to. The component always *reads* state from the URL.

This creates a unidirectional data flow:

```
User Action → URL Update → Observable Emission → Component Update → UI Render
```

Figure 301.1 illustrates this URL-First pattern visually:



URL-First state management data flow

Figure 301.1: The URL-First pattern. User actions update the URL, which triggers state updates that flow to all subscribed components. The URL serves as the single source of truth.

Angular Router Integration

The Angular Router provides `ActivatedRoute.queryParams` for reading query parameters. However, there's a subtlety: `ActivatedRoute` is scoped to the route hierarchy. A root-level service doesn't receive query param updates from child routes.

`UrlStateService` solves this by:

1. Using `Router.events` to watch all navigation events globally
2. Parsing query parameters from the router's URL directly
3. Emitting changes through a `BehaviorSubject` that any component can subscribe to

RxJS Patterns Used

This service uses patterns from Interlude B (Section 250):

Pattern	Usage
`BehaviorSubject`	Holds current URL params with synchronous access via `.value`

`filter()`	Only process `NavigationEnd` events
`map()`	Transform navigation events to query param objects
`distinctUntilChanged()`	Prevent duplicate emissions for identical params

What

Step 301.1: Create the Services Directory Index

Before creating individual service files, create a barrel export file that will aggregate all service exports.

Create the file `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 1 (Section 301) - Barrel file for framework services

// This barrel file exports all framework services.
// Import from '@app/framework/services' instead of individual files.

export * from './url-state.service';
```

Delete the `.gitkeep` file since the directory now has real content:

```
$ rm src/app/framework/services/.gitkeep
```

Step 301.2: Create the URL State Service

Create the file `src/app/framework/services/url-state.service.ts`:

```
// src/app/framework/services/url-state.service.ts
// VERSION 1 (Section 301) - URL-First state management foundation

import { Injectable, NgZone } from '@angular/core';
import { Router, ActivatedRoute, Params, NavigationEnd } from '@angular/router';
import { Observable, BehaviorSubject } from 'rxjs';
import { map, distinctUntilChanged, filter } from 'rxjs/operators';

/**
 * Domain-agnostic URL state management service
 *
 * Provides bidirectional synchronization between application state and URL query parameters.
 * The URL serves as the single source of truth for application state.
 *
```

```
 * **Key Design Decisions:**
 *
 * 1. Uses Router.events instead of ActivatedRoute.queryParams because this is a
 *    root-level singleton service. ActivatedRoute at root level doesn't receive
 *    query param updates from child routes (like /discover).
 *
 * 2. Uses Router.url and parseUrl() instead of ActivatedRoute.snapshot.queryParams
 *    for the same reason — to capture the full URL including child route params.
 *
 * 3. Uses BehaviorSubject to provide:
 *    - Synchronous access to current params via getParams()
 *    - Observable stream for reactive updates via watchParams()
 *    - Immediate value emission to new subscribers
 *
 * @example
 * ```typescript
 * // In a component
 * constructor(private urlState: UrlStateService) {
 *   // Watch for changes reactively
 *   this.urlState.watchParams<MyFilters>().subscribe(filters => {
 *     console.log('Filters changed:', filters);
 *   });
 *
 *   // Get current value synchronously
 *   const current = this.urlState.getParams<MyFilters>();
 *
 *   // Update URL (triggers navigation)
 *   this.urlState.setParams({ page: 2, search: 'ford' });
 * }
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class UrlStateService {
  /**
   * BehaviorSubject holding current URL query parameters
   *
   * BehaviorSubject is used instead of Subject because:
   * 1. It holds the current value, enabling synchronous getParams()
   * 2. New subscribers immediately receive the current value
   * 3. It requires an initial value (empty object)
   */
  private paramsSubject = new BehaviorSubject<Params>({});

  /**
   * Public observable stream of URL query parameters
   *
   * Components subscribe to this to react to URL changes.
   */
  public params$: Observable<Params> = this.paramsSubject.asObservable();

  /**
   * Constructor - sets up URL synchronization
   *
   * @param router - Angular Router for navigation and URL parsing
   * @param route - ActivatedRoute for relative navigation
   * @param ngZone - NgZone for ensuring change detection runs
   */
```

```
constructor(
  private router: Router,
  private route: ActivatedRoute,
  private ngZone: NgZone
) {
  // Initialize from current URL on service creation
  this.initializeFromRoute();

  // Watch for all future URL changes
  this.watchRouteChanges();
}

/**
 * Get current URL query parameters as a typed object
 *
 * Provides synchronous access to the current URL state.
 * Use this when you need the current value without subscribing.
 *
 * @template TParams - The shape of the parameters object
 * @returns Current query parameters cast to TParams
 *
 * @example
 * ```typescript
 * interface MyFilters {
 *   search: string;
 *   page: number;
 * }
 *
 * const filters = urlState.getParams<MyFilters>();
 * console.log(filters.search); // Type-safe access
 * ```
 */
getParams<TParams = Params>(): TParams {
  return this.paramsSubject.value as TParams;
}

/**
 * Update URL query parameters
 *
 * Performs a shallow merge with existing parameters and navigates to the new URL.
 * Use null or undefined to remove a parameter.
 *
 * @template TParams - The shape of the parameters object
 * @param params - Partial parameters to update
 * @param replaceUrl - If true, replaces current history entry instead of pushing
 * @returns Promise that resolves when navigation completes
 *
 * @example
 * ```typescript
 * // Update specific params
 * await urlState.setParams({ page: 2, search: 'test' });
 *
 * // Remove a param by setting to null
 * await urlState.setParams({ search: null });
 *
 * // Replace history entry (no new back button entry)
 * await urlState.setParams({ page: 1 }, true);
 * ```
 */
```

```
async setParams<TParams = Params>(
  params: Partial<TParams>,
  replaceUrl = false
): Promise<boolean> {
  const currentParams = this.paramsSubject.value;
  const mergedParams = { ...currentParams };

  // Merge new params, removing null/undefined values
  Object.keys(params).forEach(key => {
    const value = (params as any)[key];
    if (value === null || value === undefined) {
      delete mergedParams[key];
    } else {
      mergedParams[key] = value;
    }
  });

  return await this.router.navigate([], {
    relativeTo: this.route,
    queryParams: mergedParams,
    replaceUrl,
    queryParamsHandling: '' // Use exact params, don't preserve
  });
}

/**
 * Watch URL query parameters as an observable stream
 *
 * Returns an observable that emits whenever URL query parameters change.
 * Uses distinctUntilChanged to prevent duplicate emissions.
 *
 * @template TParams - The shape of the parameters object
 * @returns Observable of query parameters
 *
 * @example
 * ```typescript
 * urlState.watchParams<MyFilters>().subscribe(filters => {
 *   console.log('URL changed:', filters);
 *   this.loadData(filters);
 * });
 * ```
 */
watchParams<TParams = Params>(): Observable<TParams> {
  return this.params$.pipe(
    map(params => params as TParams),
    distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b))
  );
}

/**
 * Clear all URL query parameters
 *
 * Navigates to the current path with empty query string.
 *
 * @param replaceUrl - If true, replaces current history entry
 * @returns Promise that resolves when navigation completes
 */
async clearParams(replaceUrl = false): Promise<boolean> {
  return this.router.navigate([], {
```

```
    relativeTo: this.route,
    queryParams: {},
    replaceUrl
  });
}

/**
 * Get a specific query parameter value
 *
 * @param key - Parameter key
 * @returns Parameter value or null if not found
 */
getParam(key: string): any {
  return this.paramsSubject.value[key] || null;
}

/**
 * Set a specific query parameter
 *
 * Convenience method for updating a single parameter.
 *
 * @param key - Parameter key
 * @param value - Parameter value (null to remove)
 * @param replaceUrl - If true, replaces current history entry
 * @returns Promise that resolves when navigation completes
 */
async setParam(
  key: string,
  value: any,
  replaceUrl = false
): Promise<boolean> {
  return this.setParams({ [key]: value } as any, replaceUrl);
}

/**
 * Check if a specific parameter exists in the URL
 *
 * @param key - Parameter key
 * @returns True if parameter exists
 */
hasParam(key: string): boolean {
  return key in this.paramsSubject.value;
}

/**
 * Watch a specific parameter for changes
 *
 * @param key - Parameter key to watch
 * @returns Observable of parameter value
 */
watchParam(key: string): Observable<any> {
  return this.params$.pipe(
    map(params => params[key] || null),
    distinctUntilChanged()
  );
}

/**
 * Serialize parameters to URL query string
```

```
 *
 * Utility method for converting params object to query string format.
 *
 * @param params - Parameters object
 * @returns Query string (without leading '?')
 */
serializeParams(params: Params): string {
  const queryParams = new URLSearchParams();

  Object.keys(params).forEach(key => {
    const value = params[key];

    // Skip null/undefined
    if (value === null || value === undefined) {
      return;
    }

    // Handle arrays (comma-separated)
    if (Array.isArray(value)) {
      queryParams.set(key, value.join(','));
      return;
    }

    // Convert to string
    queryParams.set(key, String(value));
  });

  return queryParams.toString();
}

/**
 * Deserialize URL query string to parameters object
 *
 * Utility method for parsing query string to params object.
 * Automatically converts numeric and boolean strings.
 *
 * @param queryString - Query string (with or without leading '?')
 * @returns Parameters object
 */
deserializeParams(queryString: string): Params {
  const params: Params = {};
  const urlParams = new URLSearchParams(
    queryString.startsWith('?') ? queryString.slice(1) : queryString
  );

  urlParams.forEach((value, key) => {
    // Try to parse as number
    if (!isNaN(Number(value)) && value !== '') {
      params[key] = Number(value);
      return;
    }

    // Try to parse as boolean
    if (value === 'true' || value === 'false') {
      params[key] = value === 'true';
      return;
    }

    // Check for comma-separated values (arrays)
```

```
    if (value.includes(',')) {
      params[key] = value.split(',');
      return;
    }

    // Keep as string
    params[key] = value;
  });

  return params;
}

/**
 * Initialize from current route
 *
 * Called once in constructor to set initial state from URL.
 *
 * IMPORTANT: Uses router.url instead of route.snapshot because UrlStateService
 * is a root singleton. Root-level ActivatedRoute.snapshot may not have query
 * params from child routes (like /discover). router.url contains the full URL.
 */
private initializeFromRoute(): void {
  const params = this.extractQueryParams();
  this.ngZone.run(() => {
    this.paramsSubject.next(params);
  });
}

/**
 * Watch for route changes and update internal state
 *
 * Sets up subscription to Router.events for ongoing synchronization.
 *
 * IMPORTANT: Uses Router.events instead of ActivatedRoute.queryParams because
 * UrlStateService is a root singleton. Root-level ActivatedRoute doesn't receive
 * query param updates from child routes (like /discover). Router.events is global
 * and captures all navigation events including query parameter changes.
 */
private watchRouteChanges(): void {
  this.router.events
    .pipe(
      // Only process NavigationEnd events (route change complete)
      filter((event): event is NavigationEnd => event instanceof NavigationEnd),
      // Extract query params from the new URL
      map(() => this.extractQueryParams()),
      // Only emit if params actually changed
      distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b))
    )
    .subscribe(params => {
      // Use NgZone.run to ensure change detection runs
      this.ngZone.run(() => {
        this.paramsSubject.next(params);
      });
    });
}

/**
 * Extract query parameters from current router state
 *
```

```
   * Uses router.parseUrl() to reliably extract query params
   * regardless of the route hierarchy.
   */
  private extractQueryParams(): Params {
    const urlTree = this.router.parseUrl(this.router.url);
    return urlTree.queryParams;
  }
}
```

Step 301.3: Update the Barrel File

Update `src/app/framework/services/index.ts` to ensure the export is correct:

```
// src/app/framework/services/index.ts
// VERSION 1 (Section 301) - Barrel file for framework services

export * from './url-state.service';
```

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/
```

Expected output:

```
total 12
drwxr-xr-x 2 user user 4096 Feb  9 12:00 .
drwxr-xr-x 5 user user 4096 Feb  9 12:00 ..
-rw-r--r-- 1 user user  123 Feb  9 12:00 index.ts
-rw-r--r-- 1 user user 8456 Feb  9 12:00 url-state.service.ts
```

2. TypeScript Compilation Check

```
$ cd ~/projects/vvroom
$ npx tsc --noEmit src/app/framework/services/url-state.service.ts
```

Expected: No output (no compilation errors).

3. Verify Service Injectability

Add a temporary test to `src/app/app.component.ts`:

```
// src/app/app.component.ts
// TEMPORARY TEST - Remove after verification

import { Component, OnInit } from '@angular/core';
import { UrlStateService } from './framework/services';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
export class AppComponent implements OnInit {
  title = 'vvroom';

  constructor(private urlState: UrlStateService) {}

  ngOnInit(): void {
    // Log URL changes to console
    this.urlState.watchParams().subscribe(params => {
      console.log('[UrlStateService] URL params:', params);
    });
  }
}
```

4. Run the Application

```
$ ng serve
```

Open browser to `http://localhost:4200` and:

1. Open browser DevTools (F12) and go to Console tab
2. Navigate to `http://localhost:4200?search=ford&page=1`
3. You should see in console: `[UrlStateService] URL params: {search: 'ford', page: '1'}`
4. Modify the URL manually to `http://localhost:4200?search=toyota&page=2`
5. Console should show the new params

5. Verify Browser History

1. Navigate through several URL param changes
2. Click browser Back button
3. Console should show params reverting to previous values
4. This confirms URL-First state management is working

After verification, remove the temporary test code from AppComponent.

Common Problems

Symptom	Cause	Solution
`Cannot find module '@angular/router'`	RouterModule not imported	Ensure RouterModule is in app.module.ts imports
Console shows empty object `{}`	Route not configured	Ensure routes exist in app-routing.module.ts
Params not updating on child route navigation	Wrong ActivatedRoute	Using Router.events (as implemented) solves this
Change detection not running	Missing NgZone	Ensure ngZone.run() wraps paramsSubject.next()
Duplicate console logs	Multiple emissions	distinctUntilChanged() should prevent this
URL not updating on setParams()	Navigation blocked	Check for route guards or canDeactivate

---

Key Takeaways

1. **The URL is the single source of truth** — Components read state from the URL, they don't maintain their own copy
2. **BehaviorSubject enables both sync and async access** — `.value` for sync, `.asObservable()` for reactive streams
3. **Root services need special URL handling** — Use `Router.events` and `router.parseUrl()` instead of `ActivatedRoute.queryParams`

Figure 301.2 shows the service hierarchy and how UrlStateService fits into the broader architecture:

```
RequestCoordinatorService (Infrastructure Layer)
    ↓ (used by)
ResourceManagementService<TFilters, TData> (Generic Layer)
    ↓ (extended/configured by)
DomainConfig + ResourceManagementService (Domain Layer)
    ↓ (injected into)
Components (Presentation Layer)
```

Service hierarchy showing URL State Service position

Figure 301.2: Service hierarchy. UrlStateService sits at the foundation, providing URL-based state that higher-level services (RequestCoordinator, ResourceManagement) build upon.

Acceptance Criteria

- [ ] `src/app/framework/services/url-state.service.ts` exists with complete implementation
- [ ] `src/app/framework/services/index.ts` exports the service
- [ ] Service is `@Injectable({ providedIn: 'root' })` for singleton behavior
- [ ] `getParams<T>()` returns typed URL parameters synchronously
- [ ] `setParams<T>()` updates URL and triggers navigation
- [ ] `watchParams<T>()` returns observable stream of URL changes
- [ ] `distinctUntilChanged()` prevents duplicate emissions
- [ ] Service handles child route query params correctly via Router.events
- [ ] NgZone.run() ensures Angular change detection runs
- [ ] TypeScript compilation succeeds with no errors
- [ ] JSDoc comments document all public methods

Next Step

Proceed to `302-api-service.md` to create the service for making HTTP requests to the API.

Chapter 302

API Service

302: API Service

Status: Complete

Depends On: 206-api-response-interface

Blocks: 303-request-coordinator, 310-filter-options-service, 502-api-adapter

---

Learning Objectives

After completing this section, you will:

- Understand the role of a thin HTTP wrapper service in Angular applications
- Know how to serialize query parameters for HTTP requests
- Recognize when to handle errors at the service level vs. propagate them
- Be able to implement a domain-agnostic API service

Objective

Create the `ApiService` that provides a type-safe, domain-agnostic wrapper around Angular's `HttpClient`. This service handles common patterns like query parameter serialization, error transformation, and response typing.

Why

Every Angular application needs to make HTTP requests. The question is: *how do we structure these requests?*

Option 1: Direct HttpClient Usage

```
// In component
this.http.get<Vehicle[]>('/api/vehicles', {
  params: new HttpParams()
    .set('page', '1')
    .set('size', '20')
    .set('manufacturer', 'Ford,Toyota')
}).subscribe(data => {
  // handle response
});
```

This works, but has problems:

- Verbose param building repeated everywhere
- Error handling duplicated in each call
- No consistent typing strategy
- Hard to add cross-cutting concerns (logging, auth)

Option 2: Domain-Specific Services

```
// In VehicleService
getVehicles(filters: VehicleFilters): Observable<Vehicle[]> {
  return this.http.get<Vehicle[]>('/api/vehicles', {
    params: this.buildParams(filters)
  });
}
```

Better, but now every domain needs its own service with repeated boilerplate.

Option 3: Thin HTTP Wrapper (Our Approach)

```
// ApiService provides thin wrapper
this.api.get<Vehicle>('/api/vehicles', { params: filters })

// Domain adapter uses it
this.api.get<Vehicle>(this.baseUrl + '/vehicles', { params: filters })
```

`ApiService` handles:

- Query parameter serialization (including arrays)
- Error transformation to consistent format
- Response typing

Domain adapters (created in Phase 5) use `ApiService` for domain-specific logic.

Why Not More Abstraction?

You might wonder: *Why not build a full data layer with repositories, unit of work, etc.?*

The answer is **YAGNI (You Aren't Gonna Need It)**. The vvroom application:

- Reads data from a REST API

- Displays it in tables and charts
- Filters via URL parameters

A thin HTTP wrapper is all we need. Adding more abstraction would increase complexity without benefit.

Error Handling Strategy

`ApiService` catches errors and transforms them into consistent Error objects. It does NOT:

- Show UI notifications (that's the component's job)
- Retry failed requests (that's `RequestCoordinator`'s job)
- Log to external services (that's the error handler's job)

Each layer has its responsibility.

What

Step 302.1: Create the API Service

Create the file `src/app/framework/services/api.service.ts`:

```
// src/app/framework/services/api.service.ts
// VERSION 1 (Section 302) - Domain-agnostic HTTP wrapper

import { Injectable } from '@angular/core';
import { HttpClient, HttpParams, HttpErrorResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, map } from 'rxjs/operators';
import { ApiResponse, StandardApiResponse } from '../models/api-response.interface';

/**
 * Options for API requests
 *
 * Provides a simplified interface for common HTTP options.
 */
export interface ApiRequestOptions {
  /**
   * Query parameters to append to the URL
   * Arrays are serialized as comma-separated values
   */
  params?: Record<string, any>;

  /**
   * HTTP headers to include in the request
   */
  headers?: Record<string, string>;

  /**
```

```
   * Whether to include credentials (cookies) in the request
   */
  withCredentials?: boolean;

  /**
   * Response type (default: 'json')
   */
  responseType?: 'json' | 'text' | 'blob' | 'arraybuffer';
}

/**
 * Domain-agnostic API service for making HTTP requests
 *
 * This service provides a thin, type-safe wrapper around Angular's HttpClient.
 * It handles common patterns like query parameter serialization and error handling.
 *
 * **Design Philosophy:**
 *
 * 1. **Thin wrapper** - Doesn't add unnecessary abstraction over HttpClient
 * 2. **Type-safe** - Generic methods return typed observables
 * 3. **Domain-agnostic** - No domain-specific logic here
 * 4. **Error transformation** - Converts HTTP errors to consistent Error objects
 *
 * Domain-specific logic (like URL construction, response transformation) belongs
 * in domain adapters (created in Phase 5), not here.
 *
 * @example
 * ```typescript
 * // GET request with pagination
 * this.api.get<VehicleResult>('/api/vehicles', {
 *   params: { page: 1, size: 20, manufacturer: ['Ford', 'Toyota'] }
 * }).subscribe(response => {
 *   console.log(response.results);
 * });
 *
 * // POST request
 * this.api.post<VehicleResult>('/api/vehicles', {
 *   manufacturer: 'Ford',
 *   model: 'F-150'
 * }).subscribe(vehicle => {
 *   console.log('Created:', vehicle);
 * });
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class ApiService {
  /**
   * Constructor - injects HttpClient
   *
   * Using `readonly` to ensure the dependency isn't reassigned.
   */
  constructor(private readonly http: HttpClient) {}

  /**
   * Execute a GET request
   *
   * Returns the standard paginated API response format.
```

```
 *
 * @template TData - The type of items in the response
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of paginated response
 *
 * @example
 * ```typescript
 * this.api.get<VehicleResult>('/api/vehicles', {
 *   params: { page: 1, manufacturer: 'Ford' }
 * }).subscribe(response => {
 *   console.log(`Found ${response.total} vehicles`);
 *   console.log('Results:', response.results);
 * });
 * ```
 */
get<TData>(
  endpoint: string,
  options?: ApiRequestOptions
): Observable<ApiResponse<TData>> {
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .get<ApiResponse<TData>>(endpoint, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(catchError(this.handleError));
}

/**
 * Execute a POST request
 *
 * @template TData - The type of response data
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param body - Request body
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of response data
 */
post<TData>(
  endpoint: string,
  body: any,
  options?: ApiRequestOptions
): Observable<TData> {
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .post<TData>(endpoint, body, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(catchError(this.handleError));
}

/**
 * Execute a PUT request
 *
```

```
 * @template TData - The type of response data
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param body - Request body
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of response data
 */
put<TData>(
  endpoint: string,
  body: any,
  options?: ApiRequestOptions
): Observable<TData> {
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .put<TData>(endpoint, body, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(catchError(this.handleError));
}

/**
 * Execute a PATCH request
 *
 * @template TData - The type of response data
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param body - Request body
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of response data
 */
patch<TData>(
  endpoint: string,
  body: any,
  options?: ApiRequestOptions
): Observable<TData> {
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .patch<TData>(endpoint, body, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(catchError(this.handleError));
}

/**
 * Execute a DELETE request
 *
 * @template TData - The type of response data
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of response data
 */
delete<TData>(
  endpoint: string,
  options?: ApiRequestOptions
): Observable<TData> {
```

```
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .delete<TData>(endpoint, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(catchError(this.handleError));
}

/**
 * Execute a GET request that returns a standard success/error response
 *
 * Some APIs wrap responses in a success/error envelope:
 * ```json
 * { "success": true, "data": { ... } }
 * { "success": false, "error": { "message": "..." } }
 * ```
 *
 * This method unwraps successful responses and throws on error.
 *
 * @template TData - The type of data in the response
 * @param endpoint - API endpoint (absolute or relative URL)
 * @param options - Request options (params, headers, etc.)
 * @returns Observable of response data (unwrapped from success envelope)
 */
getStandard<TData>(
  endpoint: string,
  options?: ApiRequestOptions
): Observable<TData> {
  const httpParams = this.buildHttpParams(options?.params);

  return this.http
    .get<StandardApiResponse<TData>>(endpoint, {
      params: httpParams,
      headers: options?.headers,
      withCredentials: options?.withCredentials
    })
    .pipe(
      map(response => {
        if (response.success) {
          return response.data;
        } else {
          throw new Error(response.error.message);
        }
      }),
      catchError(this.handleError)
    );
}

/**
 * Build HttpParams from a plain object
 *
 * Handles:
 * - Null/undefined value filtering (skipped)
 * - Array serialization (comma-separated)
 * - Boolean/number to string conversion
 *
```

```
 * @param params - Plain object of query parameters
 * @returns HttpParams instance
 *
 * @example
 * ```typescript
 * // Input
 * { page: 1, manufacturer: ['Ford', 'Toyota'], active: true, empty: null }
 *
 * // Output HttpParams
 * page=1&manufacturer=Ford,Toyota&active=true
 * // Note: 'empty' is skipped because it's null
 * ```
 */
private buildHttpParams(params?: Record<string, any>): HttpParams {
  let httpParams = new HttpParams();

  if (!params) {
    return httpParams;
  }

  Object.keys(params).forEach(key => {
    const value = params[key];

    // Skip null/undefined values
    if (value === null || value === undefined) {
      return;
    }

    // Handle arrays (comma-separated)
    if (Array.isArray(value)) {
      httpParams = httpParams.set(key, value.join(','));
      return;
    }

    // Convert to string
    httpParams = httpParams.set(key, String(value));
  });

  return httpParams;
}

/**
 * Handle HTTP errors
 *
 * Transforms HttpErrorResponse into a consistent Error format.
 * Logs the error for debugging, then re-throws for upstream handling.
 *
 * @param error - HTTP error response
 * @returns Observable that throws a formatted error
 */
private handleError(error: HttpErrorResponse): Observable<never> {
  let errorMessage = 'An unknown error occurred';

  if (error.error instanceof ErrorEvent) {
    // Client-side or network error
    errorMessage = `Network error: ${error.error.message}`;
  } else {
    // Backend error
    if (error.error?.error?.message) {
```

```
        // Structured error response: { error: { message: "..." } }
        errorMessage = error.error.error.message;
      } else if (error.error?.message) {
        // Simple error response: { message: "..." }
        errorMessage = error.error.message;
      } else if (error.message) {
        // HttpErrorResponse message
        errorMessage = error.message;
      } else {
        // Fallback to status
        errorMessage = `Server error: ${error.status} ${error.statusText}`;
      }
    }

    console.error('API Error:', errorMessage, error);

    return throwError(() => new Error(errorMessage));
  }
}
```

Step 302.2: Update the Barrel File

Update `src/app/framework/services/index.ts` to export the new service:

```
// src/app/framework/services/index.ts
// VERSION 2 (Section 302) - Added ApiService

export * from './url-state.service';
export * from './api.service';
```

Step 302.3: Ensure HttpClientModule is Imported

The `ApiService` depends on `HttpClient`, which requires `HttpClientModule` to be imported in the app module.

Verify `src/app/app.module.ts` includes:

```
// src/app/app.module.ts
// Ensure HttpClientModule is imported

import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';  // <-- Required

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';

@NgModule({
```

```
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule  // <-- Must be here
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

If `HttpClientModule` is missing, add it as shown.

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/
```

Expected output includes:

```
-rw-r--r-- 1 user user 7234 Feb  9 12:00 api.service.ts
```

2. TypeScript Compilation Check

```
$ cd ~/projects/vvroom
$ npx tsc --noEmit src/app/framework/services/api.service.ts
```

Expected: No output (no compilation errors).

3. Build the Application

```
$ ng build
```

Expected: Build succeeds with no errors.

4. Verify in Browser (Optional)

If you have a working API endpoint, you can test:

```
// Temporary test in any component
```

```
constructor(private api: ApiService) {
  this.api.get<any>('http://generic-prime.minilab/api/specs/v1/automobiles', {
    params: { page: 1, size: 5 }
  }).subscribe({
    next: response => console.log('API Response:', response),
    error: err => console.error('API Error:', err)
  });
}
```

Common Problems

Symptom	Cause	Solution
`NullInjectorError: No provider for HttpClient`	HttpClientModule not imported	Add HttpClientModule to app.module.ts imports
`Cannot find module '../models/api-response.interface'`	Interface file missing	Create it in Section 206 first
CORS errors in browser	API doesn't allow cross-origin	Configure API server or use proxy
`Type 'Observable<Object>' is not assignable to...`	Generic type mismatch	Ensure response type matches expected type
Arrays serialized as `[object Object]`	Nested objects in params	Flatten objects before passing to params

---

Key Takeaways

1. **Thin wrappers reduce boilerplate** — ApiService handles serialization and errors so each call site doesn't have to
2. **Arrays serialize as comma-separated values** — This matches common REST API conventions
3. **Error handling transforms, doesn't swallow** — Errors are logged and re-thrown for upstream handling

---

Acceptance Criteria

- [ ] `src/app/framework/services/api.service.ts` exists with complete implementation

- [ ] Barrel file exports the service
- [ ] Service is `@Injectable({ providedIn: 'root' })`
- [ ] `get<T>()` method returns typed Observable
- [ ] `post<T>()`, `put<T>()`, `patch<T>()`, `delete<T>()` methods implemented
- [ ] `getStandard<T>()` unwraps success/error envelope responses
- [ ] `buildHttpParams()` correctly serializes arrays as comma-separated
- [ ] `handleError()` transforms errors consistently
- [ ] HttpClientModule is imported in app.module.ts
- [ ] TypeScript compilation succeeds
- [ ] JSDoc comments document all public methods

Next Step

Proceed to `303-request-coordinator.md` to create the service that coordinates API requests with caching and deduplication.

Chapter 303

Request Coordinator Service

303: Request Coordinator Service

Status: Complete

Depends On: 302-api-service

Blocks: 306-resource-management-service

Learning Objectives

After completing this section, you will:

- Understand the three-layer request processing model (cache, dedup, HTTP)
- Know how to implement TTL-based response caching
- Recognize the in-flight deduplication pattern for preventing duplicate requests
- Be able to implement exponential backoff retry logic

Objective

Create the `RequestCoordinatorService` that provides intelligent request coordination with three processing layers: response caching, in-flight deduplication, and HTTP execution with retry.

Why

When users interact with the vvroom application, they generate many API requests:

- Changing filters → new data fetch

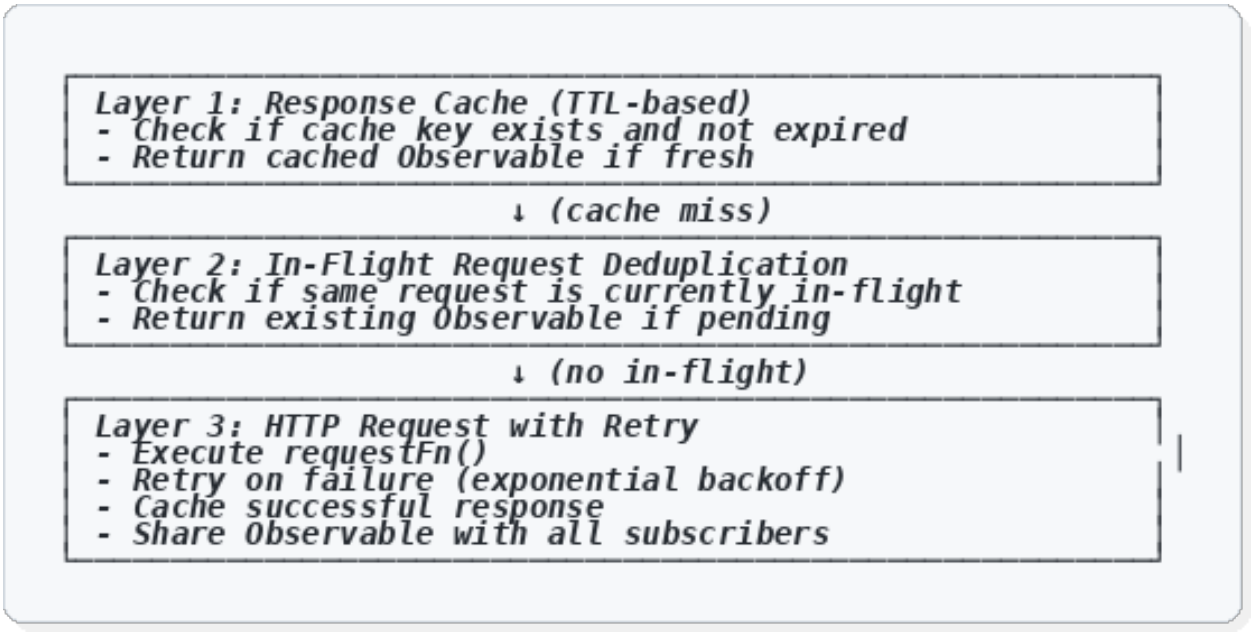
- Navigating between pages → data fetch
- Browser back/forward → URL change → data fetch
- Pop-out windows → parallel data fetch

Without coordination, these requests can:

1. **Overwhelm the API** — Same request sent multiple times
2. **Waste bandwidth** — Re-fetching data that hasn't changed
3. **Create race conditions** — Responses arriving out of order
4. **Fail silently** — Transient network errors losing data

`RequestCoordinatorService` solves these problems with a three-layer approach:

Figure 303.1 illustrates the three-layer request processing model:



Three-layer request processing: cache, dedup, HTTP

Figure 303.1: The three processing layers. Each request first checks the cache, then checks for in-flight duplicates, and finally executes HTTP with retry logic. This dramatically reduces redundant API calls.

Layer 1: Response Cache (TTL-Based)

```
Request comes in → Check cache → If fresh, return cached → Skip HTTP
```

- Stores successful responses with timestamp
- Returns cached response if within TTL
- Reduces API load for repeated queries

Layer 2: In-Flight Deduplication

```
Request comes in → Check if same request in progress → If yes, share observable → One HTTP call serves many sub
```

- Tracks ongoing requests by key
- Multiple calls to same endpoint share one observable
- Prevents duplicate concurrent requests

Layer 3: HTTP Request with Retry

```
Execute HTTP → On failure, wait → Retry with exponential backoff → After N failures, give up
```

- Executes the actual HTTP request
- Retries transient failures with exponential backoff
- Gives up after configurable number of attempts

Visual Flow

```
■■■
```

RxJS Patterns Used

Pattern	Usage
`BehaviorSubject`	Tracks loading states per request key
`shareReplay(1)`	Shares observable among multiple subscribers
`retry()`	Retries failed requests with custom delay
`timer()`	Creates delay between retries
`finalize()`	Cleanup when observable completes or errors
`tap()`	Side effect to cache successful responses

What

Step 303.1: Create the Request Coordinator Service

Create the file `src/app/framework/services/request-coordinator.service.ts`:

```
// src/app/framework/services/request-coordinator.service.ts
// VERSION 1 (Section 303) - Three-layer request coordination

import { Injectable } from '@angular/core';
import { Observable, BehaviorSubject, timer } from 'rxjs';
import {
  tap,
  finalize,
  shareReplay,
  retry,
  map,
  distinctUntilChanged
} from 'rxjs/operators';

/**
 * Configuration options for request coordination
 */
export interface RequestConfig {
  /**
   * Cache time-to-live in milliseconds
   * Responses older than this are considered stale and refetched
   * @default 30000 (30 seconds)
   */
  cacheTTL?: number;

  /**
```

```
   * Number of retry attempts on failure
   * Set to 0 to disable retries
   * @default 3
   */
  retryAttempts?: number;

  /**
   * Initial retry delay in milliseconds
   * Subsequent retries use exponential backoff (delay * 2^attempt)
   * @default 1000 (1 second)
   */
  retryDelay?: number;

  /**
   * Force bypass cache and execute fresh request
   * Use for "refresh" functionality
   * @default false
   */
  skipCache?: boolean;
}

/**
 * Cache entry structure
 */
interface CacheEntry<T> {
  /** Cached response data */
  data: T;
  /** Timestamp when cached (ms since epoch) */
  timestamp: number;
  /** Time-to-live in milliseconds */
  ttl: number;
}

/**
 * Domain-agnostic request coordination service
 *
 * Provides three-layer request processing:
 * 1. **Response Cache** (TTL-based) - Return cached response if fresh
 * 2. **In-Flight Deduplication** - Share ongoing requests
 * 3. **HTTP Request with Retry** - Execute with exponential backoff
 *
 * **Why This Matters:**
 *
 * Without coordination:
 * - User changes filter → 5 requests fire → 5 responses arrive → UI flickers
 * - User hits back button → request for data we just had → wasted bandwidth
 * - Network blip → request fails → data lost
 *
 * With coordination:
 * - User changes filter → 1 request fires → 1 response → smooth UI
 * - User hits back → cache hit → instant data → no network call
 * - Network blip → retry → data recovered
 *
 * @example
 * ```typescript
 * // Basic usage
 * this.coordinator.execute(
 *   'vehicles-page-1',
 *   () => this.api.get<Vehicle[]>('/vehicles?page=1')
```

```
 * ).subscribe(vehicles => {
 *   console.log('Vehicles:', vehicles);
 * });
 *
 * // With custom configuration
 * this.coordinator.execute(
 *   'stats-latest',
 *   () => this.api.get<Stats>('/stats'),
 *   { cacheTTL: 60000, retryAttempts: 5 }
 * ).subscribe(stats => {
 *   console.log('Statistics:', stats);
 * });
 *
 * // Force fresh fetch (skip cache)
 * this.coordinator.execute(
 *   'vehicles-page-1',
 *   () => this.api.get<Vehicle[]>('/vehicles?page=1'),
 *   { skipCache: true }
 * ).subscribe();
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class RequestCoordinatorService {
  /**
   * Response cache storage
   *
   * Maps request keys to CacheEntry objects containing:
   * - Cached data
   * - Timestamp when cached
   * - TTL for this entry
   */
  private cache = new Map<string, CacheEntry<any>>();

  /**
   * In-flight requests storage
   *
   * Maps request keys to Observable instances of ongoing HTTP requests.
   * If a second request comes in with the same key while the first is
   * still in progress, we return the same observable (deduplication).
   */
  private inFlightRequests = new Map<string, Observable<any>>();

  /**
   * Loading state per request key
   *
   * Tracks which requests are currently loading.
   * Components can subscribe to know when specific data is loading.
   */
  private loadingStateSubject = new BehaviorSubject<Map<string, boolean>>(
    new Map()
  );

  /**
   * Observable of loading states
   *
   * Emits Map of request keys to loading booleans.
   * Use getLoadingState$() or getGlobalLoading$() for filtered access.
```

```
 */
public loadingState$ = this.loadingStateSubject.asObservable();

/**
 * Execute HTTP request with caching, deduplication, and retry
 *
 * The three-layer processing:
 * 1. Check cache (return immediately if fresh)
 * 2. Check in-flight (share if same request in progress)
 * 3. Execute HTTP with retry and caching
 *
 * @template T - Response type
 * @param requestKey - Unique identifier for this request
 * @param requestFn - Function that returns Observable (lazy execution)
 * @param config - Optional request configuration
 * @returns Observable of response data (shared, cached, deduplicated)
 */
execute<T>(
  requestKey: string,
  requestFn: () => Observable<T>,
  config?: RequestConfig
): Observable<T> {
  const effectiveConfig = this.getEffectiveConfig(config);

  // Layer 1: Check response cache (unless skipCache)
  if (!effectiveConfig.skipCache) {
    const cachedResponse = this.getCachedResponse<T>(requestKey);
    if (cachedResponse !== null) {
      // Return cached data wrapped in observable
      return new Observable(observer => {
        observer.next(cachedResponse);
        observer.complete();
      });
    }
  }

  // Layer 2: Check in-flight requests (deduplication)
  if (this.inFlightRequests.has(requestKey)) {
    // Return existing observable (same request in progress)
    return this.inFlightRequests.get(requestKey)!;
  }

  // Layer 3: Execute HTTP request with retry
  this.setLoadingState(requestKey, true);

  const request$ = requestFn().pipe(
    // Retry with exponential backoff
    retry({
      count: effectiveConfig.retryAttempts!,
      delay: (error, retryCount) => {
        const delay =
          effectiveConfig.retryDelay! * Math.pow(2, retryCount - 1);
        console.log(
          `[RequestCoordinator] Retry ${retryCount} for ${requestKey} in ${delay}ms`
        );
        return timer(delay);
      }
    }),
    // Cache successful response
```

```
    tap(data => {
      if (!effectiveConfig.skipCache) {
        this.setCachedResponse(
          requestKey,
          data,
          effectiveConfig.cacheTTL!
        );
      }
    }),
    // Cleanup on complete/error
    finalize(() => {
      this.inFlightRequests.delete(requestKey);
      this.setLoadingState(requestKey, false);
    }),
    // Share with multiple subscribers (deduplication)
    shareReplay(1)
  );

  // Store in-flight request for deduplication
  this.inFlightRequests.set(requestKey, request$);

  return request$;
}

/**
 * Get loading state for specific request
 *
 * @param requestKey - Request identifier
 * @returns Observable of loading state for this request
 */
getLoadingState$(requestKey: string): Observable<boolean> {
  return this.loadingState$.pipe(
    map(stateMap => stateMap.get(requestKey) || false),
    distinctUntilChanged()
  );
}

/**
 * Get global loading state (true if any request is loading)
 *
 * @returns Observable of global loading state
 */
getGlobalLoading$(): Observable<boolean> {
  return this.loadingState$.pipe(
    map(stateMap => Array.from(stateMap.values()).some(loading => loading)),
    distinctUntilChanged()
  );
}

/**
 * Clear all cached responses or specific request cache
 *
 * @param requestKey - Optional specific request key to clear
 */
clearCache(requestKey?: string): void {
  if (requestKey) {
    this.cache.delete(requestKey);
  } else {
    this.cache.clear();
```

```
  }
}

/**
 * Invalidate specific cached response
 *
 * Alias for clearCache(requestKey) for semantic clarity.
 *
 * @param requestKey - Request identifier
 */
invalidateCache(requestKey: string): void {
  this.cache.delete(requestKey);
}

/**
 * Invalidate all cached responses matching pattern
 *
 * Useful for invalidating related caches (e.g., all vehicle pages).
 *
 * @param pattern - String pattern to match (simple substring match)
 *
 * @example
 * ```typescript
 * // Invalidate all vehicle-related caches
 * coordinator.invalidateCachePattern('vehicles');
 * // Clears: 'vehicles-page-1', 'vehicles-page-2', 'vehicles-stats'
 * ```
 */
invalidateCachePattern(pattern: string): void {
  const keysToDelete: string[] = [];

  this.cache.forEach((_, key) => {
    if (key.includes(pattern)) {
      keysToDelete.push(key);
    }
  });

  keysToDelete.forEach(key => this.cache.delete(key));
}

/**
 * Get current cache size
 *
 * @returns Number of cached entries
 */
getCacheSize(): number {
  return this.cache.size;
}

/**
 * Get number of in-flight requests
 *
 * @returns Number of ongoing requests
 */
getInFlightCount(): number {
  return this.inFlightRequests.size;
}

/**
```

```
   * Get cached response if exists and not expired
   *
   * @returns Cached data or null if not found/expired
   */
  private getCachedResponse<T>(key: string): T | null {
    const entry = this.cache.get(key);
    if (!entry) {
      return null;
    }

    // Check if expired
    const age = Date.now() - entry.timestamp;
    if (age > entry.ttl) {
      // Expired - remove and return null
      this.cache.delete(key);
      return null;
    }

    return entry.data;
  }

  /**
   * Store response in cache
   */
  private setCachedResponse<T>(key: string, data: T, ttl: number): void {
    this.cache.set(key, {
      data,
      timestamp: Date.now(),
      ttl
    });
  }

  /**
   * Update loading state for request
   */
  private setLoadingState(requestKey: string, loading: boolean): void {
    const currentStates = new Map(this.loadingStateSubject.value);

    if (loading) {
      currentStates.set(requestKey, true);
    } else {
      currentStates.delete(requestKey);
    }

    this.loadingStateSubject.next(currentStates);
  }

  /**
   * Get effective configuration with defaults
   */
  private getEffectiveConfig(config?: RequestConfig): Required<RequestConfig> {
    return {
      cacheTTL: config?.cacheTTL ?? 30000,
      retryAttempts: config?.retryAttempts ?? 3,
      retryDelay: config?.retryDelay ?? 1000,
      skipCache: config?.skipCache ?? false
    };
  }
}
```

Step 303.2: Update the Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 3 (Section 303) - Added RequestCoordinatorService

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
```

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/request-coordinator.service.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/framework/services/request-coordinator.service.ts
```

Expected: No output (no compilation errors).

3. Build the Application

```
$ ng build
```

Expected: Build succeeds with no errors.

4. Verify Deduplication (Optional)

Add temporary test code:

```
// In any component
constructor(
  private api: ApiService,
  private coordinator: RequestCoordinatorService
) {
```

```
  // Fire same request 5 times simultaneously
  for (let i = 0; i < 5; i++) {
    this.coordinator.execute(
      'test-dedup',
      () => this.api.get<any>('http://generic-prime.minilab/api/specs/v1/automobiles')
    ).subscribe(response => {
      console.log(`Request ${i} received:`, response);
    });
  }
}
```

In browser DevTools Network tab, you should see only **1** HTTP request, but 5 console logs.

5. Verify Caching (Optional)

```
// First request
this.coordinator.execute('test-cache', () => this.api.get<any>(url))
  .subscribe(() => console.log('First request complete'));

// Second request after 1 second (should be cached)
setTimeout(() => {
  this.coordinator.execute('test-cache', () => this.api.get<any>(url))
    .subscribe(() => console.log('Second request complete (from cache)'));
}, 1000);

// Third request after 35 seconds (cache expired, new request)
setTimeout(() => {
  this.coordinator.execute('test-cache', () => this.api.get<any>(url))
    .subscribe(() => console.log('Third request complete (fresh)'));
}, 35000);
```

Network tab should show 2 requests total (first and third).

Common Problems

Symptom	Cause	Solution
Multiple HTTP requests despite deduplication	Different request keys	Ensure same key for same request
Cache never expires	TTL too high	Reduce cacheTTL value
Cache always misses	TTL too low	Increase cacheTTL value
Retries not working	retryAttempts set to 0	Use default or positive number
Loading state stuck on true	Request never completes	Check for hung HTTP requests
`shareReplay` memory leak	Observable never completes	finalize() handles cleanup

Key Takeaways

1. **Three-layer processing prevents redundant requests** — Cache → Dedup → HTTP
2. `shareReplay(1)` **enables deduplication** — Multiple subscribers share one HTTP call
3. **Exponential backoff handles transient failures** — Each retry waits longer than the last

Acceptance Criteria

- [ ] `src/app/framework/services/request-coordinator.service.ts` exists
- [ ] Barrel file exports the service
- [ ] Service is `@Injectable({ providedIn: 'root' })`
- [ ] `execute()` implements three-layer processing
- [ ] Response cache stores with TTL and expires correctly
- [ ] In-flight deduplication shares observable for duplicate requests
- [ ] Retry logic uses exponential backoff
- [ ] `getLoadingState$()` returns per-request loading observable
- [ ] `getGlobalLoading$()` returns any-loading observable
- [ ] Cache invalidation methods work correctly
- [ ] TypeScript compilation succeeds
- [ ] JSDoc comments document all public methods

Next Step

Proceed to `304-domain-config-registry.md` to create the service that manages domain configurations.

Chapter 304

Domain Config Registry

304: Domain Config Registry

Status: Complete

Depends On: 201-domain-config-interface

Blocks: 305-domain-config-validator, 306-resource-management-service, 310-filter-options-service, 311-picker-config-registry

---

Learning Objectives

After completing this section, you will:

- Understand the Registry pattern and when to use it
- Know how to use Angular InjectionTokens for flexible configuration
- Recognize how runtime registration enables multi-domain applications
- Be able to implement a type-safe configuration registry

Objective

Create the `DomainConfigRegistry` service that provides centralized management of domain configurations. This registry allows domains to be registered, retrieved, and switched at runtime.

Why

The vvroom application is designed to be domain-agnostic. The framework doesn't know about automobiles — it knows about `DomainConfig`. This raises a question: *how does the framework access domain-specific*

configuration?

Option 1: Import Directly

```
// In a component
import { AUTOMOBILE_DOMAIN_CONFIG } from './domain-config/automobile';

@Component({...})
export class DiscoverComponent {
  config = AUTOMOBILE_DOMAIN_CONFIG;
}
```

This works, but:

- Hard-codes the domain in the component
- Can't switch domains at runtime
- Can't add new domains without modifying framework code

Option 2: Injection Token

```
// In a module
providers: [
  { provide: DOMAIN_CONFIG, useValue: AUTOMOBILE_DOMAIN_CONFIG }
]

// In a component
constructor(@Inject(DOMAIN_CONFIG) private config: DomainConfig<...>) {}
```

Better, but:

- Only one domain at a time
- No runtime switching
- No validation

Option 3: Registry Pattern (Our Approach)

```
// At app startup
registry.register(AUTOMOBILE_DOMAIN_CONFIG);
registry.register(REAL_ESTATE_DOMAIN_CONFIG);

// In a component
const config = registry.getActive();

// Switch domain
registry.setActive('real-estate');
```

The registry:

- Stores multiple domain configurations

- Provides runtime switching
- Validates configurations on registration
- Offers type-safe retrieval

Why Both Token AND Registry?

We use BOTH:

1. `DOMAIN_CONFIG` **token** — For component injection (simpler DI)
2. `DomainConfigRegistry` — For runtime management

The token is *populated from* the registry. Components inject `DOMAIN_CONFIG`, and the module provider gets it from the registry.

```
providers: [
  {
    provide: DOMAIN_CONFIG,
    useFactory: (registry: DomainConfigRegistry) => registry.getActive(),
    deps: [DomainConfigRegistry]
  }
]
```

What

Step 304.1: Create the Domain Config Registry Service

Create the file `src/app/framework/services/domain-config-registry.service.ts`:

```
// src/app/framework/services/domain-config-registry.service.ts
// VERSION 1 (Section 304) - Domain configuration registry

import { Injectable, InjectionToken, Injector, Provider } from '@angular/core';
import { DomainConfig } from '../models/domain-config.interface';
import { DomainConfigValidator } from './domain-config-validator.service';

/**
 * Injection token for domain configuration
 *
 * Used to provide domain-specific configuration to components.
 * The value is typically retrieved from DomainConfigRegistry.
 *
 * @example
 * ```typescript
 * // In domain module providers
 * providers: [
 *   {
 *     provide: DOMAIN_CONFIG,
 *     useFactory: (registry: DomainConfigRegistry) => registry.getActive(),
```

```
 *     deps: [DomainConfigRegistry]
 *   }
 * ]
 *
 * // In component
 * constructor(
 *   @Inject(DOMAIN_CONFIG) private domainConfig: DomainConfig<any, any, any>
 * ) {}
 * ```
 */
export const DOMAIN_CONFIG = new InjectionToken<DomainConfig<any, any, any>>(
  'Domain Configuration'
);

/**
 * Domain configuration registry service
 *
 * Centralized registry for managing multiple domain configurations.
 * Supports registering, retrieving, and switching between domains.
 *
 * **Why a Registry?**
 *
 * 1. **Multi-domain support** — Register multiple domains, switch at runtime
 * 2. **Validation** — Catch configuration errors early
 * 3. **Type safety** — Generic methods preserve type information
 * 4. **Centralized access** — Single point for all domain config access
 *
 * **Typical Usage Flow:**
 *
 * 1. App initializes → Register domain configs
 * 2. User navigates → Framework gets active config
 * 3. User switches domain → Registry changes active
 * 4. Framework components → Inject DOMAIN_CONFIG token
 *
 * @example
 * ```typescript
 * // Register domains at app initialization
 * registry.register(AUTOMOBILE_DOMAIN_CONFIG);
 * registry.register(REAL_ESTATE_DOMAIN_CONFIG);
 *
 * // Get active domain config
 * const config = registry.getActive();
 *
 * // Switch active domain
 * registry.setActive('real-estate');
 *
 * // List all registered domains
 * const domains = registry.getAllDomainNames(); // ['automobile', 'real-estate']
 *
 * // Get specific domain by name
 * const autoConfig = registry.get<AutoFilters, AutoData>('automobile');
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class DomainConfigRegistry {
  /**
   * Storage for registered domain configurations
```

```
 *
 * Maps domain name (e.g., 'automobile') to DomainConfig instance.
 */
private configs = new Map<string, DomainConfig<any, any, any>>();

/**
 * Currently active domain name
 *
 * Set to first registered domain by default.
 * Can be changed via setActive().
 */
private activeDomainName?: string;

/**
 * Constructor - injects validator
 *
 * @param validator - Service for validating domain configurations
 */
constructor(private validator: DomainConfigValidator) {}

/**
 * Register a domain configuration
 *
 * Validates the configuration (unless disabled) and adds it to the registry.
 * First registered domain becomes the active domain.
 *
 * @template TFilters - Filter model type
 * @template TData - Data model type
 * @template TStatistics - Statistics model type
 * @param config - Domain configuration to register
 * @param validate - Whether to validate configuration (default: true)
 * @throws Error if configuration is invalid (when validate=true)
 *
 * @example
 * ```typescript
 * // Register with validation
 * registry.register(AUTOMOBILE_DOMAIN_CONFIG);
 *
 * // Register without validation (e.g., in tests)
 * registry.register(TEST_CONFIG, false);
 * ```
 */
register<TFilters, TData, TStatistics>(
  config: DomainConfig<TFilters, TData, TStatistics>,
  validate: boolean = true
): void {
  // Validate if requested
  if (validate) {
    const sanitizedConfig = this.validator.validateAndSanitize(config);
    config = sanitizedConfig as DomainConfig<TFilters, TData, TStatistics>;
  }

  // Check for duplicate (warn but allow overwrite)
  if (this.configs.has(config.domainName)) {
    console.warn(
      `Domain '${config.domainName}' already registered. Overwriting.`
    );
  }
```

```
  // Register the config
  this.configs.set(config.domainName, config);

  // Set as active if first domain
  if (!this.activeDomainName) {
    this.activeDomainName = config.domainName;
  }

  console.log(`Domain '${config.domainName}' registered successfully`);
}

/**
 * Register multiple domain configurations
 *
 * Convenience method for registering several domains at once.
 *
 * @param configs - Array of domain configurations
 * @param validate - Whether to validate configurations (default: true)
 */
registerMultiple(
  configs: DomainConfig<any, any, any>[],
  validate: boolean = true
): void {
  configs.forEach((config) => this.register(config, validate));
}

/**
 * Get domain configuration by name
 *
 * @template TFilters - Filter model type
 * @template TData - Data model type
 * @template TStatistics - Statistics model type
 * @param domainName - Domain name to retrieve
 * @returns Domain configuration
 * @throws Error if domain not found
 *
 * @example
 * ```typescript
 * const config = registry.get<AutoFilters, VehicleResult>('automobile');
 * // config is typed as DomainConfig<AutoFilters, VehicleResult, any>
 * ```
 */
get<TFilters, TData, TStatistics>(
  domainName: string
): DomainConfig<TFilters, TData, TStatistics> {
  const config = this.configs.get(domainName);

  if (!config) {
    const available = this.getAllDomainNames().join(', ');
    throw new Error(
      `Domain '${domainName}' not found. Available domains: ${available}`
    );
  }

  return config as DomainConfig<TFilters, TData, TStatistics>;
}

/**
 * Get active domain configuration
```

```
 *
 * Returns the currently active domain's configuration.
 *
 * @template TFilters - Filter model type
 * @template TData - Data model type
 * @template TStatistics - Statistics model type
 * @returns Active domain configuration
 * @throws Error if no domain is active
 */
getActive<TFilters, TData, TStatistics>(): DomainConfig<
  TFilters,
  TData,
  TStatistics
> {
  if (!this.activeDomainName) {
    throw new Error('No active domain. Register a domain first.');
  }

  return this.get<TFilters, TData, TStatistics>(this.activeDomainName);
}

/**
 * Set active domain
 *
 * Changes which domain is returned by getActive().
 *
 * @param domainName - Domain name to activate
 * @throws Error if domain not found
 */
setActive(domainName: string): void {
  if (!this.configs.has(domainName)) {
    const available = this.getAllDomainNames().join(', ');
    throw new Error(
      `Cannot activate domain '${domainName}'. Available domains: ${available}`
    );
  }

  this.activeDomainName = domainName;
  console.log(`Active domain set to '${domainName}'`);
}

/**
 * Get active domain name
 *
 * @returns Active domain name or undefined if none
 */
getActiveDomainName(): string | undefined {
  return this.activeDomainName;
}

/**
 * Check if domain is registered
 *
 * @param domainName - Domain name to check
 * @returns True if domain is registered
 */
has(domainName: string): boolean {
  return this.configs.has(domainName);
}
```

```
/**
 * Get all registered domain names
 *
 * @returns Array of domain names
 */
getAllDomainNames(): string[] {
  return Array.from(this.configs.keys());
}

/**
 * Get all registered domain configurations
 *
 * @returns Array of domain configurations
 */
getAll(): DomainConfig<any, any, any>[] {
  return Array.from(this.configs.values());
}

/**
 * Unregister a domain
 *
 * Removes domain from registry. If it was active, activates the next available.
 *
 * @param domainName - Domain name to unregister
 * @returns True if domain was unregistered, false if not found
 */
unregister(domainName: string): boolean {
  const result = this.configs.delete(domainName);

  // Handle active domain removal
  if (this.activeDomainName === domainName) {
    this.activeDomainName = undefined;

    // Set first remaining domain as active
    const remaining = this.getAllDomainNames();
    if (remaining.length > 0) {
      this.setActive(remaining[0]);
    }
  }

  if (result) {
    console.log(`Domain '${domainName}' unregistered`);
  }

  return result;
}

/**
 * Clear all registered domains
 */
clear(): void {
  this.configs.clear();
  this.activeDomainName = undefined;
  console.log('All domains cleared');
}

/**
 * Get count of registered domains
 *
```

```
   * @returns Number of registered domains
   */
  getCount(): number {
    return this.configs.size;
  }

  /**
   * Validate a domain configuration without registering
   *
   * Useful for pre-validation before registration.
   *
   * @template TFilters - Filter model type
   * @template TData - Data model type
   * @template TStatistics - Statistics model type
   * @param config - Domain configuration to validate
   * @returns Validation result
   */
  validate<TFilters, TData, TStatistics>(
    config: DomainConfig<TFilters, TData, TStatistics>
  ) {
    return this.validator.validate(config);
  }

  /**
   * Get human-readable validation summary for a domain
   *
   * @param domainName - Domain name
   * @returns Validation summary string
   */
  getValidationSummary(domainName: string): string {
    const config = this.get(domainName);
    const result = this.validator.validate(config);
    return this.validator.getValidationSummary(result);
  }
}
```

Step 304.2: Update the Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 4 (Section 304) - Added DomainConfigRegistry

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
export * from './domain-config-registry.service';
```

Step 304.3: Understand the Dependency

Note that `DomainConfigRegistry` depends on `DomainConfigValidator` (Section 305). The registry won't compile until the validator exists.

This creates a circular documentation dependency:

- Registry (304) uses Validator (305)
- We document Registry first to show the pattern

In practice, you'll create both files, then build.

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/domain-config-registry.service.ts
```

2. TypeScript Compilation Check

After creating Section 305 (Validator), run:

```
$ npx tsc --noEmit src/app/framework/services/domain-config-registry.service.ts
```

Expected: No output (no compilation errors).

3. Build the Application

After Section 305:

```
$ ng build
```

Expected: Build succeeds with no errors.

4. Verify Registration (Optional)

Add temporary test code:

```
// In app.component.ts ngOnInit
import { DomainConfigRegistry } from './framework/services';

constructor(private registry: DomainConfigRegistry) {}

ngOnInit(): void {
```

```
  // Mock config for testing (real config comes in Phase 6)
  const mockConfig = {
    domainName: 'test-automobile',
    domainLabel: 'Test Automobiles',
    apiBaseUrl: 'http://example.com/api',
    filterModel: class {},
    dataModel: class {},
    apiAdapter: { fetchData: () => {} },
    urlMapper: { toUrlParams: () => ({}), fromUrlParams: () => ({}) },
    cacheKeyBuilder: { buildKey: () => '' },
    tableConfig: { tableId: 'test', dataKey: 'id', columns: [{ field: 'id' }] },
    pickers: [],
    filters: [],
    queryControlFilters: [],
    charts: [],
    features: { highlights: true, popOuts: true, rowExpansion: true }
  };

  this.registry.register(mockConfig as any, false); // Skip validation for test
  console.log('Registered domains:', this.registry.getAllDomainNames());
  console.log('Active domain:', this.registry.getActiveDomainName());
}
```

Console should show:

```
Domain 'test-automobile' registered successfully
Registered domains: ['test-automobile']
Active domain: test-automobile
```

Common Problems

Symptom	Cause	Solution
`Cannot find module './domain-config-validator.service'`	Validator not created yet	Complete Section 305 first
`No active domain` error	No domains registered	Register at least one domain
`Domain 'X' not found`	Domain not registered	Check spelling, ensure registration happened
Validation errors on register	Config missing required fields	Check DomainConfig interface requirements
Console warnings about overwriting	Same domain registered twice	Remove duplicate registration

---

Key Takeaways

1. **Registry pattern enables runtime flexibility** — Register, retrieve, switch domains dynamically
2. **Injection tokens simplify component DI** — Components inject DOMAIN_CONFIG, not the registry
3. **Validation catches errors early** — Invalid configs fail at registration, not at runtime

---

Acceptance Criteria

- [ ] `src/app/framework/services/domain-config-registry.service.ts` exists
- [ ] Barrel file exports the service and DOMAIN_CONFIG token
- [ ] Service is `@Injectable({ providedIn: 'root' })`
- [ ] `register()` validates and stores configurations
- [ ] `get()` retrieves configurations by name
- [ ] `getActive()` returns the active domain configuration
- [ ] `setActive()` changes the active domain
- [ ] First registered domain becomes active automatically
- [ ] `getAllDomainNames()` lists all registered domains
- [ ] `unregister()` removes domains and handles active domain gracefully
- [ ] DOMAIN_CONFIG InjectionToken is exported
- [ ] TypeScript compilation succeeds (after Section 305)
- [ ] JSDoc comments document all public methods

Next Step

Proceed to `305-domain-config-validator.md` to create the validation service that the registry depends on.

Chapter 305

Domain Config Validator

305: Domain Config Validator

Status: Complete

Depends On: 201-domain-config-interface, 304-domain-config-registry

Blocks: 306-resource-management-service

Learning Objectives

After completing this section, you will:

- Understand runtime validation patterns in TypeScript
- Know how to create comprehensive validation with error categorization
- Recognize the value of catching configuration errors early
- Be able to implement a reusable validation service

Objective

Create the `DomainConfigValidator` service that validates domain configurations at runtime, ensuring all required fields are present and correctly typed before the framework attempts to use them.

Why

TypeScript provides compile-time type checking, but it can't catch everything:

What TypeScript Catches

```
const config: DomainConfig<F, D, S> = {
  domainName: 123, // Error: Type 'number' is not assignable to type 'string'
  // Missing required properties - TypeScript error
};
```

What TypeScript Misses

```
const config: DomainConfig<any, any, any> = {
  domainName: '',        // Empty string - valid type, invalid value
  apiBaseUrl: 'not-url', // Not a valid URL - valid type, invalid value
  tableConfig: {
    tableId: 'test',
    columns: []          // Empty array - valid type, likely an error
  }
};
```

TypeScript checks types. It doesn't check:

- Empty strings vs meaningful strings
- Valid URLs vs arbitrary strings
- Empty arrays vs populated arrays
- Interface compliance (does `apiAdapter` have `fetchData`?)

Why Not Just Trust TypeScript?

In a perfect world, every `DomainConfig` would be created with correct types and values. In practice:

1. **Dynamic data** — Configs may come from JSON files or APIs
2. **User error** — Developers make mistakes
3. **Refactoring** — Interface changes can break existing configs
4. **Third-party configs** — Can't trust external data

Runtime validation catches these issues **at registration time**, with clear error messages, instead of cryptic failures deep in framework code.

Validation vs. Sanitization

The validator does two things:

1. **Validate** — Check for errors, return result object
2. **Validate and Sanitize** — Check for errors, apply defaults, return clean config

```
// Validation only
const result = validator.validate(config);
if (!result.valid) {
```

```
  console.error(result.errors);
}

// Validation with sanitization (applies defaults)
const cleanConfig = validator.validateAndSanitize(config);
// Throws if invalid, returns sanitized config if valid
```

What

Step 305.1: Create the Domain Config Validator Service

Create the file `src/app/framework/services/domain-config-validator.service.ts`:

```
// src/app/framework/services/domain-config-validator.service.ts
// VERSION 1 (Section 305) - Runtime configuration validation

import { Injectable } from '@angular/core';
import {
  DomainConfig,
  ConfigValidationError,
  ConfigValidationResult,
  ConfigErrorType,
  mergeDomainFeatures
} from '../models/domain-config.interface';

/**
 * Domain configuration validator service
 *
 * Validates domain configurations at runtime to ensure they meet all
 * requirements before being used by the framework. Catches errors that
 * TypeScript's compile-time checks miss.
 *
 * **Why Runtime Validation?**
 *
 * TypeScript checks types but not values:
 * - Empty strings pass type check but aren't valid domain names
 * - Invalid URLs pass type check but won't work for API calls
 * - Missing interface methods pass type check with 'any' types
 *
 * **Error Categories:**
 *
 * | Category | Description |
 * |----------|-------------|
 * | MISSING_REQUIRED | Required field is null/undefined |
 * | INVALID_TYPE | Field has wrong type |
 * | INVALID_VALUE | Field has valid type but invalid value |
 * | EMPTY_ARRAY | Required array is empty |
 * | DUPLICATE_ID | Duplicate ID in array (pickers, filters, charts) |
 *
 * @example
 * ```typescript
 * const validator = inject(DomainConfigValidator);
```

```
 *
 * // Validate without modifying
 * const result = validator.validate(config);
 * if (!result.valid) {
 *   console.error('Errors:', result.errors);
 * }
 *
 * // Validate and apply defaults
 * try {
 *   const cleanConfig = validator.validateAndSanitize(config);
 * } catch (error) {
 *   console.error('Invalid config:', error.message);
 * }
 *
 * // Get human-readable summary
 * console.log(validator.getValidationSummary(result));
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class DomainConfigValidator {
  /**
   * Validate domain configuration
   *
   * Performs comprehensive validation and returns result object.
   * Does not modify the configuration.
   *
   * @template TFilters - Filter model type
   * @template TData - Data model type
   * @template TStatistics - Statistics model type
   * @param config - Domain configuration to validate
   * @returns Validation result with errors and warnings
   */
  validate<TFilters, TData, TStatistics>(
    config: DomainConfig<TFilters, TData, TStatistics>
  ): ConfigValidationResult {
    const errors: ConfigValidationError[] = [];
    const warnings: ConfigValidationError[] = [];

    // Validate required string fields
    this.validateRequiredString(config, 'domainName', errors);
    this.validateRequiredString(config, 'domainLabel', errors);
    this.validateRequiredString(config, 'apiBaseUrl', errors);

    // Validate domain name format (lowercase, alphanumeric with hyphens)
    this.validateDomainNameFormat(config.domainName, errors);

    // Validate API base URL format
    this.validateApiBaseUrl(config.apiBaseUrl, errors);

    // Validate type models
    this.validateRequiredField(config, 'filterModel', errors);
    this.validateRequiredField(config, 'dataModel', errors);

    // Validate adapters exist
    this.validateRequiredField(config, 'apiAdapter', errors);
    this.validateRequiredField(config, 'urlMapper', errors);
    this.validateRequiredField(config, 'cacheKeyBuilder', errors);
```

```
  // Validate adapter interfaces (have required methods)
  this.validateApiAdapter(config.apiAdapter, errors);
  this.validateUrlMapper(config.urlMapper, errors);
  this.validateCacheKeyBuilder(config.cacheKeyBuilder, errors);

  // Validate table configuration
  this.validateRequiredField(config, 'tableConfig', errors);
  this.validateTableConfig(config.tableConfig, errors);

  // Validate arrays (may be empty, but should be arrays)
  this.validateArray(config, 'pickers', errors, warnings);
  this.validateArray(config, 'filters', errors, warnings);
  this.validateArray(config, 'charts', errors, warnings);

  // Validate array contents for duplicates
  if (config.pickers && config.pickers.length > 0) {
    this.validatePickers(config.pickers, errors);
  }

  if (config.filters && config.filters.length > 0) {
    this.validateFilters(config.filters, errors);
  }

  if (config.charts && config.charts.length > 0) {
    this.validateCharts(config.charts, errors);
  }

  // Validate features
  this.validateRequiredField(config, 'features', errors);
  this.validateFeatures(config.features, errors);

  return {
    valid: errors.length === 0,
    errors,
    warnings
  };
}

/**
 * Validate and sanitize domain configuration
 *
 * Validates configuration and applies defaults for optional fields.
 * Throws if validation fails.
 *
 * @template TFilters - Filter model type
 * @template TData - Data model type
 * @template TStatistics - Statistics model type
 * @param config - Domain configuration to validate
 * @returns Sanitized configuration with defaults applied
 * @throws Error if configuration is invalid
 */
validateAndSanitize<TFilters, TData, TStatistics>(
  config: DomainConfig<TFilters, TData, TStatistics>
): DomainConfig<TFilters, TData, TStatistics> {
  const result = this.validate(config);

  if (!result.valid) {
    const errorMessages = result.errors
      .map((e) => `${e.field}: ${e.message}`)
```

```
      .join('\n');
    throw new Error(
      `Domain configuration validation failed:\n${errorMessages}`
    );
  }

  // Apply defaults for optional fields
  return {
    ...config,
    features: mergeDomainFeatures(config.features),
    pickers: config.pickers || [],
    filters: config.filters || [],
    charts: config.charts || []
  };
}

/**
 * Get human-readable validation summary
 *
 * Formats validation result into readable multi-line string.
 *
 * @param result - Validation result from validate()
 * @returns Formatted summary string
 */
getValidationSummary(result: ConfigValidationResult): string {
  if (result.valid) {
    return 'Configuration is valid';
  }

  const lines: string[] = ['Configuration validation failed:'];

  if (result.errors.length > 0) {
    lines.push(`\nErrors (${result.errors.length}):`);
    result.errors.forEach((error, index) => {
      lines.push(
        `  ${index + 1}. [${error.type}] ${error.field}: ${error.message}`
      );
    });
  }

  if (result.warnings && result.warnings.length > 0) {
    lines.push(`\nWarnings (${result.warnings.length}):`);
    result.warnings.forEach((warning, index) => {
      lines.push(
        `  ${index + 1}. [${warning.type}] ${warning.field}: ${warning.message}`
      );
    });
  }

  return lines.join('\n');
}

// ============================================================================
// Private Validation Methods
// ============================================================================

/**
 * Validate required string field (non-empty)
 */
```

```
private validateRequiredString(
  config: any,
  field: string,
  errors: ConfigValidationError[]
): void {
  if (!config[field]) {
    errors.push({
      type: ConfigErrorType.MISSING_REQUIRED,
      field,
      message: `Required field '${field}' is missing`,
      expected: 'string'
    });
  } else if (typeof config[field] !== 'string') {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field,
      message: `Field '${field}' must be a string`,
      expected: 'string',
      actual: typeof config[field]
    });
  } else if (config[field].trim().length === 0) {
    errors.push({
      type: ConfigErrorType.INVALID_VALUE,
      field,
      message: `Field '${field}' cannot be empty`,
      expected: 'non-empty string',
      actual: config[field]
    });
  }
}

/**
 * Validate required field exists
 */
private validateRequiredField(
  config: any,
  field: string,
  errors: ConfigValidationError[]
): void {
  if (!config[field]) {
    errors.push({
      type: ConfigErrorType.MISSING_REQUIRED,
      field,
      message: `Required field '${field}' is missing`
    });
  }
}

/**
 * Validate domain name format
 */
private validateDomainNameFormat(
  domainName: string,
  errors: ConfigValidationError[]
): void {
  if (!domainName) return;

  // Domain name: lowercase, starts with letter, alphanumeric with hyphens
  const validPattern = /^[a-z][a-z0-9-]*$/;
```

```
  if (!validPattern.test(domainName)) {
    errors.push({
      type: ConfigErrorType.INVALID_VALUE,
      field: 'domainName',
      message:
        'Domain name must be lowercase, start with a letter, and contain only letters, numbers, and hyphens',
      expected: 'lowercase alphanumeric with hyphens (e.g., "automobile")',
      actual: domainName
    });
  }
}

/**
 * Validate API base URL format
 */
private validateApiBaseUrl(
  apiBaseUrl: string,
  errors: ConfigValidationError[]
): void {
  if (!apiBaseUrl) return;

  try {
    new URL(apiBaseUrl);
  } catch (e) {
    errors.push({
      type: ConfigErrorType.INVALID_VALUE,
      field: 'apiBaseUrl',
      message: 'API base URL is not a valid URL',
      expected: 'valid URL (e.g., "http://api.example.com")',
      actual: apiBaseUrl
    });
  }
}

/**
 * Validate API adapter has required methods
 */
private validateApiAdapter(
  adapter: any,
  errors: ConfigValidationError[]
): void {
  if (!adapter) return;

  if (typeof adapter.fetchData !== 'function') {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field: 'apiAdapter.fetchData',
      message: 'API adapter must implement fetchData method',
      expected: 'function'
    });
  }
}

/**
 * Validate URL mapper has required methods
 */
private validateUrlMapper(
  mapper: any,
  errors: ConfigValidationError[]
```

```
): void {
  if (!mapper) return;

  if (typeof mapper.toUrlParams !== 'function') {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field: 'urlMapper.toUrlParams',
      message: 'URL mapper must implement toUrlParams method',
      expected: 'function'
    });
  }

  if (typeof mapper.fromUrlParams !== 'function') {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field: 'urlMapper.fromUrlParams',
      message: 'URL mapper must implement fromUrlParams method',
      expected: 'function'
    });
  }
}

/**
 * Validate cache key builder has required methods
 */
private validateCacheKeyBuilder(
  builder: any,
  errors: ConfigValidationError[]
): void {
  if (!builder) return;

  if (typeof builder.buildKey !== 'function') {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field: 'cacheKeyBuilder.buildKey',
      message: 'Cache key builder must implement buildKey method',
      expected: 'function'
    });
  }
}

/**
 * Validate table configuration
 */
private validateTableConfig(
  tableConfig: any,
  errors: ConfigValidationError[]
): void {
  if (!tableConfig) return;

  if (!tableConfig.tableId) {
    errors.push({
      type: ConfigErrorType.MISSING_REQUIRED,
      field: 'tableConfig.tableId',
      message: 'Table config must have tableId'
    });
  }

  if (!tableConfig.dataKey) {
```

```
    errors.push({
      type: ConfigErrorType.MISSING_REQUIRED,
      field: 'tableConfig.dataKey',
      message: 'Table config must have dataKey'
    });
  }

  if (!tableConfig.columns || !Array.isArray(tableConfig.columns)) {
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field: 'tableConfig.columns',
      message: 'Table config must have columns array',
      expected: 'array'
    });
  } else if (tableConfig.columns.length === 0) {
    errors.push({
      type: ConfigErrorType.EMPTY_ARRAY,
      field: 'tableConfig.columns',
      message: 'Table config must have at least one column'
    });
  }
}

/**
 * Validate array field
 */
private validateArray(
  config: any,
  field: string,
  errors: ConfigValidationError[],
  warnings: ConfigValidationError[]
): void {
  if (config[field] === undefined || config[field] === null) {
    // Missing array is a warning (will be defaulted to [])
    warnings.push({
      type: ConfigErrorType.MISSING_REQUIRED,
      field,
      message: `Field '${field}' is missing, will default to empty array`,
      expected: 'array'
    });
  } else if (!Array.isArray(config[field])) {
    // Wrong type is an error
    errors.push({
      type: ConfigErrorType.INVALID_TYPE,
      field,
      message: `Field '${field}' must be an array`,
      expected: 'array',
      actual: typeof config[field]
    });
  }
}

/**
 * Validate pickers array (check for duplicate IDs)
 */
private validatePickers(
  pickers: any[],
  errors: ConfigValidationError[]
): void {
```

```
  const ids = new Set<string>();

  pickers.forEach((picker, index) => {
    if (!picker.id) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `pickers[${index}].id`,
        message: `Picker at index ${index} must have id`
      });
    } else {
      if (ids.has(picker.id)) {
        errors.push({
          type: ConfigErrorType.DUPLICATE_ID,
          field: `pickers[${index}].id`,
          message: `Duplicate picker id: ${picker.id}`,
          actual: picker.id
        });
      }
      ids.add(picker.id);
    }
  });
}

/**
 * Validate filters array (check for duplicate IDs and required fields)
 */
private validateFilters(
  filters: any[],
  errors: ConfigValidationError[]
): void {
  const ids = new Set<string>();

  filters.forEach((filter, index) => {
    if (!filter.id) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `filters[${index}].id`,
        message: `Filter at index ${index} must have id`
      });
    } else {
      if (ids.has(filter.id)) {
        errors.push({
          type: ConfigErrorType.DUPLICATE_ID,
          field: `filters[${index}].id`,
          message: `Duplicate filter id: ${filter.id}`,
          actual: filter.id
        });
      }
      ids.add(filter.id);
    }

    if (!filter.type) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `filters[${index}].type`,
        message: `Filter at index ${index} must have type`
      });
    }
  });
```

```
}

/**
 * Validate charts array (check for duplicate IDs and required fields)
 */
private validateCharts(
  charts: any[],
  errors: ConfigValidationError[]
): void {
  const ids = new Set<string>();

  charts.forEach((chart, index) => {
    if (!chart.id) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `charts[${index}].id`,
        message: `Chart at index ${index} must have id`
      });
    } else {
      if (ids.has(chart.id)) {
        errors.push({
          type: ConfigErrorType.DUPLICATE_ID,
          field: `charts[${index}].id`,
          message: `Duplicate chart id: ${chart.id}`,
          actual: chart.id
        });
      }
      ids.add(chart.id);
    }

    if (!chart.type) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `charts[${index}].type`,
        message: `Chart at index ${index} must have type`
      });
    }

    if (!chart.dataSourceId) {
      errors.push({
        type: ConfigErrorType.MISSING_REQUIRED,
        field: `charts[${index}].dataSourceId`,
        message: `Chart at index ${index} must have dataSourceId`
      });
    }
  });
}

/**
 * Validate features object
 */
private validateFeatures(
  features: any,
  errors: ConfigValidationError[]
): void {
  if (!features) return;

  const requiredFeatures = ['highlights', 'popOuts', 'rowExpansion'];
```

```
    requiredFeatures.forEach((feature) => {
      if (typeof features[feature] !== 'boolean') {
        errors.push({
          type: ConfigErrorType.INVALID_TYPE,
          field: `features.${feature}`,
          message: `Feature '${feature}' must be a boolean`,
          expected: 'boolean',
          actual: typeof features[feature]
        });
      }
    });
  }
}
```

Step 305.2: Update the Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 5 (Section 305) - Added DomainConfigValidator

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
export * from './domain-config-registry.service';
export * from './domain-config-validator.service';
```

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/domain-config-validator.service.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/framework/services/domain-config-validator.service.ts
```

Expected: No output (no compilation errors).

3. Build the Application

```
$ ng build
```

Expected: Build succeeds with no errors.

4. Verify Validation (Optional)

Add temporary test code:

```
// In any component
constructor(private validator: DomainConfigValidator) {
  // Test invalid config
  const badConfig = {
    domainName: '', // Empty - invalid
    domainLabel: 'Test',
    apiBaseUrl: 'not-a-url', // Invalid URL
    filterModel: null, // Missing
    dataModel: class {},
    apiAdapter: {}, // Missing fetchData
    urlMapper: { toUrlParams: () => ({}) }, // Missing fromUrlParams
    cacheKeyBuilder: { buildKey: () => '' },
    tableConfig: { tableId: 'test', dataKey: 'id', columns: [] }, // Empty columns
    features: { highlights: 'yes' } // Wrong type
  } as any;

  const result = this.validator.validate(badConfig);
  console.log('Valid:', result.valid);
  console.log('Summary:\n', this.validator.getValidationSummary(result));
}
```

Console should show multiple validation errors.

Common Problems

Symptom	Cause	Solution
`Cannot find module '../models/domain-config.interface'`	Interface file missing	Complete Section 201 first
`ConfigErrorType is not defined`	Enum not exported from interface	Check domain-config.interface.ts exports
No errors shown for bad config	Validation too lenient	Check validateRequiredField is called
validateAndSanitize doesn't throw	result.valid is true	Check all validation rules are running

---

Key Takeaways

1. **Runtime validation catches what TypeScript misses** — Type checks pass for empty strings and invalid URLs
2. **Categorized errors enable targeted fixes** — MISSING_REQUIRED vs INVALID_VALUE vs DUPLICATE_ID
3. **Validation summary provides actionable feedback** — Human-readable error messages with field paths

---

Acceptance Criteria

- [ ] `src/app/framework/services/domain-config-validator.service.ts` exists
- [ ] Barrel file exports the service
- [ ] Service is `@Injectable({ providedIn: 'root' })`
- [ ] `validate()` returns ConfigValidationResult
- [ ] `validateAndSanitize()` throws on invalid, returns clean config on valid
- [ ] `getValidationSummary()` returns human-readable string
- [ ] Validates required string fields (non-empty)
- [ ] Validates domain name format (lowercase, alphanumeric)
- [ ] Validates API base URL is valid URL
- [ ] Validates adapter interfaces have required methods
- [ ] Validates table config has columns
- [ ] Validates arrays for duplicate IDs
- [ ] Validates features are booleans
- [ ] TypeScript compilation succeeds
- [ ] JSDoc comments document all public methods

Next Step

Proceed to `306-resource-management-service.md` to create the core state orchestration service.

Chapter 306

Resource Management Service

306: Resource Management Service

Status: Complete

Depends On: 301-url-state-service, 302-api-service, 303-request-coordinator, 304-domain-config-registry, 305-domain-config-validator

Blocks: Phase 3B (307-311), Phase 8 (Framework Components), Phase 9 (Feature Components)

Learning Objectives

After completing this section, you will:

- Understand the role of a state orchestrator in URL-First architecture
- Know how BehaviorSubject enables reactive state management
- Recognize the pattern of converting URL changes to data fetches
- Be able to implement a generic, domain-agnostic resource management service

---

Objective

Create the `ResourceManagementService` that orchestrates application state with the URL as the single source of truth. This service coordinates filter changes, API calls, state updates, and cross-window synchronization.

Why

We've built the foundation:

- **UrlStateService** — Reads/writes URL parameters

- **ApiService** — Makes HTTP requests
- **RequestCoordinator** — Caches, deduplicates, retries
- **DomainConfigRegistry** — Provides domain configuration

Now we need a service that ties them together. When a user changes a filter:

```
User clicks filter → URL updates → Data fetches → UI updates
```

`ResourceManagementService` is the orchestrator that makes this flow work.

Figure 306.1 shows the complete data flow from user action through to UI update:

Complete data flow from user action to UI update

Figure 306.1: The URL-First data flow. When a user clicks a filter, the URL updates, which triggers ResourceManagementService to fetch data. All subscribed components automatically receive the updated state.

The State Flow

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                URL (Source of Truth)                                             │
│                              ?manufacturer=Ford&page=1                                           │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                          │
```

```
                                    ▼ watchParams()
■■■
```

- **Pop-out window**: Does NOT fetch (receives state from main window)

This prevents duplicate API calls and ensures consistency.

Component-Level Injection

Unlike singleton services, `ResourceManagementService` is **not** `providedIn: 'root'`. Each component that needs resource management provides its own instance:

```
@Component({
  providers: [ResourceManagementService]
})
export class DiscoverComponent {
  constructor(private resources: ResourceManagementService<F, D, S>) {}
}
```

This enables different components to manage different resources independently.

What

Step 306.1: Create the Popout Token (Placeholder)

Before creating `ResourceManagementService`, we need the `IS_POPOUT_TOKEN`. This will be fully implemented in Section 315, but we need a placeholder now.

Create `src/app/framework/tokens/popout.token.ts`:

```
// src/app/framework/tokens/popout.token.ts
// VERSION 1 (Section 306) - Placeholder, fully implemented in Section 315

import { InjectionToken } from '@angular/core';

/**
 * Injection token to indicate if component is in a pop-out window
 *
 * When true, ResourceManagementService disables auto-fetching
 * and waits for state to be synced from the main window.
 */
export const IS_POPOUT_TOKEN = new InjectionToken<boolean>('IS_POPOUT_TOKEN');
```

Create the tokens barrel file `src/app/framework/tokens/index.ts`:

```
// src/app/framework/tokens/index.ts
// VERSION 1 (Section 306)

export * from './popout.token';
```

Step 306.2: Create Placeholder for PopOutContextService

We also need `PopOutContextService`. Create a placeholder:

```
// src/app/framework/services/popout-context.service.ts
// VERSION 1 (Section 306) - Placeholder, fully implemented in Section 307

import { Injectable } from '@angular/core';

/**
 * Pop-out context service (placeholder)
 *
 * Determines if the current window is a pop-out.
 * Full implementation in Section 307.
 */
@Injectable({
  providedIn: 'root'
})
export class PopOutContextService {
  /**
   * Check if current window is a pop-out
   *
   * @returns True if in pop-out window
   */
  isInPopOut(): boolean {
    // Placeholder implementation
    // Real implementation in Section 307 checks window.opener
    return false;
  }
}
```

Step 306.3: Create the Resource Management Service

Create the file `src/app/framework/services/resource-management.service.ts`:

```
// src/app/framework/services/resource-management.service.ts
// VERSION 1 (Section 306) - Core state orchestrator

import {
  Inject,
  Injectable,
  NgZone,
  OnDestroy,
  Optional
} from '@angular/core';
import { BehaviorSubject, Observable, of, Subject } from 'rxjs';
import {
  catchError,
  distinctUntilChanged,
```

```
  finalize,
  map,
  takeUntil
} from 'rxjs/operators';
import { DomainConfig } from '../models/domain-config.interface';
import {
  ResourceManagementConfig,
  ResourceState
} from '../models/resource-management.interface';
import { DOMAIN_CONFIG } from './domain-config-registry.service';
import { PopOutContextService } from './popout-context.service';
import { UrlStateService } from './url-state.service';
import { IS_POPOUT_TOKEN } from '../tokens/popout.token';

/**
 * Deep equality check for filter objects
 *
 * Replaces JSON.stringify comparison with proper structural equality.
 * Handles nested objects and arrays correctly.
 */
function deepEqual(a: unknown, b: unknown): boolean {
  if (a === b) return true;
  if (a == null || b == null) return a === b;
  if (typeof a !== typeof b) return false;
  if (typeof a !== 'object') return a === b;

  const aObj = a as Record<string, unknown>;
  const bObj = b as Record<string, unknown>;
  const aKeys = Object.keys(aObj);
  const bKeys = Object.keys(bObj);

  if (aKeys.length !== bKeys.length) return false;

  return aKeys.every(key => deepEqual(aObj[key], bObj[key]));
}

/**
 * Generic resource management service - Core state orchestrator
 *
 * **Purpose:** Manages application state with URL as single source of truth.
 * Coordinates filter changes, API calls, state updates, and cross-window sync.
 *
 * **Architecture:** URL → Filters → API → Data → Components
 *
 * **Key Features:**
 *
 * 1. **URL-first design** — URL parameters are the single source of truth
 * 2. **BehaviorSubject state** — Current values with Observable streams
 * 3. **Domain-agnostic** — Works with any domain via DOMAIN_CONFIG injection
 * 4. **Component-level injection** — New instance per component (not singleton)
 * 5. **Pop-out aware** — Disables API calls in pop-out windows
 *
 * **Usage Pattern:**
 *
 * ```typescript
 * @Component({
 *   providers: [ResourceManagementService] // New instance for this component
 * })
 * export class DiscoverComponent {
```

```
 *   filters$ = this.resources.filters$;
 *   results$ = this.resources.results$;
 *   loading$ = this.resources.loading$;
 *
 *   constructor(
 *     private resources: ResourceManagementService<AutoFilters, VehicleResult>
 *   ) {}
 *
 *   onFilterChange(filters: Partial<AutoFilters>): void {
 *     this.resources.updateFilters(filters);
 *   }
 * }
 * ```
 *
 * @template TFilters - Domain-specific filter model type
 * @template TData - Domain-specific data model type
 * @template TStatistics - Domain-specific statistics model type
 */
@Injectable() // NOT providedIn: 'root' — component-level injection
export class ResourceManagementService<TFilters, TData, TStatistics = any>
  implements OnDestroy {

  // ============================================================================
  // Internal State
  // ============================================================================

  /** Subject for cleanup on destroy */
  private readonly destroy$ = new Subject<void>();

  /** Configuration derived from DOMAIN_CONFIG */
  private readonly config: ResourceManagementConfig<TFilters, TData, TStatistics>;

  // ============================================================================
  // BehaviorSubject-Based State
  // ============================================================================

  /** Main state holder — all state in one object */
  private readonly stateSubject: BehaviorSubject<ResourceState<TFilters, TData, TStatistics>>;

  // ============================================================================
  // Observable Streams (Public API)
  // ============================================================================

  /** Full state observable */
  public readonly state$: Observable<ResourceState<TFilters, TData, TStatistics>>;

  /** Current filters (from URL) */
  public readonly filters$: Observable<TFilters>;

  /** Current results (from API) */
  public readonly results$: Observable<TData[]>;

  /** Total result count (for pagination) */
  public readonly totalResults$: Observable<number>;

  /** Loading state */
  public readonly loading$: Observable<boolean>;

  /** Current error (null if none) */
```

```
public readonly error$: Observable<Error | null>;

/** Statistics data (optional, depends on API) */
public readonly statistics$: Observable<TStatistics | undefined>;

/** Highlight filters (for chart segmentation) */
public readonly highlights$: Observable<any>;

// ==========================================================================
// Constructor
// ==========================================================================

/**
 * Constructor — wires up state management
 *
 * @param urlState - Service for URL state management
 * @param domainConfig - Domain configuration (injected via DOMAIN_CONFIG)
 * @param popOutContext - Service to check if in pop-out window
 * @param ngZone - Angular zone for change detection
 * @param isPopOutToken - Optional token indicating pop-out mode
 */
constructor(
  private readonly urlState: UrlStateService,
  @Inject(DOMAIN_CONFIG) private readonly domainConfig: DomainConfig<TFilters, TData, TStatistics>,
  private readonly popOutContext: PopOutContextService,
  private readonly ngZone: NgZone,
  @Optional() @Inject(IS_POPOUT_TOKEN) private readonly isPopOutToken: boolean
) {
  // Determine if in pop-out (token takes precedence)
  const isPopOut = this.isPopOutToken ?? false;

  // Build configuration from domain config
  this.config = {
    filterMapper: this.domainConfig.urlMapper,
    apiAdapter: this.domainConfig.apiAdapter,
    cacheKeyBuilder: this.domainConfig.cacheKeyBuilder,
    defaultFilters: (this.domainConfig.defaultFilters || {}) as TFilters,
    supportsHighlights: this.domainConfig.features?.highlights ?? false,
    highlightPrefix: 'h_',
    // Disable auto-fetch in pop-out windows
    autoFetch: isPopOut ? false : !this.popOutContext.isInPopOut()
  };

  // Initialize state with defaults
  this.stateSubject = new BehaviorSubject<ResourceState<TFilters, TData, TStatistics>>({
    filters: this.config.defaultFilters,
    results: [],
    totalResults: 0,
    loading: false,
    error: null,
    statistics: undefined
  });

  // Create derived observables from state
  this.state$ = this.stateSubject.asObservable();

  this.filters$ = this.state$.pipe(
    map(s => s.filters),
    distinctUntilChanged((a, b) => deepEqual(a, b))
```

```
  );

  this.results$ = this.state$.pipe(
    map(s => s.results),
    distinctUntilChanged()
  );

  this.totalResults$ = this.state$.pipe(
    map(s => s.totalResults),
    distinctUntilChanged()
  );

  this.loading$ = this.state$.pipe(
    map(s => s.loading),
    distinctUntilChanged()
  );

  this.error$ = this.state$.pipe(
    map(s => s.error),
    distinctUntilChanged()
  );

  this.statistics$ = this.state$.pipe(
    map(s => s.statistics),
    distinctUntilChanged()
  );

  this.highlights$ = this.state$.pipe(
    map(s => s.highlights ?? {}),
    distinctUntilChanged((a, b) => deepEqual(a, b))
  );

  // Initialize from current URL
  this.initializeFromUrl();

  // Watch for URL changes
  this.watchUrlChanges();
}

// ============================================================================
// Public API — State Mutation
// ============================================================================

/**
 * Update filters (triggers URL update → data fetch)
 *
 * Merges partial filters with current filters, updates URL.
 * URL change triggers automatic data fetch (in main window).
 *
 * @param partial - Partial filter object to merge
 *
 * @example
 * ```typescript
 * // Update manufacturer filter
 * this.resources.updateFilters({ manufacturer: 'Ford' });
 *
 * // Update multiple filters
 * this.resources.updateFilters({ manufacturer: 'Ford', page: 1 });
 *
```

```
 * // Clear a filter by setting to null/undefined
 * this.resources.updateFilters({ manufacturer: null });
 * ```
 */
updateFilters(partial: Partial<TFilters>): void {
  const currentFilters = this.stateSubject.value.filters;
  const merged = { ...currentFilters, ...partial };

  // Clean up empty values
  const newFilters: Record<string, any> = {};
  for (const key of Object.keys(merged)) {
    const value = (merged as Record<string, any>)[key];
    if (value !== undefined && value !== null && value !== '') {
      newFilters[key] = value;
    }
  }

  // Convert to URL parameters
  const newUrlParams = this.config.filterMapper.toUrlParams(newFilters as TFilters);

  // Get current URL params to identify removals
  const currentUrlParams = this.config.filterMapper.toUrlParams(currentFilters);

  // Build final params with null for removed params
  const finalParams: Record<string, any> = { ...newUrlParams };
  for (const key of Object.keys(currentUrlParams)) {
    if (!(key in newUrlParams)) {
      finalParams[key] = null; // Mark for removal
    }
  }

  // Update URL (triggers watchUrlChanges → fetchData)
  this.urlState.setParams(finalParams);
}

/**
 * Clear all filters (reset to defaults)
 *
 * @example
 * ```typescript
 * // Reset to initial state
 * this.resources.clearFilters();
 * ```
 */
clearFilters(): void {
  const currentFilters = this.stateSubject.value.filters;
  const currentUrlParams = this.config.filterMapper.toUrlParams(currentFilters);
  const defaultUrlParams = this.config.filterMapper.toUrlParams(this.config.defaultFilters);

  // Build params with null for all current params
  const finalParams: Record<string, any> = { ...defaultUrlParams };
  for (const key of Object.keys(currentUrlParams)) {
    if (!(key in defaultUrlParams)) {
      finalParams[key] = null;
    }
  }

  this.urlState.setParams(finalParams, true); // Replace URL
}
```

```
/**
 * Refresh data with current filters
 *
 * Forces a new API call with current filter state.
 *
 * @example
 * ```typescript
 * // Force refresh after external data change
 * this.resources.refresh();
 * ```
 */
refresh(): void {
  this.fetchData(this.stateSubject.value.filters);
}

// ============================================================================
// Public API — State Access
// ============================================================================

/**
 * Get current state snapshot (synchronous)
 *
 * @returns Current resource state
 */
getCurrentState(): ResourceState<TFilters, TData, TStatistics> {
  return this.stateSubject.value;
}

/**
 * Get current filters snapshot (synchronous)
 *
 * @returns Current filters
 */
getCurrentFilters(): TFilters {
  return this.stateSubject.value.filters;
}

// ============================================================================
// Public API — Cross-Window Sync
// ============================================================================

/**
 * Sync state from external source
 *
 * Used by pop-out windows to receive state from main window.
 * Bypasses URL → fetch flow and directly updates state.
 *
 * @param externalState - State to sync from main window
 *
 * @example
 * ```typescript
 * // In pop-out window, receiving message from main window
 * window.addEventListener('message', (event) => {
 *   if (event.data.type === 'STATE_SYNC') {
 *     this.resources.syncStateFromExternal(event.data.state);
 *   }
 * });
 * ```
 */
```

```
public syncStateFromExternal(
  externalState: ResourceState<TFilters, TData, TStatistics>
): void {
  this.ngZone.run(() => {
    this.stateSubject.next(externalState);
  });
}

// ============================================================================
// Private Methods — Initialization
// ============================================================================

/**
 * Initialize filters from current URL
 */
private initializeFromUrl(): void {
  const urlParams = this.urlState.getParams();
  const filters = this.config.filterMapper.fromUrlParams(urlParams);
  const highlights = this.extractHighlights(urlParams);

  this.updateState({ filters, highlights });
}

/**
 * Watch for URL changes and update state
 */
private watchUrlChanges(): void {
  this.urlState
    .watchParams()
    .pipe(takeUntil(this.destroy$))
    .subscribe(urlParams => {
      const filters = this.config.filterMapper.fromUrlParams(urlParams);
      const highlights = this.extractHighlights(urlParams);
      this.updateState({ filters, highlights });

      // Only fetch in main window (not in pop-out)
      if (this.config.autoFetch) {
        this.fetchData(filters);
      }
    });
}

// ============================================================================
// Private Methods — Data Fetching
// ============================================================================

/**
 * Fetch data from API
 */
private fetchData(filters: TFilters): void {
  this.updateState({ loading: true, error: null });

  const highlights = this.stateSubject.value.highlights;

  this.config.apiAdapter
    .fetchData(filters, highlights)
    .pipe(
      takeUntil(this.destroy$),
      catchError(error => {
```

```
        console.error('[ResourceManagementService] Fetch error:', error);
        this.updateState({
          loading: false,
          error: error instanceof Error ? error : new Error(String(error)),
          results: [],
          totalResults: 0
        });
        return of(null);
      }),
      finalize(() => {
        // Ensure loading is false even if observable completes without value
        if (this.stateSubject.value.loading) {
          this.updateState({ loading: false });
        }
      })
    )
    .subscribe(response => {
      if (response) {
        this.updateState({
          results: response.results,
          totalResults: response.total,
          statistics: response.statistics,
          loading: false,
          error: null
        });
      }
    });
}

// ============================================================================
// Private Methods — Highlights
// ============================================================================

/**
 * Extract highlight filters from URL parameters
 */
private extractHighlights(urlParams: Record<string, any>): any {
  // Prefer domain-specific mapper strategy
  if (this.config.filterMapper.extractHighlights) {
    return this.config.filterMapper.extractHighlights(urlParams);
  }

  // Fallback: prefix-based extraction
  if (!this.config.supportsHighlights) {
    return {};
  }

  const prefix = this.config.highlightPrefix || 'h_';
  const highlights: Record<string, any> = {};

  Object.keys(urlParams).forEach(key => {
    if (key.startsWith(prefix)) {
      const highlightKey = key.substring(prefix.length);
      let value = urlParams[key];

      // Convert pipe-separated to comma-separated
      if (typeof value === 'string' && value.includes('|')) {
        value = value.replace(/\|/g, ',');
      }
```

```
        highlights[highlightKey] = value;
      }
    });

    return highlights;
  }

  // ============================================================================
  // Private Methods — State Management
  // ============================================================================

  /**
   * Update state immutably
   */
  private updateState(partial: Partial<ResourceState<TFilters, TData, TStatistics>>): void {
    this.stateSubject.next({ ...this.stateSubject.value, ...partial });
  }

  // ============================================================================
  // Lifecycle
  // ============================================================================

  /**
   * Clean up subscriptions (public alias)
   */
  destroy(): void {
    this.ngOnDestroy();
  }

  /**
   * Clean up subscriptions on component destroy
   */
  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Step 306.4: Update the Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 6 (Section 306) - Added ResourceManagementService, PopOutContextService

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
export * from './domain-config-registry.service';
export * from './domain-config-validator.service';
export * from './popout-context.service';
export * from './resource-management.service';
```

Phase 3A Milestone: Demo Component

After completing Section 306, create a temporary demo component that displays URL state changes.

Create `src/app/features/demo/url-state-demo.component.ts`:

```
// src/app/features/demo/url-state-demo.component.ts
// TEMPORARY - Remove after verification

import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';
import { UrlStateService } from '../../framework/services';

@Component({
  selector: 'app-url-state-demo',
  template: `
    <div style="padding: 20px; font-family: monospace;">
      <h2>URL State Demo</h2>

      <div style="margin: 20px 0;">
        <h3>Current URL Params:</h3>
        <pre>{{ currentParams | json }}</pre>
      </div>

      <div style="margin: 20px 0;">
        <h3>Test Controls:</h3>
        <button (click)="setManufacturer('Ford')">Set manufacturer=Ford</button>
        <button (click)="setManufacturer('Toyota')">Set manufacturer=Toyota</button>
        <button (click)="setPage(1)">Set page=1</button>
        <button (click)="setPage(2)">Set page=2</button>
        <button (click)="clearAll()">Clear All</button>
      </div>

      <div style="margin: 20px 0;">
        <h3>Log:</h3>
        <div *ngFor="let log of logs" style="font-size: 12px;">{{ log }}</div>
      </div>
    </div>
  `
})
export class UrlStateDemoComponent implements OnInit, OnDestroy {
  currentParams: any = {};
  logs: string[] = [];
  private destroy$ = new Subject<void>();

  constructor(private urlState: UrlStateService) {}

  ngOnInit(): void {
    this.urlState.watchParams()
      .pipe(takeUntil(this.destroy$))
      .subscribe(params => {
        this.currentParams = params;
```

```
        this.logs.unshift(`${new Date().toISOString()}: ${JSON.stringify(params)}`);
        if (this.logs.length > 10) this.logs.pop();
      });
  }

  setManufacturer(value: string): void {
    this.urlState.setParam('manufacturer', value);
  }

  setPage(value: number): void {
    this.urlState.setParam('page', value);
  }

  clearAll(): void {
    this.urlState.clearParams();
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Add route for demo (temporary):

```
// In app-routing.module.ts
{ path: 'demo', component: UrlStateDemoComponent }
```

Navigate to `/demo` and test the URL-First pattern in action.

Verification

1. Check Files Exist

```
$ ls -la src/app/framework/services/resource-management.service.ts
$ ls -la src/app/framework/services/popout-context.service.ts
$ ls -la src/app/framework/tokens/popout.token.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/framework/services/resource-management.service.ts
```

3. Build the Application

```
$ ng build
```

Expected: Build succeeds with no errors.

4. Run Demo Component

```
$ ng serve
```

Navigate to `http://localhost:4200/demo` and:

1. Click "Set manufacturer=Ford"
2. Observe URL changes to `?manufacturer=Ford`
3. Observe Current URL Params updates
4. Use browser back button
5. Observe params revert

This is the Phase 3A Aha Moment in action:

"The URL is the single source of truth. Components react to URL changes, they don't control them."

Common Problems

Symptom	Cause	Solution
`Cannot find module '../tokens/popout.token'`	Token file not created	Create the token file in Step 306.1
`No provider for DOMAIN_CONFIG`	Missing provider	Add provider in component or module
State not updating	Missing subscription	Ensure component subscribes to observables
Infinite loop on filter change	Filter update triggers URL, URL triggers fetch	Check distinctUntilChanged is working
Pop-out doesn't receive state	syncStateFromExternal not called	Implement message passing (Phase 3B)

Key Takeaways

1. **URL is the single source of truth** — updateFilters() modifies URL, not internal state

2. **State flows one direction** — URL → filters → API → results → components
3. **Component-level injection enables isolation** — Each component gets its own instance
4. **Pop-out awareness prevents duplicate fetches** — Main window fetches, pop-outs sync

---

Acceptance Criteria

- [ ] `src/app/framework/services/resource-management.service.ts` exists
- [ ] `src/app/framework/services/popout-context.service.ts` exists (placeholder)
- [ ] `src/app/framework/tokens/popout.token.ts` exists
- [ ] Service is `@Injectable()` (not providedIn: 'root')
- [ ] Observable streams: filters$, results$, loading$, error$, statistics$
- [ ] `updateFilters()` updates URL, not internal state directly
- [ ] `clearFilters()` resets to default filters
- [ ] `refresh()` forces new data fetch
- [ ] `syncStateFromExternal()` enables pop-out state sync
- [ ] autoFetch disabled in pop-out windows
- [ ] Proper cleanup via ngOnDestroy()
- [ ] TypeScript compilation succeeds
- [ ] Demo component works as described

Phase 3A Complete

Congratulations! You have completed Phase 3A: Core Services.

What you built:

- UrlStateService — URL as source of truth
- ApiService — Thin HTTP wrapper
- RequestCoordinatorService — Cache, dedup, retry
- DomainConfigRegistry — Multi-domain support
- DomainConfigValidator — Runtime validation
- ResourceManagementService — State orchestration

The Aha Moment:

"The URL is the single source of truth. Components react to URL changes, they don't control them."

Next Step

Proceed to `307-popout-context-service.md` to begin Phase 3B: Popout & Specialized Services.

Chapter 307

Highlight State Service

Route-Based Detection

Pop-outs are identified by their URL pattern:

```
/panel/:gridId/:panelId/:panelType
```

Example: `/panel/main-grid/statistics-1/statistics-panel-2`

The service parses this URL to extract:

- `isPopOut`: true (URL starts with `/panel`)
- `gridId`: "main-grid"
- `panelId`: "statistics-1"
- `panelType`: "statistics-panel-2"

What

Step 307.1: Replace the Placeholder Service

Replace the placeholder created in Section 306 with the full implementation.

Update `src/app/framework/services/popout-context.service.ts`:

```
// src/app/framework/services/popout-context.service.ts
// VERSION 2 (Section 307) - Full implementation with BroadcastChannel

import { Injectable, OnDestroy, NgZone } from '@angular/core';
import { Router } from '@angular/router';
import { Observable, ReplaySubject } from 'rxjs';
import {
  PopOutMessage,
  PopOutMessageType,
  PopOutContext,
  parsePopOutRoute
} from '../models/popout.interface';

/**
 * Pop-out context service
```

```
 *
 * Determines whether the current window is a pop-out and provides
 * messaging infrastructure for parent-child window communication.
 *
 * **Two Modes:**
 *
 * 1. **Pop-out mode** — Running in a pop-out window
 *    - `isInPopOut()` returns true
 *    - Sets up BroadcastChannel to receive state from main window
 *    - Sends PANEL_READY message when initialized
 *
 * 2. **Parent mode** — Running in main window
 *    - `isInPopOut()` returns false
 *    - Manages channels for each pop-out panel
 *    - Broadcasts state updates to all pop-outs
 *
 * **BroadcastChannel Pattern:**
 *
 * Each panel gets its own channel named `panel-{panelId}`.
 * This allows targeted messaging to specific panels.
 *
 * @example
 * ```typescript
 * // In pop-out window component
 * if (this.context.isInPopOut()) {
 *   const ctx = this.context.getContext();
 *   this.context.initializeAsPopOut(ctx.panelId);
 *
 *   this.context.getMessages$().subscribe(message => {
 *     if (message.type === PopOutMessageType.STATE_UPDATE) {
 *       this.handleStateUpdate(message.payload);
 *     }
 *   });
 * }
 *
 * // In main window
 * this.context.initializeAsParent();
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class PopOutContextService implements OnDestroy {
  /**
   * BroadcastChannel for this window
   *
   * In pop-out: channel for receiving messages from main window
   * In parent: may be null (uses per-panel channels via PopOutManager)
   */
  private channel: BroadcastChannel | null = null;

  /**
   * ReplaySubject for incoming messages
   *
   * ReplaySubject with buffer of 10 ensures late subscribers
   * can receive recent messages they might have missed.
   */
  private messagesSubject = new ReplaySubject<PopOutMessage>(10);
```

```
/**
 * Parsed context from URL
 *
 * Contains isPopOut, gridId, panelId, panelType
 */
private context: PopOutContext | null = null;

/**
 * Initialization flag to prevent double-init
 */
private initialized = false;

/**
 * Constructor - parses URL to determine context
 *
 * @param router - Angular Router for URL access
 * @param ngZone - NgZone for ensuring change detection
 */
constructor(
  private router: Router,
  private ngZone: NgZone
) {
  // Parse context from current URL
  this.context = parsePopOutRoute(this.router.url);
}

/**
 * Check if current window is a pop-out
 *
 * Detection: Checks if router.url starts with '/panel/'
 *
 * @returns True if in pop-out window
 */
isInPopOut(): boolean {
  // Re-parse if context not set (defensive)
  if (!this.context) {
    this.context = parsePopOutRoute(this.router.url);
  }
  return this.context?.isPopOut || false;
}

/**
 * Get parsed pop-out context
 *
 * @returns Context object with gridId, panelId, panelType, or null
 */
getContext(): PopOutContext | null {
  if (!this.context) {
    this.context = parsePopOutRoute(this.router.url);
  }
  return this.context;
}

/**
 * Initialize as pop-out window
 *
 * Sets up BroadcastChannel for receiving messages from main window.
 * Sends PANEL_READY message to notify main window.
 *
```

```
 * @param panelId - Panel identifier for channel naming
 */
initializeAsPopOut(panelId: string): void {
  if (this.initialized) {
    return;
  }

  this.initialized = true;
  this.setupChannel(panelId);

  // Notify main window that pop-out is ready
  this.sendMessage({
    type: PopOutMessageType.PANEL_READY,
    timestamp: Date.now()
  });
}

/**
 * Initialize as parent window
 *
 * Called by main window to set initialized flag.
 * Actual channel management is handled by PopOutManagerService.
 */
initializeAsParent(): void {
  if (this.initialized) {
    return;
  }

  this.initialized = true;
  // Parent doesn't set up a channel here - PopOutManager handles per-panel channels
}

/**
 * Set up BroadcastChannel for this panel
 *
 * @param panelId - Panel identifier for channel naming
 */
private setupChannel(panelId: string): void {
  const channelName = `panel-${panelId}`;

  // Close existing channel if any
  if (this.channel) {
    this.channel.close();
  }

  // Create new channel
  this.channel = new BroadcastChannel(channelName);

  // Handle incoming messages
  this.channel.onmessage = (event: MessageEvent) => {
    const message = event.data as PopOutMessage;
    // Use NgZone to ensure Angular change detection runs
    this.ngZone.run(() => {
      this.messagesSubject.next(message);
    });
  };

  // Handle message errors (rare, but log them)
  this.channel.onmessageerror = () => {
```

```
    // Silently ignore message errors
  };
}

/**
 * Send message to channel
 *
 * Used by pop-out to send messages to main window.
 *
 * @template T - Payload type
 * @param message - Message to send
 */
sendMessage<T = any>(message: PopOutMessage<T>): void {
  if (!this.channel) {
    return;
  }

  // Add timestamp if not present
  if (!message.timestamp) {
    message.timestamp = Date.now();
  }

  try {
    this.channel.postMessage(message);
  } catch (error) {
    // Silently ignore send errors
  }
}

/**
 * Get observable of incoming messages
 *
 * @returns Observable of PopOutMessage
 */
getMessages$(): Observable<PopOutMessage> {
  return this.messagesSubject.asObservable();
}

/**
 * Create a BroadcastChannel for a specific panel
 *
 * Used by PopOutManagerService to create channels for pop-out windows.
 *
 * @param panelId - Panel identifier
 * @returns New BroadcastChannel instance
 */
createChannelForPanel(panelId: string): BroadcastChannel {
  const channelName = `panel-${panelId}`;
  return new BroadcastChannel(channelName);
}

/**
 * Close the channel
 *
 * Call when pop-out is closing or service is destroyed.
 */
close(): void {
  if (this.channel) {
    this.channel.close();
```

```
      this.channel = null;
    }

    this.initialized = false;
  }

  /**
   * Cleanup on service destroy
   */
  ngOnDestroy(): void {
    this.close();
    this.messagesSubject.complete();
  }
}
```

Chapter 308

Statistics Processor Service

```
// src/app/framework/services/popout-manager.service.ts
// VERSION 1 (Section 308) - Pop-out window manager

import { Injectable, NgZone, OnDestroy } from '@angular/core';
import { Subject } from 'rxjs';
import {
  buildWindowFeatures,
  PopOutMessage,
  PopOutMessageType,
  PopOutWindowFeatures,
  PopOutWindowRef
} from '../models/popout.interface';
import { PopOutContextService } from './popout-context.service';
import { FilterOptionsCache } from './filter-options.service';

/**
 * Pop-out manager service
 *
 * Manages pop-out windows from the main window. Opens windows,
 * broadcasts state, and tracks lifecycle.
 *
 * **Important:** This is a component-level service, not a singleton.
 * Each component that needs pop-out management provides its own instance.
 *
 * **Lifecycle:**
 *
 * 1. Component creates → Manager injected
 * 2. Component calls initialize() with grid ID
 * 3. User clicks pop-out button → openPopOut() called
 * 4. Filter changes → broadcastState() called
 * 5. Pop-out closes → Manager detects and cleans up
 * 6. Component destroys → All pop-outs closed
 *
 * @example
 * ```typescript
 * @Component({
 *   providers: [PopOutManagerService]
 * })
 * export class DiscoverComponent implements OnInit, OnDestroy {
 *   constructor(
 *     private popOutManager: PopOutManagerService,
 *     private resources: ResourceManagementService
 *   ) {}
 *
 *   ngOnInit(): void {
 *     this.popOutManager.initialize('discover-grid');
 *
 *     // Broadcast state to pop-outs when it changes
 *     this.resources.state$.subscribe(state => {
 *       this.popOutManager.broadcastState(state);
 *     });
```

```
 *
 *     // Handle pop-out closure
 *     this.popOutManager.closed$.subscribe(panelId => {
 *       console.log(`Pop-out ${panelId} closed`);
 *     });
 *   }
 *
 *   onPopOutClick(panelId: string, panelType: string): void {
 *     this.popOutManager.openPopOut(panelId, panelType);
 *   }
 * }
 * ```
 */
@Injectable() // Component-level, not providedIn: 'root'
export class PopOutManagerService implements OnDestroy {
  /**
   * Grid ID for URL construction
   */
  private gridId = '';

  /**
   * Set of currently popped-out panel IDs
   */
  private poppedOutPanels = new Set<string>();

  /**
   * Map of panel ID to window reference/channel
   */
  private popoutWindows = new Map<string, PopOutWindowRef>();

  /**
   * Subject for messages from pop-outs
   */
  private messagesSubject = new Subject<{ panelId: string; message: PopOutMessage }>();

  /**
   * Subject for pop-out close events
   */
  private closedSubject = new Subject<string>();

  /**
   * Subject for blocked pop-up events (browser blocked window.open)
   */
  private blockedSubject = new Subject<string>();

  /**
   * Handler for beforeunload to close all pop-outs
   */
  private beforeUnloadHandler = () => this.closeAllPopOuts();

  /**
   * Initialization flag
   */
  private initialized = false;

  // Public observables
  readonly messages$ = this.messagesSubject.asObservable();
  readonly closed$ = this.closedSubject.asObservable();
  readonly blocked$ = this.blockedSubject.asObservable();
```

```
/**
 * Constructor
 *
 * @param popOutContext - Context service for channel creation
 * @param ngZone - NgZone for change detection
 */
constructor(
  private popOutContext: PopOutContextService,
  private ngZone: NgZone
) {}

/**
 * Initialize the manager
 *
 * Must be called before any pop-out operations.
 *
 * @param gridId - Grid identifier for URL construction
 */
initialize(gridId: string): void {
  if (this.initialized) {
    return;
  }

  this.gridId = gridId;
  this.initialized = true;

  // Initialize context as parent
  this.popOutContext.initializeAsParent();

  // Close all pop-outs when main window closes
  window.addEventListener('beforeunload', this.beforeUnloadHandler);

  // Subscribe to messages from pop-outs
  this.popOutContext.getMessages$().subscribe(message => {
    this.messagesSubject.next({ panelId: '', message });
  });
}

/**
 * Check if a panel is currently popped out
 *
 * @param panelId - Panel identifier
 * @returns True if panel is popped out
 */
isPoppedOut(panelId: string): boolean {
  return this.poppedOutPanels.has(panelId);
}

/**
 * Get all currently popped-out panel IDs
 *
 * @returns Array of panel IDs
 */
getPoppedOutPanels(): string[] {
  return Array.from(this.poppedOutPanels);
}

/**
 * Open a panel in a pop-out window
```

```
 *
 * @param panelId - Panel identifier
 * @param panelType - Panel type (e.g., 'statistics-panel-2')
 * @param features - Optional window features
 * @returns True if opened successfully, false if blocked or already open
 */
openPopOut(
  panelId: string,
  panelType: string,
  features?: Partial<PopOutWindowFeatures>
): boolean {
  // Don't open if already popped out
  if (this.poppedOutPanels.has(panelId)) {
    return false;
  }

  // Construct URL: /panel/:gridId/:panelId/:panelType
  const url = `/panel/${this.gridId}/${panelId}/${panelType}`;

  // Build window features string
  const windowFeatures = buildWindowFeatures({
    width: 1200,
    height: 800,
    left: 100,
    top: 100,
    resizable: true,
    scrollbars: true,
    ...features
  });

  // Open the window
  const popoutWindow = window.open(url, `panel-${panelId}`, windowFeatures);

  // Handle blocked popup
  if (!popoutWindow) {
    this.blockedSubject.next(panelId);
    return false;
  }

  // Track the pop-out
  this.poppedOutPanels.add(panelId);

  // Create channel for this panel
  const channel = this.popOutContext.createChannelForPanel(panelId);

  // Listen for messages from this panel
  channel.onmessage = event => {
    this.ngZone.run(() => {
      this.messagesSubject.next({ panelId, message: event.data });
    });
  };

  // Poll to detect when window closes
  const checkInterval = window.setInterval(() => {
    if (popoutWindow.closed) {
      this.ngZone.run(() => {
        this.handlePopOutClosed(panelId, channel, checkInterval);
      });
    }
```

```
  }, 500);

  // Store reference
  this.popoutWindows.set(panelId, {
    window: popoutWindow,
    channel,
    checkInterval,
    panelId,
    panelType
  });

  return true;
}

/**
 * Broadcast state to all pop-out windows
 *
 * Call this whenever state changes in the main window.
 *
 * @param state - Application state from ResourceManagementService
 * @param filterOptionsCache - Optional cached filter options
 */
broadcastState(state: any, filterOptionsCache?: FilterOptionsCache): void {
  if (this.popoutWindows.size === 0) {
    return; // No pop-outs to broadcast to
  }

  const message: PopOutMessage = {
    type: PopOutMessageType.STATE_UPDATE,
    payload: {
      state,
      filterOptionsCache: filterOptionsCache || null
    },
    timestamp: Date.now()
  };

  // Send to all pop-out channels
  this.popoutWindows.forEach(({ channel }) => {
    try {
      channel.postMessage(message);
    } catch {
      // Silently ignore posting errors (channel may be closed)
    }
  });
}

/**
 * Close a specific pop-out window
 *
 * @param panelId - Panel identifier
 */
closePopOut(panelId: string): void {
  const ref = this.popoutWindows.get(panelId);
  if (ref) {
    // Send close message
    ref.channel.postMessage({
      type: PopOutMessageType.CLOSE_POPOUT,
      timestamp: Date.now()
    });
```

```
  }
}

/**
 * Close all pop-out windows
 *
 * Called when main window is closing.
 */
closeAllPopOuts(): void {
  this.popoutWindows.forEach(({ channel }) => {
    channel.postMessage({
      type: PopOutMessageType.CLOSE_POPOUT,
      timestamp: Date.now()
    });
  });
}

/**
 * Handle pop-out window closed
 *
 * @param panelId - Panel that closed
 * @param channel - BroadcastChannel to close
 * @param checkInterval - Interval to clear
 */
private handlePopOutClosed(
  panelId: string,
  channel: BroadcastChannel,
  checkInterval: number
): void {
  // Clear polling interval
  clearInterval(checkInterval);

  // Close channel
  channel.close();

  // Remove from tracking
  this.popoutWindows.delete(panelId);
  this.poppedOutPanels.delete(panelId);

  // Emit closed event
  this.closedSubject.next(panelId);
}

/**
 * Cleanup on service destroy
 */
ngOnDestroy(): void {
  // Remove beforeunload listener
  window.removeEventListener('beforeunload', this.beforeUnloadHandler);

  // Close all pop-outs and cleanup
  this.popoutWindows.forEach(({ window: win, channel, checkInterval }) => {
    clearInterval(checkInterval);
    channel.close();
    if (win && !win.closed) {
      win.close();
    }
  });
```

```
    // Complete subjects
    this.messagesSubject.complete();
    this.closedSubject.complete();
    this.blockedSubject.complete();
  }
}
```

Chapter 309

Export Service

```
// src/app/framework/services/user-preferences.service.ts
// VERSION 1 (Section 309) - User preferences with localStorage and backend API

import { Injectable, isDevMode } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, of } from 'rxjs';
import { catchError, timeout } from 'rxjs/operators';

/**
 * User preferences service
 *
 * Persists user preferences using localStorage with domain-aware namespacing.
 * Handles storage failures gracefully with in-memory fallback.
 * Supports backend API for cross-device sync with localStorage fallback.
 *
 * **Features:**
 *
 * 1. **Domain namespacing** — Each domain has its own preferences
 * 2. **Reactive state** — BehaviorSubjects for real-time updates
 * 3. **Graceful degradation** — Works even if localStorage unavailable
 * 4. **Panel merge logic** — New panels inserted at correct positions
 * 5. **Backend API support** — Cross-device sync with localStorage fallback
 *
 * **Storage Keys:**
 *
 * ```
 * prefs:{domain}:panelOrder → ['panel-1', 'panel-2', ...]
 * prefs:{domain}:collapsedPanels → ['panel-1']
 * ```
 *
 * @example
 * ```typescript
 * // In component
 * constructor(private prefs: UserPreferencesService) {
 *   // Subscribe to panel order changes
 *   this.prefs.getPanelOrder().subscribe(order => {
 *     this.panelOrder = order;
 *   });
 *
 *   // Save new order after drag-drop
 *   this.prefs.savePanelOrder(['panel-2', 'panel-1', 'panel-3']);
 * }
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class UserPreferencesService {
  /**
   * Default panel order when no preferences saved
   */
```

```
private readonly DEFAULT_PANEL_ORDER = [
  'query-control',
  'query-panel',
  'manufacturer-model-picker',
  'statistics-panel-2',
  'basic-results-table'
];

/**
 * Default collapsed panels (none)
 */
private readonly DEFAULT_COLLAPSED_PANELS: string[] = [];

/**
 * Current domain extracted from URL
 */
private currentDomain = this.extractCurrentDomain();

/**
 * BehaviorSubject for panel order
 */
private panelOrderSubject = new BehaviorSubject<string[]>(
  this.loadPanelOrder()
);

/**
 * BehaviorSubject for collapsed panels
 */
private collapsedPanelsSubject = new BehaviorSubject<string[]>(
  this.loadCollapsedPanels()
);

/**
 * Storage availability flag
 */
private storageAvailable = this.checkStorageAvailable();

/**
 * Full preferences object for backend API storage
 */
private fullPreferences: any = {};

/**
 * User ID for backend preferences service
 */
private userId = 'default';

constructor(private http: HttpClient) {
  // Try loading from backend API first, fall back to localStorage
  this.loadFromBackendApi().pipe(
    timeout(5000), // 5 second timeout
    catchError(() => {
      // Fall back to localStorage
      return of(this.loadFromLocalStorage());
    })
  ).subscribe(prefs => {
    this.fullPreferences = prefs || {};
    this.initializeFromPreferences(this.fullPreferences);
  });
```

```
}

/**
 * Get panel order as observable
 *
 * Emits current order immediately, then on every change.
 *
 * @returns Observable of panel order array
 */
getPanelOrder(): Observable<string[]> {
  return this.panelOrderSubject.asObservable();
}

/**
 * Get collapsed panels as observable
 *
 * @returns Observable of collapsed panel IDs
 */
getCollapsedPanels(): Observable<string[]> {
  return this.collapsedPanelsSubject.asObservable();
}

/**
 * Get current panel order synchronously
 *
 * @returns Current panel order array
 */
getCurrentPanelOrder(): string[] {
  return this.panelOrderSubject.value;
}

/**
 * Get current collapsed panels synchronously
 *
 * @returns Current collapsed panel IDs
 */
getCurrentCollapsedPanels(): string[] {
  return this.collapsedPanelsSubject.value;
}

/**
 * Save panel order
 *
 * Updates BehaviorSubject and persists to backend API or localStorage.
 *
 * @param order - New panel order array
 */
savePanelOrder(order: string[]): void {
  this.panelOrderSubject.next(order);

  // Update full preferences object
  const domain = this.currentDomain;
  if (!this.fullPreferences[domain]) {
    this.fullPreferences[domain] = {};
  }
  this.fullPreferences[domain].panelOrder = order;

  // Try saving to backend API first
  this.savePreferencesToBackend(this.fullPreferences).pipe(
```

```
    catchError(() => {
      // Fall back to localStorage
      this.saveToLocalStorage('panelOrder', order);
      return of(null);
    })
  ).subscribe();
}

/**
 * Save collapsed panels
 *
 * @param panels - Array of collapsed panel IDs
 */
saveCollapsedPanels(panels: string[]): void {
  this.collapsedPanelsSubject.next(panels);

  // Update full preferences object
  const domain = this.currentDomain;
  if (!this.fullPreferences[domain]) {
    this.fullPreferences[domain] = {};
  }
  this.fullPreferences[domain].collapsedPanels = panels;

  // Try saving to backend API first
  this.savePreferencesToBackend(this.fullPreferences).pipe(
    catchError(() => {
      // Fall back to localStorage
      this.saveToLocalStorage('collapsedPanels', panels);
      return of(null);
    })
  ).subscribe();
}

/**
 * Reset preferences for current domain
 *
 * @param domain - Optional domain (defaults to current)
 */
reset(domain?: string): void {
  const targetDomain = domain || this.currentDomain;

  if (this.storageAvailable) {
    try {
      localStorage.removeItem(this.getKey('panelOrder', targetDomain));
      localStorage.removeItem(this.getKey('collapsedPanels', targetDomain));
    } catch (e) {
      // Ignore errors on reset
    }
  }

  this.panelOrderSubject.next(this.DEFAULT_PANEL_ORDER);
  this.collapsedPanelsSubject.next(this.DEFAULT_COLLAPSED_PANELS);
}

/**
 * Get current domain
 *
 * @returns Current domain name
 */
```

```
getCurrentDomain(): string {
  return this.currentDomain;
}

// ==============================================================================
// Private Methods
// ==============================================================================

/**
 * Load panel order from localStorage
 */
private loadPanelOrder(): string[] {
  if (!this.storageAvailable) {
    return this.DEFAULT_PANEL_ORDER;
  }

  try {
    const key = this.getKey('panelOrder');
    const stored = localStorage.getItem(key);

    if (stored) {
      const parsed = JSON.parse(stored);
      if (Array.isArray(parsed)) {
        // Merge with defaults to include new panels
        return this.mergePanelOrder(parsed, this.DEFAULT_PANEL_ORDER);
      }
    }
  } catch (e) {
    this.logError('loadPanelOrder', e);
  }

  return this.DEFAULT_PANEL_ORDER;
}

/**
 * Load collapsed panels from localStorage
 */
private loadCollapsedPanels(): string[] {
  if (!this.storageAvailable) {
    return this.DEFAULT_COLLAPSED_PANELS;
  }

  try {
    const key = this.getKey('collapsedPanels');
    const stored = localStorage.getItem(key);

    if (stored) {
      const parsed = JSON.parse(stored);
      if (Array.isArray(parsed)) {
        return parsed;
      }
    }
  } catch (e) {
    this.logError('loadCollapsedPanels', e);
  }

  return this.DEFAULT_COLLAPSED_PANELS;
}
```

```
/**
 * Load preferences from backend API
 */
private loadFromBackendApi(): Observable<any> {
  return this.http.get<any>(`/api/preferences/v1/${this.userId}`).pipe(
    catchError((error) => {
      if (isDevMode()) {
        console.debug('[UserPreferencesService] Backend API not available, falling back to localStorage');
      }
      throw error;
    })
  );
}

/**
 * Save preferences to backend API
 */
private savePreferencesToBackend(prefs: any): Observable<any> {
  return this.http.post<any>(`/api/preferences/v1/${this.userId}`, prefs).pipe(
    catchError((error) => {
      if (isDevMode()) {
        console.debug('[UserPreferencesService] Failed to save to backend API:', error);
      }
      throw error;
    })
  );
}

/**
 * Load preferences from localStorage
 */
private loadFromLocalStorage(): any {
  const prefs: any = {};

  ['automobiles', 'physics', 'agriculture', 'chemistry', 'math'].forEach(domain => {
    const orderKey = this.getKey('panelOrder', domain);
    const collapsedKey = this.getKey('collapsedPanels', domain);

    try {
      const orderStr = localStorage.getItem(orderKey);
      const collapsedStr = localStorage.getItem(collapsedKey);

      prefs[domain] = {
        panelOrder: orderStr ? JSON.parse(orderStr) : this.DEFAULT_PANEL_ORDER,
        collapsedPanels: collapsedStr ? JSON.parse(collapsedStr) : this.DEFAULT_COLLAPSED_PANELS
      };
    } catch (e) {
      prefs[domain] = {
        panelOrder: this.DEFAULT_PANEL_ORDER,
        collapsedPanels: this.DEFAULT_COLLAPSED_PANELS
      };
    }
  });

  return prefs;
}

/**
 * Initialize service state from loaded preferences
```

```
 */
private initializeFromPreferences(prefs: any): void {
  const domain = this.currentDomain;
  const domainPrefs = prefs[domain];

  if (domainPrefs) {
    const storedOrder = domainPrefs.panelOrder;
    const mergedOrder = storedOrder
      ? this.mergePanelOrder(storedOrder, this.DEFAULT_PANEL_ORDER)
      : this.DEFAULT_PANEL_ORDER;
    this.panelOrderSubject.next(mergedOrder);
    this.collapsedPanelsSubject.next(domainPrefs.collapsedPanels || this.DEFAULT_COLLAPSED_PANELS);
  } else {
    this.panelOrderSubject.next(this.DEFAULT_PANEL_ORDER);
    this.collapsedPanelsSubject.next(this.DEFAULT_COLLAPSED_PANELS);
  }
}

/**
 * Merge stored order with defaults
 *
 * - Removes panels no longer in defaults
 * - Inserts new panels at their default relative position
 *
 * @param stored - User's stored order
 * @param defaults - Default order (source of truth for valid panels)
 * @returns Merged order
 */
private mergePanelOrder(stored: string[], defaults: string[]): string[] {
  const defaultsSet = new Set(defaults);

  // Filter out panels that no longer exist
  const result = stored.filter(id => defaultsSet.has(id));
  const resultSet = new Set(result);

  // Insert new panels at their relative position
  for (let i = 0; i < defaults.length; i++) {
    const panelId = defaults[i];
    if (!resultSet.has(panelId)) {
      // Find insertion point based on previous panel in defaults
      let insertIndex = result.length;

      for (let j = i - 1; j >= 0; j--) {
        const prevPanel = defaults[j];
        const prevIndex = result.indexOf(prevPanel);
        if (prevIndex !== -1) {
          insertIndex = prevIndex + 1;
          break;
        }
      }

      result.splice(insertIndex, 0, panelId);
      resultSet.add(panelId);
    }
  }

  return result;
}
```

```
/**
 * Save to localStorage
 */
private saveToLocalStorage(preference: string, value: any): void {
  if (!this.storageAvailable) {
    return;
  }

  try {
    const key = this.getKey(preference);
    localStorage.setItem(key, JSON.stringify(value));
  } catch (e) {
    this.handleStorageError(e);
  }
}

/**
 * Get storage key with domain prefix
 */
private getKey(preference: string, domain?: string): string {
  const targetDomain = domain || this.currentDomain;
  return `prefs:${targetDomain}:${preference}`;
}

/**
 * Check if localStorage is available
 */
private checkStorageAvailable(): boolean {
  try {
    const test = '__localStorage_test__';
    localStorage.setItem(test, 'test');
    localStorage.removeItem(test);
    return true;
  } catch (e) {
    return false;
  }
}

/**
 * Handle storage errors
 */
private handleStorageError(error: any): void {
  this.storageAvailable = false;
  this.logError('storage', error);
}

/**
 * Log error in dev mode only
 */
private logError(context: string, error: any): void {
  if (isDevMode()) {
    console.debug(`[UserPreferencesService] ${context} error:`, error);
  }
}

/**
 * Extract current domain from URL
 */
private extractCurrentDomain(): string {
```

```
    // Get domain from current URL path
    const path = window.location.pathname;

    // Match patterns like /automobiles/, /physics/, etc.
    const match = path.match(/\/([a-z]+)\//);
    if (match && match[1]) {
      return match[1];
    }

    // Default to 'automobiles' if extraction fails
    return 'automobiles';
  }
}
```

Chapter 310

Keyboard Shortcuts Service

```
// src/app/framework/services/filter-options.service.ts
// VERSION 1 (Section 310) - Filter options caching

import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable, of } from 'rxjs';
import { tap, catchError, map } from 'rxjs/operators';
import { ApiService } from './api.service';
import { FilterOption } from '../models/filter-definition.interface';

/**
 * Cached options for a single filter
 */
export interface CachedFilterOptions {
  /** Filter field identifier */
  field: string;
  /** API endpoint used */
  endpoint: string;
  /** Transformed options */
  options: FilterOption[];
  /** Raw API response (for custom transformers) */
  rawResponse?: any;
  /** Cache timestamp */
  cachedAt: number;
}

/**
 * Complete filter options cache
 */
export interface FilterOptionsCache {
  [endpoint: string]: CachedFilterOptions;
}

/**
 * Filter options service
 *
 * Caches filter dropdown options and supports cross-window sharing.
 *
 * **Key Features:**
 *
 * 1. **Cache-first loading** — Returns cached options if available
 * 2. **Transformer support** — Adapts API responses to FilterOption[]
 * 3. **Pop-out support** — syncFromExternal() receives cache from main window
 * 4. **Reactive cache** — Observable of cache changes
 *
 * **Usage Pattern:**
 *
 * 1. Main window loads options (API call + cache)
 * 2. Main window broadcasts state (includes filterOptionsCache)
 * 3. Pop-out receives state
 * 4. Pop-out calls syncFromExternal(cache)
 * 5. Pop-out has options without API calls
```

```
 *
 * **Note:** FilterOption is imported from filter-definition.interface.ts:
 * ```typescript
 * export interface FilterOption {
 *   value: string | number;
 *   label: string;
 *   count?: number;
 * }
 * ```
 *
 * @example
 * ```typescript
 * // Load with transformer
 * this.filterOptions.getOptions(
 *   'http://api/manufacturers',
 *   'manufacturer',
 *   response => response.data.map(m => ({ value: m.id, label: m.name }))
 * ).subscribe(options => {
 *   this.manufacturerOptions = options;
 * });
 *
 * // In pop-out, sync from main window
 * this.filterOptions.syncFromExternal(message.payload.filterOptionsCache);
 *
 * // Check if cached
 * if (this.filterOptions.isCached('http://api/manufacturers')) {
 *   // Use cached value
 * }
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class FilterOptionsService {
  /**
   * Cache storage
   */
  private cache: FilterOptionsCache = {};

  /**
   * Observable of cache state
   */
  private cache$ = new BehaviorSubject<FilterOptionsCache>({});

  /**
   * Constructor
   *
   * @param apiService - API service for fetching options
   */
  constructor(private apiService: ApiService) {}

  /**
   * Get current cache for state broadcast
   *
   * @returns Copy of current cache
   */
  getCache(): FilterOptionsCache {
    return { ...this.cache };
  }
```

```
/**
 * Get observable of cache changes
 *
 * @returns Observable of cache state
 */
getCache$(): Observable<FilterOptionsCache> {
  return this.cache$.asObservable();
}

/**
 * Get options (from cache or API)
 *
 * @param endpoint - API endpoint URL
 * @param field - Filter field identifier
 * @param transformer - Optional function to transform API response
 * @returns Observable of filter options
 */
getOptions(
  endpoint: string,
  field: string,
  transformer?: (response: any) => FilterOption[]
): Observable<FilterOption[]> {
  // Check cache first
  const cached = this.cache[endpoint];
  if (cached) {
    // If transformer provided and we have raw response, re-transform
    if (transformer && cached.rawResponse) {
      return of(transformer(cached.rawResponse));
    }
    return of(cached.options);
  }

  // Fetch from API
  return this.apiService.get(endpoint).pipe(
    tap(response => {
      const options = transformer ? transformer(response) : [];
      this.cache[endpoint] = {
        field,
        endpoint,
        options,
        rawResponse: response,
        cachedAt: Date.now()
      };
      this.cache$.next({ ...this.cache });
    }),
    map(response => transformer ? transformer(response) : []),
    catchError(error => {
      console.error(`[FilterOptionsService] Failed to load from ${endpoint}:`, error);
      throw error;
    })
  );
}

/**
 * Get raw API response from cache
 *
 * Used for filters that need the full response (e.g., range filters).
 *
 * @param endpoint - API endpoint URL
```

```
 * @returns Cached raw response or null
 */
getRawResponse(endpoint: string): any | null {
  return this.cache[endpoint]?.rawResponse ?? null;
}

/**
 * Get raw response asynchronously (from cache or API)
 *
 * @param endpoint - API endpoint URL
 * @param field - Filter field identifier
 * @returns Observable of raw API response
 */
getRawResponseAsync(endpoint: string, field: string): Observable<any> {
  // Check cache first
  const cached = this.cache[endpoint];
  if (cached?.rawResponse) {
    return of(cached.rawResponse);
  }

  // Fetch from API
  return this.apiService.get(endpoint).pipe(
    tap(response => {
      this.cache[endpoint] = {
        field,
        endpoint,
        options: [],
        rawResponse: response,
        cachedAt: Date.now()
      };
      this.cache$.next({ ...this.cache });
    }),
    catchError(error => {
      console.error(`[FilterOptionsService] Failed to load from ${endpoint}:`, error);
      throw error;
    })
  );
}

/**
 * Sync cache from external source
 *
 * Used by pop-out windows to receive cache from main window.
 *
 * @param externalCache - Cache from main window
 */
syncFromExternal(externalCache: FilterOptionsCache): void {
  if (!externalCache) {
    return;
  }

  // Merge external cache into local cache
  this.cache = { ...this.cache, ...externalCache };
  this.cache$.next({ ...this.cache });
}

/**
 * Check if options are cached
 *
```

```
   * @param endpoint - API endpoint URL
   * @returns True if cached
   */
  isCached(endpoint: string): boolean {
    return !!this.cache[endpoint];
  }

  /**
   * Clear entire cache
   */
  clearCache(): void {
    this.cache = {};
    this.cache$.next({});
  }
}
```

Chapter 311

Theme Service

```
// src/app/framework/services/picker-config-registry.service.ts
// VERSION 1 (Section 311) - Picker configuration registry

import { Injectable } from '@angular/core';
import { PickerConfig } from '../models/picker-config.interface';

/**
 * Picker configuration registry service
 *
 * Centralized registry for managing picker configurations.
 * Allows registration and retrieval of picker configs by ID.
 *
 * **Why a Registry?**
 *
 * 1. **O(1) lookup** — Get config by ID without searching arrays
 * 2. **Type safety** — Generic retrieval preserves types
 * 3. **Centralized** — Single source of truth for picker configs
 * 4. **Decoupled** — Components don't need to know about domain config
 *
 * **Typical Flow:**
 *
 * 1. Domain module registers picker configs at init
 * 2. Components/templates reference pickers by ID
 * 3. BasePicker component looks up config from registry
 *
 * @example
 * ```typescript
 * // Registration (in domain module)
 * constructor(private registry: PickerConfigRegistry) {
 *   registry.registerMultiple(AUTOMOBILE_PICKER_CONFIGS);
 * }
 *
 * // Lookup (in component)
 * const config = registry.get<VehicleResult>('vehicle-picker');
 *
 * // Template usage
 * <app-base-picker [configId]="'vehicle-picker'"></app-base-picker>
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class PickerConfigRegistry {
  /**
   * Storage for registered picker configurations
   */
  private configs = new Map<string, PickerConfig<any>>();

  /**
   * Register a picker configuration
   *
```

```
 * @template T - The data model type
 * @param config - Picker configuration to register
 *
 * @example
 * ```typescript
 * const vehicleConfig: PickerConfig<VehicleResult> = {
 *   id: 'vehicle-picker',
 *   displayName: 'Vehicle Selection',
 *   columns: [...],
 *   // ...
 * };
 *
 * registry.register(vehicleConfig);
 * ```
 */
register<T>(config: PickerConfig<T>): void {
  if (this.configs.has(config.id)) {
    console.warn(
      `Picker config with ID '${config.id}' is already registered. ` +
      `Overwriting existing configuration.`
    );
  }

  this.configs.set(config.id, config);
}

/**
 * Register multiple picker configurations
 *
 * @param configs - Array of picker configurations
 *
 * @example
 * ```typescript
 * registry.registerMultiple([
 *   VEHICLE_PICKER_CONFIG,
 *   MANUFACTURER_MODEL_PICKER_CONFIG,
 *   VIN_PICKER_CONFIG
 * ]);
 * ```
 */
registerMultiple(configs: PickerConfig<any>[]): void {
  configs.forEach(config => this.register(config));
}

/**
 * Get picker configuration by ID
 *
 * @template T - The data model type
 * @param id - Picker configuration ID
 * @returns Picker configuration
 * @throws Error if picker ID not found
 *
 * @example
 * ```typescript
 * const config = registry.get<VehicleResult>('vehicle-picker');
 * // config is typed as PickerConfig<VehicleResult>
 * ```
 */
get<T>(id: string): PickerConfig<T> {
```

```
  const config = this.configs.get(id);

  if (!config) {
    throw new Error(
      `Picker configuration with ID '${id}' not found. ` +
      `Available pickers: ${Array.from(this.configs.keys()).join(', ')}`
    );
  }

  return config as PickerConfig<T>;
}

/**
 * Check if picker configuration exists
 *
 * @param id - Picker configuration ID
 * @returns True if picker exists
 */
has(id: string): boolean {
  return this.configs.has(id);
}

/**
 * Get all registered picker IDs
 *
 * @returns Array of picker IDs
 */
getAllIds(): string[] {
  return Array.from(this.configs.keys());
}

/**
 * Get all registered picker configurations
 *
 * @returns Array of all picker configurations
 */
getAll(): PickerConfig<any>[] {
  return Array.from(this.configs.values());
}

/**
 * Unregister a picker configuration
 *
 * @param id - Picker configuration ID
 * @returns True if removed, false if not found
 */
unregister(id: string): boolean {
  return this.configs.delete(id);
}

/**
 * Clear all registered picker configurations
 */
clear(): void {
  this.configs.clear();
}

/**
 * Get count of registered pickers
```

```
   *
   * @returns Number of registered pickers
   */
  getCount(): number {
    return this.configs.size;
  }
}
```

Chapter 312

Notification Service

```
// src/app/framework/models/error-notification.interface.ts
// VERSION 1 (Section 312) - Error notification types

/**
 * Error category enumeration
 *
 * Categorizes errors by their source and nature for appropriate handling
 * and user messaging.
 */
export enum ErrorCategory {
  /**
   * Network-related errors (connection issues, timeouts, etc.)
   */
  NETWORK = 'NETWORK',

  /**
   * Validation errors (invalid input, business rule violations)
   */
  VALIDATION = 'VALIDATION',

  /**
   * Authorization/authentication errors (401, 403)
   */
  AUTHORIZATION = 'AUTHORIZATION',

  /**
   * Server-side errors (5xx status codes)
   */
  SERVER = 'SERVER',

  /**
   * Client-side errors (JavaScript errors, component errors)
   */
  CLIENT = 'CLIENT',

  /**
   * Application-level errors (business logic, state errors)
   */
  APPLICATION = 'APPLICATION',

  /**
   * Unknown or uncategorized errors
   */
  UNKNOWN = 'UNKNOWN'
}

/**
 * Error severity levels
 *
 * Maps to PrimeNG Toast severity levels for visual feedback
 */
```

```
export type ErrorSeverity = 'success' | 'info' | 'warn' | 'error';

/**
 * Error notification data structure
 *
 * Contains all information needed to display an error to the user
 */
export interface ErrorNotification {
  /**
   * Error category for categorization and routing
   */
  category: ErrorCategory;

  /**
   * Display severity (maps to PrimeNG Toast severity)
   */
  severity: ErrorSeverity;

  /**
   * Brief error summary (shown as toast title)
   */
  summary: string;

  /**
   * Detailed error message (shown as toast body)
   */
  detail: string;

  /**
   * Optional error code for debugging
   */
  code?: string;

  /**
   * Optional timestamp of when error occurred
   */
  timestamp?: string;

  /**
   * Optional URL where error occurred
   */
  url?: string;

  /**
   * Optional HTTP status code (for network errors)
   */
  status?: number;

  /**
   * Original error object (for logging/debugging)
   */
  originalError?: any;
}

/**
 * Error display options
 *
 * Configuration for how errors should be displayed to users
 */
```

```
export interface ErrorDisplayOptions {
  /**
   * Auto-hide duration in milliseconds
   * Set to 0 or null to prevent auto-hide
   * Default: 5000 (5 seconds)
   */
  life?: number;

  /**
   * Whether to show close button
   * Default: true
   */
  closable?: boolean;

  /**
   * Whether to show in sticky mode (no auto-hide)
   * Default: false
   */
  sticky?: boolean;

  /**
   * Custom CSS class for the toast
   */
  styleClass?: string;

  /**
   * Toast position key
   * Default: 'app-toast'
   */
  key?: string;
}

/**
 * Default error display options
 */
export const DEFAULT_ERROR_DISPLAY_OPTIONS: ErrorDisplayOptions = {
  life: 5000,
  closable: true,
  sticky: false,
  key: 'app-toast'
};

/**
 * Error severity mapping configuration
 *
 * Maps error categories to default severity levels
 */
export const ERROR_CATEGORY_SEVERITY_MAP: Record<ErrorCategory, ErrorSeverity> = {
  [ErrorCategory.NETWORK]: 'error',
  [ErrorCategory.VALIDATION]: 'warn',
  [ErrorCategory.AUTHORIZATION]: 'warn',
  [ErrorCategory.SERVER]: 'error',
  [ErrorCategory.CLIENT]: 'error',
  [ErrorCategory.APPLICATION]: 'error',
  [ErrorCategory.UNKNOWN]: 'error'
};

/**
 * HTTP status code to error category mapping
```

```
 *
 * @param status - HTTP status code
 * @returns Appropriate error category
 */
export function getErrorCategoryFromStatus(status: number): ErrorCategory {
  if (status === 0) {
    return ErrorCategory.NETWORK;
  }

  if (status === 401 || status === 403) {
    return ErrorCategory.AUTHORIZATION;
  }

  if (status === 400 || status === 422) {
    return ErrorCategory.VALIDATION;
  }

  if (status >= 500 && status < 600) {
    return ErrorCategory.SERVER;
  }

  if (status >= 400 && status < 500) {
    return ErrorCategory.CLIENT;
  }

  return ErrorCategory.UNKNOWN;
}

/**
 * Error code to error category mapping
 *
 * @param code - Error code string
 * @returns Appropriate error category
 */
export function getErrorCategoryFromCode(code: string): ErrorCategory {
  const upperCode = code.toUpperCase();

  if (upperCode.includes('NETWORK') || upperCode.includes('TIMEOUT')) {
    return ErrorCategory.NETWORK;
  }

  if (upperCode.includes('VALIDATION') || upperCode.includes('INVALID')) {
    return ErrorCategory.VALIDATION;
  }

  if (
    upperCode.includes('UNAUTHORIZED') ||
    upperCode.includes('FORBIDDEN') ||
    upperCode.includes('AUTH')
  ) {
    return ErrorCategory.AUTHORIZATION;
  }

  if (upperCode.includes('SERVER') || upperCode.includes('INTERNAL')) {
    return ErrorCategory.SERVER;
  }

  if (upperCode.includes('CLIENT')) {
    return ErrorCategory.CLIENT;
```

```
  }

  return ErrorCategory.UNKNOWN;
}

/**
 * Get user-friendly summary for error category
 *
 * Note: This is an internal utility function (not exported)
 *
 * @param category - Error category
 * @returns Summary text
 */
function getSummaryForCategory(category: ErrorCategory): string {
  switch (category) {
    case ErrorCategory.NETWORK:
      return 'Connection Error';
    case ErrorCategory.VALIDATION:
      return 'Validation Error';
    case ErrorCategory.AUTHORIZATION:
      return 'Access Denied';
    case ErrorCategory.SERVER:
      return 'Server Error';
    case ErrorCategory.CLIENT:
      return 'Application Error';
    case ErrorCategory.APPLICATION:
      return 'Operation Failed';
    case ErrorCategory.UNKNOWN:
    default:
      return 'Error';
  }
}

/**
 * Create error notification from HTTP error
 *
 * @param error - HTTP error object (from interceptor)
 * @returns ErrorNotification object
 */
export function createErrorNotificationFromHttpError(error: any): ErrorNotification {
  const status = error.status || 0;
  const code = error.code || 'UNKNOWN_ERROR';
  const category = getErrorCategoryFromStatus(status);
  const severity = ERROR_CATEGORY_SEVERITY_MAP[category];

  return {
    category,
    severity,
    summary: getSummaryForCategory(category),
    detail: error.message || 'An unexpected error occurred',
    code,
    timestamp: error.timestamp,
    url: error.url,
    status,
    originalError: error
  };
}

/**
```

```
 * Create error notification from generic error
 *
 * @param error - Generic error object
 * @returns ErrorNotification object
 */
export function createErrorNotificationFromError(error: Error): ErrorNotification {
  const code = (error as any).code;
  const category = code
    ? getErrorCategoryFromCode(code)
    : ErrorCategory.CLIENT;
  const severity = ERROR_CATEGORY_SEVERITY_MAP[category];

  return {
    category,
    severity,
    summary: getSummaryForCategory(category),
    detail: error.message || 'An unexpected error occurred',
    code,
    timestamp: new Date().toISOString(),
    originalError: error
  };
}
```

Chapter 313

Window Communication Service

313: HTTP Error Interceptor

Status: Complete

Depends On: 302-api-service, 312-error-notification-service

Blocks: 314-global-error-handler

Learning Objectives

After completing this section, you will:

- Understand how Angular HTTP interceptors work
- Know how to implement automatic retry for transient errors
- Recognize the importance of consistent error formatting
- Be able to create user-friendly error messages from HTTP status codes

---

Objective

Create the `HttpErrorInterceptor` that provides global error handling for all HTTP requests. This interceptor automatically retries transient errors, formats error responses consistently, and logs errors for debugging.

Why

Every HTTP request can fail. Without centralized handling, each component must:

- Catch errors in its subscription

- Determine the error type
- Format user-friendly messages
- Log for debugging
- Decide whether to retry

This leads to duplicated logic and inconsistent error handling.

The Interceptor Pattern

Angular's HTTP interceptor pattern allows you to intercept and modify HTTP requests and responses globally. For error handling:

```
Request → Interceptor → HTTP → Server Response
                              ↓
                         Error? → Retry? → Format → Component
```

Automatic Retry for Transient Errors

Some errors are **transient** — they succeed on retry:

Status Code	Meaning	Retryable?
429	Too Many Requests	Yes — wait and retry
500	Internal Server Error	Maybe — server might recover
502	Bad Gateway	Yes — proxy might recover
503	Service Unavailable	Yes — server restarting
504	Gateway Timeout	Yes — might just be slow
400	Bad Request	No — client error won't change
401	Unauthorized	No — need authentication
404	Not Found	No — resource doesn't exist

The interceptor automatically retries for status codes 429, 500, 502, 503, 504 up to 2 times before giving up.

Consistent Error Formatting

The interceptor transforms Angular's `HttpErrorResponse` into a consistent format:

```
// Input: HttpErrorResponse
{
  status: 500,
  statusText: 'Internal Server Error',
```

```
  error: { message: 'Database connection failed' },
  url: '/api/vehicles'
}

// Output: Formatted error object
{
  code: 'INTERNAL_SERVER_ERROR',
  message: 'Internal server error. Please try again later.',
  status: 500,
  statusText: 'Internal Server Error',
  url: '/api/vehicles',
  timestamp: '2024-02-09T12:00:00.000Z'
}
```

This consistent format makes error handling predictable for:

- The ErrorNotificationService
- Component error handlers
- Error logging systems

Error Code Mapping

Each HTTP status maps to a semantic error code:

Status	Error Code
400	`BAD_REQUEST`
401	`UNAUTHORIZED`
403	`FORBIDDEN`
404	`NOT_FOUND`
409	`CONFLICT`
422	`VALIDATION_ERROR`
429	`RATE_LIMIT_EXCEEDED`
500	`INTERNAL_SERVER_ERROR`
502	`BAD_GATEWAY`
503	`SERVICE_UNAVAILABLE`
504	`GATEWAY_TIMEOUT`

What

Step 313.1: Create the HTTP Error Interceptor

Create the file `src/app/framework/services/http-error.interceptor.ts`:

```
// src/app/framework/services/http-error.interceptor.ts
// VERSION 1 (Section 313) - HTTP error interceptor

import { Injectable } from '@angular/core';
import {
  HttpErrorResponse,
  HttpEvent,
  HttpHandler,
  HttpInterceptor,
  HttpRequest
} from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';

/**
 * Configuration for error interceptor retry behavior
 */
export interface RetryConfig {
  /** Maximum number of retry attempts */
  maxRetries: number;
  /** HTTP status codes that should trigger retry */
  retryableStatusCodes: number[];
}

/**
 * Default retry configuration
 *
 * Retries transient server errors (5xx) and rate limiting (429).
 * Does not retry client errors (4xx) as they won't succeed without changes.
 */
const DEFAULT_RETRY_CONFIG: RetryConfig = {
  maxRetries: 2,
  retryableStatusCodes: [429, 500, 502, 503, 504]
};

/**
 * Map HTTP status code to semantic error code
 *
 * @param status - HTTP status code
 * @returns Semantic error code string
 */
function getErrorCode(status: number): string {
  switch (status) {
    case 400:
      return 'BAD_REQUEST';
    case 401:
      return 'UNAUTHORIZED';
    case 403:
      return 'FORBIDDEN';
    case 404:
```

```
      return 'NOT_FOUND';
    case 409:
      return 'CONFLICT';
    case 422:
      return 'VALIDATION_ERROR';
    case 429:
      return 'RATE_LIMIT_EXCEEDED';
    case 500:
      return 'INTERNAL_SERVER_ERROR';
    case 502:
      return 'BAD_GATEWAY';
    case 503:
      return 'SERVICE_UNAVAILABLE';
    case 504:
      return 'GATEWAY_TIMEOUT';
    default:
      return status >= 500 ? 'SERVER_ERROR' : 'CLIENT_ERROR';
  }
}

/**
 * Get user-friendly error message from HTTP error
 *
 * Attempts to extract message from error response body,
 * falls back to standard messages based on status code.
 *
 * @param error - HTTP error response
 * @returns User-friendly error message
 */
function getErrorMessage(error: HttpErrorResponse): string {
  // Try to get message from nested error object
  if (error.error?.error?.message) {
    return error.error.error.message;
  }

  // Try to get message from error object
  if (error.error?.message) {
    return error.error.message;
  }

  // Fall back to status-based messages
  switch (error.status) {
    case 0:
      return 'Unable to connect to server. Please check your network connection.';
    case 400:
      return 'Invalid request. Please check your input.';
    case 401:
      return 'Authentication required. Please log in.';
    case 403:
      return 'Access denied. You do not have permission to access this resource.';
    case 404:
      return 'Resource not found.';
    case 409:
      return 'Conflict. The resource already exists or is in an invalid state.';
    case 422:
      return 'Validation failed. Please check your input.';
    case 429:
      return 'Too many requests. Please wait and try again.';
    case 500:
```

```
      return 'Internal server error. Please try again later.';
    case 502:
      return 'Bad gateway. The server is temporarily unavailable.';
    case 503:
      return 'Service unavailable. Please try again later.';
    case 504:
      return 'Gateway timeout. The server took too long to respond.';
    default:
      return error.message || `HTTP error: ${error.status} ${error.statusText}`;
  }
}

/**
 * Log error details for debugging
 *
 * @param error - HTTP error response
 * @param request - Original HTTP request
 * @param errorCode - Formatted error code
 * @param errorMessage - Formatted error message
 */
function logError(
  error: HttpErrorResponse,
  request: HttpRequest<unknown>,
  errorCode: string,
  errorMessage: string
): void {
  const logDetails = {
    code: errorCode,
    message: errorMessage,
    status: error.status,
    statusText: error.statusText,
    url: request.url,
    method: request.method,
    timestamp: new Date().toISOString()
  };

  console.error('HTTP Error:', logDetails);

  // Log response body for additional context
  if (error.error) {
    console.error('Error details:', error.error);
  }
}

/**
 * Handle HTTP error and format response
 *
 * @param error - HTTP error response
 * @param request - Original HTTP request
 * @returns Observable that throws formatted error
 */
function handleError(error: HttpErrorResponse, request: HttpRequest<unknown>) {
  let errorMessage: string;
  let errorCode: string;

  // Check if error is client-side (ErrorEvent) or server-side
  if (error.error instanceof ErrorEvent) {
    // Client-side error (network issue, etc.)
    errorCode = 'CLIENT_ERROR';
```

```
    errorMessage = `Network error: ${error.error.message}`;
  } else {
    // Server-side error
    errorCode = getErrorCode(error.status);
    errorMessage = getErrorMessage(error);
  }

  // Log for debugging
  logError(error, request, errorCode, errorMessage);

  // Return formatted error object
  return throwError(() => ({
    code: errorCode,
    message: errorMessage,
    status: error.status,
    statusText: error.statusText,
    url: request.url,
    timestamp: new Date().toISOString()
  }));
}

/**
 * HTTP error interceptor
 *
 * Handles global error processing for all HTTP requests:
 * - Automatic retry for transient errors (5xx, 429)
 * - Consistent error formatting
 * - Error logging
 *
 * **Retry Behavior:**
 *
 * The interceptor retries failed requests up to 2 times for specific
 * status codes (429, 500, 502, 503, 504). This handles transient
 * failures that often succeed on retry.
 *
 * **Error Format:**
 *
 * All errors are transformed to a consistent format:
 * ```typescript
 * {
 *   code: string;      // Semantic error code (e.g., 'UNAUTHORIZED')
 *   message: string;   // User-friendly message
 *   status: number;    // HTTP status code
 *   statusText: string;
 *   url: string;
 *   timestamp: string;
 * }
 * ```
 *
 * @example
 * ```typescript
 * // In app.module.ts
 * providers: [
 *   {
 *     provide: HTTP_INTERCEPTORS,
 *     useClass: HttpErrorInterceptor,
 *     multi: true
 *   }
 * ]
```

```
 * ```
 */
@Injectable()
export class HttpErrorInterceptor implements HttpInterceptor {
  /**
   * Intercept HTTP requests and handle errors
   *
   * @param request - Outgoing HTTP request
   * @param next - HTTP handler for the next interceptor
   * @returns Observable of HTTP events
   */
  intercept(
    request: HttpRequest<unknown>,
    next: HttpHandler
  ): Observable<HttpEvent<unknown>> {
    const retryConfig = DEFAULT_RETRY_CONFIG;

    return next.handle(request).pipe(
      // Retry transient errors
      retry(retryConfig.maxRetries),
      // Catch and format errors
      catchError((error: HttpErrorResponse) => {
        return handleError(error, request);
      })
    );
  }
}
```

Step 313.2: Update the Services Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 12 (Section 313) - Added HttpErrorInterceptor

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
export * from './domain-config-registry.service';
export * from './domain-config-validator.service';
export * from './popout-context.service';
export * from './popout-manager.service';
export * from './user-preferences.service';
export * from './filter-options.service';
export * from './picker-config-registry.service';
export * from './resource-management.service';
export * from './error-notification.service';
export * from './http-error.interceptor';
```

Step 313.3: Register the Interceptor

Update `src/app/app.module.ts` to register the interceptor:

```
// src/app/app.module.ts (partial - add to existing)
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { HttpErrorInterceptor } from './framework/services';

@NgModule({
  // ...
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: HttpErrorInterceptor,
      multi: true
    }
  ],
  // ...
})
export class AppModule { }
```

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/http-error.interceptor.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/framework/services/http-error.interceptor.ts
```

3. Build the Application

```
$ ng build
```

4. Test Error Handling

Create a test that triggers an HTTP error:

```
// In any component
constructor(private api: ApiService) {
  // Request a non-existent endpoint
  this.api.get('/api/non-existent-endpoint').subscribe({
    next: (data) => console.log('Data:', data),
    error: (error) => {
```

```
      console.log('Caught error:', error);
      // Error should be formatted as:
      // { code: 'NOT_FOUND', message: 'Resource not found.', status: 404, ... }
    }
  });
}
```

5. Verify Retry Behavior

To test retry, you can temporarily modify your API to return 503:

```
// In browser DevTools Network tab:
// 1. Enable "Offline" mode briefly
// 2. Watch for retry attempts in console logs
// 3. See that after 2 retries, error is thrown
```

Common Problems

Symptom	Cause	Solution
Interceptor not running	Not registered in providers	Add HTTP_INTERCEPTORS provider to app.module.ts
`multi: true` missing	Single interceptor overwrites others	Always use `multi: true` with HTTP_INTERCEPTORS
Infinite retry loop	All errors trigger retry	Only retry specific status codes (429, 5xx)
Error format wrong	Custom error handler	Ensure components expect the formatted error object
Retry too fast	No exponential backoff	Consider adding delay with `timer()` in retry

---

Key Takeaways

1. **Interceptors provide global error handling** — Every HTTP request flows through the interceptor
2. **Transient errors deserve retries** — 5xx and 429 errors often succeed on retry
3. **Consistent formatting simplifies handling** — Components receive the same error shape regardless of source

Acceptance Criteria

- [ ] `src/app/framework/services/http-error.interceptor.ts` exists
- [ ] Interceptor implements `HttpInterceptor` interface
- [ ] `intercept()` method handles requests and errors
- [ ] Retry logic attempts 2 retries for 429, 500, 502, 503, 504
- [ ] Error formatting produces consistent object shape
- [ ] Error codes map correctly (400 → BAD_REQUEST, etc.)
- [ ] User-friendly messages for each status code
- [ ] Client-side errors (ErrorEvent) handled separately
- [ ] Error logging includes request URL and method
- [ ] Interceptor registered with `HTTP_INTERCEPTORS` provider
- [ ] TypeScript compilation succeeds
- [ ] JSDoc comments document all functions

Next Step

Proceed to `314-global-error-handler.md` to create the global error handler for unhandled exceptions.

Chapter 314

Popout Manager Service

314: Global Error Handler

Status: Complete

Depends On: 312-error-notification-service, 313-http-error-interceptor

Blocks: Phase 8 (Framework Components)

Learning Objectives

After completing this section, you will:

- Understand how Angular's ErrorHandler interface works
- Know how to catch unhandled exceptions globally
- Recognize different error types (HTTP, chunk load, promise rejection)
- Be able to provide user-friendly messages for JavaScript errors

---

Objective

Create the `GlobalErrorHandler` that catches all unhandled exceptions in the Angular application and displays user-friendly error messages using the ErrorNotificationService.

Why

Errors escape individual component handlers for many reasons:

- Developers forget to add `.catch()` to promises

- Observable subscriptions don't include error callbacks
- Third-party libraries throw unexpectedly
- Template expressions throw during rendering

Without global error handling, these errors:

- Crash silently with cryptic console messages
- Leave users staring at broken UIs with no feedback
- Make debugging difficult without context

Angular's ErrorHandler

Angular provides an `ErrorHandler` class that catches all unhandled errors. By default, it just logs to console. We replace it with our `GlobalErrorHandler` that:

1. **Categorizes errors** — HTTP vs chunk load vs promise rejection vs generic
2. **Shows user feedback** — Toast notifications explain what went wrong
3. **Logs context** — URL, route, stack trace for debugging

Error Types

The global handler categorizes errors by type:

Type	Detection	Example
HTTP Error	`instanceof HttpErrorResponse`	API returned 500
Chunk Load Error	Message contains "loading chunk"	Lazy module failed to load
Promise Rejection	Error has `.rejection` property	Unhandled async error
Generic Error	Default	TypeError, ReferenceError

Each type gets appropriate handling:

Type	Display	Duration
HTTP Error	Error toast	5 seconds
Chunk Load Error	Sticky toast (requires action)	Until dismissed
Promise Rejection	Error toast	7 seconds
Generic Error	Sticky toast	Until dismissed

The Lazy-Loading Trick

The GlobalErrorHandler uses Angular's `Injector` to get the `ErrorNotificationService` instead of constructor injection:

```
// Wrong - can cause circular dependency
constructor(private errorNotification: ErrorNotificationService) { }

// Right - lazy loading via injector
constructor(private injector: Injector) { }

private get errorNotificationService(): ErrorNotificationService {
  return this.injector.get(ErrorNotificationService);
}
```

This avoids circular dependency issues because:

1. ErrorHandler is created very early in Angular's bootstrap
2. ErrorNotificationService may depend on services not yet created
3. Lazy injection defers resolution until first use

Chunk Load Errors

A special case is **chunk load errors** — when lazy-loaded modules fail to load:

```
Error: Loading chunk 5 failed.
```

These typically happen when:

- Network connection drops during navigation
- Server deployment changed chunk hashes
- Browser cache serves stale chunk

The handler shows a sticky message prompting the user to refresh.

What

Step 314.1: Create the Global Error Handler

Create the file `src/app/framework/services/global-error.handler.ts`:

```
// src/app/framework/services/global-error.handler.ts
// VERSION 1 (Section 314) - Global error handler

import { ErrorHandler, Injectable, Injector } from '@angular/core';
import { HttpErrorResponse } from '@angular/common/http';
import { ErrorNotificationService } from './error-notification.service';
```

```
import { ErrorCategory } from '../models/error-notification.interface';

/**
 * Global error handler
 *
 * Catches all unhandled errors in the Angular application and displays
 * user-friendly error messages using the ErrorNotificationService.
 *
 * This handler:
 * - Intercepts all unhandled exceptions
 * - Categorizes errors by type (HTTP, Promise rejection, component error, etc.)
 * - Displays appropriate user-facing messages
 * - Logs detailed error information for debugging
 *
 * **Why Lazy Injection?**
 *
 * Uses Injector.get() instead of constructor injection to avoid circular
 * dependency issues. ErrorHandler is created very early in Angular's
 * bootstrap process, before some services are fully initialized.
 *
 * @example
 * ```typescript
 * // In app.module.ts
 * providers: [
 *   {
 *     provide: ErrorHandler,
 *     useClass: GlobalErrorHandler
 *   }
 * ]
 * ```
 */
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  /**
   * ErrorNotificationService instance
   *
   * Lazy-loaded via getter to avoid circular dependency issues.
   * Service is requested from injector on first access.
   */
  private get errorNotificationService(): ErrorNotificationService {
    return this.injector.get(ErrorNotificationService);
  }

  /**
   * Constructor with injector for lazy service loading
   *
   * @param injector - Angular injector for lazy-loading services
   */
  constructor(private injector: Injector) {}

  /**
   * Handle error
   *
   * Main error handling method called by Angular for all unhandled errors
   *
   * @param error - The error object
   */
  handleError(error: any): void {
    // Extract actual error from Angular's wrapper
```

```
  const actualError = this.unwrapError(error);

  // Log to console for debugging
  this.logErrorToConsole(actualError);

  // Handle different error types
  if (this.isHttpError(actualError)) {
    this.handleHttpError(actualError);
  } else if (this.isChunkLoadError(actualError)) {
    this.handleChunkLoadError(actualError);
  } else if (this.isPromiseRejection(error)) {
    this.handlePromiseRejection(actualError);
  } else {
    this.handleGenericError(actualError);
  }
}

/**
 * Unwrap error from Angular's ErrorEvent wrapper
 *
 * Angular sometimes wraps errors in objects with `.rejection` or `.error`
 * properties. This extracts the actual error object.
 *
 * @param error - Potentially wrapped error
 * @returns Actual error object
 */
private unwrapError(error: any): any {
  // Angular wraps promise rejections in rejection field
  if (error && error.rejection) {
    return error.rejection;
  }

  // Angular wraps errors in error field
  if (error && error.error) {
    return error.error;
  }

  return error;
}

/**
 * Check if error is HTTP error
 *
 * @param error - Error object
 * @returns True if HTTP error
 */
private isHttpError(error: any): boolean {
  return error instanceof HttpErrorResponse;
}

/**
 * Check if error is chunk load error (lazy-loaded module)
 *
 * Chunk load errors occur when lazy-loaded JavaScript files fail to load.
 * This typically happens due to network issues or server deployments
 * that change chunk hashes.
 *
 * @param error - Error object
 * @returns True if chunk load error
```

```
 */
private isChunkLoadError(error: any): boolean {
  if (!(error instanceof Error)) {
    return false;
  }

  const message = error.message.toLowerCase();
  return (
    message.includes('loading chunk') ||
    message.includes('failed to fetch') ||
    message.includes('dynamically imported module')
  );
}

/**
 * Check if error is promise rejection
 *
 * @param error - Error object
 * @returns True if promise rejection
 */
private isPromiseRejection(error: any): boolean {
  return error && error.promise && error.rejection;
}

/**
 * Handle HTTP errors
 *
 * HTTP errors are already handled by the HTTP interceptor, but may
 * bubble up if not caught in component subscriptions.
 *
 * @param error - HTTP error response
 */
private handleHttpError(error: HttpErrorResponse): void {
  // Check if error was already formatted by interceptor
  if (error.error && error.error.code) {
    // Already formatted by interceptor
    this.errorNotificationService.showHttpError(error.error);
  } else {
    // Not formatted - format now
    this.errorNotificationService.showHttpError({
      code: 'HTTP_ERROR',
      message: error.message || 'An HTTP error occurred',
      status: error.status,
      statusText: error.statusText,
      url: error.url || undefined,
      timestamp: new Date().toISOString()
    });
  }
}

/**
 * Handle chunk load errors (lazy-loaded modules)
 *
 * These occur when lazy-loaded JavaScript chunks fail to load,
 * typically due to network issues or deployments.
 *
 * Shows a sticky toast prompting the user to refresh.
 *
 * @param error - Chunk load error
```

```
 */
private handleChunkLoadError(error: Error): void {
  this.errorNotificationService.show(
    {
      category: ErrorCategory.NETWORK,
      severity: 'error',
      summary: 'Loading Error',
      detail:
        'Failed to load application module. Please refresh the page. ' +
        'If the problem persists, clear your browser cache.',
      code: 'CHUNK_LOAD_ERROR',
      timestamp: new Date().toISOString(),
      originalError: error
    },
    {
      sticky: true, // Don't auto-hide - requires user action
      life: 0
    }
  );
}

/**
 * Handle promise rejections
 *
 * Unhandled promise rejections that weren't caught with .catch()
 *
 * @param error - Rejected promise value
 */
private handlePromiseRejection(error: any): void {
  // If it's an HTTP error, handle as HTTP
  if (this.isHttpError(error)) {
    this.handleHttpError(error);
    return;
  }

  // Generic promise rejection
  this.errorNotificationService.show(
    {
      category: ErrorCategory.APPLICATION,
      severity: 'error',
      summary: 'Operation Failed',
      detail:
        error?.message ||
        'An asynchronous operation failed. Please try again.',
      code: 'PROMISE_REJECTION',
      timestamp: new Date().toISOString(),
      originalError: error
    },
    {
      life: 7000 // Longer display for async errors
    }
  );
}

/**
 * Handle generic errors
 *
 * All other unhandled errors (component errors, TypeError, etc.)
 *
```

```
 * Shows a sticky toast for serious programming errors.
 *
 * @param error - Error object
 */
private handleGenericError(error: any): void {
  const errorMessage =
    error?.message || error?.toString() || 'An unexpected error occurred';

  // Check for known error patterns
  const isTypeError = error instanceof TypeError;
  const isReferenceError = error instanceof ReferenceError;

  let detail = errorMessage;
  if (isTypeError || isReferenceError) {
    detail +=
      ' This is likely a programming error. Please report this issue.';
  }

  this.errorNotificationService.show(
    {
      category: ErrorCategory.CLIENT,
      severity: 'error',
      summary: 'Application Error',
      detail,
      code: 'UNHANDLED_ERROR',
      timestamp: new Date().toISOString(),
      originalError: error
    },
    {
      sticky: true, // Keep visible for debugging
      life: 0
    }
  );
}

/**
 * Log error to console for debugging
 *
 * Groups error information for easier debugging in DevTools.
 *
 * @param error - Error object
 */
private logErrorToConsole(error: any): void {
  const timestamp = new Date().toISOString();

  console.group(`[Global Error Handler] ${timestamp}`);
  console.error('Error caught by global handler:', error);

  // Log stack trace if available
  if (error instanceof Error && error.stack) {
    console.error('Stack trace:', error.stack);
  }

  // Log additional context
  console.error('Error type:', error?.constructor?.name || typeof error);

  // Log location information
  console.error('Location:', {
    url: window.location.href,
```

```
      route: window.location.pathname
    });

    console.groupEnd();
  }
}
```

Step 314.2: Update the Services Barrel File

Update `src/app/framework/services/index.ts`:

```
// src/app/framework/services/index.ts
// VERSION 13 (Section 314) - Added GlobalErrorHandler

export * from './url-state.service';
export * from './api.service';
export * from './request-coordinator.service';
export * from './domain-config-registry.service';
export * from './domain-config-validator.service';
export * from './popout-context.service';
export * from './popout-manager.service';
export * from './user-preferences.service';
export * from './filter-options.service';
export * from './picker-config-registry.service';
export * from './resource-management.service';
export * from './error-notification.service';
export * from './http-error.interceptor';
export * from './global-error.handler';
```

Step 314.3: Register the Global Error Handler

Update `src/app/app.module.ts` to register the handler:

```
// src/app/app.module.ts (partial - add to existing)
import { ErrorHandler } from '@angular/core';
import { GlobalErrorHandler } from './framework/services';

@NgModule({
  // ...
  providers: [
    {
      provide: ErrorHandler,
      useClass: GlobalErrorHandler
    }
  ],
  // ...
})
export class AppModule { }
```

Verification

1. Check File Exists

```
$ ls -la src/app/framework/services/global-error.handler.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/framework/services/global-error.handler.ts
```

3. Build the Application

```
$ ng build
```

4. Test Unhandled Error

Add temporary code to throw an error:

```
// In any component ngOnInit
ngOnInit(): void {
  // Uncomment to test - should show sticky toast
  // throw new Error('Test unhandled error');
}
```

5. Test Promise Rejection

```
// In any component
ngOnInit(): void {
  // Uncomment to test - should show error toast
  // Promise.reject(new Error('Test promise rejection'));
}
```

6. Test Chunk Load Error

Simulate by modifying the error check temporarily, or:

1. Build the application
2. Delete a chunk file from dist
3. Navigate to the route that loads that chunk

7. Verify Console Logging

When an error is caught, check the browser console for grouped log output:

```
▼ [Global Error Handler] 2024-02-09T12:00:00.000Z
  Error caught by global handler: Error: Test error
  Stack trace: Error: Test error
    at Component.ngOnInit (component.ts:15)
    ...
  Error type: Error
  Location: {url: "http://localhost:4200/discover", route: "/discover"}
```

Common Problems

Symptom	Cause	Solution
Circular dependency error	ErrorHandler created too early	Use `Injector.get()` instead of constructor injection
No toast appears	ErrorNotificationService not provided	Ensure MessageService is in app.module.ts providers
Error handled twice	HTTP interceptor + global handler	Check if error already formatted (has `.code` property)
Console shows "NullInjectorError"	Service not available	Ensure all dependencies are provided
Chunk load error not detected	Message pattern changed	Update `isChunkLoadError()` detection patterns

---

Key Takeaways

1. **Global error handling catches everything** — Even errors developers forgot to handle
2. **Lazy injection avoids circular dependencies** — Use `Injector.get()` for early-loaded services
3. **Error categorization enables appropriate responses** — Chunk errors need refresh, promise errors need retry

---

Acceptance Criteria

- [ ] `src/app/framework/services/global-error.handler.ts` exists
- [ ] Handler extends Angular's `ErrorHandler` interface
- [ ] `handleError()` method catches all unhandled errors
- [ ] HTTP errors detected and formatted
- [ ] Chunk load errors show sticky refresh prompt
- [ ] Promise rejections handled with 7-second display
- [ ] Generic errors show sticky toast
- [ ] Console logging includes stack trace and location
- [ ] Lazy injection via `Injector.get()` to avoid circular deps
- [ ] Handler registered with `provide: ErrorHandler` in app.module.ts
- [ ] TypeScript compilation succeeds
- [ ] JSDoc comments document all methods

Phase 3C Complete

Congratulations! You have completed Phase 3C: Error Handling.

What you built:

- ErrorNotification interface — Error categorization and display types
- ErrorNotificationService — Centralized toast notifications with deduplication
- HttpErrorInterceptor — Automatic retry and consistent error formatting
- GlobalErrorHandler — Catch-all for unhandled exceptions

The Aha Moment:

"Errors are inevitable. User-friendly error messages are not."

Next Step

Proceed to `315-popout-token.md` to create the injection token for pop-out window detection.

Chapter 315

Analytics Service

```
// src/app/framework/tokens/popout.token.ts
// VERSION 1 (Section 315) - Pop-out injection token

import { InjectionToken } from '@angular/core';

/**
 * Injection token to signal that the current component context is a pop-out window.
 * Used by services (like ResourceManagementService) to adjust behavior when provided
 * by a pop-out component (e.g., disabling auto-fetch).
 *
 * **Usage:**
 *
 * ```typescript
 * constructor(@Inject(IS_POPOUT_TOKEN) public isPopOut: boolean) {}
 * ```
 *
 * **In Templates:**
 *
 * ```html
 * <nav *ngIf="!isPopOut">Main navigation</nav>
 * <button *ngIf="isPopOut" (click)="close()">Close</button>
 * ```
 *
 * **Note:** This token is provided at the component level, not via a factory.
 * Components that render in pop-out windows provide this token with value `true`.
 *
 * @example
 * ```typescript
 * // In pop-out component
 * @Component({
 *   selector: 'app-panel-popout',
 *   providers: [
 *     { provide: IS_POPOUT_TOKEN, useValue: true }
 *   ]
 * })
 * export class PanelPopoutComponent { }
 *
 * // In main window component
 * @Component({
 *   selector: 'app-discover',
 *   providers: [
 *     { provide: IS_POPOUT_TOKEN, useValue: false }
 *   ]
 * })
 * export class DiscoverComponent { }
 * ```
 */
export const IS_POPOUT_TOKEN = new InjectionToken<boolean>('IS_POPOUT_TOKEN');
```

Step 315.4: Update the Barrel File

Update `src/app/framework/tokens/index.ts`:

```
// src/app/framework/tokens/index.ts
// VERSION 1 (Section 315) - Framework injection tokens

export * from './popout.token';
```

Step 315.5: Provide the Token at Component Level

The `IS_POPOUT_TOKEN` is provided at the component level rather than via a factory function. This approach allows explicit control over the token value in different contexts:

```
// In pop-out window component
@Component({
  selector: 'app-panel-popout',
  providers: [
    { provide: IS_POPOUT_TOKEN, useValue: true }
  ]
})
export class PanelPopoutComponent {
  // Services injected here will receive isPopOut = true
}

// In main window component
@Component({
  selector: 'app-discover',
  providers: [
    { provide: IS_POPOUT_TOKEN, useValue: false }
  ]
})
export class DiscoverComponent {
  // Services injected here will receive isPopOut = false
}
```

This pattern is preferred over a factory function because:

1. It provides explicit control over the token value
2. It works reliably across different routing scenarios
3. It avoids relying on URL parsing at bootstrap time

Phase 4: Domain Models

Chapter 401: Automobile Filter Model

Chapter 402: Automobile Data Model

Chapter 403: Automobile Statistics Model

Chapter 401

Automobile Filter Model

Step 401.2: Create the Domain Models Directory

Create the directory structure for automobile domain models.

```
$ mkdir -p src/app/domain-config/automobile/models
$ touch src/app/domain-config/automobile/models/index.ts
```

Create the barrel file `src/app/domain-config/automobile/models/index.ts`:

```
// src/app/domain-config/automobile/models/index.ts
// VERSION 1 (Section 401) - Automobile domain models barrel

// Data models (Section 402)
// export * from './automobile.data';

// Filter and statistics models (Section 403)
// export * from './automobile.filters';
// export * from './automobile.statistics';
```

Step 401.3: Document the Model Pattern

Create a reference document showing the complete model pattern.

The complete model pattern looks like this:

```
/**
 * [Model Name]
 *
 * [Description of what this model represents]
 *
 * Domain: [Domain Name]
 *
 * @example
 * ```typescript
 * const instance: ModelClass = {
 *   field1: 'value1',
 *   field2: 123
 * };
 * ```
 */
```

```
export class ModelClass {
  /**
   * [Field description]
   *
   * @example 'example value'
   */
  field1!: string;

  /**
   * [Optional field description]
   *
   * @example 123
   */
  field2?: number;

  /**
   * Constructor with partial data
   *
   * @param partial - Partial ModelClass object
   */
  constructor(partial?: Partial<ModelClass>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  /**
   * Create ModelClass from API response
   *
   * @param data - Raw API response data
   * @returns ModelClass instance
   */
  static fromApiResponse(data: any): ModelClass {
    return new ModelClass({
      field1: data.field1 || data.field_1,
      field2: data.field2 !== undefined ? Number(data.field2) : undefined
    });
  }

  /**
   * Get display name
   *
   * @returns Formatted display string
   */
  getDisplayName(): string {
    return this.field1;
  }
}
```

Verification

1. Check Directory Exists

```
$ ls -la src/app/domain-config/automobile/models/
```

Expected output:

```
total 8
drwxr-xr-x 2 user user 4096 Feb  9 12:00 .
drwxr-xr-x 3 user user 4096 Feb  9 12:00 ..
-rw-r--r-- 1 user user  200 Feb  9 12:00 index.ts
```

2. TypeScript Check

```
$ npx tsc --noEmit src/app/domain-config/automobile/models/index.ts
```

Common Problems

Symptom	Cause	Solution
`Property has no initializer`	Missing `!` on required field	Add `!` or make optional with `?`
`Partial<T>` not accepting field	Wrong field name	Ensure field names match exactly
API fields not mapping	Missing transformation	Add case in `fromApiResponse()`
Methods not available	Using interface instead of class	Convert to class with methods

---

Key Takeaways

1. **Classes over interfaces for domain models** — Enable methods, factories, and instanceof checks
2. **Partial constructor pattern** — Flexible initialization with type safety
3. **Static factory for API data** — Single point for API field normalization
4. **Instance methods for computed values** — Encapsulate business logic in the model

---

Acceptance Criteria

- [ ] Domain models directory structure created
- [ ] `src/app/domain-config/automobile/models/index.ts` exists
- [ ] Model conventions documented and understood:
- [ ] Non-null assertion for required fields
- [ ] Optional marker for optional fields
- [ ] Partial constructor pattern
- [ ] Static `fromApiResponse()` factory
- [ ] Instance methods for computed values
- [ ] JSDoc documentation on all members

Next Step

Proceed to `402-domain-data-models.md` to create the vehicle and VIN instance data models.

Chapter 402

Automobile Data Model

What

Step 402.1: Create the Vehicle Result Model

Create the file `src/app/domain-config/automobile/models/automobile.data.ts`:

```
// src/app/domain-config/automobile/models/automobile.data.ts
// VERSION 1 (Section 402) - Automobile domain data models

/**
 * Vehicle result data
 *
 * Represents a unique vehicle configuration with aggregated VIN instance count.
 * Each record represents a distinct combination of manufacturer, model, year, and body class.
 *
 * Domain: Automobile Discovery
 *
 * @example
 * ```typescript
 * const vehicle: VehicleResult = {
 *   vehicle_id: 'TOY-CAM-2024-SED',
 *   manufacturer: 'Toyota',
 *   model: 'Camry',
 *   year: 2024,
 *   body_class: 'Sedan',
 *   instance_count: 156,
 *   first_seen: '2024-01-15T10:30:00Z',
 *   last_seen: '2024-11-20T14:22:00Z'
 * };
 * ```
 */
export class VehicleResult {
  /**
   * Unique vehicle identifier
   * Composite key: manufacturer-model-year-bodyclass
   *
   * @example 'TOY-CAM-2024-SED', 'HON-ACC-2023-SED'
   */
  vehicle_id!: string;

  /**
   * Vehicle manufacturer name
   *
   * @example 'Toyota', 'Honda', 'Ford', 'Chevrolet'
```

```
 */
manufacturer!: string;

/**
 * Vehicle model name
 *
 * @example 'Camry', 'Accord', 'F-150', 'Silverado'
 */
model!: string;

/**
 * Vehicle model year
 *
 * @example 2024, 2023, 2022
 */
year!: number;

/**
 * Vehicle body class/type
 *
 * @example 'Sedan', 'SUV', 'Truck', 'Coupe', 'Wagon', 'Van'
 */
body_class!: string;

/**
 * Number of VIN instances for this vehicle configuration
 * Represents how many individual vehicles (VINs) exist for this configuration
 *
 * @example 156 (means 156 unique VINs for this vehicle config)
 */
instance_count!: number;

/**
 * Date/time this vehicle configuration was first seen in the system
 * ISO 8601 format
 *
 * @example '2024-01-15T10:30:00Z'
 */
first_seen?: string;

/**
 * Date/time this vehicle configuration was last updated
 * ISO 8601 format
 *
 * @example '2024-11-20T14:22:00Z'
 */
last_seen?: string;

/**
 * Drive type
 * @example 'FWD', 'RWD', 'AWD', '4WD'
 */
drive_type?: string;

/**
 * Engine configuration
 * @example 'V6', 'I4', 'V8', 'Electric'
 */
engine?: string;
```

```
/**
 * Transmission type
 * @example 'Automatic', 'Manual', 'CVT'
 */
transmission?: string;

/**
 * Fuel type
 * @example 'Gasoline', 'Diesel', 'Electric', 'Hybrid'
 */
fuel_type?: string;

/**
 * Vehicle class/category
 * @example 'Passenger Car', 'Light Truck', 'Commercial Vehicle'
 */
vehicle_class?: string;

/**
 * Constructor with partial data
 *
 * @param partial - Partial VehicleResult object
 */
constructor(partial?: Partial<VehicleResult>) {
  if (partial) {
    Object.assign(this, partial);
  }
}

/**
 * Create VehicleResult from API response
 *
 * Handles both snake_case and camelCase field names from API.
 *
 * @param data - Raw API response data
 * @returns VehicleResult instance
 */
static fromApiResponse(data: any): VehicleResult {
  return new VehicleResult({
    vehicle_id: data.vehicle_id || data.vehicleId || data.id,
    manufacturer: data.manufacturer,
    model: data.model,
    year: Number(data.year),
    body_class: data.body_class || data.bodyClass,
    instance_count: Number(data.instance_count || data.instanceCount || 0),
    first_seen: data.first_seen || data.firstSeen,
    last_seen: data.last_seen || data.lastSeen,
    drive_type: data.drive_type || data.driveType,
    engine: data.engine,
    transmission: data.transmission,
    fuel_type: data.fuel_type || data.fuelType,
    vehicle_class: data.vehicle_class || data.vehicleClass
  });
}

/**
 * Get display name for vehicle
 *
 * @returns Formatted display string
```

```
   * @example 'Toyota Camry 2024'
   */
  getDisplayName(): string {
    return `${this.manufacturer} ${this.model} ${this.year}`;
  }

  /**
   * Get full description for vehicle
   *
   * @returns Detailed description string
   * @example 'Toyota Camry 2024 Sedan (156 instances)'
   */
  getFullDescription(): string {
    return `${this.manufacturer} ${this.model} ${this.year} ${this.body_class} (${this.instance_count} instances)
  }

  /**
   * Check if vehicle has VIN instances
   *
   * @returns True if instance_count > 0
   */
  hasInstances(): boolean {
    return this.instance_count > 0;
  }

  /**
   * Get age of vehicle in years
   *
   * @returns Age in years from current year
   */
  getAge(): number {
    const currentYear = new Date().getFullYear();
    return currentYear - this.year;
  }

  /**
   * Check if vehicle is current year
   *
   * @returns True if year matches current year
   */
  isCurrentYear(): boolean {
    return this.year === new Date().getFullYear();
  }
}

/**
 * VIN instance detail
 *
 * Represents a single VIN instance for a vehicle configuration.
 * Used in row expansion to show individual VINs.
 *
 * @example
 * ```typescript
 * const vin: VinInstance = {
 *   vin: '1HGCM82633A123456',
 *   vehicle_id: 'HON-ACC-2023-SED',
 *   registration_date: '2023-05-10',
 *   registration_state: 'CA',
 *   odometer_reading: 15234
```

```
 * };
 * ```
 */
export class VinInstance {
  /**
   * Vehicle Identification Number (17 characters)
   *
   * @example '1HGCM82633A123456'
   */
  vin!: string;

  /**
   * Associated vehicle configuration ID
   * Links to VehicleResult.vehicle_id
   *
   * @example 'HON-ACC-2023-SED'
   */
  vehicle_id!: string;

  /**
   * Registration date
   * @example '2023-05-10'
   */
  registration_date?: string;

  /**
   * Registration state/province
   * @example 'CA', 'TX', 'NY'
   */
  registration_state?: string;

  /**
   * Odometer reading (miles)
   * @example 15234
   */
  odometer_reading?: number;

  /**
   * Vehicle status
   * @example 'Active', 'Salvage', 'Totaled', 'Stolen'
   */
  status?: string;

  /**
   * Color
   * @example 'White', 'Black', 'Silver'
   */
  color?: string;

  /**
   * Current owner/registrant (anonymized)
   * @example 'Owner-12345'
   */
  owner_id?: string;

  /**
   * Last update timestamp
   * @example '2024-11-20T10:30:00Z'
   */
```

```
  last_updated?: string;

  /**
   * Constructor with partial data
   *
   * @param partial - Partial VinInstance object
   */
  constructor(partial?: Partial<VinInstance>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  /**
   * Create VinInstance from API response
   *
   * @param data - Raw API response data
   * @returns VinInstance instance
   */
  static fromApiResponse(data: any): VinInstance {
    return new VinInstance({
      vin: data.vin,
      vehicle_id: data.vehicle_id || data.vehicleId,
      registration_date: data.registration_date || data.registrationDate,
      registration_state: data.registration_state || data.registrationState,
      odometer_reading: data.odometer_reading || data.odometerReading,
      status: data.status,
      color: data.color,
      owner_id: data.owner_id || data.ownerId,
      last_updated: data.last_updated || data.lastUpdated
    });
  }

  /**
   * Get formatted VIN (groups of 4 characters)
   *
   * @returns Formatted VIN string
   * @example '1HGC M826 33A1 2345 6'
   */
  getFormattedVin(): string {
    return this.vin.match(/.{1,4}/g)?.join(' ') || this.vin;
  }

  /**
   * Check if VIN is valid length
   *
   * @returns True if VIN is 17 characters
   */
  isValidLength(): boolean {
    return this.vin?.length === 17;
  }
}
```

Step 402.2: Update the Barrel File

Update `src/app/domain-config/automobile/models/index.ts`:

```
// src/app/domain-config/automobile/models/index.ts
// VERSION 2 (Section 402) - Added data models

export * from './automobile.data';

// Filter and statistics models (Section 403)
// export * from './automobile.filters';
// export * from './automobile.statistics';
```

Verification

1. Check File Exists

```
$ ls -la src/app/domain-config/automobile/models/automobile.data.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/domain-config/automobile/models/automobile.data.ts
```

3. Test Model Usage

Create a temporary test:

```
// In any component or test file
import { VehicleResult, VinInstance } from '@app/domain-config/automobile/models';

// Test VehicleResult
const vehicle = VehicleResult.fromApiResponse({
  vehicle_id: 'TOY-CAM-2024-SED',
  manufacturer: 'Toyota',
  model: 'Camry',
  year: '2024', // String from API
  body_class: 'Sedan',
  instance_count: '156' // String from API
});

console.log('Display name:', vehicle.getDisplayName());
// Output: 'Toyota Camry 2024'

console.log('Full description:', vehicle.getFullDescription());
// Output: 'Toyota Camry 2024 Sedan (156 instances)'

console.log('Age:', vehicle.getAge());
// Output: 0 (if current year is 2024)
```

```
// Test VinInstance
const vin = new VinInstance({
  vin: '1HGCM82633A123456',
  vehicle_id: 'HON-ACC-2023-SED'
});

console.log('Formatted VIN:', vin.getFormattedVin());
// Output: '1HGC M826 33A1 2345 6'

console.log('Valid length:', vin.isValidLength());
// Output: true
```

Common Problems

Symptom	Cause	Solution		
`year` is string not number	API returns string	Use `Number()` in `fromApiResponse()`		
`instance_count` is `NaN`	Missing or null from API	Default to 0: `Number(data.instance_count		0)`
`body_class` undefined	Different API field name	Check for both `body_class` and `bodyClass`		
Methods not available on object	Plain object, not class instance	Use `fromApiResponse()` or `new VehicleResult()`		

Chapter 403

Automobile Statistics Model

What

Step 403.1: Create the Filter Models

Create the file `src/app/domain-config/automobile/models/automobile.filters.ts`:

```
// src/app/domain-config/automobile/models/automobile.filters.ts
// VERSION 1 (Section 403) - Automobile domain filter models

/**
 * Highlight Filters
 *
 * Parameters for segmented statistics computation.
 * Sent to backend API as h_* query parameters.
 *
 * Purpose: Request segmented statistics with {total, highlighted} format
 * for chart visualization showing highlighted vs total data.
 *
 * @example
 * ```typescript
 * const highlights: HighlightFilters = {
 *   manufacturer: 'Ford',
 *   yearMin: 2020,
 *   yearMax: 2024
 * };
 * // URL: ?h_manufacturer=Ford&h_yearMin=2020&h_yearMax=2024
 * ```
 */
export interface HighlightFilters {
  /**
   * Year range highlighting (minimum)
   * URL parameter: h_yearMin
   */
  yearMin?: number;

  /**
   * Year range highlighting (maximum)
   * URL parameter: h_yearMax
   */
  yearMax?: number;

  /**
   * Manufacturer highlighting
   * URL parameter: h_manufacturer
```

```
   */
  manufacturer?: string;

  /**
   * Model combinations highlighting
   * Format: Manufacturer:Model,Manufacturer:Model
   * URL parameter: h_modelCombos
   */
  modelCombos?: string;

  /**
   * Body class highlighting
   * URL parameter: h_bodyClass
   */
  bodyClass?: string;
}

/**
 * Automobile search filters
 *
 * Comprehensive filter model for searching and filtering vehicle data.
 * All fields are optional to support partial filtering.
 *
 * @example
 * ```typescript
 * const filters: AutoSearchFilters = {
 *   manufacturer: 'Toyota',
 *   yearMin: 2020,
 *   yearMax: 2024,
 *   bodyClass: 'SUV',
 *   page: 1,
 *   size: 20,
 *   sort: 'year',
 *   sortDirection: 'desc'
 * };
 * ```
 */
export class AutoSearchFilters {
  /**
   * Vehicle manufacturer name
   * Case-insensitive partial match
   *
   * @example 'Toyota', 'Honda', 'Ford'
   */
  manufacturer?: string;

  /**
   * Vehicle model name
   * Case-insensitive partial match
   *
   * @example 'Camry', 'Accord', 'F-150'
   */
  model?: string;

  /**
   * Minimum year (inclusive)
   *
   * @example 2020
   */
```

```
yearMin?: number;

/**
 * Maximum year (inclusive)
 *
 * @example 2024
 */
yearMax?: number;

/**
 * Vehicle body class/type (supports multiple selections)
 * Case-insensitive partial match
 * Can be a single value or array for multi-select
 *
 * @example 'Sedan', ['SUV', 'Truck'], 'Coupe'
 */
bodyClass?: string | string[];

/**
 * Minimum VIN instance count
 * Filter vehicles with at least this many VIN instances
 *
 * @example 10
 */
instanceCountMin?: number;

/**
 * Maximum VIN instance count
 * Filter vehicles with at most this many VIN instances
 *
 * @example 1000
 */
instanceCountMax?: number;

/**
 * Page number (1-indexed)
 * Used for pagination
 *
 * @default 1
 */
page?: number;

/**
 * Page size (number of results per page)
 * Used for pagination
 *
 * @default 20
 */
size?: number;

/**
 * Sort field
 * Field name to sort by
 *
 * @example 'manufacturer', 'model', 'year', 'instance_count'
 */
sort?: string;

/**
```

```
 * Sort direction
 * Ascending or descending order
 *
 * @default 'asc'
 */
sortDirection?: 'asc' | 'desc';

/**
 * Search query (global search)
 * Searches across multiple fields (manufacturer, model, body class)
 *
 * @example 'Toyota Camry'
 */
search?: string;

/**
 * Model combinations (from picker)
 * Comma-separated manufacturer:model pairs
 *
 * @example 'Ford:F-150,Toyota:Camry,Honda:Accord'
 */
modelCombos?: string;

/**
 * Constructor with default values
 *
 * @param partial - Partial AutoSearchFilters object
 */
constructor(partial?: Partial<AutoSearchFilters>) {
  Object.assign(this, partial);
}

/**
 * Create filters from partial object
 *
 * @param partial - Partial filter object
 * @returns AutoSearchFilters instance
 */
static fromPartial(partial: Partial<AutoSearchFilters>): AutoSearchFilters {
  return new AutoSearchFilters(partial);
}

/**
 * Get default filters
 *
 * @returns Default filter values
 */
static getDefaults(): AutoSearchFilters {
  return new AutoSearchFilters({
    page: 1,
    size: 20,
    sort: 'manufacturer',
    sortDirection: 'asc'
  });
}

/**
 * Check if filters are empty (no active filters except pagination/sort)
 *
```

```
    * @returns True if no search filters are active
    */
   isEmpty(): boolean {
     // Handle bodyClass as potentially array
     const hasBodyClass = Array.isArray(this.bodyClass)
       ? this.bodyClass.length > 0
       : !!this.bodyClass;

     return (
       !this.manufacturer &&
       !this.model &&
       !this.yearMin &&
       !this.yearMax &&
       !hasBodyClass &&
       !this.instanceCountMin &&
       !this.instanceCountMax &&
       !this.search
     );
   }

   /**
    * Clone filters
    *
    * @returns New AutoSearchFilters instance with same values
    */
   clone(): AutoSearchFilters {
     return new AutoSearchFilters({ ...this });
   }

   /**
    * Merge with other filters
    *
    * @param other - Filters to merge
    * @returns New AutoSearchFilters with merged values
    */
   merge(other: Partial<AutoSearchFilters>): AutoSearchFilters {
     return new AutoSearchFilters({
       ...this,
       ...other
     });
   }

   /**
    * Clear all filters except pagination and sort
    *
    * @returns New AutoSearchFilters with only pagination/sort
    */
   clearSearch(): AutoSearchFilters {
     return new AutoSearchFilters({
       page: this.page,
       size: this.size,
       sort: this.sort,
       sortDirection: this.sortDirection
     });
   }
 }
```

Step 403.2: Create the Statistics Models

Create the file `src/app/domain-config/automobile/models/automobile.statistics.ts`:

```
// src/app/domain-config/automobile/models/automobile.statistics.ts
// VERSION 1 (Section 403) - Automobile domain statistics models

/**
 * Manufacturer statistic
 *
 * Aggregated data for a single manufacturer
 */
export class ManufacturerStat {
  /**
   * Manufacturer name
   * @example 'Toyota'
   */
  name!: string;

  /**
   * Number of vehicle configurations
   * @example 234
   */
  count!: number;

  /**
   * Total VIN instances
   * @example 8456
   */
  instanceCount?: number;

  /**
   * Percentage of total
   * @example 18.8
   */
  percentage!: number;

  /**
   * Number of unique models
   * @example 42
   */
  modelCount?: number;

  constructor(partial?: Partial<ManufacturerStat>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  static fromApiResponse(data: any): ManufacturerStat {
    return new ManufacturerStat({
      name: data.name || data.manufacturer,
      count: Number(data.count || data.vehicle_count || 0),
      instanceCount: data.instance_count || data.instanceCount,
      percentage: Number(data.percentage || 0),
      modelCount: data.model_count || data.modelCount
```

```
    });
  }
}

/**
 * Model statistic
 *
 * Aggregated data for a single vehicle model
 */
export class ModelStat {
  /**
   * Model name
   * @example 'Camry'
   */
  name!: string;

  /**
   * Manufacturer name
   * @example 'Toyota'
   */
  manufacturer!: string;

  /**
   * Number of vehicle configurations
   * @example 15
   */
  count!: number;

  /**
   * Total VIN instances
   * @example 3456
   */
  instanceCount!: number;

  /**
   * Percentage of total instances
   * @example 7.6
   */
  percentage!: number;

  constructor(partial?: Partial<ModelStat>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  static fromApiResponse(data: any): ModelStat {
    return new ModelStat({
      name: data.name || data.model,
      manufacturer: data.manufacturer,
      count: Number(data.count || data.vehicle_count || 0),
      instanceCount: Number(data.instance_count || data.instanceCount || 0),
      percentage: Number(data.percentage || 0)
    });
  }

  /**
   * Get full model name including manufacturer
   * @returns Full name (e.g., "Toyota Camry")
```

```
   */
  getFullName(): string {
    return `${this.manufacturer} ${this.name}`;
  }
}

/**
 * Body class statistic
 *
 * Aggregated data for a single body class
 */
export class BodyClassStat {
  /**
   * Body class name
   * @example 'Sedan'
   */
  name!: string;

  /**
   * Number of vehicle configurations
   * @example 456
   */
  count!: number;

  /**
   * Total VIN instances
   * @example 16789
   */
  instanceCount?: number;

  /**
   * Percentage of total
   * @example 36.6
   */
  percentage!: number;

  constructor(partial?: Partial<BodyClassStat>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  static fromApiResponse(data: any): BodyClassStat {
    return new BodyClassStat({
      name: data.name || data.body_class || data.bodyClass,
      count: Number(data.count || data.vehicle_count || 0),
      instanceCount: data.instance_count || data.instanceCount,
      percentage: Number(data.percentage || 0)
    });
  }
}

/**
 * Year statistic
 *
 * Aggregated data for a single vehicle year
 */
export class YearStat {
  /**
```

```
   * Vehicle year
   * @example 2024
   */
  year!: number;

  /**
   * Number of vehicle configurations
   * @example 89
   */
  count!: number;

  /**
   * Total VIN instances
   * @example 3245
   */
  instanceCount?: number;

  /**
   * Percentage of total
   * @example 7.1
   */
  percentage!: number;

  constructor(partial?: Partial<YearStat>) {
    if (partial) {
      Object.assign(this, partial);
    }
  }

  static fromApiResponse(data: any): YearStat {
    return new YearStat({
      year: Number(data.year),
      count: Number(data.count || data.vehicle_count || 0),
      instanceCount: data.instance_count || data.instanceCount,
      percentage: Number(data.percentage || 0)
    });
  }

  /**
   * Check if this year is current year
   */
  isCurrentYear(): boolean {
    return this.year === new Date().getFullYear();
  }

  /**
   * Get age of vehicles from this year
   */
  getAge(): number {
    return new Date().getFullYear() - this.year;
  }
}

/**
 * Vehicle statistics
 *
 * Aggregated statistics across all filtered vehicles.
 * Provides high-level metrics and distributions for analysis.
 *
```

```
 * @example
 * ```typescript
 * const stats: VehicleStatistics = {
 *   totalVehicles: 1247,
 *   totalInstances: 45623,
 *   manufacturerCount: 23,
 *   modelCount: 412,
 *   yearRange: { min: 2010, max: 2024 },
 *   averageInstancesPerVehicle: 36.6
 * };
 * ```
 */
export class VehicleStatistics {
  /**
   * Total number of unique vehicle configurations
   * @example 1247
   */
  totalVehicles!: number;

  /**
   * Total number of VIN instances across all vehicles
   * @example 45623
   */
  totalInstances!: number;

  /**
   * Number of unique manufacturers
   * @example 23
   */
  manufacturerCount!: number;

  /**
   * Number of unique models
   * @example 412
   */
  modelCount!: number;

  /**
   * Number of unique body classes
   * @example 8
   */
  bodyClassCount?: number;

  /**
   * Year range (min and max years in dataset)
   */
  yearRange!: {
    min: number;
    max: number;
  };

  /**
   * Average VIN instances per vehicle configuration
   * @example 36.6
   */
  averageInstancesPerVehicle!: number;

  /**
   * Median VIN instances per vehicle configuration
```

```
 * @example 28
 */
medianInstancesPerVehicle?: number;

/**
 * Top manufacturers by vehicle count (top 20)
 */
topManufacturers?: ManufacturerStat[];

/**
 * Top models by instance count (top 20)
 */
topModels?: ModelStat[];

/**
 * Distribution by body class
 */
bodyClassDistribution?: BodyClassStat[];

/**
 * Distribution by year
 */
yearDistribution?: YearStat[];

/**
 * Complete manufacturer distribution
 */
manufacturerDistribution?: ManufacturerStat[];

/**
 * Raw segmented statistics by manufacturer
 * Format: { "Toyota": { total: 234, highlighted: 45 } }
 */
byManufacturer?: Record<string, { total: number; highlighted: number }>;

/**
 * Raw segmented statistics by body class
 */
byBodyClass?: Record<string, { total: number; highlighted: number }>;

/**
 * Raw segmented statistics by year
 */
byYearRange?: Record<string, { total: number; highlighted: number }>;

/**
 * Raw segmented statistics by model per manufacturer
 */
modelsByManufacturer?: Record<string, Record<string, { total: number; highlighted: number }>>;

constructor(partial?: Partial<VehicleStatistics>) {
  if (partial) {
    Object.assign(this, partial);
  }
}

/**
 * Create VehicleStatistics from API response
 *
```

```
 * Handles two formats:
 * 1. Array-based statistics (topManufacturers, etc.)
 * 2. Segmented statistics (byManufacturer, modelsByManufacturer, etc.)
 *
 * @param data - Raw API response data
 * @returns VehicleStatistics instance
 */
static fromApiResponse(data: any): VehicleStatistics {
  // Check if segmented format
  const byYearData = data.byYearRange || data.byYear;
  if (data.byManufacturer || data.modelsByManufacturer || data.byBodyClass || byYearData) {
    return VehicleStatistics.fromSegmentedStats(data);
  }

  // Array-based format
  return new VehicleStatistics({
    totalVehicles: Number(data.total_vehicles || data.totalVehicles || 0),
    totalInstances: Number(data.total_instances || data.totalInstances || 0),
    manufacturerCount: Number(data.manufacturer_count || data.manufacturerCount || 0),
    modelCount: Number(data.model_count || data.modelCount || 0),
    bodyClassCount: data.body_class_count || data.bodyClassCount,
    yearRange: {
      min: Number(data.year_range?.min || data.yearRange?.min || 0),
      max: Number(data.year_range?.max || data.yearRange?.max || 0)
    },
    averageInstancesPerVehicle: Number(
      data.average_instances_per_vehicle ||
      data.averageInstancesPerVehicle ||
      0
    ),
    medianInstancesPerVehicle: data.median_instances_per_vehicle || data.medianInstancesPerVehicle,
    topManufacturers: data.top_manufacturers?.map((m: any) =>
      ManufacturerStat.fromApiResponse(m)
    ) || data.topManufacturers?.map((m: any) =>
      ManufacturerStat.fromApiResponse(m)
    ),
    topModels: data.top_models?.map((m: any) =>
      ModelStat.fromApiResponse(m)
    ) || data.topModels?.map((m: any) =>
      ModelStat.fromApiResponse(m)
    ),
    bodyClassDistribution: data.body_class_distribution?.map((b: any) =>
      BodyClassStat.fromApiResponse(b)
    ) || data.bodyClassDistribution?.map((b: any) =>
      BodyClassStat.fromApiResponse(b)
    ),
    yearDistribution: data.year_distribution?.map((y: any) =>
      YearStat.fromApiResponse(y)
    ) || data.yearDistribution?.map((y: any) =>
      YearStat.fromApiResponse(y)
    )
  });
}

/**
 * Create from segmented statistics format
 */
private static fromSegmentedStats(data: any): VehicleStatistics {
  const byYearData = data.byYearRange || data.byYear;
```

```
  const topManufacturers = VehicleStatistics.transformByManufacturer(data.byManufacturer);
  const topModels = VehicleStatistics.transformModelsByManufacturer(data.modelsByManufacturer);
  const bodyClassDistribution = VehicleStatistics.transformByBodyClass(data.byBodyClass);
  const yearDistribution = VehicleStatistics.transformByYearRange(byYearData);

  const totalVehicles = data.totalCount || 0;
  const manufacturerCount = topManufacturers?.length || 0;
  const modelCount = topModels?.length || 0;
  const bodyClassCount = bodyClassDistribution?.length || 0;

  const years = yearDistribution?.map(y => y.year) || [];
  const yearRange = years.length > 0
    ? { min: Math.min(...years), max: Math.max(...years) }
    : { min: 0, max: 0 };

  return new VehicleStatistics({
    totalVehicles,
    totalInstances: totalVehicles,
    manufacturerCount,
    modelCount,
    bodyClassCount,
    yearRange,
    averageInstancesPerVehicle: 0,
    topManufacturers,
    topModels,
    bodyClassDistribution,
    yearDistribution,
    manufacturerDistribution: topManufacturers,
    byManufacturer: data.byManufacturer,
    byBodyClass: data.byBodyClass,
    byYearRange: byYearData,
    modelsByManufacturer: data.modelsByManufacturer
  });
}

/**
 * Transform byManufacturer object to ManufacturerStat array
 */
private static transformByManufacturer(
  byManufacturer: Record<string, any> | undefined
): ManufacturerStat[] | undefined {
  if (!byManufacturer) return undefined;

  const stats = Object.entries(byManufacturer).map(([name, countOrStats]) => {
    const count = typeof countOrStats === 'object'
      ? (countOrStats.total || 0)
      : (countOrStats || 0);

    return new ManufacturerStat({
      name,
      count,
      instanceCount: count,
      percentage: 0,
      modelCount: 0
    });
  });

  stats.sort((a, b) => b.count - a.count);
```

```
  const totalCount = stats.reduce((sum, s) => sum + s.count, 0);
  stats.forEach(s => {
    s.percentage = totalCount > 0 ? (s.count / totalCount) * 100 : 0;
  });

  return stats.slice(0, 20);
}

/**
 * Transform modelsByManufacturer to ModelStat array
 */
private static transformModelsByManufacturer(
  modelsByManufacturer: Record<string, Record<string, any>> | undefined
): ModelStat[] | undefined {
  if (!modelsByManufacturer) return undefined;

  const stats: ModelStat[] = [];
  let totalCount = 0;

  Object.entries(modelsByManufacturer).forEach(([manufacturer, models]) => {
    Object.entries(models).forEach(([modelName, countOrStats]) => {
      const instanceCount = typeof countOrStats === 'object'
        ? (countOrStats.total || 0)
        : (countOrStats || 0);
      totalCount += instanceCount;
      stats.push(new ModelStat({
        name: modelName,
        manufacturer,
        count: 1,
        instanceCount,
        percentage: 0
      }));
    });
  });

  stats.sort((a, b) => b.instanceCount - a.instanceCount);

  stats.forEach(s => {
    s.percentage = totalCount > 0 ? (s.instanceCount / totalCount) * 100 : 0;
  });

  return stats.slice(0, 20);
}

/**
 * Transform byBodyClass to BodyClassStat array
 */
private static transformByBodyClass(
  byBodyClass: Record<string, any> | undefined
): BodyClassStat[] | undefined {
  if (!byBodyClass) return undefined;

  const stats = Object.entries(byBodyClass).map(([name, countOrStats]) => {
    const count = typeof countOrStats === 'object'
      ? (countOrStats.total || 0)
      : (countOrStats || 0);

    return new BodyClassStat({
      name,
```

```
      count,
      instanceCount: count,
      percentage: 0
    });
  });

  const totalCount = stats.reduce((sum, s) => sum + s.count, 0);
  stats.forEach(s => {
    s.percentage = totalCount > 0 ? (s.count / totalCount) * 100 : 0;
  });

  stats.sort((a, b) => b.count - a.count);

  return stats;
}

/**
 * Transform byYearRange to YearStat array
 */
private static transformByYearRange(
  byYearRange: Record<string, any> | undefined
): YearStat[] | undefined {
  if (!byYearRange) return undefined;

  const stats = Object.entries(byYearRange).map(([yearStr, countOrStats]) => {
    const count = typeof countOrStats === 'object'
      ? (countOrStats.total || 0)
      : (countOrStats || 0);

    return new YearStat({
      year: parseInt(yearStr, 10),
      count,
      instanceCount: count,
      percentage: 0
    });
  });

  const totalCount = stats.reduce((sum, s) => sum + s.count, 0);
  stats.forEach(s => {
    s.percentage = totalCount > 0 ? (s.count / totalCount) * 100 : 0;
  });

  stats.sort((a, b) => a.year - b.year);

  return stats;
}

/**
 * Get year span (number of years covered)
 */
getYearSpan(): number {
  return this.yearRange.max - this.yearRange.min + 1;
}

/**
 * Get average vehicles per manufacturer
 */
getAverageVehiclesPerManufacturer(): number {
  return this.manufacturerCount > 0
```

```
      ? this.totalVehicles / this.manufacturerCount
      : 0;
  }
}
```

Step 403.3: Update the Barrel File

Update `src/app/domain-config/automobile/models/index.ts`:

```
// src/app/domain-config/automobile/models/index.ts
// VERSION 3 (Section 403) - Complete domain models

export * from './automobile.data';
export * from './automobile.filters';
export * from './automobile.statistics';
```

Verification

1. Check Files Exist

```
$ ls -la src/app/domain-config/automobile/models/
```

Expected output:

```
total 20
drwxr-xr-x 2 user user 4096 Feb  9 12:00 .
drwxr-xr-x 3 user user 4096 Feb  9 12:00 ..
-rw-r--r-- 1 user user 4567 Feb  9 12:00 automobile.data.ts
-rw-r--r-- 1 user user 3456 Feb  9 12:00 automobile.filters.ts
-rw-r--r-- 1 user user 8901 Feb  9 12:00 automobile.statistics.ts
-rw-r--r-- 1 user user  200 Feb  9 12:00 index.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/domain-config/automobile/models/index.ts
```

3. Test Filter Usage

```
import { AutoSearchFilters } from '@app/domain-config/automobile/models';

// Create filters
```

```
const filters = new AutoSearchFilters({
  manufacturer: 'Toyota',
  yearMin: 2020,
  yearMax: 2024,
  page: 1,
  size: 20
});

console.log('Is empty:', filters.isEmpty());
// Output: false

// Clone and modify
const newFilters = filters.merge({ bodyClass: 'SUV' });
console.log('Body class:', newFilters.bodyClass);
// Output: 'SUV'
```

Common Problems

Symptom	Cause	Solution
Statistics empty	Wrong API format detection	Check for `byManufacturer` vs `topManufacturers`
Percentages all zero	Total count is zero	Check totalCount calculation
Year range wrong	Years not parsed as numbers	Use `parseInt()` for year keys
Highlight data missing	Not preserved in transformation	Keep raw `byManufacturer` etc.

Key Takeaways

1. **Filter models map to URL and API** — Same structure for URL params and query strings
2. **Statistics handle two formats** — Array-based and segmented for charts
3. **Highlight filters enable chart segmentation** — `h_*` prefix for API parameters
4. **Transformation preserves raw data** — Keep segmented data for chart highlighting

Phase 5: Domain Adapters

Chapter 501: Automobile API Adapter

Chapter 502: Automobile URL Mapper

Chapter 503: Automobile Cache Key Builder

Chapter 501

Automobile API Adapter

Step 501.2: Create the Adapters Directory

Create the directory structure for automobile domain adapters:

```
$ mkdir -p src/app/domain-config/automobile/adapters
$ touch src/app/domain-config/automobile/adapters/index.ts
```

Create the barrel file `src/app/domain-config/automobile/adapters/index.ts`:

```
// src/app/domain-config/automobile/adapters/index.ts
// VERSION 1 (Section 501) - Automobile domain adapters barrel

// URL Mapper (Section 502)
// export * from './automobile-url-mapper';

// API Adapter (Section 503)
// export * from './automobile-api.adapter';

// Cache Key Builder (Section 503)
// export * from './automobile-cache-key-builder';
```

Step 501.3: Understand Adapter Usage

The adapters are provided to `ResourceManagementService` via a configuration object:

```
// In domain config factory
const resourceConfig: ResourceManagementConfig<
  AutoSearchFilters,
  VehicleResult,
  VehicleStatistics
> = {
  filterMapper: new AutomobileUrlMapper(),
  apiAdapter: new AutomobileApiAdapter(apiService, baseUrl),
  cacheKeyBuilder: new AutomobileCacheKeyBuilder(),
  defaultFilters: AutoSearchFilters.getDefaults(),
  autoFetch: true,
  cacheTTL: 30000
};

// ResourceManagementService uses these adapters
```

```
const resourceService = new ResourceManagementService(
  urlStateService,
  requestCoordinator,
  resourceConfig
);
```

The adapters form the bridge between:

1. **URL** (user-facing state)
2. **Filters** (domain-specific parameters)
3. **API** (backend data source)
4. **Cache** (request deduplication)

Verification

1. Check Directory Exists

```
$ ls -la src/app/domain-config/automobile/adapters/
```

2. Review Interface File

```
$ cat src/app/framework/models/resource-management.interface.ts
```

Ensure the interface exports include:

- `IFilterUrlMapper<TFilters>`
- `IApiAdapter<TFilters, TData, TStatistics>`
- `ICacheKeyBuilder<TFilters>`
- `ApiAdapterResponse<TData, TStatistics>`
- `ResourceManagementConfig<TFilters, TData, TStatistics>`

Common Problems

Symptom	Cause	Solution
"Generic type requires arguments"	Missing type parameters	Provide `<TFilters, TData>` when implementing

Method signature mismatch	Wrong parameter/return types	Match interface signature exactly
Cannot find interface	Import path wrong	Import from `@app/framework/models`
"Property is missing"	Interface not fully implemented	Implement all required methods

Chapter 502

Automobile URL Mapper

Parameter Naming

URL parameters should match API parameters for consistency:

```
URL:     ?yearMin=2020&yearMax=2024&sortBy=manufacturer
API:     GET /vehicles?yearMin=2020&yearMax=2024&sortBy=manufacturer
```

This allows the URL to be directly usable as API query string (with minor adjustments).

Highlight Filters

For chart highlighting, the mapper extracts `h_` prefixed parameters:

```
URL: ?manufacturer=Toyota&h_yearMin=2022&h_yearMax=2024
```

Extracted highlights: `{ yearMin: 2022, yearMax: 2024 }`

These are sent to the API for segmented statistics: `{ total: 234, highlighted: 45 }`

What

Step 502.1: Create the URL Mapper

Create the file `src/app/domain-config/automobile/adapters/automobile-url-mapper.ts`:

```
// src/app/domain-config/automobile/adapters/automobile-url-mapper.ts
// VERSION 1 (Section 502) - Automobile URL mapper adapter

import { Injectable } from '@angular/core';
import { Params } from '@angular/router';
import { IFilterUrlMapper } from '../../../framework/models/resource-management.interface';
import { AutoSearchFilters } from '../models/automobile.filters';

/**
 * Automobile filter URL mapper
 *
 * Bidirectional conversion between filter objects and URL query parameters.
 * URL parameter names match backend API parameter names (camelCase).
```

```
 *
 * **URL Parameter Mapping:**
 * - manufacturer → manufacturer
 * - model → model
 * - yearMin → yearMin
 * - yearMax → yearMax
 * - bodyClass → bodyClass
 * - page → page
 * - size → size
 * - sortBy → sort (filter property)
 * - sortOrder → sortDirection (filter property)
 *
 * **Highlight Parameters:**
 * - h_yearMin, h_yearMax, h_manufacturer, etc.
 *
 * @example
 * ```typescript
 * const mapper = new AutomobileUrlMapper();
 *
 * // To URL
 * const filters = new AutoSearchFilters({
 *   manufacturer: 'Toyota',
 *   yearMin: 2020,
 *   page: 1
 * });
 * const params = mapper.toUrlParams(filters);
 * // { manufacturer: 'Toyota', yearMin: '2020', page: '1' }
 *
 * // From URL
 * const urlParams = { manufacturer: 'Toyota', yearMin: '2020', page: '1' };
 * const filters = mapper.fromUrlParams(urlParams);
 * // AutoSearchFilters { manufacturer: 'Toyota', yearMin: 2020, page: 1 }
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class AutomobileUrlMapper implements IFilterUrlMapper<AutoSearchFilters> {
  /**
   * URL parameter names (match backend API parameter names)
   */
  private readonly PARAM_NAMES = {
    manufacturer: 'manufacturer',
    model: 'model',
    yearMin: 'yearMin',
    yearMax: 'yearMax',
    bodyClass: 'bodyClass',
    instanceCountMin: 'instanceCountMin',
    instanceCountMax: 'instanceCountMax',
    search: 'search',
    modelCombos: 'modelCombos',
    page: 'page',
    size: 'size',
    sort: 'sortBy',
    sortDirection: 'sortOrder'
  };

  /**
   * Convert filters to URL query parameters
```

```
 *
 * Maps filter object fields to URL parameter names.
 * Only includes non-null/undefined values.
 * Converts all values to strings for URL compatibility.
 *
 * @param filters - Filter object
 * @returns URL query parameters
 */
toUrlParams(filters: AutoSearchFilters): Params {
  const params: Params = {};

  // Search filters
  if (filters.manufacturer !== undefined && filters.manufacturer !== null) {
    params[this.PARAM_NAMES.manufacturer] = filters.manufacturer;
  }

  if (filters.model !== undefined && filters.model !== null) {
    params[this.PARAM_NAMES.model] = filters.model;
  }

  if (filters.yearMin !== undefined && filters.yearMin !== null) {
    params[this.PARAM_NAMES.yearMin] = String(filters.yearMin);
  }

  if (filters.yearMax !== undefined && filters.yearMax !== null) {
    params[this.PARAM_NAMES.yearMax] = String(filters.yearMax);
  }

  if (filters.bodyClass !== undefined && filters.bodyClass !== null) {
    // Handle array values (multiselect) - join with comma
    if (Array.isArray(filters.bodyClass)) {
      if (filters.bodyClass.length > 0) {
        params[this.PARAM_NAMES.bodyClass] = filters.bodyClass.join(',');
      }
    } else {
      params[this.PARAM_NAMES.bodyClass] = filters.bodyClass;
    }
  }

  if (filters.instanceCountMin !== undefined && filters.instanceCountMin !== null) {
    params[this.PARAM_NAMES.instanceCountMin] = String(filters.instanceCountMin);
  }

  if (filters.instanceCountMax !== undefined && filters.instanceCountMax !== null) {
    params[this.PARAM_NAMES.instanceCountMax] = String(filters.instanceCountMax);
  }

  if (filters.search !== undefined && filters.search !== null) {
    params[this.PARAM_NAMES.search] = filters.search;
  }

  if (filters.modelCombos !== undefined && filters.modelCombos !== null) {
    params[this.PARAM_NAMES.modelCombos] = filters.modelCombos;
  }

  // Pagination
  if (filters.page !== undefined && filters.page !== null) {
    params[this.PARAM_NAMES.page] = String(filters.page);
  }
```

```
  if (filters.size !== undefined && filters.size !== null) {
    params[this.PARAM_NAMES.size] = String(filters.size);
  }

  // Sorting
  if (filters.sort !== undefined && filters.sort !== null) {
    params[this.PARAM_NAMES.sort] = filters.sort;
  }

  if (filters.sortDirection !== undefined && filters.sortDirection !== null) {
    params[this.PARAM_NAMES.sortDirection] = filters.sortDirection;
  }

  return params;
}

/**
 * Convert URL query parameters to filters
 *
 * Maps URL parameter names back to filter object fields.
 * Performs type conversion (strings to numbers).
 * Returns filter object with only defined values.
 *
 * @param params - URL query parameters
 * @returns Filter object
 */
fromUrlParams(params: Params): AutoSearchFilters {
  const filters = new AutoSearchFilters();

  // Search filters
  if (params[this.PARAM_NAMES.manufacturer]) {
    filters.manufacturer = String(params[this.PARAM_NAMES.manufacturer]);
  }

  if (params[this.PARAM_NAMES.model]) {
    filters.model = String(params[this.PARAM_NAMES.model]);
  }

  if (params[this.PARAM_NAMES.yearMin]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.yearMin]);
    if (value !== null) {
      filters.yearMin = value;
    }
  }

  if (params[this.PARAM_NAMES.yearMax]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.yearMax]);
    if (value !== null) {
      filters.yearMax = value;
    }
  }

  if (params[this.PARAM_NAMES.bodyClass]) {
    const bodyClassParam = String(params[this.PARAM_NAMES.bodyClass]);
    // Check if it contains comma (multiple values)
    if (bodyClassParam.includes(',')) {
      filters.bodyClass = bodyClassParam.split(',').map(v => v.trim());
    } else {
      // Single value - return as array for consistency with multiselect
```

```
      filters.bodyClass = [bodyClassParam];
    }
  }

  if (params[this.PARAM_NAMES.instanceCountMin]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.instanceCountMin]);
    if (value !== null) {
      filters.instanceCountMin = value;
    }
  }

  if (params[this.PARAM_NAMES.instanceCountMax]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.instanceCountMax]);
    if (value !== null) {
      filters.instanceCountMax = value;
    }
  }

  if (params[this.PARAM_NAMES.search]) {
    filters.search = String(params[this.PARAM_NAMES.search]);
  }

  if (params[this.PARAM_NAMES.modelCombos]) {
    filters.modelCombos = String(params[this.PARAM_NAMES.modelCombos]);
  }

  // Pagination
  if (params[this.PARAM_NAMES.page]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.page]);
    if (value !== null) {
      filters.page = value;
    }
  }

  if (params[this.PARAM_NAMES.size]) {
    const value = this.parseNumber(params[this.PARAM_NAMES.size]);
    if (value !== null) {
      filters.size = value;
    }
  }

  // Sorting
  if (params[this.PARAM_NAMES.sort]) {
    filters.sort = String(params[this.PARAM_NAMES.sort]);
  }

  if (params[this.PARAM_NAMES.sortDirection]) {
    const direction = String(params[this.PARAM_NAMES.sortDirection]);
    if (direction === 'asc' || direction === 'desc') {
      filters.sortDirection = direction;
    }
  }

  return filters;
}

/**
 * Extract highlight filters from URL parameters
 *
```

```
 * Looks for 'h_' prefixed parameters for chart highlighting.
 * These enable segmented statistics (total vs highlighted).
 *
 * @param params - URL query parameters
 * @returns Highlight filters object
 */
extractHighlights(params: Params): Record<string, any> {
  const highlights: Record<string, any> = {};
  const prefix = 'h_';

  Object.keys(params).forEach(key => {
    if (key.startsWith(prefix)) {
      const highlightKey = key.substring(prefix.length);
      let value = params[key];

      // Normalize separators: Convert pipes to commas
      if (typeof value === 'string' && value.includes('|')) {
        value = value.replace(/\|/g, ',');
      }

      highlights[highlightKey] = value;
    }
  });

  return highlights;
}

/**
 * Parse number from URL parameter value
 *
 * @param value - URL parameter value
 * @returns Parsed number or null
 */
private parseNumber(value: any): number | null {
  if (value === null || value === undefined || value === '') {
    return null;
  }

  const num = Number(value);
  return isNaN(num) ? null : num;
}

/**
 * Get parameter name mapping
 *
 * @returns Parameter name mapping object
 */
getParameterMapping(): Record<string, string> {
  return { ...this.PARAM_NAMES };
}

/**
 * Build shareable URL from filters
 *
 * @param baseUrl - Base URL (e.g., '/discover')
 * @param filters - Filter object
 * @returns Complete URL with query parameters
 */
buildShareableUrl(baseUrl: string, filters: AutoSearchFilters): string {
```

```
    const params = this.toUrlParams(filters);
    const queryString = Object.entries(params)
      .map(([key, value]) => `${key}=${encodeURIComponent(value)}`)
      .join('&');

    return queryString ? `${baseUrl}?${queryString}` : baseUrl;
  }

  /**
   * Validate URL parameters
   *
   * @param params - URL query parameters
   * @returns Validation result with errors
   */
  validateUrlParams(params: Params): { valid: boolean; errors: string[] } {
    const errors: string[] = [];

    // Check numeric fields
    const numericFields = [
      this.PARAM_NAMES.yearMin,
      this.PARAM_NAMES.yearMax,
      this.PARAM_NAMES.instanceCountMin,
      this.PARAM_NAMES.instanceCountMax,
      this.PARAM_NAMES.page,
      this.PARAM_NAMES.size
    ];

    numericFields.forEach(field => {
      if (params[field] !== undefined && params[field] !== null) {
        const value = this.parseNumber(params[field]);
        if (value === null) {
          errors.push(`Invalid numeric value for ${field}: ${params[field]}`);
        }
      }
    });

    // Check sort direction
    if (params[this.PARAM_NAMES.sortDirection]) {
      const direction = String(params[this.PARAM_NAMES.sortDirection]);
      if (direction !== 'asc' && direction !== 'desc') {
        errors.push(`Invalid sort direction: ${direction}. Must be 'asc' or 'desc'.`);
      }
    }

    // Check year range
    if (params[this.PARAM_NAMES.yearMin] && params[this.PARAM_NAMES.yearMax]) {
      const min = this.parseNumber(params[this.PARAM_NAMES.yearMin]);
      const max = this.parseNumber(params[this.PARAM_NAMES.yearMax]);
      if (min !== null && max !== null && min > max) {
        errors.push(`Year minimum (${min}) cannot be greater than maximum (${max})`);
      }
    }

    return {
      valid: errors.length === 0,
      errors
    };
  }
}
```

Step 502.2: Update the Barrel File

Update `src/app/domain-config/automobile/adapters/index.ts`:

```
// src/app/domain-config/automobile/adapters/index.ts
// VERSION 2 (Section 502) - Added URL mapper

export * from './automobile-url-mapper';

// API Adapter (Section 503)
// export * from './automobile-api.adapter';

// Cache Key Builder (Section 503)
// export * from './automobile-cache-key-builder';
```

Verification

1. Check File Exists

```
$ ls -la src/app/domain-config/automobile/adapters/automobile-url-mapper.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/domain-config/automobile/adapters/automobile-url-mapper.ts
```

3. Test Mapping

```
import { AutomobileUrlMapper } from '@app/domain-config/automobile/adapters';
import { AutoSearchFilters } from '@app/domain-config/automobile/models';

const mapper = new AutomobileUrlMapper();

// Test toUrlParams
const filters = new AutoSearchFilters({
  manufacturer: 'Toyota',
  yearMin: 2020,
  yearMax: 2024,
  bodyClass: ['Sedan', 'SUV'],
  page: 1,
  size: 20
});
```

```
const params = mapper.toUrlParams(filters);
console.log('URL params:', params);
// { manufacturer: 'Toyota', yearMin: '2020', yearMax: '2024',
//   bodyClass: 'Sedan,SUV', page: '1', size: '20' }

// Test fromUrlParams
const urlParams = {
  manufacturer: 'Honda',
  yearMin: '2022',
  page: '2'
};

const parsedFilters = mapper.fromUrlParams(urlParams);
console.log('Parsed filters:', parsedFilters);
// AutoSearchFilters { manufacturer: 'Honda', yearMin: 2022, page: 2 }

// Test highlight extraction
const highlightParams = {
  manufacturer: 'Ford',
  h_yearMin: '2023',
  h_yearMax: '2024'
};

const highlights = mapper.extractHighlights(highlightParams);
console.log('Highlights:', highlights);
// { yearMin: '2023', yearMax: '2024' }
```

Common Problems

Symptom	Cause	Solution
Numbers come back as strings	Missing `parseNumber()` call	Use `parseNumber()` for numeric fields
Array comes back as string	Missing comma split	Check for comma and split
Highlight params not extracted	Wrong prefix check	Ensure h_ prefix matching
Empty string in URL	Null/undefined not filtered	Check for null/undefined before adding

Chapter 503

Automobile Cache Key Builder

Cache Key Builder

The `RequestCoordinatorService` deduplicates identical requests. It needs a unique key for each request. The cache key builder creates this key from filters:

```
// Same filters = same cache key = deduplicated
buildKey({ manufacturer: 'Toyota', page: 1 })
// → 'auto:manufacturer=Toyota:page=1'

buildKey({ manufacturer: 'Toyota', page: 1 })
// → 'auto:manufacturer=Toyota:page=1' (same, deduplicated)

buildKey({ manufacturer: 'Honda', page: 1 })
// → 'auto:manufacturer=Honda:page=1' (different, new request)
```

Highlight Parameters in Cache Key

Highlight filters must be included in the cache key because they affect the response:

```
// Different highlights = different cache key
buildKey({ manufacturer: 'Toyota' }, { yearMin: 2020 })
// → 'auto:h_yearMin=2020:manufacturer=Toyota'

buildKey({ manufacturer: 'Toyota' }, { yearMin: 2022 })
// → 'auto:h_yearMin=2022:manufacturer=Toyota' (different)
```

What

Step 503.1: Create the API Adapter

Create the file `src/app/domain-config/automobile/adapters/automobile-api.adapter.ts`:

```
// src/app/domain-config/automobile/adapters/automobile-api.adapter.ts
// VERSION 1 (Section 503) - Automobile API adapter

import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import {
```

```
  IApiAdapter,
  ApiAdapterResponse
} from '../../../framework/models/resource-management.interface';
import { ApiResponse } from '../../../framework/models/api-response.interface';
import { ApiService } from '../../../framework/services/api.service';
import { AutoSearchFilters } from '../models/automobile.filters';
import { VehicleResult } from '../models/automobile.data';
import { VehicleStatistics } from '../models/automobile.statistics';

/**
 * Automobile API adapter
 *
 * Fetches vehicle data from the automobile discovery API.
 * Transforms API responses into domain models.
 *
 * NOTE: This is NOT an Angular service. It's instantiated manually
 * in the domain config factory with the base URL.
 *
 * @example
 * ```typescript
 * const adapter = new AutomobileApiAdapter(apiService, 'http://api.example.com/v1');
 * const filters = new AutoSearchFilters({ manufacturer: 'Toyota' });
 *
 * adapter.fetchData(filters).subscribe(response => {
 *   console.log('Vehicles:', response.results);
 *   console.log('Total:', response.total);
 *   console.log('Statistics:', response.statistics);
 * });
 * ```
 */
export class AutomobileApiAdapter
  implements IApiAdapter<AutoSearchFilters, VehicleResult, VehicleStatistics>
{
  /**
   * API endpoint for vehicle search
   */
  private readonly VEHICLES_ENDPOINT = '/vehicles/details';

  /**
   * API endpoint for statistics only
   */
  private readonly STATISTICS_ENDPOINT = '/statistics';

  /**
   * Base URL for API requests
   */
  private baseUrl: string;

  /**
   * API service for making HTTP requests
   */
  private apiService: ApiService;

  /**
   * Constructor
   *
   * @param apiService - Injected API service for HTTP requests
   * @param baseUrl - Base URL for automobile API
   */
```

```
constructor(apiService: ApiService, baseUrl: string) {
  this.apiService = apiService;
  this.baseUrl = baseUrl;
}

/**
 * Fetch vehicle data from API
 *
 * @param filters - Search filters
 * @param highlights - Optional highlight filters (h_* parameters)
 * @returns Observable of vehicle results with statistics
 */
fetchData(
  filters: AutoSearchFilters,
  highlights?: any
): Observable<ApiAdapterResponse<VehicleResult, VehicleStatistics>> {
  // Convert filters to API parameters
  const params = this.filtersToApiParams(filters, highlights);
  const url = `${this.baseUrl}${this.VEHICLES_ENDPOINT}`;

  // Fetch vehicle data
  return this.apiService.get<VehicleResult>(url, { params }).pipe(
    map((apiResponse: ApiResponse<VehicleResult>) => {
      // Transform API response to adapter response
      return {
        results: apiResponse.results.map(item =>
          VehicleResult.fromApiResponse(item)
        ),
        total: apiResponse.total,
        statistics: apiResponse.statistics
          ? VehicleStatistics.fromApiResponse(apiResponse.statistics)
          : undefined
      };
    })
  );
}

/**
 * Fetch statistics only (without vehicle data)
 *
 * Useful for refreshing statistics panel without reloading table data.
 *
 * @param filters - Search filters
 * @returns Observable of statistics
 */
fetchStatistics(filters: AutoSearchFilters): Observable<VehicleStatistics> {
  const params = this.filtersToApiParams(filters);

  return this.apiService
    .get<VehicleStatistics>(`${this.baseUrl}${this.STATISTICS_ENDPOINT}`, params)
    .pipe(
      map((response: any) => {
        // API might return statistics directly or wrapped
        const statsData = response.statistics || response;
        return VehicleStatistics.fromApiResponse(statsData);
      })
    );
}
```

```
/**
 * Convert filter object to API query parameters
 *
 * @param filters - Domain filters
 * @param highlights - Optional highlight filters (h_* parameters)
 * @returns API query parameters
 */
private filtersToApiParams(
  filters: AutoSearchFilters,
  highlights?: any
): Record<string, any> {
  const params: Record<string, any> = {};

  // Search filters
  if (filters.manufacturer) {
    params['manufacturer'] = filters.manufacturer;
  }

  if (filters.model) {
    params['model'] = filters.model;
  }

  if (filters.yearMin !== undefined && filters.yearMin !== null) {
    params['yearMin'] = filters.yearMin;
  }

  if (filters.yearMax !== undefined && filters.yearMax !== null) {
    params['yearMax'] = filters.yearMax;
  }

  if (filters.bodyClass) {
    params['bodyClass'] = filters.bodyClass;
  }

  if (filters.instanceCountMin !== undefined && filters.instanceCountMin !== null) {
    params['instanceCountMin'] = filters.instanceCountMin;
  }

  if (filters.instanceCountMax !== undefined && filters.instanceCountMax !== null) {
    params['instanceCountMax'] = filters.instanceCountMax;
  }

  if (filters.search) {
    params['search'] = filters.search;
  }

  // Model combinations from picker
  if (filters.modelCombos) {
    params['models'] = filters.modelCombos;
  }

  // Pagination
  if (filters.page !== undefined && filters.page !== null) {
    params['page'] = filters.page;
  }

  if (filters.size !== undefined && filters.size !== null) {
    params['size'] = filters.size;
  }
```

```
    // Sorting
    if (filters.sort) {
      params['sortBy'] = filters.sort;
    }

    if (filters.sortDirection) {
      params['sortOrder'] = filters.sortDirection;
    }

    // Highlight parameters (h_* prefix for segmented statistics)
    if (highlights) {
      if (highlights.yearMin !== undefined && highlights.yearMin !== null) {
        params['h_yearMin'] = String(highlights.yearMin);
      }

      if (highlights.yearMax !== undefined && highlights.yearMax !== null) {
        params['h_yearMax'] = String(highlights.yearMax);
      }

      if (highlights.manufacturer) {
        params['h_manufacturer'] = highlights.manufacturer;
      }

      if (highlights.modelCombos) {
        params['h_modelCombos'] = highlights.modelCombos;
      }

      if (highlights.bodyClass) {
        params['h_bodyClass'] = highlights.bodyClass;
      }
    }

    return params;
  }
}
```

Step 503.2: Create the Cache Key Builder

Create the file `src/app/domain-config/automobile/adapters/automobile-cache-key-builder.ts`:

```
// src/app/domain-config/automobile/adapters/automobile-cache-key-builder.ts
// VERSION 1 (Section 503) - Automobile cache key builder

import { Injectable } from '@angular/core';
import { ICacheKeyBuilder } from '../../../framework/models/resource-management.interface';
import { AutoSearchFilters } from '../models/automobile.filters';

/**
 * Automobile cache key builder
 *
 * Creates unique cache keys from filter objects for request deduplication.
 * Keys are deterministic: same filters always produce the same key.
 *
 * Key format: `auto:{filter1}={value1}:{filter2}={value2}:...`
```

```
 *
 * @example
 * ```typescript
 * const builder = new AutomobileCacheKeyBuilder();
 *
 * const key = builder.buildKey({ manufacturer: 'Toyota', page: 1 });
 * // → 'auto:manufacturer=Toyota:page=1'
 *
 * const keyWithHighlights = builder.buildKey(
 *   { manufacturer: 'Toyota' },
 *   { yearMin: 2020 }
 * );
 * // → 'auto:h_yearMin=2020:manufacturer=Toyota'
 * ```
 */
@Injectable({
  providedIn: 'root'
})
export class AutomobileCacheKeyBuilder
  implements ICacheKeyBuilder<AutoSearchFilters>
{
  /**
   * Prefix for all automobile cache keys
   */
  private readonly PREFIX = 'auto';

  /**
   * Build a unique cache key from filters
   *
   * @param filters - Filter object
   * @param highlights - Optional highlight filters
   * @returns Cache key string
   */
  buildKey(filters: AutoSearchFilters, highlights?: any): string {
    const parts: string[] = [this.PREFIX];

    // Collect all defined filter values
    const entries = this.getFilterEntries(filters);

    // Add highlight parameters with h_ prefix
    if (highlights) {
      if (highlights.yearMin !== undefined && highlights.yearMin !== null) {
        entries.push(['h_yearMin', highlights.yearMin]);
      }
      if (highlights.yearMax !== undefined && highlights.yearMax !== null) {
        entries.push(['h_yearMax', highlights.yearMax]);
      }
      if (highlights.manufacturer) {
        entries.push(['h_manufacturer', highlights.manufacturer]);
      }
      if (highlights.modelCombos) {
        entries.push(['h_modelCombos', highlights.modelCombos]);
      }
      if (highlights.bodyClass) {
        entries.push(['h_bodyClass', highlights.bodyClass]);
      }
    }

    // Sort by key for deterministic ordering
```

```
  entries.sort((a, b) => a[0].localeCompare(b[0]));

  // Build key parts
  entries.forEach(([key, value]) => {
    parts.push(`${key}=${this.serializeValue(value)}`);
  });

  return parts.join(':');
}

/**
 * Get all defined filter entries as [key, value] pairs
 *
 * @param filters - Filter object
 * @returns Array of [key, value] tuples
 */
private getFilterEntries(
  filters: AutoSearchFilters
): Array<[string, any]> {
  const entries: Array<[string, any]> = [];

  // Add each defined filter field
  if (filters.manufacturer !== undefined && filters.manufacturer !== null) {
    entries.push(['manufacturer', filters.manufacturer]);
  }

  if (filters.model !== undefined && filters.model !== null) {
    entries.push(['model', filters.model]);
  }

  if (filters.yearMin !== undefined && filters.yearMin !== null) {
    entries.push(['yearMin', filters.yearMin]);
  }

  if (filters.yearMax !== undefined && filters.yearMax !== null) {
    entries.push(['yearMax', filters.yearMax]);
  }

  if (filters.bodyClass !== undefined && filters.bodyClass !== null) {
    entries.push(['bodyClass', filters.bodyClass]);
  }

  if (filters.instanceCountMin !== undefined && filters.instanceCountMin !== null) {
    entries.push(['instanceCountMin', filters.instanceCountMin]);
  }

  if (filters.instanceCountMax !== undefined && filters.instanceCountMax !== null) {
    entries.push(['instanceCountMax', filters.instanceCountMax]);
  }

  if (filters.search !== undefined && filters.search !== null) {
    entries.push(['search', filters.search]);
  }

  if (filters.page !== undefined && filters.page !== null) {
    entries.push(['page', filters.page]);
  }

  if (filters.size !== undefined && filters.size !== null) {
```

```
      entries.push(['size', filters.size]);
    }

    if (filters.sort !== undefined && filters.sort !== null) {
      entries.push(['sort', filters.sort]);
    }

    if (filters.sortDirection !== undefined && filters.sortDirection !== null) {
      entries.push(['sortDirection', filters.sortDirection]);
    }

    return entries;
  }

  /**
   * Serialize value for cache key
   *
   * Converts values to strings for use in cache keys.
   * Handles strings, numbers, booleans, and arrays.
   *
   * @param value - Value to serialize
   * @returns Serialized string
   */
  private serializeValue(value: any): string {
    if (value === null || value === undefined) {
      return '';
    }

    if (Array.isArray(value)) {
      return value.map((v) => String(v)).join(',');
    }

    if (typeof value === 'object') {
      return JSON.stringify(value);
    }

    return String(value);
  }

  /**
   * Get cache key prefix for automobile domain
   *
   * @returns Cache key prefix
   */
  getPrefix(): string {
    return this.PREFIX;
  }
}
```

Step 503.3: Update the Barrel File

Update `src/app/domain-config/automobile/adapters/index.ts`:

```
// src/app/domain-config/automobile/adapters/index.ts
```

```
// VERSION 3 (Section 503) - Complete adapters

export * from './automobile-url-mapper';
export * from './automobile-api.adapter';
export * from './automobile-cache-key-builder';
```

Verification

1. Check Files Exist

```
$ ls -la src/app/domain-config/automobile/adapters/
```

Expected output:

```
total 20
drwxr-xr-x 2 user user 4096 Feb  9 12:00 .
drwxr-xr-x 3 user user 4096 Feb  9 12:00 ..
-rw-r--r-- 1 user user 4567 Feb  9 12:00 automobile-api.adapter.ts
-rw-r--r-- 1 user user 2345 Feb  9 12:00 automobile-cache-key-builder.ts
-rw-r--r-- 1 user user 6789 Feb  9 12:00 automobile-url-mapper.ts
-rw-r--r-- 1 user user  200 Feb  9 12:00 index.ts
```

2. TypeScript Compilation Check

```
$ npx tsc --noEmit src/app/domain-config/automobile/adapters/index.ts
```

3. Test API Adapter

```
import { AutomobileApiAdapter } from '@app/domain-config/automobile/adapters';
import { ApiService } from '@app/framework/services';
import { AutoSearchFilters } from '@app/domain-config/automobile/models';

// Create adapter (normally done in domain config factory)
const adapter = new AutomobileApiAdapter(apiService, 'http://localhost:3000/api/v1');

// Test fetch
const filters = new AutoSearchFilters({
  manufacturer: 'Toyota',
  yearMin: 2020,
  page: 1,
  size: 20
});

adapter.fetchData(filters).subscribe(response => {
  console.log('Results:', response.results.length);
  console.log('Total:', response.total);
```

```
  console.log('First vehicle:', response.results[0]?.getDisplayName());
});
```

4. Test Cache Key Builder

```
import { AutomobileCacheKeyBuilder } from '@app/domain-config/automobile/adapters';
import { AutoSearchFilters } from '@app/domain-config/automobile/models';

const builder = new AutomobileCacheKeyBuilder();

// Test basic key
const key1 = builder.buildKey(new AutoSearchFilters({
  manufacturer: 'Toyota',
  page: 1
}));
console.log('Key 1:', key1);
// → 'auto:manufacturer=Toyota:page=1'

// Test with highlights
const key2 = builder.buildKey(
  new AutoSearchFilters({ manufacturer: 'Toyota' }),
  { yearMin: 2020, yearMax: 2024 }
);
console.log('Key 2:', key2);
// → 'auto:h_yearMax=2024:h_yearMin=2020:manufacturer=Toyota'

// Test determinism (same filters = same key)
const key3 = builder.buildKey(new AutoSearchFilters({
  manufacturer: 'Toyota',
  page: 1
}));
console.log('Keys match:', key1 === key3);
// → true
```

Phase 6: Domain Configs

Chapter 601: Filter Definitions Config

Chapter 602: Table Config

Chapter 603: Picker Configs

Chapter 604: Query Control Filters Config

Chapter 605: Highlight Filters Config

Chapter 606: Chart Configs

Chapter 607: Domain Config Assembly

Chapter 608: Domain Provider Registration

Chapter 601

Filter Definitions Config

```
// src/app/domain-config/automobile/configs/automobile.filter-definitions.ts

/**
 * Automobile Domain - Filter Definitions
 *
 * Defines query control filters for the automobile discovery interface.
 * These are UI filters that users can interact with to refine their search.
 *
 * Domain: Automobile Discovery
 */

import { FilterDefinition } from '../../../framework/models/domain-config.interface';

/**
 * Automobile filter definitions
 *
 * Array of filter controls for the query panel.
 * Users can combine these filters to search for specific vehicles.
 *
 * @example
 * ```typescript
 * <div class="filter-panel">
 *   <app-filter-control
 *     *ngFor="let filter of AUTOMOBILE_FILTER_DEFINITIONS"
 *     [definition]="filter"
 *     [(value)]="filterValues[filter.id]">
 *   </app-filter-control>
 * </div>
 * ```
 */
export const AUTOMOBILE_FILTER_DEFINITIONS: FilterDefinition[] = [
  /**
   * Manufacturer filter
   */
  {
    id: 'manufacturer',
    label: 'Manufacturer',
    type: 'autocomplete',
    placeholder: 'Enter manufacturer name...',
    autocompleteEndpoint: 'filters/manufacturers',
    autocompleteMinChars: 1,
    operators: ['contains', 'equals', 'startsWith'],
    defaultOperator: 'contains',
    validation: {
      minLength: 1,
      maxLength: 100
    }
  },

  /**
   * Model filter
```

```
 *
 * Uses autocomplete with progressive refinement:
 * - User types 2+ characters
 * - Backend returns top 10 matching models
 * - Results narrow as user types more
 */
{
  id: 'model',
  label: 'Model',
  type: 'autocomplete',
  placeholder: 'Type to search models...',
  autocompleteEndpoint: 'filters/models',
  autocompleteMinChars: 1,
  operators: ['contains', 'equals', 'startsWith'],
  defaultOperator: 'contains',
  validation: {
    minLength: 1,
    maxLength: 100
  }
},

/**
 * Year range filter
 *
 * Uses format.number.useGrouping: false to prevent thousand separators
 * (displays "1980" instead of "1,980")
 *
 * Note: id is 'year' so Query Panel looks for 'yearMin'/'yearMax' in currentFilters,
 * which matches the actual filter model field names (AutoSearchFilters.yearMin/yearMax)
 */
{
  id: 'year',
  label: 'Year Range',
  type: 'range',
  min: 1900,
  max: new Date().getFullYear() + 1, // Include next year for upcoming models
  step: 1,
  format: {
    number: {
      useGrouping: false, // No commas in years
      minimumFractionDigits: 0,
      maximumFractionDigits: 0
    }
  }
},

/**
 * Body class filter (multi-select)
 *
 * Uses optionsEndpoint to load options dynamically from the backend.
 * This ensures all body classes in the data are available as options.
 * Allows selecting multiple body classes with checkboxes.
 */
{
  id: 'bodyClass',
  label: 'Body Class',
  type: 'multiselect',
  placeholder: 'Select body classes...',
  format: {
```

```
      caseSensitive: false, // Match "Coupe", "coupe", "COUPE" equally
      transform: 'titlecase' // Normalize to "Coupe" format
    },
    optionsEndpoint: 'body_class' // Loads from /api/specs/v1/agg/body_class
  },

  /**
   * Instance count range filter
   *
   * Uses format.number.useGrouping: true to show thousand separators
   * (displays "1,000" instead of "1000")
   */
  {
    id: 'instanceCountRange',
    label: 'VIN Count Range',
    type: 'range',
    min: 0,
    max: 10000,
    step: 1,
    format: {
      number: {
        useGrouping: true, // Show commas for VIN counts
        minimumFractionDigits: 0,
        maximumFractionDigits: 0
      }
    }
  },

  /**
   * Global search filter
   */
  {
    id: 'search',
    label: 'Search',
    type: 'text',
    placeholder: 'Search manufacturer, model, or body class...',
    operators: ['contains'],
    defaultOperator: 'contains',
    validation: {
      minLength: 1,
      maxLength: 200
    }
  }
];

/**
 * Quick filter presets
 *
 * Predefined filter combinations for common searches
 */
export const AUTOMOBILE_QUICK_FILTERS = {
  /**
   * Recent vehicles (last 5 years)
   */
  recent: {
    label: 'Recent Vehicles',
    filters: {
      yearMin: new Date().getFullYear() - 5,
      yearMax: new Date().getFullYear()
```

```
    }
  },

  /**
   * Popular vehicles (high instance count)
   */
  popular: {
    label: 'Popular Vehicles',
    filters: {
      instanceCountMin: 100
    }
  },

  /**
   * Classic vehicles (pre-2000)
   */
  classic: {
    label: 'Classic Vehicles',
    filters: {
      yearMax: 2000
    }
  },

  /**
   * SUVs only
   */
  suvs: {
    label: 'SUVs',
    filters: {
      bodyClass: 'SUV'
    }
  },

  /**
   * Trucks only
   */
  trucks: {
    label: 'Trucks',
    filters: {
      bodyClass: 'Truck'
    }
  },

  /**
   * Sedans only
   */
  sedans: {
    label: 'Sedans',
    filters: {
      bodyClass: 'Sedan'
    }
  }
};

/**
 * Filter groups
 *
 * Organize filters into logical groups for better UX
 */
```

```
export const AUTOMOBILE_FILTER_GROUPS = {
  /**
   * Vehicle identification filters
   */
  identification: {
    label: 'Vehicle Identification',
    filters: ['manufacturer', 'model', 'bodyClass']
  },

  /**
   * Time-based filters
   */
  temporal: {
    label: 'Year',
    filters: ['year']
  },

  /**
   * Quantity filters
   */
  quantity: {
    label: 'VIN Count',
    filters: ['instanceCountRange']
  },

  /**
   * General search
   */
  general: {
    label: 'General Search',
    filters: ['search']
  }
};

/**
 * Filter validation rules
 *
 * Additional validation beyond basic type validation
 */
export const AUTOMOBILE_FILTER_VALIDATION = {
  /**
   * Validate year range
   */
  yearRange: (min: number, max: number): boolean => {
    if (min && max && min > max) {
      return false; // Min cannot be greater than max
    }
    const currentYear = new Date().getFullYear();
    if (min && (min < 1900 || min > currentYear + 1)) {
      return false; // Year out of valid range
    }
    if (max && (max < 1900 || max > currentYear + 1)) {
      return false; // Year out of valid range
    }
    return true;
  },

  /**
   * Validate instance count range
```

```
   */
  instanceCountRange: (min: number, max: number): boolean => {
    if (min && max && min > max) {
      return false; // Min cannot be greater than max
    }
    if (min && min < 0) {
      return false; // Cannot be negative
    }
    if (max && max < 0) {
      return false; // Cannot be negative
    }
    return true;
  }
};
```

Chapter 602

Table Config

602: Table Config

Status: Planning

Depends On: 402-automobile-data-model, 204-table-config-interface

Blocks: 607-domain-config-assembly

Learning Objectives

After completing this section, you will:

- Understand how table configuration separates data structure from presentation
- Know how to define columns with sorting, filtering, and width properties
- Recognize the pattern of lazy loading for large datasets

Objective

Create the automobile table configuration that defines how vehicle results are displayed in the data table. This configuration controls columns, pagination, sorting, row expansion, and state persistence.

Why

Data tables are one of the most common UI patterns in enterprise applications. Without configuration, you might hard-code table structure:

```
<!-- Hard-coded table (anti-pattern) -->
<table>
```

```
    <tr>
      <th>Manufacturer</th>
      <th>Model</th>
      <th>Year</th>
    </tr>
    <tr *ngFor="let v of vehicles">
      <td>{{ v.manufacturer }}</td>
      <td>{{ v.model }}</td>
      <td>{{ v.year }}</td>
    </tr>
  </table>
```

This approach has problems:

1. **No reusability** — Every domain needs its own table template
2. **No flexibility** — Adding columns requires template changes
3. **Missing features** — Sorting, filtering, pagination require custom code

The configuration-driven approach:

```
<app-results-table
  [config]="AUTOMOBILE_TABLE_CONFIG"
  [data]="vehicles">
</app-results-table>
```

One generic component, unlimited configurations.

Angular Style Guide References

- Style 05-14: Put logic in services, not components
- Configuration objects follow this principle by moving table structure to data

---

What

Step 602.1: Create the Table Configuration File

Create the file that will define the automobile results table.

Create `src/app/domain-config/automobile/configs/automobile.table-config.ts`:

```
// src/app/domain-config/automobile/configs/automobile.table-config.ts

/**
 * Automobile Domain - Table Configuration
 *
 * Defines the main data table for displaying vehicle results.
```

```
 * Configures columns, pagination, sorting, filtering, and row expansion.
 *
 * Domain: Automobile Discovery
 */

import { TableConfig } from '../../../framework/models/table-config.interface';
import { VehicleResult } from '../models/automobile.data';

/**
 * Automobile table configuration
 *
 * Main table for displaying vehicle search results.
 * Supports pagination, sorting, filtering, and row expansion for VIN details.
 *
 * @example
 * ```typescript
 * <p-table
 *   [value]="vehicles"
 *   [columns]="AUTOMOBILE_TABLE_CONFIG.columns"
 *   [dataKey]="AUTOMOBILE_TABLE_CONFIG.dataKey"
 *   [stateStorage]="AUTOMOBILE_TABLE_CONFIG.stateStorage"
 *   [stateKey]="AUTOMOBILE_TABLE_CONFIG.stateKey">
 * </p-table>
 * ```
 */
export const AUTOMOBILE_TABLE_CONFIG: TableConfig<VehicleResult> = {
  /**
   * Unique table identifier
   */
  tableId: 'automobile-vehicles-table',

  /**
   * State persistence key
   * Saves column widths, order, visibility, filters, sorting, pagination
   */
  stateKey: 'auto-vehicles-state',

  /**
   * Data key field (must be unique per row)
   * Used for row expansion, selection, and state management
   */
  dataKey: 'vehicle_id',

  /**
   * Table columns configuration
   */
  columns: [
    {
      field: 'manufacturer',
      header: 'Manufacturer',
      sortable: true,
      filterable: true,
      filterMatchMode: 'contains',
      reorderable: true,
      width: '150px'
    },
    {
      field: 'model',
      header: 'Model',
```

```
    sortable: true,
    filterable: true,
    filterMatchMode: 'contains',
    reorderable: true,
    width: '150px'
  },
  {
    field: 'year',
    header: 'Year',
    sortable: true,
    filterable: true,
    filterMatchMode: 'equals',
    reorderable: true,
    width: '100px'
  },
  {
    field: 'body_class',
    header: 'Body Class',
    sortable: true,
    filterable: true,
    filterMatchMode: 'contains',
    reorderable: true,
    width: '120px'
  },
  {
    field: 'instance_count',
    header: 'VIN Count',
    sortable: true,
    filterable: false,
    reorderable: true,
    width: '100px'
  }
],

/**
 * Row expansion enabled
 * Clicking expand button shows VIN instances for the vehicle
 */
expandable: true,

/**
 * Row selection disabled
 * Enable if you need multi-select functionality
 */
selectable: false,

/**
 * Selection mode (if selectable=true)
 */
selectionMode: undefined,

/**
 * Pagination enabled
 */
paginator: true,

/**
 * Default rows per page
 */
```

```
  rows: 20,

  /**
   * Rows per page options
   */
  rowsPerPageOptions: [10, 20, 50, 100],

  /**
   * Lazy loading enabled
   * Data fetched on demand (pagination, sorting, filtering)
   */
  lazy: true,

  /**
   * State persistence
   * Saves table state to localStorage
   */
  stateStorage: 'local',

  /**
   * Table style class
   * PrimeNG classes for styling
   */
  styleClass: 'p-datatable-striped p-datatable-gridlines',

  /**
   * Responsive layout
   */
  responsiveLayout: 'scroll',

  /**
   * Show grid lines
   */
  gridlines: true,

  /**
   * Striped rows
   */
  stripedRows: true,

  /**
   * Loading indicator
   */
  loading: false
};

/**
 * Column visibility presets
 *
 * Predefined column visibility configurations for different use cases
 */
export const AUTOMOBILE_TABLE_COLUMN_PRESETS = {
  /**
   * All columns visible
   */
  all: AUTOMOBILE_TABLE_CONFIG.columns,

  /**
   * Minimal view (core fields only)
```

```
   */
  minimal: AUTOMOBILE_TABLE_CONFIG.columns.filter((col) =>
    ['manufacturer', 'model', 'year', 'instance_count'].includes(col.field)
  ),

  /**
   * Summary view (no VIN count)
   */
  summary: AUTOMOBILE_TABLE_CONFIG.columns.filter(
    (col) => col.field !== 'instance_count'
  )
};

/**
 * Default sort configuration
 */
export const AUTOMOBILE_TABLE_DEFAULT_SORT = {
  field: 'manufacturer',
  order: 1 // 1 = ascending, -1 = descending
};

/**
 * Export format configurations
 *
 * Define which columns to include in exports
 */
export const AUTOMOBILE_TABLE_EXPORT_CONFIG = {
  /**
   * CSV export column configuration
   */
  csv: {
    columns: [
      { field: 'manufacturer', header: 'Manufacturer' },
      { field: 'model', header: 'Model' },
      { field: 'year', header: 'Year' },
      { field: 'body_class', header: 'Body Class' },
      { field: 'instance_count', header: 'VIN Count' }
    ],
    filename: 'automobile-vehicles'
  },

  /**
   * Excel export column configuration
   */
  excel: {
    columns: [
      { field: 'vehicle_id', header: 'Vehicle ID' },
      { field: 'manufacturer', header: 'Manufacturer' },
      { field: 'model', header: 'Model' },
      { field: 'year', header: 'Year' },
      { field: 'body_class', header: 'Body Class' },
      { field: 'instance_count', header: 'VIN Count' },
      { field: 'first_seen', header: 'First Seen' },
      { field: 'last_seen', header: 'Last Seen' }
    ],
    filename: 'automobile-vehicles',
    sheetName: 'Vehicles'
  }
};
```

Step 602.2: Understand the Column Configuration

Each column definition controls how that column behaves:

Property	Type	Description
`field`	`string`	Property name in data object (e.g., 'manufacturer')
`header`	`string`	Display text in column header (e.g., 'Manufacturer')
`sortable`	`boolean`	Enable column sorting
`filterable`	`boolean`	Enable column filtering
`filterMatchMode`	`string`	Filter comparison: 'contains', 'equals', 'startsWith'
`reorderable`	`boolean`	Allow drag-and-drop column reordering
`width`	`string`	Column width (CSS units)

Field Mapping:

The `field` property maps to the `VehicleResult` interface:

```
interface VehicleResult {
  vehicle_id: string;     // field: 'vehicle_id'
  manufacturer: string;   // field: 'manufacturer'
  model: string;          // field: 'model'
  year: number;           // field: 'year'
  body_class: string;     // field: 'body_class'
  instance_count: number; // field: 'instance_count'
}
```

Step 602.3: Understanding Lazy Loading

The `lazy: true` setting is critical for performance with large datasets:

```
■■■
```

```
■ 3. Table renders first 20                                  ■
■ 4. Slow initial load, high memory usage                    ■
■■■
```

```
■■■
```

```
$ grep "^export const" src/app/domain-config/automobile/configs/automobile.table-config.ts | wc -l
```

Expected: `4` (AUTOMOBILE_TABLE_CONFIG, AUTOMOBILE_TABLE_COLUMN_PRESETS, AUTOMOBILE_TABLE_DEFAULT_SORT, AUTOMOBILE_TABLE_EXPORT_CONFIG)

Common Problems

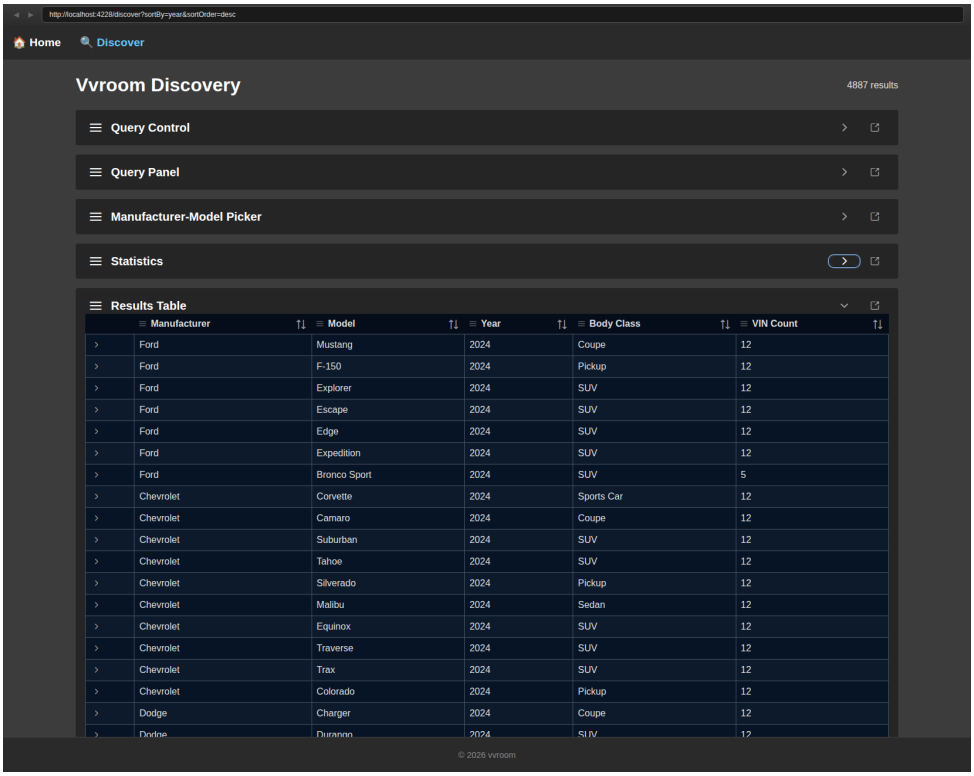
Symptom	Cause	Solution
"Cannot find module '../models/automobile.data'"	Data model not yet created	Ensure Phase 4 (document 402) is complete
"Property 'vehicle_id' does not exist on type 'VehicleResult'"	Field name mismatch	Verify field names match VehicleResult interface
Table shows "No records found"	Lazy loading not wired up	Ensure component calls API on onLazyLoad events
Column widths not persisting	State key conflict	Use unique stateKey per table
Sort not working	Missing sortable: true	Add sortable: true to column definition

---

Key Takeaways

1. **Table configuration is data** — Column definitions, pagination, and sorting are all configuration objects
2. **Lazy loading is essential for large datasets** — Only fetch what the user can see
3. **State persistence provides personalization** — Users don't lose their column preferences on reload

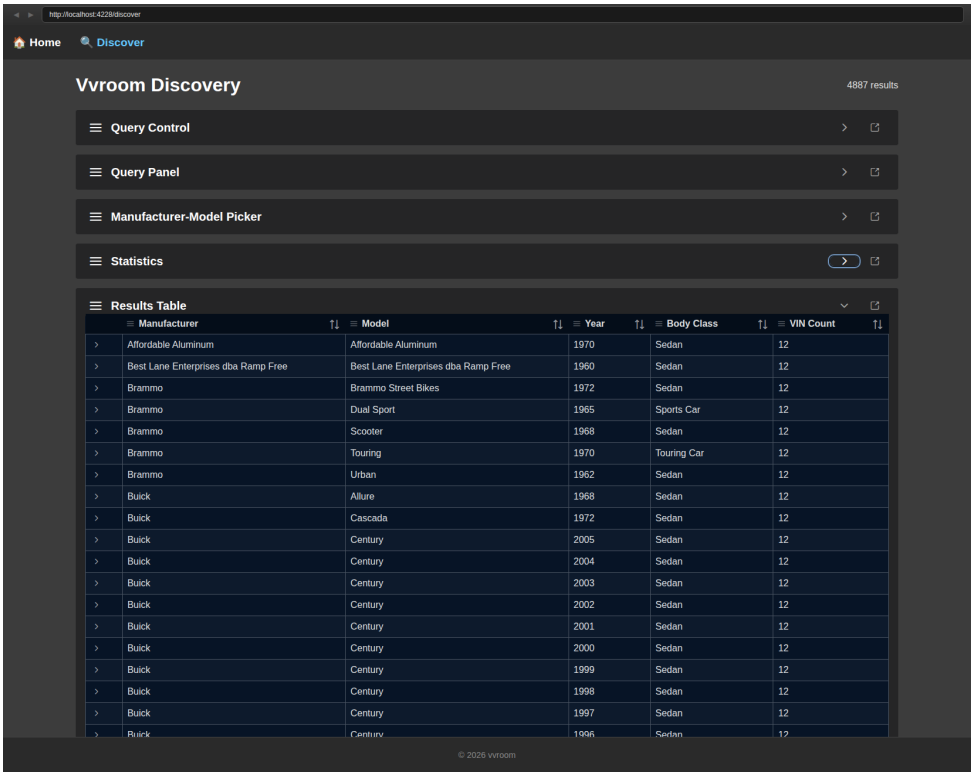
Figure 602.1 shows the results table with the configured columns displaying real vehicle data:



Results table showing vehicle data with sorting

Figure 602.1: Results table using AUTOMOBILE_TABLE_CONFIG. The table displays manufacturer, model, year, body class, and instance count columns. The year column is sorted descending, showing 2008 vehicles first. The sort indicator arrow confirms the active sort.

Figure 602.2 demonstrates pagination in action:



Results table with pagination showing page navigation

Figure 602.2: Pagination controls at the bottom of the table. Users can navigate between pages or change the number of rows per page (10, 20, 50). The current page and total pages are displayed for context.

Acceptance Criteria

- [ ] `src/app/domain-config/automobile/configs/automobile.table-config.ts` exists
- [ ] `AUTOMOBILE_TABLE_CONFIG` has 5 column definitions (manufacturer, model, year, body_class, instance_count)
- [ ] `lazy: true` is set for lazy loading
- [ ] `stateStorage: 'local'` and `stateKey` are configured for state persistence
- [ ] `expandable: true` and `dataKey: 'vehicle_id'` enable row expansion
- [ ] `AUTOMOBILE_TABLE_COLUMN_PRESETS` provides 3 presets (all, minimal, summary)
- [ ] `AUTOMOBILE_TABLE_EXPORT_CONFIG` defines CSV and Excel export formats
- [ ] File compiles without TypeScript errors

Next Step

Proceed to `603-picker-configs.md` to define the automobile picker configurations for selecting manufacturer-model combinations.

Chapter 603

Picker Configs

```
// src/app/domain-config/automobile/configs/automobile.picker-configs.ts

/**
 * Automobile Domain - Picker Configurations
 *
 * Defines picker components for selecting related data.
 * Pickers are searchable tables with multi-select capabilities.
 *
 * Domain: Automobile Discovery
 */

import { PickerConfig } from '../../../framework/models/picker-config.interface';
import { Injector } from '@angular/core';
import { ApiService } from '../../../framework/services';
import { environment } from '../../../../environments/environment';

/**
 * Manufacturer/Model row data type
 *
 * Represents a manufacturer-model combination with count
 */
export interface ManufacturerModelRow {
  /**
   * Manufacturer name (e.g., "Toyota", "Honda", "BMW")
   */
  manufacturer: string;

  /**
   * Vehicle model name (e.g., "Camry", "Civic", "X5")
   */
  model: string;

  /**
   * Number of vehicles matching this manufacturer-model combination
   */
  count: number;
}

/**
 * Create Manufacturer-Model Picker Configuration
 *
 * Factory function that creates picker config with injected dependencies.
 * Allows selection of manufacturer-model combinations for filtering.
 *
 * @param apiService - Injected API service
 * @returns Configured picker
 */
export function createManufacturerModelPickerConfig(
  apiService: ApiService
): PickerConfig<ManufacturerModelRow> {
  return {
```

```
id: 'manufacturer-model-picker',
displayName: 'Select Manufacturer & Model',

// Column definitions (PrimeNGColumn format)
columns: [
  {
    field: 'manufacturer',
    header: 'Manufacturer',
    sortable: true,
    filterable: true,
    width: '40%'
  },
  {
    field: 'model',
    header: 'Model',
    sortable: true,
    filterable: true,
    width: '40%'
  },
  {
    field: 'count',
    header: 'Count',
    sortable: true,
    filterable: false,
    width: '20%'
  }
],

// API configuration
// Server-side pagination: API returns { data, total, page, size, totalPages }
api: {
  fetchData: (params) => {
    const endpoint = `${environment.apiBaseUrl}/manufacturer-model-combinations`;
    return apiService.get<any>(endpoint, {
      params: {
        page: params.page + 1, // API is 1-indexed, picker is 0-indexed
        size: params.size,
        search: params.search || undefined,
        sortBy: params.sortField || 'manufacturer',
        sortOrder: params.sortOrder === -1 ? 'desc' : 'asc'
      }
    });
  },

  responseTransformer: (response) => {
    return {
      results: response.data || [],
      total: response.total || 0,
      page: response.page,
      size: response.size,
      totalPages: response.totalPages
    };
  }
},

// Row key configuration
row: {
  keyGenerator: (row) => `${row.manufacturer}:${row.model}`,
  keyParser: (key) => {
```

```
        const [manufacturer, model] = key.split(':');
        return { manufacturer, model } as Partial<ManufacturerModelRow>;
      }
    },

    // Selection configuration
    selection: {
      mode: 'multiple',
      urlParam: 'modelCombos',

      // Serialize selected items to URL
      serializer: (items) => {
        if (!items || items.length === 0) return '';
        return items.map(item => `${item.manufacturer}:${item.model}`).join(',');
      },

      // Deserialize URL to partial items (for hydration)
      deserializer: (urlValue) => {
        if (!urlValue) return [];
        return urlValue.split(',').map(combo => {
          const [manufacturer, model] = combo.split(':');
          return { manufacturer, model } as Partial<ManufacturerModelRow>;
        });
      },

      // Optional: Custom key generator (defaults to row.keyGenerator)
      keyGenerator: (item) => `${item.manufacturer}:${item.model}`
    },

    // Pagination configuration
    // Server-side pagination: fetches only current page from API
    pagination: {
      mode: 'server',
      defaultPageSize: 20,
      pageSizeOptions: [10, 20, 50, 100]
    },

    // Search configuration
    showSearch: true,
    searchPlaceholder: 'Search manufacturer or model...'
  };
}

/**
 * Register all automobile picker configurations
 *
 * @param injector - Angular injector for dependency resolution
 * @returns Array of picker configurations
 */
export function createAutomobilePickerConfigs(injector: Injector): PickerConfig<any>[] {
  const apiService = injector.get(ApiService);

  return [
    createManufacturerModelPickerConfig(apiService)
    // Add more pickers here as needed
  ];
}

/**
```

```
 * Static export for backwards compatibility
 * Populated dynamically by domain config factory
 */
export const AUTOMOBILE_PICKER_CONFIGS: PickerConfig<any>[] = [];
```

Chapter 604

Query Control Filters Config

```
// src/app/domain-config/automobile/configs/automobile.query-control-filters.ts

/**
 * Automobile Domain - Query Control Filter Definitions
 *
 * Defines filter definitions for the Query Control component.
 * These filters allow users to manually add/remove/edit filters via dialogs.
 *
 * Domain: Automobile Discovery
 */

import { FilterDefinition, FilterOption } from '../../../framework/models/filter-definition.interface';
import { AutoSearchFilters } from '../models/automobile.filters';
import { environment } from '../../../../environments/environment';

/**
 * Query Control filter definitions
 *
 * Each definition specifies:
 * - Which field it filters
 * - What type of UI control to display (multiselect, range, etc.)
 * - Where to fetch options from (API endpoint)
 * - How to transform API responses
 * - URL parameter names
 *
 * These are used by the QueryControlComponent to dynamically render filter dialogs.
 */
export const AUTOMOBILE_QUERY_CONTROL_FILTERS: FilterDefinition<AutoSearchFilters>[] = [
  /**
   * Manufacturer filter (Multiselect)
   */
  {
    field: 'manufacturer',
    label: 'Manufacturer',
    type: 'multiselect',
    optionsEndpoint: `${environment.apiBaseUrl}/filters/manufacturers`,
    optionsTransformer: (response: any): FilterOption[] => {
      if (response && response.manufacturers) {
        return response.manufacturers.map((m: string) => ({
          value: m,
          label: m
        }));
      }
      return [];
    },
    urlParams: 'manufacturer',
    searchPlaceholder: 'Type to search manufacturers...',
    dialogSubtitle: 'Select one or more manufacturers to filter results.'
  },

  /**
```

```
 * Model filter (Multiselect)
 */
{
  field: 'model',
  label: 'Model',
  type: 'multiselect',
  optionsEndpoint: `${environment.apiBaseUrl}/filters/models`,
  optionsTransformer: (response: any): FilterOption[] => {
    if (response && response.models) {
      return response.models.map((m: string) => ({
        value: m,
        label: m
      }));
    }
    return [];
  },
  urlParams: 'model',
  searchPlaceholder: 'Type to search models...',
  dialogSubtitle: 'Select one or more models to filter results.'
},

/**
 * Body Class filter (Multiselect)
 */
{
  field: 'bodyClass',
  label: 'Body Class',
  type: 'multiselect',
  optionsEndpoint: `${environment.apiBaseUrl}/filters/body-classes`,
  optionsTransformer: (response: any): FilterOption[] => {
    if (response && response.body_classes) {
      return response.body_classes.map((b: string) => ({
        value: b,
        label: b
      }));
    }
    return [];
  },
  urlParams: 'bodyClass',
  searchPlaceholder: 'Type to search body classes...',
  dialogSubtitle: 'Select one or more body classes to filter results.'
},

/**
 * Year Range filter (Range)
 *
 * Note: This uses yearMin as the field, but actually manages both yearMin and yearMax
 */
{
  field: 'yearMin',
  label: 'Year',
  type: 'range',
  optionsEndpoint: `${environment.apiBaseUrl}/filters/year-range`,
  urlParams: { min: 'yearMin', max: 'yearMax' },
  dialogTitle: 'Select Year Range',
  dialogSubtitle: 'Select a year range to filter results. You can select just a start year, end year, or both.'
  rangeConfig: {
    valueType: 'integer',
    minLabel: 'Start Year',
```

```
      maxLabel: 'End Year',
      minPlaceholder: 'e.g., 1980',
      maxPlaceholder: 'e.g., 2023',
      step: 1,
      useGrouping: false,
      defaultRange: { min: 1900, max: new Date().getFullYear() }
    }
  },

  /**
   * Manufacturer-Model Combinations filter (Multiselect)
   *
   * Used to display chips when selections are made via Manufacturer-Model Picker
   * Format: "Manufacturer:Model" (e.g., "Ford:F-150")
   */
  {
    field: 'modelCombos',
    label: 'Manufacturer & Model',
    type: 'multiselect',
    optionsEndpoint: `${environment.apiBaseUrl}/manufacturer-model-combinations?page=1&size=100`,
    optionsTransformer: (response: any): FilterOption[] => {
      // Response structure: { total, data: [ { manufacturer, count, models: [ { model, count } ] } ] }
      if (response && response.data) {
        const options: FilterOption[] = [];
        for (const mfr of response.data) {
          for (const modelObj of mfr.models || []) {
            options.push({
              value: `${mfr.manufacturer}:${modelObj.model}`,
              label: `${mfr.manufacturer}: ${modelObj.model}`
            });
          }
        }
        return options;
      }
      return [];
    },
    urlParams: 'modelCombos',
    searchPlaceholder: 'Type to search manufacturer-model combinations...',
    dialogSubtitle: 'Select one or more manufacturer-model combinations. Tip: Use the Manufacturer-Model Picker p
  }
];
```

Chapter 605

Highlight Filters Config

```
// src/app/domain-config/automobile/configs/automobile.highlight-filters.ts

/**
 * Automobile Domain - Highlight Filter Definitions
 *
 * Defines highlight filter definitions for the Query Control component.
 * Highlight filters allow users to add h_* parameters to segment statistics
 * in charts (showing highlighted vs other data in stacked bars).
 *
 * Domain: Automobile Discovery
 */

import { FilterDefinition, FilterOption } from '../../../framework/models/filter-definition.interface';
import { HighlightFilters } from '../models/automobile.filters';
import { environment } from '../../../../environments/environment';

/**
 * Highlight filter definitions
 *
 * Each definition specifies:
 * - Which highlight field it manages (h_manufacturer, h_modelCombos, etc.)
 * - What type of UI control to display (multiselect, range, etc.)
 * - Where to fetch options from (API endpoint)
 * - How to transform API responses
 * - URL parameter names (with h_ prefix)
 *
 * These are used by the QueryControlComponent to dynamically render highlight filter dialogs.
 *
 * @example
 * ```typescript
 * // User adds "Highlight Manufacturer: Ford" filter
 * // URL updates to: ?h_manufacturer=Ford
 * // API receives: GET /vehicles/details?h_manufacturer=Ford
 * // Returns segmented statistics: {"Ford": {"total": 665, "highlighted": 665}, ...}
 * // Charts render stacked bars showing highlighted (Ford) vs other manufacturers
 * ```
 */
export const AUTOMOBILE_HIGHLIGHT_FILTERS: FilterDefinition<HighlightFilters>[] = [
  /**
   * Highlight Manufacturer filter (Multiselect)
   * URL parameter: h_manufacturer
   */
  {
    field: 'manufacturer',
    label: 'Highlight Manufacturer',
    type: 'multiselect',
    optionsEndpoint: `${environment.apiBaseUrl}/filters/manufacturers`,
    optionsTransformer: (response: any): FilterOption[] => {
      if (response && response.manufacturers) {
        return response.manufacturers.map((m: string) => ({
          value: m,
```

```
        label: m
      }));
    }
    return [];
  },
  urlParams: 'h_manufacturer',
  searchPlaceholder: 'Type to search manufacturers...',
  dialogSubtitle: 'Select one or more manufacturers to highlight in charts.'
},

/**
 * Highlight Model Combinations filter (Multiselect)
 * URL parameter: h_modelCombos
 * Format: Manufacturer:Model,Manufacturer:Model (e.g., Ford:F-150,Toyota:Camry)
 */
{
  field: 'modelCombos',
  label: 'Highlight Models',
  type: 'multiselect',
  optionsEndpoint: `${environment.apiBaseUrl}/manufacturer-model-combinations`,
  optionsTransformer: (response: any): FilterOption[] => {
    const options: FilterOption[] = [];
    if (response && response.data) {
      // Flatten nested structure: data[].manufacturer + data[].models[].model
      for (const manufacturerGroup of response.data) {
        const manufacturer = manufacturerGroup.manufacturer;
        if (manufacturerGroup.models) {
          for (const modelObj of manufacturerGroup.models) {
            const model = modelObj.model;
            options.push({
              value: `${manufacturer}:${model}`,
              label: `${manufacturer}:${model}`
            });
          }
        }
      }
    }
    return options;
  },
  urlParams: 'h_modelCombos',
  searchPlaceholder: 'Type to search model combinations...',
  dialogSubtitle: 'Select one or more model combinations to highlight in charts.'
},

/**
 * Highlight Body Class filter (Multiselect)
 * URL parameter: h_bodyClass
 */
{
  field: 'bodyClass',
  label: 'Highlight Body Class',
  type: 'multiselect',
  optionsEndpoint: `${environment.apiBaseUrl}/filters/body-classes`,
  optionsTransformer: (response: any): FilterOption[] => {
    if (response && response.body_classes) {
      return response.body_classes.map((b: string) => ({
        value: b,
        label: b
      }));
```

```
      }
      return [];
    },
    urlParams: 'h_bodyClass',
    searchPlaceholder: 'Type to search body classes...',
    dialogSubtitle: 'Select one or more body classes to highlight in charts.'
  },

  /**
   * Highlight Year Range filter (Range)
   * URL parameters: h_yearMin, h_yearMax
   *
   * Note: This uses yearMin as the field, but actually manages both yearMin and yearMax
   */
  {
    field: 'yearMin',
    label: 'Highlight Year',
    type: 'range',
    optionsEndpoint: `${environment.apiBaseUrl}/filters/year-range`,
    urlParams: { min: 'h_yearMin', max: 'h_yearMax' },
    dialogTitle: 'Highlight Year Range',
    dialogSubtitle: 'Select a year range to highlight in charts. You can select just a start year, end year, or b
    rangeConfig: {
      valueType: 'integer',
      minLabel: 'Start Year',
      maxLabel: 'End Year',
      minPlaceholder: 'e.g., 1980',
      maxPlaceholder: 'e.g., 2023',
      step: 1,
      useGrouping: false,
      defaultRange: { min: 1900, max: new Date().getFullYear() }
    }
  }
];
```

Chapter 606

Chart Configs

606: Chart Configs

Status: Planning

Depends On: 201-domain-config-interface

Blocks: 607-domain-config-assembly, 651-manufacturer-chart-source

---

Learning Objectives

After completing this section, you will:

- Understand how chart configurations separate visualization metadata from data transformation
- Know the relationship between chart configs and chart data sources
- Recognize the pattern of declarative chart definition

Objective

Create the automobile chart configurations that define which charts appear in the statistics panel. These configurations specify chart identity, type, dimensions, and which data source transforms the statistics into chart data.

Why

Visualizations help users understand data patterns. For automobiles, we want charts showing:

1. **Manufacturer distribution** — Which manufacturers have the most vehicles?

2. **Model distribution** — Which specific models are most common?
3. **Year distribution** — How are vehicles distributed across years?
4. **Body class distribution** — What types of vehicles are in the data?

Without configuration, you might create each chart as a separate component:

```
@Component({
  template: `
    <div class="chart-container">
      <h3>Manufacturers</h3>
      <plotly-plot [data]="manufacturerData" [layout]="layout"></plotly-plot>
    </div>
  `
})
export class ManufacturerChartComponent {
  // Transform statistics to chart data
  // Handle Plotly configuration
  // Manage visibility
  // ...50+ lines of code
}
```

Four charts = four components = 200+ lines of similar code.

With configuration:

```
export const AUTOMOBILE_CHART_CONFIGS: ChartConfig[] = [
  { id: 'manufacturer-distribution', title: 'Manufacturers', dataSourceId: 'manufacturer', ... },
  { id: 'year-distribution', title: 'Year', dataSourceId: 'year', ... },
  // ...
];
```

One generic `BaseChartComponent` + configuration = minimal code.

This is the Phase 6 Aha Moment again: Configuration is declarative code. You describe what charts you want, not how to render them.

Angular Style Guide References

- Configuration objects follow the same pattern as Angular's built-in configurations (routes, providers)

What

Step 606.1: Create the Chart Configurations File

Create the file that will define automobile chart configurations.

Create `src/app/domain-config/automobile/configs/automobile.chart-configs.ts`:

```
// src/app/domain-config/automobile/configs/automobile.chart-configs.ts

/**
 * Automobile Domain - Chart Configurations (Plotly.js)
 *
 * Defines chart visualizations for automobile statistics using Plotly.js.
 * Charts display aggregated data and distributions.
 *
 * Domain: Automobile Discovery
 */

import { ChartConfig } from '../../../framework/models/domain-config.interface';

/**
 * Automobile chart configurations
 *
 * Array of chart definitions for the statistics panel.
 * Each chart visualizes a different aspect of the vehicle data.
 *
 * NOTE: These configs work with Plotly.js via BaseChartComponent.
 * Data transformation is handled by chart data sources in chart-sources/ directory.
 *
 * @example
 * ```typescript
 * <app-statistics-panel-2 [domainConfig]="domainConfig">
 * </app-statistics-panel-2>
 * ```
 */
export const AUTOMOBILE_CHART_CONFIGS: ChartConfig[] = [
  /**
   * Manufacturer distribution (vertical stacked bar chart)
   */
  {
    id: 'manufacturer-distribution',
    title: 'Manufacturers',
    type: 'bar',
    dataSourceId: 'manufacturer',
    height: 400,
    width: '100%',
    visible: true,
    collapsible: true
  },

  /**
   * Top models by VIN count (vertical stacked bar chart)
   */
  {
    id: 'top-models',
    title: 'Models',
    type: 'bar',
    dataSourceId: 'top-models',
    height: 400,
    width: '100%',
    visible: true,
    collapsible: true
  },
```

```
    /**
     * Year distribution (vertical stacked bar chart)
     */
    {
      id: 'year-distribution',
      title: 'Year',
      type: 'bar',
      dataSourceId: 'year',
      height: 400,
      width: '100%',
      visible: true,
      collapsible: true
    },

    /**
     * Body class distribution (vertical stacked bar chart)
     */
    {
      id: 'body-class-distribution',
      title: 'Body Class',
      type: 'bar',
      dataSourceId: 'body-class',
      height: 400,
      width: '100%',
      visible: true,
      collapsible: true
    }
  ];
```

Step 606.2: Understand the Chart Configuration Structure

Each chart configuration defines a visualization:

Property	Type	Description			
`id`	`string`	Unique identifier for the chart			
`title`	`string`	Display title shown above the chart			
`type`	`''bar' \`	'line' \	'pie' \	'scatter'`	Plotly chart type
`dataSourceId`	`string`	Key to look up in chartDataSources map			
`height`	`number`	Chart height in pixels			

`width`	`string`	Chart width (CSS value)			
`visible`	`boolean`	Initial visibility state			
`collapsible`	`boolean`	Can user collapse/expand the chart			

Step 606.3: Understanding the Chart/Data Source Relationship

Chart configs and chart data sources are separate concerns:

Chart Config (this file):

- Defines chart metadata (id, title, type, dimensions)
- Says "render a bar chart called 'Manufacturers'"
- Does NOT know how to transform statistics to chart data

Chart Data Source (Phase 7, documents 651-654):

- Transforms statistics into Plotly trace format
- Knows the structure of VehicleStatistics
- Handles highlight segmentation

The connection is the `dataSourceId`:

```
// Chart config (document 606)
{ id: 'manufacturer-distribution', dataSourceId: 'manufacturer', ... }

// Domain config (document 607) connects them
chartDataSources: {
  'manufacturer': new ManufacturerChartDataSource(),  // Phase 7
  // ...
}
```

Data Flow:

```
■■■■■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■
■  ChartConfig    ■     ■ ChartDataSource ■     ■ BaseChartComp   ■
■                 ■     ■                 ■     ■                 ■
■ dataSourceId:   ■■■■■■■■ transform()     ■■■■■■■■ Plotly.js       ■
■ 'manufacturer'  ■     ■ returns traces  ■     ■ render          ■
■■■■■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■     ■■■■■■■■■■■■■■■■■■■■■■
```

Step 606.4: Understanding Chart Types

All automobile charts use `type: 'bar'`, but the framework supports multiple types:

Bar Charts (used here):

```
    ■
 50 ■ ■■■■
    ■ ■■■■ ■■■■
 25 ■ ■■■■ ■■■■ ■■■■
    ■ ■■■■ ■■■■ ■■■■ ■■■■
  0 ■■■■■■■■■■■■■■■■■■■■■■■■
      Ford Toyota Honda BMW
```

Best for: Comparing categories (manufacturers, body classes)

Line Charts:

```
    ■
 50 ■         /\
    ■    /\  /  \
 25 ■   /  \/    \
    ■  /          \____
  0 ■■■■■■■■■■■■■■■■■■■■■■■■
     2018 2019 2020 2021 2022
```

Best for: Showing trends over time

Pie Charts:

```
       ■■■■■■■■■■■■■■■■
     /    Ford     \
    ■     25%       ■
    ■               ■
    \    Toyota    /
     \   20%      /
       ■■■■■■■■■■■■
```

Best for: Showing proportions of a whole

Step 606.5: Understanding Stacked Bar Charts with Highlights

When highlights are active, bar charts become stacked:

Without highlights:

```
Manufacturers
    ■
100 ■ ■■■■
    ■ ■■■■ ■■■■
```

```
  50 ■ ■■■■ ■■■■ ■■■■
     ■ ■■■■ ■■■■ ■■■■ ■■■■
   0 ■■■■■■■■■■■■■■■■■■■■■■■
       Ford Toyota Honda BMW
```

With h_manufacturer=Ford:

```
Manufacturers
     ■
 100 ■ ■■■■ (orange: highlighted)
     ■ ■■■■ ■■■■
  50 ■ ■■■■ ■■■■ ■■■■
     ■ ■■■■ ■■■■ ■■■■ ■■■■
   0 ■■■■■■■■■■■■■■■■■■■■■■■
       Ford Toyota Honda BMW

Legend: ■■■■ = Highlighted (Ford)
        ■■■■ = Other
```

The stacking is handled by the chart data source (Phase 7), not the chart config.

Step 606.6: Understanding Chart Dimensions

The `height` and `width` properties control chart sizing:

```
{
  height: 400,   // Fixed 400px height
  width: '100%'  // Responsive width (fills container)
}
```

Why fixed height, responsive width?

- **Fixed height**: Charts need consistent vertical space for readability
- **Responsive width**: Charts should adapt to container width (desktop vs mobile)

The statistics panel uses a grid layout:

```
■■■■■■■■■■■■■■■■■■■
```

```
■■■
```

3. Verify Chart Count

```
$ grep -c "id:" src/app/domain-config/automobile/configs/automobile.chart-configs.ts
```

Expected: `4` (manufacturer-distribution, top-models, year-distribution, body-class-distribution)

4. Verify Data Source IDs

```
$ grep "dataSourceId:" src/app/domain-config/automobile/configs/automobile.chart-configs.ts
```

Expected: Four dataSourceId values (manufacturer, top-models, year, body-class)

Common Problems

Symptom	Cause	Solution
"Cannot find module '../../../framework/models/domain-config.interface'"	Interface not yet created	Ensure Phase 2 (document 201) is complete
Chart not rendering	dataSourceId not found in chartDataSources	Verify dataSourceId matches key in domain config
Chart too tall/short	Incorrect height value	Adjust height to appropriate pixel value
Chart not collapsing	collapsible: false	Set collapsible: true
"ChartConfig is not a type"	Import missing	Add import for ChartConfig from domain-config.interface

---

Key Takeaways

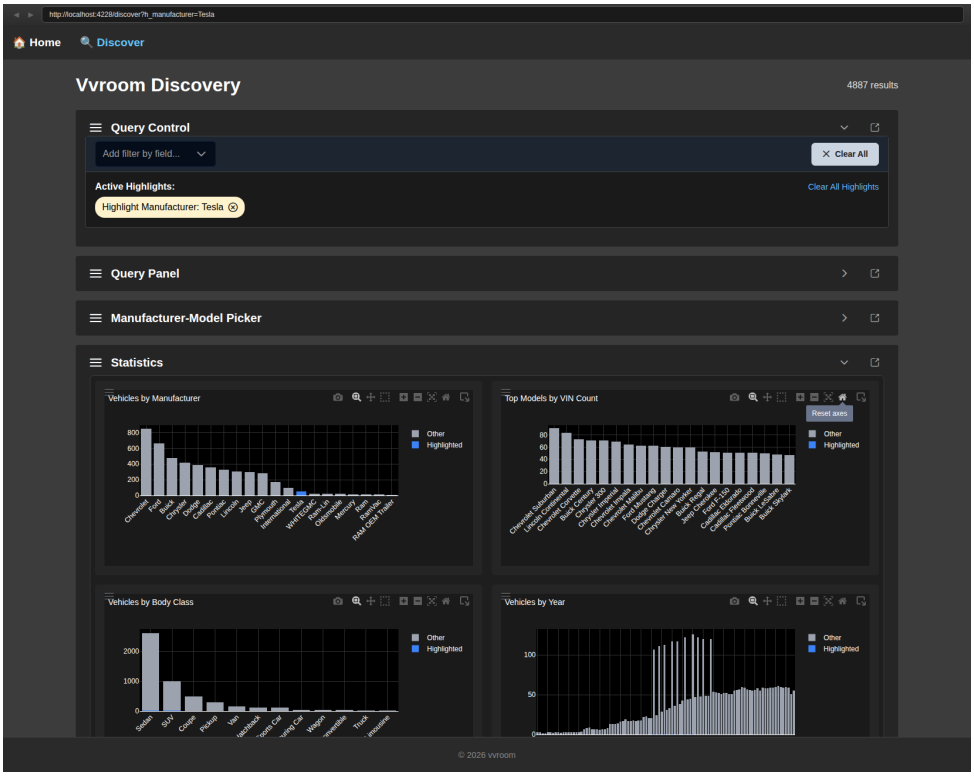
1. **Chart configs are pure metadata** — They describe what to render, not how
2. **Data sources handle transformation** — The dataSourceId links to code that transforms statistics
3. **All charts use consistent dimensions** — height: 400, width: '100%' for uniform appearance

Figure 606.1 shows the statistics panel with the configured bar charts displaying aggregated vehicle data:

Statistics panel showing bar charts for manufacturers and body classes

Figure 606.1: The statistics panel rendering all four chart configurations: manufacturer distribution, top models, year distribution, and body class distribution. Each chart uses the horizontal bar type with consistent 400px height.

Figure 606.2 demonstrates the highlight feature with Tesla vehicles highlighted:



Statistics panel with Tesla manufacturer highlighted

Figure 606.2: Highlight statistics showing Tesla selected. The segmented bars display "highlighted" (filled) vs "total" (outlined) counts. This enables visual comparison between a subset and the full dataset.

Acceptance Criteria

- [ ] `src/app/domain-config/automobile/configs/automobile.chart-configs.ts` exists
- [ ] `AUTOMOBILE_CHART_CONFIGS` array contains 4 chart definitions
- [ ] Each chart has id, title, type, dataSourceId, height, width, visible, collapsible
- [ ] All charts use type: 'bar' for consistent visualization
- [ ] dataSourceId values are: manufacturer, top-models, year, body-class
- [ ] All charts have height: 400 and width: '100%'
- [ ] All charts have visible: true and collapsible: true
- [ ] File compiles without TypeScript errors

Next Step

Proceed to `607-domain-config-assembly.md` to assemble all configurations into the complete automobile domain config.

Chapter 607

Domain Config Assembly

```
// src/app/domain-config/automobile/automobile.domain-config.ts

/**
 * Automobile Domain Configuration
 *
 * Complete domain configuration combining all models, adapters, and UI configs
 * from milestones D1-D4.
 */

import { Injector } from '@angular/core';
import { DomainConfig } from '../../framework/models';
import { ApiService } from '../../framework/services';
import { environment } from '../../../environments/environment';
import {
  AutoSearchFilters,
  VehicleResult,
  VehicleStatistics
} from './models';
import {
  AutomobileApiAdapter,
  AutomobileUrlMapper,
  AutomobileCacheKeyBuilder
} from './adapters';
import {
  AUTOMOBILE_TABLE_CONFIG,
  AUTOMOBILE_FILTER_DEFINITIONS,
  AUTOMOBILE_QUERY_CONTROL_FILTERS,
  AUTOMOBILE_HIGHLIGHT_FILTERS,
  AUTOMOBILE_CHART_CONFIGS,
  AUTOMOBILE_PICKER_CONFIGS
} from './configs';
import {
  ManufacturerChartDataSource,
  TopModelsChartDataSource,
  BodyClassChartDataSource,
  YearChartDataSource
} from './chart-sources';
import { Provider } from '@angular/core';
import { DOMAIN_CONFIG } from '../../framework/services';

/**
 * Factory function to create Automobile Domain Configuration
 *
 * This factory creates the domain configuration with properly injected dependencies.
 * Must be called with Angular's Injector to resolve service dependencies.
 *
 * @param injector - Angular injector for resolving dependencies
 * @returns Complete automobile domain configuration
 *
 * @example
 * // In app module
```

```
 * providers: [
 *   {
 *     provide: DOMAIN_CONFIG,
 *     useFactory: createAutomobileDomainConfig,
 *     deps: [Injector]
 *   }
 * ]
 */
export function createAutomobileDomainConfig(injector: Injector): DomainConfig<
  AutoSearchFilters,
  VehicleResult,
  VehicleStatistics
> {
  const apiService = injector.get(ApiService);
  const apiBaseUrl = environment.apiBaseUrl;

  return {
    // ==================== Identity ====================
    domainName: 'automobile',
    domainLabel: 'Vvroom Discovery',
    apiBaseUrl: apiBaseUrl,

    // ==================== Type Models ====================
    filterModel: AutoSearchFilters,
    dataModel: VehicleResult,
    statisticsModel: VehicleStatistics,

    // ==================== Adapters ====================
    apiAdapter: new AutomobileApiAdapter(apiService, apiBaseUrl),
    urlMapper: new AutomobileUrlMapper(),
    cacheKeyBuilder: new AutomobileCacheKeyBuilder(),

    // ==================== UI Configuration ====================
    tableConfig: AUTOMOBILE_TABLE_CONFIG,
    pickers: AUTOMOBILE_PICKER_CONFIGS,
    filters: AUTOMOBILE_FILTER_DEFINITIONS,
    queryControlFilters: AUTOMOBILE_QUERY_CONTROL_FILTERS,
    highlightFilters: AUTOMOBILE_HIGHLIGHT_FILTERS,
    charts: AUTOMOBILE_CHART_CONFIGS,
    chartDataSources: {
      'manufacturer': new ManufacturerChartDataSource(),
      'top-models': new TopModelsChartDataSource(),
      'body-class': new BodyClassChartDataSource(),
      'year': new YearChartDataSource()
    },

    // ==================== Feature Flags ====================
    features: {
      // Required features
      highlights: true,
      popOuts: true,
      rowExpansion: true,

      // Optional features
      statistics: true,
      export: true,
      columnManagement: true,
      statePersistence: true
    },
```

```
    // ==================== Metadata ====================
    metadata: {
      version: '1.0.0',
      description: 'Automobile vehicle discovery and analysis',
      author: 'Generic Discovery Framework Team',
      createdAt: '2025-11-20',
      updatedAt: '2025-11-20'
    }
  };
}

/**
 * Angular dependency injection provider for Automobile Domain Configuration
 *
 * Pre-configured provider that can be used directly in Angular module declarations
 * to register the automobile domain configuration with the dependency injection container.
 *
 * @constant {Provider} DOMAIN_PROVIDER
 * @remarks
 * This is an Angular Provider object that:
 * - Provides the DOMAIN_CONFIG injection token
 * - Uses a factory function to create the configuration instance
 * - Automatically resolves the Injector dependency
 *
 * **Usage in Module**:
 * ```typescript
 * @NgModule({
 *   providers: [DOMAIN_PROVIDER] // Add to any module
 * })
 * export class AppModule { }
 * ```
 *
 * **Internally**:
 * - provide: Points to the DOMAIN_CONFIG injection token
 * - useFactory: References createAutomobileDomainConfig function
 * - deps: Specifies that Injector should be injected into the factory
 *
 * @see createAutomobileDomainConfig - The factory function that creates the configuration
 * @see DomainConfig - The interface describing configuration structure
 * @see DOMAIN_CONFIG - The injection token this provider uses
 */
export const DOMAIN_PROVIDER: Provider = {
  provide: DOMAIN_CONFIG,
  useFactory: createAutomobileDomainConfig,
  deps: [Injector],
};
```

Chapter 608

Domain Provider Registration

608: Domain Providers

Status: Planning

Depends On: 607-domain-config-assembly

Blocks: 906-app-config

Learning Objectives

After completing this section, you will:

- Understand the purpose of a domain providers registry
- Know how to add new domains to the application
- Recognize the pattern of collecting providers from multiple sources

---

Objective

Create the domain providers file that exports an array of all domain configuration providers. This is the single point where all domains are registered for the application.

Why

The vvroom application is designed to support multiple domains. While we only have automobiles now, the architecture anticipates:

- Agriculture (crops, yields, regions)

- Chemistry (compounds, reactions)
- Physics (particles, experiments)
- Finance (stocks, portfolios)

Each domain has its own `DOMAIN_PROVIDER`. The domain providers file collects them all:

```
// Without domain providers registry
@NgModule({
  providers: [
    automobileDomainProvider,
    agricultureDomainProvider,
    chemistryDomainProvider,
    // ... manually list each one
  ]
})


// With domain providers registry
import { DOMAIN_PROVIDERS } from '../domain-config/domain-providers';

@NgModule({
  providers: [
    ...DOMAIN_PROVIDERS  // Spread all domain providers
  ]
})
```

Benefits:

1. **Single source of truth** — All domains listed in one file
2. **Easy to add domains** — Add one line to add a new domain
3. **Clear inventory** — Easy to see what domains exist

For Vvroom (Automobile Only)

Since vvroom only has automobiles, the domain providers file is simple. But the pattern prepares for future expansion.

What

Step 608.1: Create the Domain Providers File

Create the file that exports all domain providers.

Create `src/app/domain-config/domain-providers.ts`:

```
// src/app/domain-config/domain-providers.ts

import { Provider } from '@angular/core';
import { DOMAIN_PROVIDER as automobileDomainProvider } from './automobile';

/**
 * Array of all domain configuration providers.
 *
 * Each domain should export a `DOMAIN_PROVIDER` that can be added to this array.
 * This allows for dynamic registration of domains at application startup.
 *
 * @example
 * ```typescript
 * // Add a new domain
 * import { DOMAIN_PROVIDER as newDomainProvider } from './new-domain';
 *
 * export const DOMAIN_PROVIDERS: Provider[] = [
 *   automobileDomainProvider,
 *   newDomainProvider
 * ];
 * ```
 */
export const DOMAIN_PROVIDERS: Provider[] = [
  automobileDomainProvider,
];
```

Step 608.2: Understand the Provider Array Pattern

The `DOMAIN_PROVIDERS` array is a collection of Angular Provider objects:

```
// Each item is a Provider
const automobileDomainProvider: Provider = {
  provide: DOMAIN_CONFIG,
  useFactory: createAutomobileDomainConfig,
  deps: [Injector]
};

// The array collects all providers
export const DOMAIN_PROVIDERS: Provider[] = [
  automobileDomainProvider,
  // future: agricultureDomainProvider,
  // future: chemistryDomainProvider,
];
```

When used in a module:

```
@NgModule({
  providers: [
    ...DOMAIN_PROVIDERS  // Spreads all providers into the module
  ]
})
```

This is equivalent to:

```
@NgModule({
  providers: [
    {
      provide: DOMAIN_CONFIG,
      useFactory: createAutomobileDomainConfig,
      deps: [Injector]
    }
    // All other domain providers...
  ]
})
```

Step 608.3: Understanding Single vs Multi Providers

Currently, `DOMAIN_PROVIDERS` contains one provider. In a multi-domain app, you might want the `DOMAIN_CONFIG` token to provide multiple configs.

Angular supports this with `multi: true`:

```
// Single provider (current approach)
{ provide: DOMAIN_CONFIG, useFactory: createAutomobileDomainConfig, deps: [Injector] }

// Multi-provider (for multiple domains)
{ provide: DOMAIN_CONFIG, useFactory: createAutomobileDomainConfig, deps: [Injector], multi: true },
{ provide: DOMAIN_CONFIG, useFactory: createAgricultureDomainConfig, deps: [Injector], multi: true }
```

With `multi: true`, injecting `DOMAIN_CONFIG` provides an array:

```
constructor(@Inject(DOMAIN_CONFIG) private configs: DomainConfig[]) {
  // configs is an array of all registered domain configs
}
```

For vvroom, we use single providers because we only have one domain. The pattern is established for future expansion.

Step 608.4: Understanding the Import Pattern

Note the import pattern:

```
import { DOMAIN_PROVIDER as automobileDomainProvider } from './automobile';
```

Each domain exports a `DOMAIN_PROVIDER`. We rename them on import to avoid conflicts:

```
// If we had multiple domains:
import { DOMAIN_PROVIDER as automobileDomainProvider } from './automobile';
import { DOMAIN_PROVIDER as agricultureDomainProvider } from './agriculture';
import { DOMAIN_PROVIDER as chemistryDomainProvider } from './chemistry';

export const DOMAIN_PROVIDERS: Provider[] = [
  automobileDomainProvider,
  agricultureDomainProvider,
  chemistryDomainProvider,
];
```

The `as` keyword prevents naming conflicts when all domains export the same `DOMAIN_PROVIDER` name.

Step 608.5: Understanding Usage in App Config

The domain providers are used in the application configuration:

```
// src/app/app.config.ts (document 906)
import { DOMAIN_PROVIDERS } from '../domain-config/domain-providers';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(),
    ...DOMAIN_PROVIDERS,  // Register all domain configs
  ]
};
```

This makes the domain configuration available throughout the application via dependency injection.

Step 608.6: Adding a New Domain (Future)

To add a new domain in the future, you would:

1. Create the domain directory: `src/app/domain-config/new-domain/`
2. Implement models, adapters, and configs (Phases 4-6 for new domain)
3. Export `DOMAIN_PROVIDER` from `new-domain/index.ts`
4. Add to domain-providers.ts:

```
import { DOMAIN_PROVIDER as automobileDomainProvider } from './automobile';
import { DOMAIN_PROVIDER as newDomainProvider } from './new-domain';

export const DOMAIN_PROVIDERS: Provider[] = [
  automobileDomainProvider,
  newDomainProvider,  // Add new domain
];
```

5. Add routes for the new domain in `app.routes.ts`

That's it. No changes to framework code.

Verification

1. Verify File Created

```
$ ls -la src/app/domain-config/domain-providers.ts
```

Expected: File exists.

2. Verify TypeScript Compilation

```
$ cd ~/projects/vvroom
$ npx tsc --noEmit src/app/domain-config/domain-providers.ts
```

Expected: No compilation errors.

3. Verify Export

```
$ grep "^export const DOMAIN_PROVIDERS" src/app/domain-config/domain-providers.ts
```

Expected: `export const DOMAIN_PROVIDERS: Provider[] = [`

4. Verify Provider Count

```
$ grep -c "DomainProvider" src/app/domain-config/domain-providers.ts
```

Expected: `1` (automobileDomainProvider only)

Common Problems

Symptom	Cause	Solution

"Cannot find module './automobile'"	Barrel export missing	Ensure automobile/index.ts exists and exports DOMAIN_PROVIDER
"DOMAIN_PROVIDER is not exported"	Export missing from domain config	Add `export const DOMAIN_PROVIDER` to automobile.domain-config.ts
"Provider[] is not a type"	Missing import	Add `import { Provider } from '@angular/core';`
Circular dependency error	Domain imports from framework that imports domain	Check import paths; domain should only import from framework

---

Key Takeaways

1. **Domain providers registry centralizes registration** — One file lists all domain providers
2. **The pattern scales to multiple domains** — Add imports and array entries to add domains
3. **Framework code remains unchanged** — New domains only require domain-config changes

---

Acceptance Criteria

- [ ] `src/app/domain-config/domain-providers.ts` exists
- [ ] File imports `DOMAIN_PROVIDER` from automobile domain
- [ ] `DOMAIN_PROVIDERS` array is exported
- [ ] Array contains automobile domain provider
- [ ] Documentation comments explain the pattern
- [ ] Example shows how to add new domains
- [ ] File compiles without TypeScript errors

Phase 6 Checkpoint

Congratulations! You have completed Phase 6: Automobile Domain Configs.

What you created:

Document	File	Purpose
601	automobile.filter-definitions.ts	Query panel filter controls
602	automobile.table-config.ts	Results table columns and behavior
603	automobile.picker-configs.ts	Manufacturer-model picker
604	automobile.query-control-filters.ts	Add filter dialog definitions
605	automobile.highlight-filters.ts	Chart segmentation filters
606	automobile.chart-configs.ts	Statistics panel charts
607	automobile.domain-config.ts	Assembly of all domain pieces
608	domain-providers.ts	Domain registration

The Phase 6 Aha Moment:

> *"Configuration is declarative code. You describe what you want, not how to get it."*

You defined filters, tables, pickers, and charts using configuration objects. The framework components (which you'll build in Phase 8) will interpret these configurations to generate the actual UI.

What's next:

- Phase 7: Chart Data Sources (documents 651-654) — Transform statistics into Plotly trace format
- Phase 8: Framework Components (documents 801-809) — Build the generic UI components

---

Next Step

Proceed to `651-manufacturer-chart-source.md` to implement the first chart data source.

Phase 6B: Chart Data Sources

Chapter 651: Manufacturer Chart Source

Chapter 652: Year Chart Source

Chapter 653: Body Class Chart Source

Chapter 654: Top Models Chart Source

Chapter 651

Manufacturer Chart Source

651: Manufacturer Chart Source

Status: Planning

Depends On: 403-automobile-statistics-model, 606-chart-configs

Blocks: 801-base-chart-component

Learning Objectives

After completing this section, you will:

- Understand how chart data sources transform domain statistics into visualization-ready formats
- Know how to implement the abstract `ChartDataSource` class for a specific data type
- Be able to handle both simple and segmented (highlighted) statistics in chart transformations

---

Objective

Create the manufacturer chart data source that transforms vehicle statistics into a Plotly.js vertical stacked bar chart showing vehicle count by manufacturer. This chart source handles both simple counts and segmented statistics (with highlighted vs. non-highlighted data).

Why

Charts are one of the most powerful ways to communicate data insights. However, raw statistics from an API rarely match the exact format that charting libraries expect. We need a transformation layer between our domain data and our visualization library.

The Chart Data Source Pattern:

```
VehicleStatistics  →  ManufacturerChartDataSource  →  Plotly Chart Data
     (domain)              (transformer)                 (visualization)
```

This separation provides several benefits:

1. **Single Responsibility** — The chart component handles rendering; the data source handles transformation
2. **Testability** — Data transformations can be unit tested without rendering actual charts
3. **Reusability** — The same `BaseChartComponent` works with any data source
4. **Flexibility** — Changing the chart appearance only requires modifying the data source, not the component

URL-First Architecture Reference

Chart sources connect to the URL-First pattern through click handling. When a user clicks a bar in the manufacturer chart, the data source converts that click into URL parameters (e.g., `manufacturer=Toyota`), which then updates the URL and triggers a new data fetch. This creates an interactive filtering experience where charts become navigation controls.

What

Step 651.1: Create the Manufacturer Chart Source File

Create the file `src/app/domain-config/automobile/chart-sources/manufacturer-chart-source.ts`:

```
// src/app/domain-config/automobile/chart-sources/manufacturer-chart-source.ts

/**
 * Manufacturer Chart Data Source
 *
 * Transforms vehicle statistics into Plotly.js vertical stacked bar chart
 * showing vehicle count by manufacturer with highlighted vs other.
 *
 * Domain: Automobile
 */

import { ChartDataSource, ChartData } from '../../../framework/components/base-chart/base-chart.component';
import { VehicleStatistics } from '../models/automobile.statistics';

/**
 * Manufacturer distribution chart data source
 *
 * Creates a vertical stacked bar chart of manufacturers by vehicle count.
 * Matches the visual style from the reference application.
```

```
 */
export class ManufacturerChartDataSource extends ChartDataSource<VehicleStatistics> {
  /**
   * Transform statistics into Plotly chart data
   */
  transform(
    statistics: VehicleStatistics | null,
    highlights: any,
    _selectedValue: string | null,
    _containerWidth: number
  ): ChartData | null {
    if (!statistics || !statistics.byManufacturer) {
      return null;
    }

    const entries = Object.entries(statistics.byManufacturer);

    // Check if data has server-side segmented format ({total, highlighted})
    const isSegmented = entries.length > 0 &&
      typeof entries[0][1] === 'object' &&
      'total' in entries[0][1];

    let traces: Plotly.Data[] = [];

    if (isSegmented) {
      // Server-side segmented statistics: use backend data directly
      const sorted = entries
        .sort((a, b) => {
          const aTotal = (a[1] as any).total || 0;
          const bTotal = (b[1] as any).total || 0;
          return bTotal - aTotal;
        })
        .slice(0, 20);

      const manufacturers = sorted.map(([name]) => name);
      const highlightedCounts = sorted.map(([, stats]: [string, any]) => stats.highlighted || 0);
      const otherCounts = sorted.map(([, stats]: [string, any]) =>
        (stats.total || 0) - (stats.highlighted || 0)
      );

      // Create stacked bar traces (Highlighted first at bottom, then Other on top)
      traces = [
        {
          type: 'bar',
          name: 'Highlighted',
          x: manufacturers,
          y: highlightedCounts,
          marker: { color: '#3B82F6' }, // Blue
          hovertemplate: '<b>%{x}</b><br>Highlighted: %{y}<extra></extra>'
        },
        {
          type: 'bar',
          name: 'Other',
          x: manufacturers,
          y: otherCounts,
          marker: { color: '#9CA3AF' }, // Gray
          hovertemplate: '<b>%{x}</b><br>Other: %{y}<extra></extra>'
        }
      ];
```

```
  } else {
    // No highlights: simple blue bars using simple number format
    const sorted = entries
      .map(([name, count]) => [name, typeof count === 'number' ? count : 0] as [string, number])
      .sort((a, b) => b[1] - a[1])
      .slice(0, 20);

    const manufacturers = sorted.map(([name]) => name);
    const counts = sorted.map(([, count]) => count);

    traces = [{
      type: 'bar',
      x: manufacturers,
      y: counts,
      marker: { color: '#3B82F6' },
      hovertemplate: '<b>%{x}</b><br>Count: %{y}<br><extra></extra>'
    }];
  }

  // Create layout
  const layout: Partial<Plotly.Layout> = {
    barmode: isSegmented ? 'stack' : undefined,
    xaxis: {
      tickangle: -45,
      automargin: true,
      color: '#FFFFFF',
      gridcolor: '#333333'
    },
    yaxis: {
      title: { text: '' },
      gridcolor: '#333333',
      automargin: true,
      color: '#FFFFFF'
    },
    margin: {
      l: 60,
      r: 40,
      t: 40,
      b: 120
    },
    plot_bgcolor: '#000000',
    paper_bgcolor: '#1a1a1a',
    font: { color: '#FFFFFF' },
    showlegend: isSegmented
  };

  return {
    traces: traces,
    layout: layout
  };
}

/**
 * Get chart title
 */
getTitle(): string {
  return 'Vehicles by Manufacturer';
}
```

```
  /**
   * Handle chart click event
   *
   * Supports both single-click and box selection.
   * Returns comma-separated manufacturers for OR filtering.
   * Backend API supports comma-separated values.
   */
  handleClick(event: any): string | null {
    if (event.points && event.points.length > 0) {
      // Extract all manufacturer names from selected points
      const manufacturers: string[] = event.points.map((point: any) => point.x as string);

      // Remove duplicates (box selection may select both stacked bars)
      const uniqueManufacturers: string[] = [...new Set(manufacturers)];

      // Return comma-separated list (backend supports OR logic)
      return uniqueManufacturers.join(',') || null;
    }
    return null;
  }

  /**
   * Convert clicked value to URL parameters
   */
  toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
    const paramName = isHighlightMode ? 'h_manufacturer' : 'manufacturer';
    return { [paramName]: value };
  }
}
```

Step 651.2: Understanding the Transform Method

The `transform` method is the heart of any chart data source. Let's break down its logic:

Input Parameters:

Parameter	Type	Purpose	
`statistics`	`VehicleStatistics \	null`	Domain statistics from the API
`highlights`	`any`	Highlight filter state (for visual distinction)	
`_selectedValue`	`string \	null`	Currently selected value (for active state)
`_containerWidth`	`number`	Container width for responsive sizing	

Null Check:

```
if (!statistics || !statistics.byManufacturer) {
  return null;
}
```

If statistics are missing or the `byManufacturer` property is absent, return `null`. The chart component will handle this gracefully by showing an empty state.

Segmentation Detection:

```
const isSegmented = entries.length > 0 &&
  typeof entries[0][1] === 'object' &&
  'total' in entries[0][1];
```

The API can return manufacturer data in two formats:

1. **Simple format:** `{ "Toyota": 234, "Honda": 187 }` — just counts
2. **Segmented format:** `{ "Toyota": { total: 234, highlighted: 45 }, ... }` — with highlight breakdown

We detect which format we have by checking if the first value is an object with a `total` property.

Step 651.3: Understanding the Trace Structure

Plotly charts use "traces" — each trace is a data series. For the segmented case, we create two traces:

```
traces = [
  {
    type: 'bar',
    name: 'Highlighted',
    x: manufacturers,       // Category labels on X axis
    y: highlightedCounts,   // Values on Y axis
    marker: { color: '#3B82F6' },  // Blue color
    hovertemplate: '<b>%{x}</b><br>Highlighted: %{y}<extra></extra>'
  },
  {
    type: 'bar',
    name: 'Other',
    x: manufacturers,
    y: otherCounts,
    marker: { color: '#9CA3AF' },  // Gray color
    hovertemplate: '<b>%{x}</b><br>Other: %{y}<extra></extra>'
  }
];
```

Why Two Traces?

When highlight filters are active, we want to show how much of each manufacturer's data matches the highlight criteria. The blue "Highlighted" bars show matching data; the gray "Other" bars show non-matching data.

Stacking them shows the total while distinguishing the segments.

The Hover Template:

```
<b>%{x}</b><br>Highlighted: %{y}<extra></extra>
```

- `<b>%{x}</b>` — Bold manufacturer name
- `<br>` — Line break
- `Highlighted: %{y}` — Label and value
- `<extra></extra>` — Hides the trace name (which would otherwise appear)

Step 651.4: Understanding URL Parameter Mapping

The `toUrlParams` method converts a clicked chart value into URL query parameters:

```
toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
  const paramName = isHighlightMode ? 'h_manufacturer' : 'manufacturer';
  return { [paramName]: value };
}
```

Filter Mode vs Highlight Mode:

- **Filter mode** (default): Click filters the dataset → `manufacturer=Toyota`
- **Highlight mode** (h key held): Click adds a highlight → `h_manufacturer=Toyota`

This dual-mode interaction follows the URL-First pattern: user actions translate directly to URL changes.

Verification

1. TypeScript Compilation

```
$ cd ~/projects/vvroom
$ ng build
```

Expected: Build succeeds with no errors.

2. File Location

```
$ ls -la src/app/domain-config/automobile/chart-sources/
```

Expected:

```
-rw-r--r-- 1 user user  4567 Feb  9 17:00 manufacturer-chart-source.ts
```

3. Export Check

Once the chart configs are updated (document 606), verify the class is importable:

```
import { ManufacturerChartDataSource } from './chart-sources/manufacturer-chart-source';
```

Common Problems

Symptom	Cause	Solution
`Cannot find module '../../../framework/components/base-chart/base-chart.component'`	BaseChartComponent not yet created	This is expected until Phase 8 (document 801)
`Cannot find module '../models/automobile.statistics'`	VehicleStatistics not created	Complete document 403 first
`Property 'byManufacturer' does not exist on type 'VehicleStatistics'`	Missing property in statistics model	Add `byManufacturer` property to VehicleStatistics
TypeScript error on `Plotly.Data`	Plotly types not installed	Run `npm install --save-dev @types/plotly.js`

Key Takeaways

1. **Chart data sources are transformers** — They convert domain data into visualization-ready formats
2. **Handle both simple and segmented data** — Statistics can come in multiple formats; check dynamically
3. **URL parameters enable interactivity** — Clicks on charts become filter or highlight URL changes

---

Acceptance Criteria

- [ ] File exists at `src/app/domain-config/automobile/chart-sources/manufacturer-chart-source.ts`
- [ ] Class `ManufacturerChartDataSource` extends `ChartDataSource<VehicleStatistics>`
- [ ] `transform` method handles both simple and segmented statistics
- [ ] `getTitle` returns `'Vehicles by Manufacturer'`
- [ ] `handleClick` extracts manufacturer names from Plotly click events
- [ ] `toUrlParams` maps values to `manufacturer` or `h_manufacturer` parameters
- [ ] Code compiles without TypeScript errors (after dependencies are created)

Next Step

Proceed to `652-year-chart-source.md` to create the year distribution chart source.

Chapter 652

Year Chart Source

652: Year Chart Source

Status: Planning

Depends On: 403-automobile-statistics-model, 651-manufacturer-chart-source

Blocks: 801-base-chart-component

Learning Objectives

After completing this section, you will:

- Understand how to handle time-series data in chart transformations
- Know how to implement range selection for year-based filtering
- Be able to convert multi-value selections into URL parameters

---

Objective

Create the year chart data source that transforms vehicle statistics into a Plotly.js bar chart showing vehicle distribution over time. This chart source handles year-based data and supports both single-year clicks and range selections.

Why

Time-series data presents unique visualization and interaction challenges:

1. **Chronological ordering** — Years must be sorted ascending (oldest to newest), unlike other charts that sort by count

2. **Range selection** — Users often want to filter by a range of years, not just a single year
3. **Different URL parameter format** — Year ranges require two parameters (`yearMin` and `yearMax`) instead of one

The year chart demonstrates how a single abstract interface (`ChartDataSource`) can accommodate domain-specific requirements through concrete implementations.

Design Decision: Bar Chart for Years

While line charts are common for time series, we use a bar chart for years because:

- **Discrete values** — Vehicle model years are discrete integers, not continuous data
- **Highlight support** — Stacked bars clearly show highlighted vs. non-highlighted segments
- **Consistency** — Matches the visual style of other charts in the application

URL-First Architecture Reference

The year chart implements range selection by returning a pipe-delimited string (`2010|2020`) from `handleClick`, which `toUrlParams` then splits into separate `yearMin` and `yearMax` parameters. This demonstrates how chart sources can produce complex URL structures from simple user interactions.

What

Step 652.1: Create the Year Chart Source File

Create the file `src/app/domain-config/automobile/chart-sources/year-chart-source.ts`:

```
// src/app/domain-config/automobile/chart-sources/year-chart-source.ts

/**
 * Year Chart Data Source
 *
 * Transforms vehicle statistics into Plotly.js line chart
 * showing vehicle distribution over time.
 *
 * Domain: Automobile
 */

import { ChartDataSource, ChartData } from '../../../framework/components/base-chart/base-chart.component';
import { VehicleStatistics } from '../models/automobile.statistics';

/**
 * Year distribution chart data source
 *
```

```
 * Creates a line chart showing vehicle count by year.
 */
export class YearChartDataSource extends ChartDataSource<VehicleStatistics> {
  /**
   * Transform statistics into Plotly chart data
   */
  transform(
    statistics: VehicleStatistics | null,
    _highlights: any,
    _selectedValue: string | null,
    _containerWidth: number
  ): ChartData | null {
    if (!statistics || !statistics.byYearRange) {
      return null;
    }

    const entries = Object.entries(statistics.byYearRange);

    // Check if data has server-side segmented format ({total, highlighted})
    const isSegmented = entries.length > 0 &&
      typeof entries[0][1] === 'object' &&
      'total' in entries[0][1];

    let traces: Plotly.Data[] = [];

    if (isSegmented) {
      // Server-side segmented statistics: use backend data directly
      const sorted = entries
        .sort((a, b) => parseInt(a[0], 10) - parseInt(b[0], 10)); // Sort by year ascending

      const years = sorted.map(([year]) => year);
      const highlightedCounts = sorted.map(([, stats]: [string, any]) => stats.highlighted || 0);
      const otherCounts = sorted.map(([, stats]: [string, any]) =>
        (stats.total || 0) - (stats.highlighted || 0)
      );

      // Create stacked bar traces (Highlighted first at bottom, then Other on top)
      traces = [
        {
          type: 'bar',
          name: 'Highlighted',
          x: years,
          y: highlightedCounts,
          marker: { color: '#3B82F6' },
          hovertemplate: '<b>%{x}</b><br>Highlighted: %{y}<extra></extra>'
        },
        {
          type: 'bar',
          name: 'Other',
          x: years,
          y: otherCounts,
          marker: { color: '#9CA3AF' },
          hovertemplate: '<b>%{x}</b><br>Other: %{y}<extra></extra>'
        }
      ];
    } else {
      // No highlights: simple blue bars using simple number format
      const sorted = entries
        .map(([year, count]) => [year, typeof count === 'number' ? count : 0] as [string, number])
```

```
      .sort((a, b) => parseInt(a[0], 10) - parseInt(b[0], 10));

    const years = sorted.map(([year]) => year);
    const counts = sorted.map(([, count]) => count);

    traces = [{
      type: 'bar',
      x: years,
      y: counts,
      marker: { color: '#3B82F6' },
      hovertemplate: '<b>%{x}</b><br>Count: %{y}<br><extra></extra>'
    }];
  }

  // Create layout
  const layout: Partial<Plotly.Layout> = {
    barmode: isSegmented ? 'stack' : undefined,
    xaxis: {
      title: { text: '' },
      gridcolor: '#333333',
      type: 'category',
      color: '#FFFFFF'
    },
    yaxis: {
      title: { text: '' },
      gridcolor: '#333333',
      rangemode: 'tozero',
      automargin: true,
      color: '#FFFFFF'
    },
    margin: {
      l: 60,
      r: 40,
      t: 40,
      b: 60
    },
    plot_bgcolor: '#000000',
    paper_bgcolor: '#1a1a1a',
    font: { color: '#FFFFFF' },
    showlegend: isSegmented
  };

  return {
    traces: traces,
    layout: layout
  };
}

/**
 * Get chart title
 */
getTitle(): string {
  return 'Vehicles by Year';
}

/**
 * Handle chart click event
 *
 * Supports both single-click and box selection.
```

```
   * For box selection, returns year range as "min|max".
   */
  handleClick(event: any): string | null {
    if (event.points && event.points.length > 0) {
      // Extract all years from selected points
      const years: number[] = event.points.map((point: any) => parseInt(point.x, 10));

      // Remove duplicates (box selection may select both stacked bars)
      const uniqueYears: number[] = [...new Set(years)].sort((a, b) => a - b);

      if (uniqueYears.length === 1) {
        // Single year selected
        return uniqueYears[0].toString();
      } else {
        // Multiple years: return as range min|max
        const min = uniqueYears[0];
        const max = uniqueYears[uniqueYears.length - 1];
        return `${min}|${max}`;
      }
    }
    return null;
  }

  /**
   * Convert clicked value to URL parameters
   *
   * Handles both single year and year range (min|max format).
   */
  toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
    if (value.includes('|')) {
      // Year range: split into min/max params
      const [min, max] = value.split('|');
      return isHighlightMode
        ? { h_yearMin: min, h_yearMax: max }
        : { yearMin: min, yearMax: max };
    }
    // Single year
    const paramName = isHighlightMode ? 'h_year' : 'year';
    return { [paramName]: value };
  }
}
```

Step 652.2: Understanding Chronological Sorting

Unlike the manufacturer chart (which sorts by count descending), the year chart sorts chronologically:

```
const sorted = entries
  .sort((a, b) => parseInt(a[0], 10) - parseInt(b[0], 10)); // Sort by year ascending
```

Why ascending order?

Time flows forward. Users expect to see older years on the left and newer years on the right. This matches mental models of timelines and makes trends (growth or decline over time) immediately visible.

Step 652.3: Understanding Category Type for X-Axis

Notice the layout configuration for the x-axis:

```
xaxis: {
  title: { text: '' },
  gridcolor: '#333333',
  type: 'category',  // Treat years as categories, not numbers
  color: '#FFFFFF'
}
```

Why `type: 'category'`?

Without this setting, Plotly would treat years as continuous numbers and might:

- Add decimal ticks (2015.5)
- Create gaps for missing years
- Misalign bars with labels

By specifying `type: 'category'`, each year becomes a discrete label with evenly spaced bars.

Step 652.4: Understanding Range Selection

The `handleClick` method demonstrates sophisticated selection handling:

```
handleClick(event: any): string | null {
  if (event.points && event.points.length > 0) {
    // Extract all years from selected points
    const years: number[] = event.points.map((point: any) => parseInt(point.x, 10));

    // Remove duplicates (box selection may select both stacked bars)
    const uniqueYears: number[] = [...new Set(years)].sort((a, b) => a - b);

    if (uniqueYears.length === 1) {
      // Single year selected
      return uniqueYears[0].toString();
    } else {
      // Multiple years: return as range min|max
      const min = uniqueYears[0];
      const max = uniqueYears[uniqueYears.length - 1];
      return `${min}|${max}`;
    }
  }
  return null;
}
```

Single Click vs Box Selection:

Selection Type	Points Array	Return Value	
Single click on 2020	`[{x: '2020'}]`	`'2020'`	
Box select 2015-2020	`[{x: '2015'}, {x: '2016'}, ...]`	`'2015	2020'`

Duplicate Removal:

When a stacked bar chart has box selection, clicking might return the same year twice (once for each trace). Using `[...new Set(years)]` ensures each year appears only once.

Step 652.5: Understanding URL Parameter Conversion

The `toUrlParams` method handles both formats:

```
toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
  if (value.includes('|')) {
    // Year range: split into min/max params
    const [min, max] = value.split('|');
    return isHighlightMode
      ? { h_yearMin: min, h_yearMax: max }
      : { yearMin: min, yearMax: max };
  }
  // Single year
  const paramName = isHighlightMode ? 'h_year' : 'year';
  return { [paramName]: value };
}
```

URL Examples:

User Action	Return Value	URL Parameters	
Click 2020	`'2020'`	`?year=2020`	
Click 2020 (highlight mode)	`'2020'`	`?h_year=2020`	
Box select 2015-2020	`'2015	2020'`	`?yearMin=2015&yearMax=2020`
Box select 2015-2020 (highlight mode)	`'2015	2020'`	`?h_yearMin=2015&h_yearMax=2020`

This demonstrates how a single abstract method can produce varied URL structures based on the interaction pattern.

Verification

1. TypeScript Compilation

```
$ cd ~/projects/vvroom
$ ng build
```

Expected: Build succeeds with no errors.

2. File Location

```
$ ls -la src/app/domain-config/automobile/chart-sources/
```

Expected:

```
-rw-r--r-- 1 user user  4567 Feb  9 17:00 manufacturer-chart-source.ts
-rw-r--r-- 1 user user  4123 Feb  9 17:05 year-chart-source.ts
```

3. Sorting Verification

Mentally trace through the code with sample data:

```
// Input (unsorted):
byYearRange: { "2022": 50, "2020": 100, "2021": 75 }

// After sorting:
// years = ["2020", "2021", "2022"]
// counts = [100, 75, 50]
```

The chart will show 2020 on the left with 100 vehicles, progressing to 2022 on the right with 50.

Common Problems

Symptom	Cause	Solution	
Years appear in random order	Missing or incorrect sort	Verify `parseInt(a[0], 10) - parseInt(b[0], 10)` sorts ascending	

Decimal year labels (2020.5)	X-axis type not set to category	Add `type: 'category'` to xaxis config	
Range selection returns duplicates	Missing duplicate removal	Ensure `[...new Set(years)]` is present	
`yearMin=2020&yearMax=2020` for single click	Pipe split on non-pipe string	Check for `	` before splitting
`Cannot find module './models/automobile.statistics'`	VehicleStatistics not created	Complete document 403 first	

Key Takeaways

1. **Time data requires chronological sorting** — Unlike counts, years have an inherent order that users expect
2. **Range selections need special handling** — Convert multi-point selections into min/max pairs
3. **URL parameters can be polymorphic** — The same method produces single or multi-parameter outputs based on input

---

Acceptance Criteria

- [ ] File exists at `src/app/domain-config/automobile/chart-sources/year-chart-source.ts`
- [ ] Class `YearChartDataSource` extends `ChartDataSource<VehicleStatistics>`
- [ ] Years are sorted chronologically (ascending) in the chart
- [ ] X-axis uses category type for discrete year labels
- [ ] `handleClick` returns single year or `min|max` format for ranges
- [ ] `toUrlParams` handles both single year and range formats
- [ ] Highlight mode produces `h_year`, `h_yearMin`, `h_yearMax` parameters
- [ ] Code compiles without TypeScript errors (after dependencies are created)

Next Step

Proceed to `653-body-class-chart-source.md` to create the body class distribution chart source.

Chapter 653

Body Class Chart Source

653: Body Class Chart Source

Status: Planning

Depends On: 403-automobile-statistics-model, 652-year-chart-source

Blocks: 801-base-chart-component

Learning Objectives

After completing this section, you will:

- Understand how to maintain unused code for future flexibility (color mappings)
- Know how to implement multi-value selection for categorical data
- Recognize patterns that repeat across chart sources and opportunities for refactoring

---

Objective

Create the body class chart data source that transforms vehicle statistics into a Plotly.js bar chart showing distribution by body class (Sedan, SUV, Truck, etc.). This chart source handles categorical data and supports comma-separated multi-selection.

Why

Body class represents a categorical dimension of vehicle data. Unlike manufacturers (which can number in the dozens) or years (which span decades), body classes form a small, fixed set of categories. This affects our design choices:

1. **Color scheme preparation** — Although we currently use uniform blue bars, we define a color mapping for potential future use (e.g., distinct colors per body class)
2. **Full display** — We show all body classes rather than limiting to "top 20" since the set is naturally small
3. **Descending count sort** — Like manufacturers, we sort by count (most common body classes first)

Design Decision: Bar Chart Over Pie Chart

The original file comment mentions "pie chart," but the implementation uses a bar chart. Why?

- **Comparison accuracy** — Humans judge bar heights more accurately than pie slice angles
- **Highlight support** — Stacked bars work better than segmented pie slices for showing highlighted vs. non-highlighted data
- **Consistency** — Matches the visual style of manufacturer and year charts

URL-First Architecture Reference

Body class filtering uses comma-separated values for OR logic: `bodyClass=Sedan,SUV` matches vehicles that are either Sedans OR SUVs. This supports intuitive multi-selection where clicking multiple bars expands rather than narrows the result set.

What

Step 653.1: Create the Body Class Chart Source File

Create the file `src/app/domain-config/automobile/chart-sources/body-class-chart-source.ts`:

```
// src/app/domain-config/automobile/chart-sources/body-class-chart-source.ts

/**
 * Body Class Chart Data Source
 *
 * Transforms vehicle statistics into Plotly.js pie chart
 * showing distribution by body class.
 *
 * Domain: Automobile
 */

import { ChartDataSource, ChartData } from '../../../framework/components/base-chart/base-chart.component';
import { VehicleStatistics } from '../models/automobile.statistics';

/**
 * Body class distribution chart data source
 *
 * Creates a pie chart showing vehicle distribution by body class.
 */
```

```
export class BodyClassChartDataSource extends ChartDataSource<VehicleStatistics> {
  /**
   * Color scheme for body classes
   */
  private readonly BODY_CLASS_COLORS: Record<string, string> = {
    'Sedan': '#3B82F6',
    'SUV': '#10B981',
    'Truck': '#F59E0B',
    'Pickup': '#F59E0B',
    'Coupe': '#EF4444',
    'Wagon': '#8B5CF6',
    'Van': '#EC4899',
    'Minivan': '#06B6D4',
    'Convertible': '#84CC16',
    'Hatchback': '#F97316'
  };

  /**
   * Transform statistics into Plotly chart data
   */
  transform(
    statistics: VehicleStatistics | null,
    _highlights: any,
    _selectedValue: string | null,
    _containerWidth: number
  ): ChartData | null {
    if (!statistics || !statistics.byBodyClass) {
      return null;
    }

    const entries = Object.entries(statistics.byBodyClass);

    // Check if data has server-side segmented format ({total, highlighted})
    const isSegmented = entries.length > 0 &&
      typeof entries[0][1] === 'object' &&
      'total' in entries[0][1];

    let traces: Plotly.Data[] = [];

    if (isSegmented) {
      // Server-side segmented statistics: use backend data directly
      const sorted = entries
        .sort((a, b) => {
          const aTotal = (a[1] as any).total || 0;
          const bTotal = (b[1] as any).total || 0;
          return bTotal - aTotal;
        });

      const labels = sorted.map(([name]) => name);
      const highlightedCounts = sorted.map(([, stats]: [string, any]) => stats.highlighted || 0);
      const otherCounts = sorted.map(([, stats]: [string, any]) =>
        (stats.total || 0) - (stats.highlighted || 0)
      );

      // Create stacked bar traces (Highlighted first at bottom, then Other on top)
      traces = [
        {
          type: 'bar',
          name: 'Highlighted',
```

```
      x: labels,
      y: highlightedCounts,
      marker: { color: '#3B82F6' },
      hovertemplate: '<b>%{x}</b><br>Highlighted: %{y}<extra></extra>'
    },
    {
      type: 'bar',
      name: 'Other',
      x: labels,
      y: otherCounts,
      marker: { color: '#9CA3AF' },
      hovertemplate: '<b>%{x}</b><br>Other: %{y}<extra></extra>'
    }
  ];
} else {
  // No highlights: simple blue bars using simple number format
  const sorted = entries
    .map(([name, count]) => [name, typeof count === 'number' ? count : 0] as [string, number])
    .sort((a, b) => b[1] - a[1]);

  const labels = sorted.map(([name]) => name);
  const counts = sorted.map([, count]) => count);

  traces = [{
    type: 'bar',
    x: labels,
    y: counts,
    marker: { color: '#3B82F6' },
    hovertemplate: '<b>%{x}</b><br>Count: %{y}<br><extra></extra>'
  }];
}

// Create layout
const layout: Partial<Plotly.Layout> = {
  barmode: isSegmented ? 'stack' : undefined,
  xaxis: {
    tickangle: -45,
    automargin: true,
    color: '#FFFFFF',
    gridcolor: '#333333'
  },
  yaxis: {
    title: { text: '' },
    gridcolor: '#333333',
    automargin: true,
    color: '#FFFFFF'
  },
  margin: {
    l: 60,
    r: 40,
    t: 40,
    b: 100
  },
  plot_bgcolor: '#000000',
  paper_bgcolor: '#1a1a1a',
  font: { color: '#FFFFFF' },
  showlegend: isSegmented
};
```

```
    return {
      traces: traces,
      layout: layout
    };
  }

  /**
   * Get chart title
   */
  getTitle(): string {
    return 'Vehicles by Body Class';
  }

  /**
   * Handle chart click event
   *
   * Supports both single-click and box selection.
   * Returns comma-separated body classes for OR filtering.
   * Backend API v1.0.1+ supports comma-separated values.
   */
  handleClick(event: any): string | null {
    if (event.points && event.points.length > 0) {
      // Extract all body class names from selected points
      const bodyClasses: string[] = event.points.map((point: any) => point.x as string);

      // Remove duplicates (box selection may select both stacked bars)
      const uniqueBodyClasses: string[] = [...new Set(bodyClasses)];

      // Return comma-separated list (backend supports OR logic as of v1.0.1)
      return uniqueBodyClasses.join(',') || null;
    }
    return null;
  }

  /**
   * Convert clicked value to URL parameters
   */
  toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
    const paramName = isHighlightMode ? 'h_bodyClass' : 'bodyClass';
    return { [paramName]: value };
  }
}
```

Step 653.2: Understanding the Color Scheme

Notice the unused `BODY_CLASS_COLORS` property:

```
private readonly BODY_CLASS_COLORS: Record<string, string> = {
  'Sedan': '#3B82F6',
  'SUV': '#10B981',
  'Truck': '#F59E0B',
  'Pickup': '#F59E0B',
  'Coupe': '#EF4444',
  'Wagon': '#8B5CF6',
```

```
    'Van': '#EC4899',
    'Minivan': '#06B6D4',
    'Convertible': '#84CC16',
    'Hatchback': '#F97316'
  };
```

Why include unused code?

This is a deliberate design choice. The color mapping:

1. **Documents intent** — Shows that distinct colors per body class were considered
2. **Enables future enhancement** — Can be activated without research into appropriate colors
3. **Provides consistency** — Truck and Pickup share the same orange (#F59E0B), indicating semantic similarity

Currently, the code uses uniform blue bars for consistency with other charts. The color scheme remains available for future features like:

- Color-coded bars when highlights are not active
- Legend entries matching body class colors
- Tooltips with color indicators

Step 653.3: Comparing Chart Sources

By now, you may notice similarities between chart sources. Let's compare:

Aspect	Manufacturer	Year	Body Class
Data property	`byManufacturer`	`byYearRange`	`byBodyClass`
Sort order	Count descending	Year ascending	Count descending
Limit	Top 20	All	All
X-axis labels	Angled (-45)	Normal	Angled (-45)
Bottom margin	120px	60px	100px
URL parameter	`manufacturer`	`year/yearMin/yearMax`	`bodyClass`

Patterns to Notice:

1. **Same structure** — All three follow the same `transform` → `getTitle` → `handleClick` → `toUrlParams` pattern
2. **Same segmentation logic** — The `isSegmented` check is identical
3. **Same trace structure** — Blue for highlighted, gray for other

Why not abstract further?

You might be tempted to create a generic chart source factory. Resist this urge initially:

- **Explicit is better than implicit** — Each file is self-contained and understandable
- **Customization is easy** — Modifying one chart doesn't risk breaking others
- **Patterns emerge naturally** — After building several, refactoring opportunities become clear

The fourth chart source (Top Models) will show a more complex case that would break a simple abstraction.

Step 653.4: Understanding Bottom Margin Differences

Notice the different bottom margins:

```
// Manufacturer chart
margin: { l: 60, r: 40, t: 40, b: 120 }

// Year chart
margin: { l: 60, r: 40, t: 40, b: 60 }

// Body class chart
margin: { l: 60, r: 40, t: 40, b: 100 }
```

Why the differences?

- **Manufacturer (120px)** — Long names like "Mercedes-Benz" need more space when angled
- **Year (60px)** — 4-digit years are short; no angling needed
- **Body Class (100px)** — Medium-length names like "Convertible" need moderate space

These seemingly minor details significantly affect chart readability. When labels overlap or get cut off, users lose information.

Verification

1. TypeScript Compilation

```
$ cd ~/projects/vvroom
$ ng build
```

Expected: Build succeeds with no errors.

2. File Location

```
$ ls -la src/app/domain-config/automobile/chart-sources/
```

Expected:

```
-rw-r--r-- 1 user user  4567 Feb  9 17:00 manufacturer-chart-source.ts
-rw-r--r-- 1 user user  4123 Feb  9 17:05 year-chart-source.ts
-rw-r--r-- 1 user user  4234 Feb  9 17:10 body-class-chart-source.ts
```

3. Color Scheme Verification

Verify the color scheme covers expected body classes:

```
$ grep -c "'" src/app/domain-config/automobile/chart-sources/body-class-chart-source.ts
```

Expected: Multiple single quotes indicating the color mapping entries.

Common Problems

Symptom	Cause	Solution
Labels cut off at bottom	Margin too small	Increase b (bottom) margin value
"Pickup" and "Truck" have same color	Intentional design	These are semantically similar categories
Color scheme not applied	Code uses uniform blue	The color scheme is for future use; current implementation uses blue
Multi-selection returns duplicates	Missing `Set` deduplication	Ensure `[...new Set(bodyClasses)]` is present
`Cannot find module '../models/automobile.statistics'`	VehicleStatistics not created	Complete document 403 first

---

Key Takeaways

1. **Unused code can be intentional** — Color schemes document design decisions for future enhancement
2. **Small sets need different treatment** — No "top 20" limit needed for naturally small categories

3. **Margins matter for readability** — Adjust bottom margins based on label length and angle

---

Acceptance Criteria

- [ ] File exists at `src/app/domain-config/automobile/chart-sources/body-class-chart-source.ts`
- [ ] Class `BodyClassChartDataSource` extends `ChartDataSource<VehicleStatistics>`
- [ ] Color scheme defined for common body classes (even if not currently used)
- [ ] Body classes sorted by count descending
- [ ] `handleClick` returns comma-separated body class names
- [ ] `toUrlParams` maps values to `bodyClass` or `h_bodyClass` parameters
- [ ] Code compiles without TypeScript errors (after dependencies are created)

Next Step

Proceed to `654-top-models-chart-source.md` to create the top models chart source, which demonstrates more complex data transformation.

Chapter 654

Top Models Chart Source

654: Top Models Chart Source

Status: Planning

Depends On: 403-automobile-statistics-model, 653-body-class-chart-source

Blocks: 801-base-chart-component

Learning Objectives

After completing this section, you will:

- Understand how to transform nested data structures for visualization
- Know how to implement composite URL parameters from chart clicks
- Be able to handle data from multiple API response formats

Objective

Create the top models chart data source that transforms vehicle statistics into a Plotly.js bar chart showing the top models by VIN instance count. This chart source handles nested data structures (`modelsByManufacturer`) and produces composite URL parameters (`manufacturer:model` format).

Why

The top models chart presents unique challenges that the previous chart sources did not:

1. **Nested data structure** — Models are nested within manufacturers in the API response

2. **Composite labels** — Each bar shows "Manufacturer Model" (e.g., "Toyota Camry")
3. **Composite URL parameters** — Clicking produces `modelCombos=Toyota:Camry`, not separate parameters
4. **Two data sources** — Can use either `topModels` array or `modelsByManufacturer` object

This complexity demonstrates why we keep chart sources as separate classes rather than abstracting too aggressively. Each chart source handles its unique requirements while maintaining the same interface.

Design Decision: Combined Manufacturer-Model Format

When a user clicks on a model bar, we need to filter by both manufacturer AND model. We combine these into a single URL parameter using colon separator:

```
?modelCombos=Toyota:Camry
```

This approach:

- **Avoids ambiguity** — "Camry" alone could match multiple manufacturers if the API had duplicates
- **Supports multi-select** — `modelCombos=Toyota:Camry,Honda:Accord` selects multiple models
- **Matches API expectations** — The backend parses this format for efficient filtering

URL-First Architecture Reference

The top models chart shows how chart sources can produce complex URL structures. The `handleClick` method transforms display format ("Toyota Camry") to URL format ("Toyota:Camry"), demonstrating that chart sources aren't just data transformers but also interaction translators.

What

Step 654.1: Create the Top Models Chart Source File

Create the file `src/app/domain-config/automobile/chart-sources/top-models-chart-source.ts`:

```
// src/app/domain-config/automobile/chart-sources/top-models-chart-source.ts

/**
 * Top Models Chart Data Source
 *
 * Transforms vehicle statistics into Plotly.js horizontal bar chart
 * showing top models by VIN instance count.
 *
 * Domain: Automobile
 */
```

```
import { ChartDataSource, ChartData } from '../../../framework/components/base-chart/base-chart.component';
import { VehicleStatistics } from '../models/automobile.statistics';

/**
 * Top models chart data source
 *
 * Creates a horizontal bar chart of top models by VIN instance count.
 */
export class TopModelsChartDataSource extends ChartDataSource<VehicleStatistics> {
  /**
   * Transform statistics into Plotly chart data
   */
  transform(
    statistics: VehicleStatistics | null,
    _highlights: any,
    _selectedValue: string | null,
    _containerWidth: number
  ): ChartData | null {
    if (!statistics || !statistics.topModels || statistics.topModels.length === 0) {
      return null;
    }

    // Check if we have segmented statistics from API (with total/highlighted counts)
    const hasSegmentedStats = statistics.modelsByManufacturer &&
      Object.values(statistics.modelsByManufacturer).some(models =>
        typeof models === 'object' && Object.values(models).some(v =>
          typeof v === 'object' && 'total' in v
        )
      );

    let traces: any[] = [];

    if (hasSegmentedStats && statistics.modelsByManufacturer) {
      // Use API's segmented statistics with {total, highlighted}
      const modelEntries: Array<[string, any]> = [];

      Object.entries(statistics.modelsByManufacturer).forEach(([manufacturer, models]) => {
        Object.entries(models).forEach(([modelName, stats]) => {
          modelEntries.push([`${manufacturer} ${modelName}`, stats]);
        });
      });

      // Sort by total count descending and take top 20
      const sorted = modelEntries
        .sort((a, b) => ((b[1] as any).total || 0) - ((a[1] as any).total || 0))
        .slice(0, 20);

      const modelLabels = sorted.map(([label]) => label);
      const highlightedCounts = sorted.map(([, stats]: [string, any]) => stats.highlighted || 0);
      const otherCounts = sorted.map(([, stats]: [string, any]) =>
        (stats.total || 0) - (stats.highlighted || 0)
      );

      // Create stacked bar traces (Highlighted first at bottom, then Other on top)
      traces = [
        {
          type: 'bar',
          name: 'Highlighted',
          x: modelLabels,
```

```
      y: highlightedCounts,
      marker: { color: '#3B82F6' },
      hovertemplate: '<b>%{x}</b><br>Highlighted: %{y}<extra></extra>'
    },
    {
      type: 'bar',
      name: 'Other',
      x: modelLabels,
      y: otherCounts,
      marker: { color: '#9CA3AF' },
      hovertemplate: '<b>%{x}</b><br>Other: %{y}<extra></extra>'
    }
  ];
} else {
  // Fallback: simple blue bars using topModels
  const topModels = statistics.topModels.slice(0, 20);
  const modelLabels = topModels.map(m => `${m.manufacturer} ${m.name}`);
  const counts = topModels.map(m => m.instanceCount);

  traces = [{
    type: 'bar',
    x: modelLabels,
    y: counts,
    marker: { color: '#3B82F6' },
    hovertemplate: '<b>%{x}</b><br>Count: %{y}<br><extra></extra>'
  }];
}

// Create layout
const layout: Partial<any> = {
  barmode: hasSegmentedStats ? 'stack' : undefined,
  xaxis: {
    tickangle: -45,
    automargin: true,
    color: '#FFFFFF',
    gridcolor: '#333333'
  },
  yaxis: {
    title: '',
    gridcolor: '#333333',
    automargin: true,
    color: '#FFFFFF'
  },
  margin: {
    l: 60,
    r: 40,
    t: 40,
    b: 140
  },
  plot_bgcolor: '#000000',
  paper_bgcolor: '#1a1a1a',
  font: { color: '#FFFFFF' },
  showlegend: hasSegmentedStats
};

return {
  traces: traces,
  layout: layout
};
```

```
  }

  /**
   * Get chart title
   */
  getTitle(): string {
    return 'Top Models by VIN Count';
  }

  /**
   * Handle chart click event
   *
   * Supports both single-click and box selection.
   * For box selection, returns comma-separated list of unique models.
   * Converts label format from "Manufacturer Model" to "Manufacturer:Model".
   */
  handleClick(event: any): string | null {
    if (event.points && event.points.length > 0) {
      // Extract all model labels from selected points (format: "Manufacturer Model")
      const modelLabels: string[] = event.points.map((point: any) => point.x as string);

      // Remove duplicates (box selection may select both stacked bars)
      const uniqueLabels = [...new Set(modelLabels)];

      // Convert from "Manufacturer Model" to "Manufacturer:Model" format
      const modelCombos = uniqueLabels.map(label => {
        // Replace first space with colon (manufacturer doesn't have spaces)
        return label.replace(' ', ':');
      });

      // Return comma-separated list (or single value)
      return modelCombos.join(',');
    }
    return null;
  }

  /**
   * Convert clicked value to URL parameters
   */
  toUrlParams(value: string, isHighlightMode: boolean): Record<string, any> {
    const paramName = isHighlightMode ? 'h_modelCombos' : 'modelCombos';
    return { [paramName]: value };
  }
}
```

Step 654.2: Understanding Nested Data Transformation

The top models chart handles a more complex data structure:

```
// Statistics structure
{
  modelsByManufacturer: {
    "Toyota": {
      "Camry": { total: 234, highlighted: 45 },
```

```
      "Corolla": { total: 189, highlighted: 32 }
    },
    "Honda": {
      "Accord": { total: 156, highlighted: 28 },
      "Civic": { total: 145, highlighted: 21 }
    }
  }
}
```

The transformation flattens this nested structure:

```
Object.entries(statistics.modelsByManufacturer).forEach(([manufacturer, models]) => {
  Object.entries(models).forEach(([modelName, stats]) => {
    modelEntries.push([`${manufacturer} ${modelName}`, stats]);
  });
});
```

Result:

```
modelEntries = [
  ["Toyota Camry", { total: 234, highlighted: 45 }],
  ["Toyota Corolla", { total: 189, highlighted: 32 }],
  ["Honda Accord", { total: 156, highlighted: 28 }],
  ["Honda Civic", { total: 145, highlighted: 21 }]
]
```

This creates a flat array suitable for sorting and slicing to get the top 20.

Step 654.3: Understanding Dual Data Source Fallback

Notice the two code paths:

```
if (hasSegmentedStats && statistics.modelsByManufacturer) {
  // Use nested structure with highlight support
} else {
  // Fallback: use topModels array
  const topModels = statistics.topModels.slice(0, 20);
  const modelLabels = topModels.map(m => `${m.manufacturer} ${m.name}`);
  const counts = topModels.map(m => m.instanceCount);
}
```

Why two approaches?

The API can return model data in two formats:

1. **Segmented format** (`modelsByManufacturer`) — Has highlight counts, needs flattening
2. **Array format** (`topModels`) — Pre-sorted array, simpler to use but no highlight data

The segmented format is preferred when available (for highlight support), but the fallback ensures the chart works with simpler API responses.

Step 654.4: Understanding Label Format Conversion

The click handler converts display format to URL format:

```
// Convert from "Manufacturer Model" to "Manufacturer:Model" format
const modelCombos = uniqueLabels.map(label => {
  // Replace first space with colon (manufacturer doesn't have spaces)
  return label.replace(' ', ':');
});
```

Why this conversion?

Format	Purpose	Example
Display	Human-readable chart labels	"Toyota Camry"
URL	Machine-parseable parameter	"Toyota:Camry"

The colon separator enables the backend to split the value and query by both manufacturer AND model efficiently.

Edge Case: Multi-Word Models

Consider "Ford Mustang Mach-E":

```
"Ford Mustang Mach-E".replace(' ', ':')
// Result: "Ford:Mustang Mach-E"
```

The first space becomes a colon, preserving "Mustang Mach-E" as the model name. This works because manufacturer names in this dataset don't contain spaces.

Step 654.5: Understanding the Larger Bottom Margin

The top models chart uses the largest bottom margin:

```
margin: {
  l: 60,
  r: 40,
  t: 40,
  b: 140  // Largest of all charts
}
```

Why 140px?

Model labels are the longest of any chart:

- "Toyota Camry" — Short
- "Mercedes-Benz S-Class" — Medium
- "Lamborghini Aventador LP700-4" — Long

When angled at -45 degrees, these long labels need substantial vertical space to remain readable without overlapping or being cut off.

Step 654.6: Understanding the Segmentation Check

The segmentation check for top models is more complex:

```
const hasSegmentedStats = statistics.modelsByManufacturer &&
  Object.values(statistics.modelsByManufacturer).some(models =>
    typeof models === 'object' && Object.values(models).some(v =>
      typeof v === 'object' && 'total' in v
    )
  );
```

Why so complex?

Unlike `byManufacturer` (which is one level deep), `modelsByManufacturer` is two levels deep:

```
modelsByManufacturer → manufacturer → model → { total, highlighted }
```

We need to:

1. Check that `modelsByManufacturer` exists
2. Check that at least one manufacturer has models
3. Check that at least one model has the `{ total, highlighted }` structure

This defensive checking prevents runtime errors when data is missing or in an unexpected format.

Verification

1. TypeScript Compilation

```
$ cd ~/projects/vvroom
```

```
$ ng build
```

Expected: Build succeeds with no errors.

2. File Location

```
$ ls -la src/app/domain-config/automobile/chart-sources/
```

Expected:

```
-rw-r--r-- 1 user user  4567 Feb  9 17:00 manufacturer-chart-source.ts
-rw-r--r-- 1 user user  4123 Feb  9 17:05 year-chart-source.ts
-rw-r--r-- 1 user user  4234 Feb  9 17:10 body-class-chart-source.ts
-rw-r--r-- 1 user user  4789 Feb  9 17:15 top-models-chart-source.ts
```

3. Label Conversion Test

Mentally trace the click handler:

```
// Input from Plotly click event
event.points = [{ x: "Toyota Camry" }, { x: "Honda Accord" }]

// After processing
// uniqueLabels = ["Toyota Camry", "Honda Accord"]
// modelCombos = ["Toyota:Camry", "Honda:Accord"]
// return value = "Toyota:Camry,Honda:Accord"

// URL parameter
// ?modelCombos=Toyota:Camry,Honda:Accord
```

Common Problems

Symptom	Cause	Solution
Chart shows no data	`topModels` array empty	Verify API returns model data
Labels show "undefined undefined"	Missing manufacturer or name	Check `topModels` array structure
"Ford:Mustang:Mach-E" (two colons)	Multiple `replace()` calls	Use `replace(' ', ':')` not `replaceAll`
Segmentation not detected	Nested check failing	Verify `modelsByManufacturer` has expected structure
Labels overlap	Bottom margin too small	Increase b value in margins

`Cannot find module '../models/automobile.statistics'`	VehicleStatistics not created	Complete document 403 first

---

Key Takeaways

1. **Nested data requires flattening** — Transform hierarchical API responses into flat arrays for charting
2. **Display format differs from URL format** — Convert human-readable labels to machine-parseable parameters
3. **Fallback data sources add resilience** — Handle multiple API response formats gracefully

---

Acceptance Criteria

- [ ] File exists at `src/app/domain-config/automobile/chart-sources/top-models-chart-source.ts`
- [ ] Class `TopModelsChartDataSource` extends `ChartDataSource<VehicleStatistics>`
- [ ] Handles both `modelsByManufacturer` (segmented) and `topModels` (array) formats
- [ ] Labels show "Manufacturer Model" format (e.g., "Toyota Camry")
- [ ] `handleClick` converts to "Manufacturer:Model" format
- [ ] `toUrlParams` maps values to `modelCombos` or `h_modelCombos` parameters
- [ ] Bottom margin (140px) accommodates long model names
- [ ] Code compiles without TypeScript errors (after dependencies are created)

Phase 7 Complete

You have now completed all chart data sources for the automobile domain:

Document	Chart Source	Data Property	URL Parameter
651	ManufacturerChartDataSource	`byManufacturer`	`manufacturer`
652	YearChartDataSource	`byYearRange`	`year`, `yearMin`, `yearMax`

653	BodyClassChartDataSource	`byBodyClass`	`bodyClass`
654	TopModelsChartDataSource	`modelsByManufacturer`, `topModels`	`modelCombos`

Phase 7 Aha Moment: "Chart sources transform domain data into visualization-ready formats."

Each chart source:

- Transforms domain statistics into Plotly traces and layouts
- Handles both simple and segmented (highlighted) data
- Converts user interactions into URL parameters

Next Step

Proceed to Phase 8 (document 801) to build the `BaseChartComponent` that uses these data sources to render interactive charts.

Phase 8: Framework Components

Chapter 801: Base Chart Component

Chapter 802: Base Picker Component

Chapter 803: Results Table Component

Chapter 804: Query Panel Component

Chapter 805: Query Control Component

Chapter 806: Statistics Panel Component

Chapter 807: Search Component

Chapter 808: Pagination Component

Chapter 809: Export Button Component

Chapter 801

Base Chart Component

Step 801.2: Create the Chart Data Interface

The `ChartData` interface and `ChartDataSource` abstract class are defined inline in `src/app/framework/components/base-chart/base-chart.component.ts`:

```
// src/app/framework/components/base-chart/base-chart.component.ts
// Chart data structures defined inline with the component

/**
 * Chart data structure for Plotly.js
 */
export interface ChartData {
  /**
   * Array of Plotly data traces (each trace represents a data series or element)
   */
  traces: any[];

  /**
   * Plotly layout configuration object controlling chart appearance, axes, titles, etc.
   */
  layout: Partial<any>;

  /**
   * Optional click event data from the last chart interaction
   */
  clickData?: any;
}

/**
 * Abstract chart data source
 *
 * Transforms domain statistics into Plotly-ready chart data.
 * Implemented by domain-specific chart sources.
 *
 * @template TStatistics - Domain-specific statistics type
 */
export abstract class ChartDataSource<TStatistics = any> {
  /**
   * Transform statistics into chart data
   *
   * @param statistics - Domain statistics
   * @param highlights - Highlight filters (optional)
   * @param selectedValue - Currently selected value (optional)
   * @param containerWidth - Container width for responsive sizing
   * @returns Chart data or null if no data available
   */
  abstract transform(
    statistics: TStatistics | null,
    highlights: any,
```

```
    selectedValue: string | null,
    containerWidth: number
  ): ChartData | null;

  /**
   * Get chart title
   */
  abstract getTitle(): string;

  /**
   * Handle chart click event
   *
   * @param event - Plotly click event
   * @returns Clicked value or null
   */
  abstract handleClick(event: any): string | null;

  /**
   * Convert clicked value to URL parameters
   *
   * Maps the chart's clicked value to the appropriate URL query parameters.
   * Handles both filter mode (regular params) and highlight mode (h_* params).
   *
   * @param value - The clicked value from handleClick()
   * @param isHighlightMode - Whether highlight mode is active (h key held)
   * @returns URL parameters object to merge into the query string
   *
   * @example
   * // ManufacturerChartDataSource
   * toUrlParams('Toyota', false) // { manufacturer: 'Toyota' }
   * toUrlParams('Toyota', true)  // { h_manufacturer: 'Toyota' }
   *
   * // YearChartDataSource (range selection)
   * toUrlParams('1990|2000', false) // { yearMin: '1990', yearMax: '2000' }
   */
  abstract toUrlParams(value: string, isHighlightMode: boolean): Record<string, any>;
}
```

Note: These interfaces are co-located with the `BaseChartComponent` in the same file rather than in a separate `chart-data.interface.ts` file. This simplifies imports and keeps related code together.

```
// src/app/framework/components/base-chart/base-chart.component.ts
// VERSION 1 (Section 801) - Generic Plotly.js chart container

import {
  Component,
  Input,
  Output,
  EventEmitter,
  OnInit,
  OnDestroy,
  OnChanges,
  AfterViewInit,
  SimpleChanges,
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  ElementRef,
  ViewChild,
```

```
  HostListener
} from '@angular/core';
import { CommonModule } from '@angular/common';
import { Subject } from 'rxjs';
import { ButtonModule } from 'primeng/button';

import { ChartData, ChartDataSource } from './base-chart.component';

// Import Plotly.js minified distribution
import * as Plotly from 'plotly.js-dist-min';

/**
 * Extended HTMLElement interface for Plotly charts
 *
 * Plotly augments the DOM element with chart-specific methods and properties
 * after calling Plotly.newPlot().
 */
interface PlotlyHTMLElement extends HTMLElement {
  on(event: string, callback: (data: any) => void): void;
  data?: any[];
  layout?: any;
}

/**
 * Base Chart Component
 *
 * Reusable Plotly.js chart container that works with any ChartDataSource.
 * Handles chart rendering, resizing, click events, and memory cleanup.
 *
 * @example
 * ```html
 * <app-base-chart
 *   [dataSource]="manufacturerChartSource"
 *   [statistics]="statistics$ | async"
 *   [highlights]="highlights$ | async"
 *   (chartClick)="onChartClick($event)">
 * </app-base-chart>
 * ```
 */
@Component({
  selector: 'app-base-chart',
  templateUrl: './base-chart.component.html',
  styleUrls: ['./base-chart.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class BaseChartComponent implements OnInit, AfterViewInit, OnDestroy, OnChanges {

  /**
   * Reference to the chart container div where Plotly renders
   */
  @ViewChild('chartContainer', { static: false })
  chartContainer!: ElementRef<HTMLDivElement>;

  /**
   * Chart data source implementation (required)
   */
  @Input() dataSource!: ChartDataSource;

  /**
```

```
 * Statistics data to visualize
 */
@Input() statistics: any | null = null;

/**
 * Highlight filters for visual emphasis
 */
@Input() highlights: any = {};

/**
 * Currently selected value to highlight
 */
@Input() selectedValue: string | null = null;

/**
 * Whether to hide the chart title
 */
@Input() hideTitle = false;

/**
 * Whether to show pop-out button in mode bar
 */
@Input() canPopOut = false;

/**
 * Emits when user clicks on a chart element
 */
@Output() chartClick = new EventEmitter<{
  value: string;
  isHighlightMode: boolean;
}>();

/**
 * Emits when user clicks the pop-out button
 */
@Output() popOutClick = new EventEmitter<void>();

/**
 * Whether highlight mode is active (h key held)
 */
isHighlightModeActive = false;

/**
 * Chart title from data source
 */
chartTitle = '';

/**
 * Error state flag
 */
hasError = false;

/**
 * Error message for display
 */
errorMessage = '';

private destroy$ = new Subject<void>();
private plotlyElement: PlotlyHTMLElement | null = null;
```

```
constructor(private cdr: ChangeDetectorRef) {}

// ==============================================================================
// Lifecycle Hooks
// ==============================================================================

ngOnInit(): void {
  if (!this.dataSource) {
    console.error('BaseChartComponent: dataSource is required');
  }
  this.chartTitle = this.dataSource?.getTitle() || 'Chart';
}

ngAfterViewInit(): void {
  // Render chart after view is ready
  this.renderChart();
}

ngOnChanges(changes: SimpleChanges): void {
  // Re-render when inputs change (after initial render)
  if (this.chartContainer) {
    this.renderChart();
  }
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();

  // Clean up Plotly chart to prevent memory leaks
  if (this.plotlyElement) {
    Plotly.purge(this.plotlyElement);
  }
}

// ==============================================================================
// Event Handlers
// ==============================================================================

/**
 * Handle window resize events
 *
 * Uses double-resize pattern for pop-out window support.
 * The delayed second resize ensures proper dimensions after
 * browser layout recalculation.
 */
@HostListener('window:resize')
onWindowResize(): void {
  if (this.plotlyElement) {
    Plotly.Plots.resize(this.plotlyElement);

    // Extra resize with delay for pop-out windows
    setTimeout(() => {
      if (this.plotlyElement) {
        Plotly.Plots.resize(this.plotlyElement);
      }
    }, 100);
  }
}
```

```
/**
 * Enable highlight mode when h key is pressed
 */
@HostListener('document:keydown.h')
onHighlightKeyDown(): void {
  this.isHighlightModeActive = true;
  this.cdr.markForCheck();
}

/**
 * Disable highlight mode when h key is released
 */
@HostListener('document:keyup.h')
onHighlightKeyUp(): void {
  this.isHighlightModeActive = false;
  this.cdr.markForCheck();
}

/**
 * Retry rendering after an error
 */
retryRender(): void {
  this.hasError = false;
  this.errorMessage = '';
  this.cdr.markForCheck();
  setTimeout(() => this.renderChart(), 0);
}

// ==============================================================================
// Private Methods
// ==============================================================================

/**
 * Render the Plotly chart with error boundary
 */
private renderChart(): void {
  if (!this.chartContainer || !this.dataSource) {
    return;
  }

  try {
    this.hasError = false;
    this.errorMessage = '';

    const element = this.chartContainer.nativeElement;
    const containerWidth = element.clientWidth;

    // Transform data using the data source
    const chartData = this.dataSource.transform(
      this.statistics,
      this.highlights,
      this.selectedValue,
      containerWidth
    );

    if (!chartData) {
      // No data - clear chart
      if (this.plotlyElement) {
        Plotly.purge(this.plotlyElement);
```

```
        this.plotlyElement = null;
      }
      return;
    }

    // Configure layout with title
    const layout: Record<string, any> = { ...chartData.layout };
    if (!this.hideTitle && this.chartTitle) {
      layout['title'] = {
        text: this.chartTitle,
        font: { size: 14, color: '#ffffff' },
        x: 0.01,
        xanchor: 'left',
        y: 0.97,
        yanchor: 'top'
      };
      layout['margin'] = {
        ...layout['margin'],
        t: (layout['margin']?.t || 30) + 20
      };
    }

    // Plotly configuration with pop-out button support
    const config: Partial<any> = {
      responsive: true,
      displayModeBar: true,
      displaylogo: false,
      scrollZoom: false,
      modeBarButtonsToRemove: ['sendDataToCloud', 'lasso2d'],
      modeBarButtonsToAdd: this.canPopOut ? [{
        name: 'popout',
        title: 'Pop out to separate window',
        icon: {
          width: 1024,
          height: 1024,
          path: 'M896 0H640c-35.3 0-64 28.7-64 64s28.7 64 64 64h146.7L544 370.7c-25 25-25 65.5 0 90.5s65.5 25 9
          transform: 'matrix(1 0 0 -1 0 1024)'
        },
        click: () => this.popOutClick.emit()
      }] : []
    };

    // Render or update chart
    if (this.plotlyElement) {
      Plotly.react(this.plotlyElement, chartData.traces, layout, config);
    } else {
      Plotly.newPlot(element, chartData.traces, layout, config)
        .then((gd: PlotlyHTMLElement) => {
          this.plotlyElement = gd;
          this.attachEventHandlers(gd);
        })
        .catch((err: Error) => {
          this.handleRenderError(err, 'Failed to create chart');
        });
    }
  } catch (err) {
    this.handleRenderError(err as Error, 'Chart rendering failed');
  }
}
```

```
  /**
   * Attach Plotly event handlers
   */
  private attachEventHandlers(gd: PlotlyHTMLElement): void {
    gd.on('plotly_click', (data: any) => {
      try {
        const clickedValue = this.dataSource.handleClick(data);
        if (clickedValue) {
          this.chartClick.emit({
            value: clickedValue,
            isHighlightMode: this.isHighlightModeActive
          });
        }
      } catch (err) {
        console.error('[BaseChart] Click handler error:', err);
      }
    });

    gd.on('plotly_selected', (data: any) => {
      try {
        const selectedValue = this.dataSource.handleClick(data);
        if (selectedValue) {
          this.chartClick.emit({
            value: selectedValue,
            isHighlightMode: this.isHighlightModeActive
          });
        }
      } catch (err) {
        console.error('[BaseChart] Selection handler error:', err);
      }
    });
  }

  /**
   * Handle render errors with user-friendly fallback
   */
  private handleRenderError(err: Error, context: string): void {
    console.error(`[BaseChart] ${context}:`, err);
    this.hasError = true;
    this.errorMessage = `${context}: ${err.message || 'Unknown error'}`;
    this.cdr.markForCheck();

    if (this.plotlyElement) {
      try {
        Plotly.purge(this.plotlyElement);
      } catch {
        // Ignore purge errors
      }
      this.plotlyElement = null;
    }
  }
}
```

Chapter 802

Base Picker Component

What

Step 802.1: Create the Picker State Interface

The `PickerState` interface and `getDefaultPickerState` function are defined in `src/app/framework/models/picker-config.interface.ts` alongside other picker configuration types:

```
// src/app/framework/models/picker-config.interface.ts
// Picker state and configuration interfaces

/**
 * Internal state for BasePickerComponent
 *
 * Tracks loaded data, pagination, selection, and loading/error states.
 */
export interface PickerState<T> {
  /**
   * Currently loaded data rows (current page only)
   */
  data: T[];

  /**
   * Total count of records (for pagination)
   */
  totalCount: number;

  /**
   * Current page index (0-based)
   */
  currentPage: number;

  /**
   * Number of rows per page
   */
  pageSize: number;

  /**
   * Current sort field (if any)
   */
  sortField?: string;

  /**
   * Sort order: 1 = ascending, -1 = descending
```

```
   */
  sortOrder: number;

  /**
   * Current search/filter term
   */
  searchTerm: string;

  /**
   * Loading state flag
   */
  loading: boolean;

  /**
   * Error object (if any)
   */
  error: Error | null;

  /**
   * Whether initial data has been loaded
   */
  dataLoaded: boolean;

  /**
   * Set of selected row keys (for O(1) lookup)
   */
  selectedKeys: Set<string>;

  /**
   * Array of selected row objects
   */
  selectedItems: T[];

  /**
   * Keys pending hydration from URL (before data loads)
   */
  pendingHydration: string[];
}

/**
 * Get default picker state
 */
export function getDefaultPickerState<T>(pageSize = 20): PickerState<T> {
  return {
    data: [],
    totalCount: 0,
    currentPage: 0,
    pageSize,
    sortField: undefined,
    sortOrder: 1,
    searchTerm: '',
    loading: false,
    error: null,
    dataLoaded: false,
    selectedKeys: new Set<string>(),
    selectedItems: [],
    pendingHydration: []
  };
}
```

Note: The `PickerState` interface is co-located in the `picker-config.interface.ts` model file alongside `PickerConfig`, `PickerApiParams`, and `PickerSelectionEvent` interfaces, rather than in a separate component-local file.

Step 802.2: Create the Base Picker Component TypeScript

Create the file `src/app/framework/components/base-picker/base-picker.component.ts`:

```
// src/app/framework/components/base-picker/base-picker.component.ts
// VERSION 1 (Section 802) - Configuration-driven multi-select table

import {
  AfterViewInit,
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  ElementRef,
  EventEmitter,
  Input,
  OnDestroy,
  OnInit,
  Optional,
  Output
} from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Subject } from 'rxjs';
import { distinctUntilChanged, map, takeUntil } from 'rxjs/operators';

import { TableModule } from 'primeng/table';
import { ButtonModule } from 'primeng/button';
import { CheckboxModule } from 'primeng/checkbox';
import { InputTextModule } from 'primeng/inputtext';
import { SkeletonModule } from 'primeng/skeleton';
import { MessageModule } from 'primeng/message';

import { environment } from '../../../../environments/environment';
import {
  getDefaultPickerState,
  PickerApiParams,
  PickerConfig,
  PickerSelectionEvent,
  PickerState
} from '../../models/picker-config.interface';
import { PickerConfigRegistry } from '../../services/picker-config-registry.service';
import { UrlStateService } from '../../services/url-state.service';
import { ResourceManagementService } from '../../services/resource-management.service';

/**
 * Base Picker Component
 *
 * Configuration-driven multi-select table with:
 * - Server-side pagination and sorting
 * - Search filtering
 * - Selection persistence across pages
```

```
 * - URL parameter synchronization
 *
 * @template T - The data model type for table rows
 *
 * @example
 * ```html
 * <!-- Using config from registry -->
 * <app-base-picker
 *   [configId]="'vehicle-picker'"
 *   (selectionChange)="onSelectionChange($event)">
 * </app-base-picker>
 *
 * <!-- Using direct config object -->
 * <app-base-picker
 *   [config]="vehiclePickerConfig"
 *   (selectionChange)="onSelectionChange($event)">
 * </app-base-picker>
 * ```
 */
@Component({
  selector: 'app-base-picker',
  templateUrl: './base-picker.component.html',
  styleUrls: ['./base-picker.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class BasePickerComponent<T> implements OnInit, OnDestroy, AfterViewInit {

  /**
   * Environment configuration for conditional test-id rendering
   */
  readonly environment = environment;

  /**
   * Picker configuration ID (loads from registry)
   * Either configId or config must be provided.
   */
  @Input() configId?: string;

  /**
   * Direct picker configuration object
   * Either configId or config must be provided.
   */
  @Input() config?: PickerConfig<T>;

  /**
   * Emits when user clicks "Apply" with selected items
   */
  @Output() selectionChange = new EventEmitter<PickerSelectionEvent<T>>();

  /**
   * Current picker state
   */
  state: PickerState<T> = getDefaultPickerState();

  /**
   * Active configuration (resolved from configId or config)
   */
  activeConfig?: PickerConfig<T>;
```

```
private destroy$ = new Subject<void>();

constructor(
  private registry: PickerConfigRegistry,
  private urlState: UrlStateService,
  private cdr: ChangeDetectorRef,
  private elementRef: ElementRef,
  @Optional() private resourceService?: ResourceManagementService<any, any, any>
) {}

// ============================================================================
// Lifecycle
// ============================================================================

ngOnInit(): void {
  this.loadConfiguration();

  if (!this.activeConfig) {
    throw new Error('BasePickerComponent requires either configId or config input');
  }

  this.initializeState();
  this.subscribeToUrlChanges();
  this.loadData();
}

ngAfterViewInit(): void {
  this.syncPaginatorWidth();
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}

// ============================================================================
// Configuration
// ============================================================================

private loadConfiguration(): void {
  if (this.config) {
    this.activeConfig = this.config;
  } else if (this.configId) {
    this.activeConfig = this.registry.get<T>(this.configId);
  }
}

private initializeState(): void {
  const pageSize = this.activeConfig!.pagination.defaultPageSize || 20;
  this.state = getDefaultPickerState<T>(pageSize);
}

// ============================================================================
// URL Synchronization
// ============================================================================

private subscribeToUrlChanges(): void {
  const urlParam = this.activeConfig!.selection.urlParam;
```

```
  // If ResourceManagementService is available, watch filters$ (works in both windows)
  if (this.resourceService) {
    this.resourceService.filters$
      .pipe(
        map(filters => (filters as any)[urlParam] || null),
        distinctUntilChanged(),
        takeUntil(this.destroy$)
      )
      .subscribe(filterValue => {
        if (filterValue) {
          this.hydrateFromUrl(String(filterValue));
        } else {
          // Clear selections
          this.state.selectedKeys = new Set<string>();
          this.state.selectedItems = [];
          this.state.pendingHydration = [];
          this.state.data = [...this.state.data];
          this.cdr.detectChanges();
        }
      });
  } else {
    // Fallback: watch URL params directly (legacy mode)
    this.urlState
      .watchParam(urlParam)
      .pipe(takeUntil(this.destroy$))
      .subscribe(urlValue => {
        if (urlValue) {
          this.hydrateFromUrl(urlValue);
        } else {
          this.state.selectedKeys = new Set<string>();
          this.state.selectedItems = [];
          this.state.pendingHydration = [];
          this.state.data = [...this.state.data];
          this.cdr.detectChanges();
        }
      });
  }
}

private hydrateFromUrl(urlValue: string): void {
  const config = this.activeConfig!;
  const partialItems = config.selection.deserializer(urlValue);
  const keyGenerator = config.selection.keyGenerator || config.row.keyGenerator;
  const keys = partialItems.map(item => keyGenerator(item as T));

  if (this.state.dataLoaded) {
    this.hydrateSelections(keys);
  } else {
    this.state.pendingHydration = keys;
  }

  this.cdr.markForCheck();
}

private hydrateSelections(keys: string[]): void {
  const config = this.activeConfig!;
  this.state.selectedKeys = new Set<string>(keys);

  // Preserve items from other pages
```

```
  const existingItemsByKey = new Map<string, T>();
  this.state.selectedItems.forEach(item => {
    const key = config.row.keyGenerator(item);
    existingItemsByKey.set(key, item);
  });

  // Build new selectedItems array
  const newSelectedItems: T[] = [];
  keys.forEach(key => {
    const itemInCurrentPage = this.state.data.find(
      row => config.row.keyGenerator(row) === key
    );

    if (itemInCurrentPage) {
      newSelectedItems.push(itemInCurrentPage);
    } else if (existingItemsByKey.has(key)) {
      newSelectedItems.push(existingItemsByKey.get(key)!);
    }
  });

  this.state.selectedItems = newSelectedItems;
  this.cdr.markForCheck();
}

// ==============================================================================
// Data Loading
// ==============================================================================

private loadData(): void {
  const config = this.activeConfig!;
  this.state.loading = true;
  this.state.error = null;
  this.cdr.markForCheck();

  const params: PickerApiParams = {
    page: this.state.currentPage,
    size: this.state.pageSize,
    search: this.state.searchTerm || undefined,
    sortField: this.state.sortField,
    sortOrder: this.state.sortOrder
  };

  const apiParams = config.api.paramMapper
    ? config.api.paramMapper(params)
    : params;

  config.api.fetchData(apiParams)
    .pipe(takeUntil(this.destroy$))
    .subscribe({
      next: response => {
        const transformed = config.api.responseTransformer(response);
        this.state.data = transformed.results;
        this.state.totalCount = transformed.total;
        this.state.loading = false;
        this.state.dataLoaded = true;

        // Hydrate pending selections
        if (this.state.pendingHydration.length > 0) {
          this.hydrateSelections(this.state.pendingHydration);
```

```
          this.state.pendingHydration = [];
        }

        this.cdr.markForCheck();
      },
      error: error => {
        this.state.loading = false;
        this.state.error = error;
        this.cdr.markForCheck();
      }
    });
}

// ==============================================================================
// Selection Handlers
// ==============================================================================

onRowSelectionChange(row: T, checked: boolean): void {
  const key = this.activeConfig!.row.keyGenerator(row);

  if (checked) {
    this.state.selectedKeys.add(key);
    this.state.selectedItems.push(row);
  } else {
    this.state.selectedKeys.delete(key);
    this.state.selectedItems = this.state.selectedItems.filter(
      item => this.activeConfig!.row.keyGenerator(item) !== key
    );
  }

  this.cdr.markForCheck();
}

onSelectAll(checked: boolean): void {
  if (checked) {
    this.state.data.forEach(row => {
      const key = this.activeConfig!.row.keyGenerator(row);
      if (!this.state.selectedKeys.has(key)) {
        this.state.selectedKeys.add(key);
        this.state.selectedItems.push(row);
      }
    });
  } else {
    this.state.data.forEach(row => {
      const key = this.activeConfig!.row.keyGenerator(row);
      this.state.selectedKeys.delete(key);
    });
    this.state.selectedItems = this.state.selectedItems.filter(item => {
      const key = this.activeConfig!.row.keyGenerator(item);
      return this.state.selectedKeys.has(key);
    });
  }

  this.cdr.markForCheck();
}

isRowSelected(row: T): boolean {
  const key = this.activeConfig!.row.keyGenerator(row);
  return this.state.selectedKeys.has(key);
```

```
}

get allVisibleSelected(): boolean {
  if (this.state.data.length === 0) {
    return false;
  }
  return this.state.data.every(row => this.isRowSelected(row));
}

// ============================================================================
// Pagination & Sorting
// ============================================================================

onLazyLoad(event: any): void {
  if (this.state.loading) {
    return;
  }

  this.state.currentPage = event.first / event.rows;
  this.state.pageSize = event.rows;
  this.state.sortField = event.sortField || undefined;
  this.state.sortOrder = event.sortOrder || 1;

  this.loadData();
}

onSearch(term: string): void {
  this.state.searchTerm = term;
  this.state.currentPage = 0;
  this.loadData();
}

// ============================================================================
// Actions
// ============================================================================

applySelections(): void {
  const config = this.activeConfig!;
  const urlValue = config.selection.serializer(this.state.selectedItems);

  const event: PickerSelectionEvent<T> = {
    pickerId: config.id,
    selections: this.state.selectedItems,
    selectedKeys: Array.from(this.state.selectedKeys),
    urlValue
  };

  this.selectionChange.emit(event);
}

clearSelections(): void {
  this.state.selectedKeys = new Set<string>();
  this.state.selectedItems = [];
  this.state.data = [...this.state.data];
  this.cdr.markForCheck();

  // Emit empty selection
  const event: PickerSelectionEvent<T> = {
    pickerId: this.activeConfig!.id,
```

```
      selections: [],
      selectedKeys: [],
      urlValue: ''
    };
    this.selectionChange.emit(event);
  }

  // ==============================================================================
  // Helpers
  // ==============================================================================

  fieldToString(field: keyof T): string {
    return String(field);
  }

  private syncPaginatorWidth(): void {
    const nativeEl = this.elementRef.nativeElement;
    const table = nativeEl.querySelector('.p-datatable-table') as HTMLElement;
    const paginator = nativeEl.querySelector('.p-paginator') as HTMLElement;

    if (table && paginator) {
      paginator.style.width = `${table.offsetWidth}px`;
    }
  }
}
```

Note: The `ResourceManagementService` is injected with `@Optional()` decorator since the component can work with or without it. When available, the component uses `filters$` for reactive URL synchronization. When not available, it falls back to direct URL watching via `UrlStateService`.

Chapter 803

Results Table Component

```
// src/app/framework/components/basic-results-table/basic-results-table.component.ts
// VERSION 1 (Section 803) - Configuration-driven results display

import {
  AfterViewInit,
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  ElementRef,
  EventEmitter,
  Input,
  OnDestroy,
  OnInit,
  Output
} from '@angular/core';
import { CommonModule } from '@angular/common';
import { Subject } from 'rxjs';
import { filter, takeUntil } from 'rxjs/operators';

import { TableModule } from 'primeng/table';
import { ButtonModule } from 'primeng/button';
import { RippleModule } from 'primeng/ripple';
import { SkeletonModule } from 'primeng/skeleton';

import { environment } from '../../../../environments/environment';
import { DomainConfig } from '../../models/domain-config.interface';
import { ResourceManagementService } from '../../services/resource-management.service';
import { PopOutContextService } from '../../services/popout-context.service';
import { PopOutMessageType } from '../../models/popout.interface';

/**
 * Basic Results Table Component
 *
 * Displays search results in a paginated, sortable table with expandable rows.
 * Reads configuration from DomainConfig and data from ResourceManagementService.
 *
 * @template TFilters - Domain-specific filter model type
 * @template TData - Domain-specific data model type
 * @template TStatistics - Domain-specific statistics model type
 *
 * @example
 * ```html
 * <app-basic-results-table
 *   [domainConfig]="automobileDomainConfig">
 * </app-basic-results-table>
 * ```
 */
@Component({
  selector: 'app-basic-results-table',
  templateUrl: './basic-results-table.component.html',
  styleUrls: ['./basic-results-table.component.scss'],
```

```
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class BasicResultsTableComponent<TFilters = any, TData = any, TStatistics = any>
  implements OnInit, AfterViewInit, OnDestroy {

  private readonly destroy$ = new Subject<void>();

  /**
   * Environment configuration for conditional test-id rendering
   */
  readonly environment = environment;

  /**
   * Domain configuration with table columns and settings
   */
  @Input() domainConfig!: DomainConfig<TFilters, TData, TStatistics>;

  /**
   * Emits when URL parameters should be updated (for pop-out sync)
   */
  @Output() urlParamsChange = new EventEmitter<{ [key: string]: any }>();

  /**
   * Track expanded rows by data key
   */
  expandedRows: { [key: string]: boolean } = {};

  /**
   * Reference to Object for template use
   */
  Object = Object;

  constructor(
    private readonly resourceService: ResourceManagementService<TFilters, TData, TStatistics>,
    private readonly cdr: ChangeDetectorRef,
    private readonly popOutContext: PopOutContextService,
    private readonly elementRef: ElementRef
  ) {}

  // ============================================================================
  // State Getters (from ResourceManagementService)
  // ============================================================================

  get filters(): TFilters {
    return this.resourceService.filters;
  }

  get results(): TData[] {
    return this.resourceService.results;
  }

  get totalResults(): number {
    return this.resourceService.totalResults;
  }

  get loading(): boolean {
    return this.resourceService.loading;
  }
```

```
// ============================================================================
// Computed Properties
// ============================================================================

/**
 * Calculate paginator first index from current filters
 */
get paginatorFirst(): number {
  const filters = this.filters as Record<string, any>;
  const page = filters['page'] || 1;
  const size = filters['size'] || 20;
  return (page - 1) * size;
}

/**
 * Get current filters for template bindings
 */
get currentFilters(): Record<string, any> {
  return this.filters as Record<string, any>;
}

// ============================================================================
// Lifecycle
// ============================================================================

ngOnInit(): void {
  if (!this.domainConfig) {
    throw new Error('BasicResultsTableComponent requires domainConfig input');
  }

  // Pop-out window support: sync state from parent window
  if (this.popOutContext.isInPopOut()) {
    this.popOutContext
      .getMessages$()
      .pipe(
        filter(msg => msg.type === PopOutMessageType.STATE_UPDATE),
        takeUntil(this.destroy$)
      )
      .subscribe(message => {
        if (message.payload && message.payload.state) {
          this.resourceService.syncStateFromExternal(message.payload.state);
          this.cdr.markForCheck();
        }
      });
  }
}

ngAfterViewInit(): void {
  this.syncPaginatorWidth();
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}

// ============================================================================
// Event Handlers
// ============================================================================
```

```
/**
 * Handle pagination change
 */
onPageChange(event: any): void {
  const page = event.first / event.rows + 1;
  const size = event.rows;

  if (this.popOutContext.isInPopOut()) {
    // In pop-out: emit for parent to handle
    this.urlParamsChange.emit({ page, size });
  } else {
    // In main window: update filters directly
    const currentFilters = this.filters as Record<string, any>;
    const newFilters = {
      ...currentFilters,
      page,
      size
    } as unknown as TFilters;
    this.resourceService.updateFilters(newFilters);
  }
}

/**
 * Handle sort change
 *
 * Note: Includes debug logging for development/troubleshooting pop-out scenarios.
 */
onSort(event: any): void {
  const sort = event.field;
  const sortDirection = event.order === 1 ? 'asc' : 'desc';
  const isPopOut = this.popOutContext.isInPopOut();

  console.log('[BasicResultsTable] onSort called', { sort, sortDirection, isPopOut });

  if (isPopOut) {
    console.log('[BasicResultsTable] Emitting urlParamsChange', { sort, sortDirection });
    this.urlParamsChange.emit({ sort, sortDirection });
  } else {
    console.log('[BasicResultsTable] Calling updateFilters directly');
    const currentFilters = this.filters as Record<string, any>;
    const newFilters = {
      ...currentFilters,
      sort,
      sortDirection
    } as unknown as TFilters;
    this.resourceService.updateFilters(newFilters);
  }
}

/**
 * Refresh current results
 */
refresh(): void {
  this.resourceService.refresh();
}

// ==============================================================================
// Private Methods
// ==============================================================================
```

```
  /**
   * Sync paginator width to match table width
   */
  private syncPaginatorWidth(): void {
    const nativeEl = this.elementRef.nativeElement;
    const table = nativeEl.querySelector('.p-datatable-table') as HTMLElement;
    const paginator = nativeEl.querySelector('.p-paginator') as HTMLElement;

    if (table && paginator) {
      paginator.style.width = `${table.offsetWidth}px`;
    }
  }
}
```

Note: This component uses synchronous getters (not Observable streams) for accessing state from `ResourceManagementService`. This simplifies template bindings and avoids unnecessary async pipe usage. The `DynamicResultsTableComponent` variant uses Observable streams with async pipe for scenarios requiring OnPush change detection optimization.

Chapter 804

Query Panel Component

804: Statistics Panel Component

Status: Superseded by Section 808 (StatisticsPanel2Component)

Notice

This section describes a `StatisticsPanelComponent` that was planned but not implemented. The functionality has been superseded by **StatisticsPanel2Component** documented in Section 808.

The directory `src/app/framework/components/statistics-panel/` exists as an empty placeholder in the codebase.

Migration Guide

If you are looking for statistics panel functionality, please refer to:

- **Section 808: Statistics Panel 2 Component** - CDK mixed orientation chart grid with drag-drop reordering

The StatisticsPanel2Component provides:

- CDK horizontal drag-drop grid for chart arrangement
- Integration with BaseChartComponent for chart rendering
- Pop-out window support for charts
- Domain-agnostic configuration via DomainConfig.chartDataSources

---

Original Learning Objectives (For Reference)

The planned component would have:

- Composed multiple chart components into a single panel
- Implemented drag-and-drop reordering with Angular CDK
- Coordinated chart clicks with URL state updates

These objectives are now fulfilled by StatisticsPanel2Component.

Chapter 805

Query Control Component

805: Inline Filters Component

Status: Superseded by QueryControlComponent

Notice

This section describes an `InlineFiltersComponent` that was planned but not implemented. The functionality has been superseded by **QueryControlComponent** which provides expanded functionality.

The directory `src/app/framework/components/inline-filters/` exists as an empty placeholder in the codebase.

Migration Guide

The QueryControlComponent (`src/app/framework/components/query-control/query-control.component.ts`) provides all planned inline filter functionality plus additional features:

Original Planned Features (Now in QueryControlComponent)

- Display active filters as chips
- Click to edit filter
- Click to remove filter
- Sync with URL state

Additional Features in QueryControlComponent

- Filter field dropdown for selecting filterable fields

- Multiselect dialog for multi-value filters
- Range dialog for min/max filters
- Active highlight chips (separate from filter chips)
- Full PrimeNG Dialog integration
- Keyboard navigation and accessibility features
- Comprehensive bug fix implementations

QueryControlComponent Location

```
src/app/framework/components/query-control/query-control.component.ts
```

Basic Usage

```
<app-query-control
  [domainConfig]="domainConfig"
  (urlParamsChange)="onUrlParamsChange($event)"
  (clearAllFilters)="onClearAllFilters()">
</app-query-control>
```

Key Interfaces

The component uses these internal interfaces:

```
interface ActiveFilter {
  definition: FilterDefinition;
  values: (string | number)[];
  urlValue: string;
}
```

Architecture

- **PrimeNG-First**: Uses PrimeNG Dialog, Dropdown, Checkbox directly
- **URL-First**: All state changes via UrlStateService
- **OnPush**: Requires manual change detection via ChangeDetectorRef
- **Domain-Agnostic**: Works with any domain configuration

The component is approximately 970 lines of TypeScript with comprehensive documentation and bug fix comments.

Chapter 806

Statistics Panel Component

```
// src/app/framework/components/query-panel/query-panel.component.ts
// VERSION 1 (Section 806) - Configuration-driven filter panel

import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  EventEmitter,
  Input,
  OnDestroy,
  OnInit,
  Output
} from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { HttpClient } from '@angular/common/http';
import { Observable, Subject } from 'rxjs';
import { takeUntil, filter, debounceTime } from 'rxjs/operators';

import { InputTextModule } from 'primeng/inputtext';
import { InputNumberModule } from 'primeng/inputnumber';
import { ButtonModule } from 'primeng/button';
import { DropdownModule } from 'primeng/dropdown';
import { MultiSelectModule } from 'primeng/multiselect';
import { AutoCompleteModule } from 'primeng/autocomplete';
import { CheckboxModule } from 'primeng/checkbox';
import { CalendarModule } from 'primeng/calendar';

import { environment } from '../../../../environments/environment';
import { DomainConfig, FilterDefinition, FilterOption } from '../../models/domain-config.interface';
import { ResourceManagementService } from '../../services/resource-management.service';
import { PopOutContextService } from '../../services/popout-context.service';
import { PopOutMessageType } from '../../models/popout.interface';

/**
 * Query Panel Component
 *
 * Domain-agnostic filter panel that renders controls based on configuration.
 *
 * Supported filter types:
 * - text: Text input with clear button
 * - number: Numeric input with spinner
 * - range: Dual inputs for min/max values
 * - select: Single-select dropdown
 * - multiselect: Multi-select dropdown
 * - autocomplete: Text input with suggestions from API
 * - date: Date picker
 * - boolean: Checkbox
 *
 * @template TFilters - Domain-specific filter model type
 * @template TData - Domain-specific data model type
```

```
 * @template TStatistics - Domain-specific statistics model type
 *
 * @example
 * ```html
 * <app-query-panel
 *   [domainConfig]="automobileDomainConfig">
 * </app-query-panel>
 * ```
 */
@Component({
  selector: 'app-query-panel',
  templateUrl: './query-panel.component.html',
  styleUrls: ['./query-panel.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class QueryPanelComponent<TFilters = any, TData = any, TStatistics = any>
  implements OnInit, OnDestroy {

  /**
   * Environment configuration for conditional test-id rendering
   */
  readonly environment = environment;

  /**
   * Domain configuration with filter definitions
   */
  @Input() domainConfig!: DomainConfig<TFilters, TData, TStatistics>;

  /**
   * Emits when URL parameters should be updated (for pop-out sync)
   */
  @Output() urlParamsChange = new EventEmitter<{ [key: string]: any }>();

  /**
   * Emits when all filters should be cleared
   */
  @Output() clearAllFilters = new EventEmitter<void>();

  /**
   * Observable of current filter state
   */
  filters$!: Observable<TFilters>;

  /**
   * Local filter values (for form binding)
   */
  currentFilters: Record<string, any> = {};

  /**
   * Dynamic options loaded from API
   */
  dynamicOptions: Record<string, FilterOption[]> = {};

  /**
   * Autocomplete suggestions by filter ID
   */
  autocompleteSuggestions: Record<string, string[]> = {};

  private destroy$ = new Subject<void>();
```

```
  private searchSubject = new Subject<{ field: string; value: any }>();

  constructor(
    private resourceService: ResourceManagementService<TFilters, TData, TStatistics>,
    private cdr: ChangeDetectorRef,
    private http: HttpClient,
    private popOutContext: PopOutContextService
  ) {
    // Setup debounced search for text inputs
    this.searchSubject.pipe(
      debounceTime(300),
      takeUntil(this.destroy$)
    ).subscribe(({ field, value }) => {
      this.applyFilterChange(field, value);
    });
  }

  // ==============================================================================
  // Lifecycle
  // ==============================================================================

  ngOnInit(): void {
    if (!this.domainConfig) {
      throw new Error('QueryPanelComponent requires domainConfig input');
    }

    // Load dynamic options for filters with endpoints
    this.loadDynamicFilterOptions();

    // Subscribe to filter state
    this.filters$ = this.resourceService.filters$;

    this.filters$
      .pipe(takeUntil(this.destroy$))
      .subscribe(filters => {
        this.currentFilters = { ...filters as any };
        this.cdr.markForCheck();
      });

    // Pop-out window support
    if (this.popOutContext.isInPopOut()) {
      this.popOutContext
        .getMessages$()
        .pipe(
          filter(msg => msg.type === PopOutMessageType.STATE_UPDATE),
          takeUntil(this.destroy$)
        )
        .subscribe(message => {
          if (message.payload && message.payload.state) {
            this.resourceService.syncStateFromExternal(message.payload.state);
            this.cdr.markForCheck();
          }
        });
    }
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
```

```
}

// ==============================================================================
// Filter Change Handling
// ==============================================================================

/**
 * Handle filter input change
 *
 * @param field - Filter field ID
 * @param value - New value
 * @param debounce - Whether to debounce this change
 */
onFilterChange(field: string, value: any, debounce = false): void {
  // Update local model immediately
  this.currentFilters[field] = value;

  if (debounce) {
    this.searchSubject.next({ field, value });
  } else {
    this.applyFilterChange(field, value);
  }
}

/**
 * Apply filter change to state management
 */
private applyFilterChange(field: string, value: any): void {
  const isEmpty = value === null ||
                  value === undefined ||
                  value === '' ||
                  (Array.isArray(value) && value.length === 0);

  const paramValue = isEmpty ? null : value;

  if (this.popOutContext.isInPopOut()) {
    // In pop-out: emit event for parent
    this.urlParamsChange.emit({
      [field]: paramValue,
      page: 1
    });
  } else {
    // In main window: update ResourceManagementService
    const newFilters: Record<string, any> = {
      ...this.currentFilters,
      page: 1
    };

    if (isEmpty) {
      newFilters[field] = undefined;
    } else {
      newFilters[field] = value;
    }

    this.resourceService.updateFilters(newFilters as unknown as TFilters);
  }
}

/**
```

```
 * Clear all filters
 */
clearFilters(): void {
  if (this.popOutContext.isInPopOut()) {
    this.clearAllFilters.emit();
  } else {
    this.resourceService.updateFilters({
      page: 1,
      size: this.currentFilters['size'] || 20
    } as unknown as TFilters);
  }
}

// ==============================================================================
// Options Management
// ==============================================================================

/**
 * Get options for a filter
 */
getFilterOptions(filterId: string): FilterOption[] {
  if (this.dynamicOptions[filterId]) {
    return this.dynamicOptions[filterId];
  }
  const filterDef = this.domainConfig.filters.find(f => f.id === filterId);
  return filterDef?.options || [];
}

/**
 * Load dynamic options from API
 */
private loadDynamicFilterOptions(): void {
  const filtersWithEndpoint = this.domainConfig.filters.filter(f => f.optionsEndpoint);

  filtersWithEndpoint.forEach(filterDef => {
    const endpoint = `${this.domainConfig.apiBaseUrl}/agg/${filterDef.optionsEndpoint}`;

    this.http.get<{ field: string; values: Array<{ value: string; count: number }> }>(endpoint)
      .pipe(takeUntil(this.destroy$))
      .subscribe({
        next: (response) => {
          this.dynamicOptions[filterDef.id] = response.values.map(item => ({
            value: item.value,
            label: item.value
          }));
          this.cdr.markForCheck();
        },
        error: (err) => {
          console.error(`Failed to load options for ${filterDef.id}:`, err);
        }
      });
  });
}

// ==============================================================================
// Autocomplete
// ==============================================================================

/**
```

```
 * Handle autocomplete search
 */
onAutocompleteSearch(event: { query: string }, filterDef: FilterDefinition): void {
  if (!filterDef.autocompleteEndpoint) {
    return;
  }

  const query = event.query;
  const limit = 10;
  const endpoint = `${this.domainConfig.apiBaseUrl}/${filterDef.autocompleteEndpoint}?search=${encodeURICompone

  this.http.get<Record<string, string[]>>(endpoint)
    .pipe(takeUntil(this.destroy$))
    .subscribe({
      next: (response) => {
        const values = Object.values(response)[0] || [];
        this.autocompleteSuggestions[filterDef.id] = values;
        this.cdr.detectChanges();
      },
      error: () => {
        this.autocompleteSuggestions[filterDef.id] = [];
        this.cdr.detectChanges();
      }
    });
}

/**
 * Handle autocomplete focus - load initial suggestions
 */
onAutocompleteFocus(filterDef: FilterDefinition): void {
  if (this.autocompleteSuggestions[filterDef.id]?.length > 0) {
    return;
  }
  this.onAutocompleteSearch({ query: '' }, filterDef);
}

/**
 * Handle autocomplete keyboard events
 * Allows Space key to select highlighted item (standard UX)
 *
 * @param event - Keyboard event
 * @param autocomplete - Reference to p-autoComplete component
 * @param filterId - Filter ID for updating the filter
 */
onAutocompleteKeyUp(event: KeyboardEvent, autocomplete: any, filterId: string): void {
  if (event.key === ' ' || event.code === 'Space') {
    const highlightedOption = autocomplete.highlightOption;
    if (highlightedOption) {
      event.preventDefault();
      autocomplete.selectItem(event, highlightedOption);
      this.onFilterChange(filterId, highlightedOption);
    }
  }
}

/**
 * Handle autocomplete blur event
 * Ensures the current text value is applied as a filter even if not selected from dropdown
 *
```

```
     * @param event - Blur event
     * @param filterId - Filter ID
     * @param autocomplete - Autocomplete component reference
     */
    handleAutocompleteBlur(event: any, filterId: string, autocomplete: any): void {
      const inputValue = autocomplete.inputEL.nativeElement.value;
      if (inputValue && inputValue !== this.currentFilters[filterId]) {
        this.onFilterChange(filterId, inputValue);
      }
    }
  }
```

Note: The component includes the `environment` property for conditional test-id rendering in development mode, and additional accessibility methods (`onAutocompleteKeyUp` and `handleAutocompleteBlur`) for enhanced keyboard navigation and blur-to-apply behavior.

Chapter 807

Search Component

807: Column Manager Component

Status: Planned - Not Yet Implemented

Notice

This section describes a `ColumnManagerComponent` that is planned but not yet implemented. The directory `src/app/framework/components/column-manager/` exists as an empty placeholder in the codebase.

Planned Functionality

When implemented, the Column Manager Component would provide:

1. **Column visibility toggle**: Show/hide columns without losing data
2. **Column reordering**: Drag columns to customize display order
3. **Persistence**: Settings survive browser refresh via localStorage
4. **Domain-agnostic**: Works with any table configuration

Planned Storage Key

Column preferences would be stored in localStorage using a domain-specific key:

```
vvroom_columns_{domainId}
```

Current Alternatives

Column management may be handled differently in the current implementation:

- **DynamicResultsTableComponent** may handle some column management internally
- Table configurations in DomainConfig define available columns

---

Future Implementation Notes

When this component is implemented, it should:

1. Create interface file at `src/app/framework/components/column-manager/column-manager.interface.ts`
2. Create component at `src/app/framework/components/column-manager/column-manager.component.ts`
3. Integrate with Angular CDK for drag-and-drop reordering
4. Use localStorage for preference persistence
5. Emit column configuration events to parent components

Learning Objectives (For Future Implementation)

After completing this section, you will:

- Understand how to manage table column visibility with user preferences
- Know how to persist column settings to localStorage
- Be able to implement drag-and-drop column reordering

Chapter 808

Pagination Component

```
// src/app/framework/components/statistics-panel-2/statistics-panel-2.component.ts
// VERSION 1 (Section 808) - CDK horizontal chart grid

import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  EventEmitter,
  Input,
  OnDestroy,
  OnInit,
  Output
} from '@angular/core';
import { CdkDragDrop, moveItemInArray } from '@angular/cdk/drag-drop';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';
import { environment } from '../../../../environments/environment';
import { ChartConfig, DomainConfig } from '../../models/domain-config.interface';
import { PopOutMessageType } from '../../models/popout.interface';
import { PopOutContextService } from '../../services/popout-context.service';
import { ResourceManagementService } from '../../services/resource-management.service';
import { UrlStateService } from '../../services/url-state.service';
import { ChartDataSource, BaseChartComponent } from '../base-chart/base-chart.component';

/**
 * Statistics Panel 2 Component
 *
 * Renders statistical charts in a CDK horizontal drag-drop grid.
 * Charts can be reordered by dragging.
 *
 * @example
 * ```html
 * <app-statistics-panel-2
 *   [domainConfig]="domainConfig"
 *   (chartPopOut)="onChartPopOut($event)"
 *   (chartClicked)="onChartClick($event)">
 * </app-statistics-panel-2>
 * ```
 */
@Component({
  selector: 'app-statistics-panel-2',
  templateUrl: './statistics-panel-2.component.html',
  styleUrls: ['./statistics-panel-2.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class StatisticsPanel2Component implements OnInit, OnDestroy {

  private readonly destroy$ = new Subject<void>();

  // ============================================================================
  // Configuration
```

```
// ============================================================================

readonly environment = environment;

/**
 * Domain configuration with chart data sources
 */
@Input() domainConfig!: DomainConfig<any, any, any>;

/**
 * Function to check if a chart is popped out
 * Provided by parent component (DiscoverComponent)
 */
@Input() isPanelPoppedOut: (panelId: string) => boolean = () => false;

// ============================================================================
// Outputs
// ============================================================================

/**
 * Emits when user clicks the pop-out button on a chart
 */
@Output() chartPopOut = new EventEmitter<string>();

/**
 * Emits when user clicks on a chart element
 */
@Output() chartClicked = new EventEmitter<{
  event: { value: string; isHighlightMode: boolean };
  dataSource: ChartDataSource;
}>();

// ============================================================================
// State Getters (from ResourceManagementService)
// ============================================================================

get statistics(): any | undefined {
  return this.resourceService.statistics;
}

get highlights(): any {
  return this.resourceService.highlights;
}

/**
 * Check if running in a pop-out window
 */
get isInPopOut(): boolean {
  return this.popOutContext.isInPopOut();
}

// ============================================================================
// Component-Local State
// ============================================================================

/**
 * Ordered list of chart IDs for the grid
 */
chartOrder: string[] = [];
```

```
constructor(
  private resourceService: ResourceManagementService<any, any, any>,
  private urlState: UrlStateService,
  private popOutContext: PopOutContextService,
  private cdr: ChangeDetectorRef
) {}

// ==============================================================================
// Lifecycle
// ==============================================================================

ngOnInit(): void {
  if (!this.domainConfig) {
    console.error('StatisticsPanel2Component: domainConfig is required');
    return;
  }

  // Initialize chart order from domain config
  if (this.domainConfig.chartDataSources) {
    this.chartOrder = Object.keys(this.domainConfig.chartDataSources);
  }

  // Subscribe to statistics changes to trigger change detection
  this.resourceService.statistics$
    .pipe(takeUntil(this.destroy$))
    .subscribe(() => {
      this.cdr.markForCheck();
    });
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}

// ==============================================================================
// Event Handlers
// ==============================================================================

/**
 * Handle chart drag-drop to reorder
 */
onChartDrop(event: CdkDragDrop<string[]>): void {
  moveItemInArray(this.chartOrder, event.previousIndex, event.currentIndex);
  this.cdr.markForCheck();
}

/**
 * Handle chart pop-out request
 */
onChartPopOut(chartId: string): void {
  this.chartPopOut.emit(chartId);
}

/**
 * Handle chart click
 *
 * Gets URL params from the data source and updates URL state.
 * When in a pop-out window, sends the update via message to parent.
```

```
    */
   onChartClick(event: { value: string; isHighlightMode: boolean }, chartId: string): void {
     const dataSource = this.domainConfig.chartDataSources?.[chartId];
     if (!dataSource) return;

     // Delegate URL param generation to the data source
     const newParams = dataSource.toUrlParams(event.value, event.isHighlightMode);

     // Update URL (either directly or via pop-out message)
     if (this.popOutContext.isInPopOut()) {
       this.popOutContext.sendMessage({
         type: PopOutMessageType.URL_PARAMS_CHANGED,
         payload: { params: newParams },
         timestamp: Date.now()
       });
     } else {
       this.urlState.setParams(newParams);
     }
   }

   /**
    * Get data source for a chart ID
    */
   getDataSource(chartId: string): ChartDataSource | undefined {
     return this.domainConfig.chartDataSources?.[chartId];
   }
 }
```

Note: This implementation differs from what was previously documented:

1. Uses synchronous getters (`statistics`, `highlights`) instead of Observable streams
2. Imports `ChartDataSource` from `base-chart.component.ts`, not a separate interface file
3. Has a simplified constructor without `DomainConfigRegistry`, `ActivatedRoute`, or `IS_POPOUT_TOKEN` injections
4. Simpler `ngOnInit` that relies on parent component to provide appropriate configuration
5. Subscribes to `statistics$` to trigger change detection when statistics update

Chapter 809

Export Button Component

809: Dockview Statistics Panel Component

Status: Planned - Not Yet Implemented

Notice

This section describes a `DockviewStatisticsPanelComponent` that is planned but not yet implemented. The directory `src/app/framework/components/dockview-statistics-panel/` exists as an empty placeholder in the codebase.

The application currently uses **StatisticsPanel2Component** for chart layout, which uses CDK drag-drop grid instead of Dockview.

Planned Functionality

When implemented, the Dockview Statistics Panel would provide:

1. **Tabbed panels**: Multiple charts in the same space, switch via tabs
2. **Resizable splits**: Drag borders to adjust chart sizes
3. **Drag to rearrange**: Drag tabs to reorder or split panels
4. **Consistent dark theme**: Matches the application's visual style
5. **Layout serialization**: Save and restore panel arrangements

---

Current Alternative

Use **StatisticsPanel2Component** (Section 808) for chart layout:

```
<app-statistics-panel-2
  [domainConfig]="domainConfig"
  (chartPopOut)="onChartPopOut($event)"
  (chartClicked)="onChartClick($event)">
</app-statistics-panel-2>
```

StatisticsPanel2 provides:

- CDK horizontal drag-drop grid
- Chart reordering via drag and drop
- Pop-out support for individual charts
- Integration with ResourceManagementService

Future Implementation Notes

Dockview integration requires:

1. Additional npm package installation: `npm install dockview`
2. Complex Angular-to-Dockview DOM bridging
3. DOM manipulation pattern for moving Angular components into Dockview containers

The DOM Manipulation Pattern

Dockview is framework-agnostic and expects to manage its own DOM. The integration pattern:

1. Render Angular chart components in a hidden container
2. When Dockview creates a panel, move the chart element into the panel
3. When Dockview disposes a panel, move the chart back to the hidden container

This keeps Angular happy while letting Dockview manage the layout.

Learning Objectives (For Future Implementation)

After completing this section, you will:

- Understand how to integrate Dockview with Angular components
- Know how to move Angular-rendered elements into Dockview panels via DOM manipulation

- Be able to create tabbed, resizable panel layouts for data visualization

Phase 9: Feature Components

Chapter 901: Home Component

Chapter 902: Automobile Component

Chapter 903: Automobile Discover Component

Chapter 904: Popout Component

Chapter 905: Route Configuration

Chapter 906: Module Assembly

Chapter 907: Final Integration

Chapter 901

Home Component

901: Home Component

Status: Planning

Depends On: 102-app-shell, 103-routing

Blocks: 907-final-integration

Learning Objectives

After completing this section, you will:

- Understand how to create a landing page component that serves as a domain hub
- Know how to use Angular's `RouterModule` for in-app navigation links
- Be able to apply CSS Grid to create responsive card layouts

---

Objective

Build the Home component — the application's main entry point that welcomes users and provides navigation to multiple domain discovery features. This component establishes the visual identity of the Generic-Prime framework and guides users to explore data across various domains.

Why

Every application needs a home base. The Home component serves several critical purposes:

1. **First Impression** — Users land here first; it sets expectations for the entire application

2. **Navigation Hub** — Provides clear pathways to key features without overwhelming users
3. **Domain Introduction** — Explains what the application does before diving into specifics

Angular Style Guide References

- Style 02-01: Use consistent naming conventions (`home.component.ts`)
- Style 04-07: Create a folder for each feature component

Design Principles

The Home component is intentionally simple. It contains no business logic, no API calls, and no complex state management. It's pure presentation:

- Static content
- Navigation links
- Visual styling

This simplicity is deliberate. The Home component should load instantly and never fail. It's the fallback when something goes wrong elsewhere.

What

Step 901.1: Create the Home Component Directory

First, ensure the home feature directory exists:

```
$ cd ~/projects/vvroom
$ mkdir -p src/app/features/home
```

Step 901.2: Create the Home Component

Create `src/app/features/home/home.component.ts`:

```
// src/app/features/home/home.component.ts
// VERSION 1 (Section 901) - Complete home component with navigation

import { Component } from '@angular/core';

/**
```

```
 * Home Component - Landing Page
 *
 * Serves as the main entry point and domain selector for the Generic-Prime application.
 * This component provides navigation to various domain-specific modules including
 * Automobile, Physics, Agriculture, Chemistry, and Mathematics.
 *
 * The home page acts as a hub allowing users to select their desired domain of interest
 * and navigate to the corresponding feature modules for data exploration and visualization.
 *
 * @class HomeComponent
 * @since 1.0
 */
@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.scss']
})
export class HomeComponent {}
```

What this code does:

Element	Purpose
`selector: 'app-home'`	The HTML tag used to render this component
`templateUrl`	External HTML file for better separation of concerns
`styleUrls`	External SCSS file for component-specific styles
Empty class body	No logic needed — this is pure presentation

Step 901.3: Create the Home Component Template

Create `src/app/features/home/home.component.html`:

```
<!-- src/app/features/home/home.component.html -->
<!-- VERSION 1 (Section 901) - Home page with domain navigation -->

<div class="home-container">
  <!-- Header -->
  <div class="header">
    <h1>Generic Prime Discovery Framework</h1>
    <p class="subtitle">Explore data across multiple domains</p>
  </div>

  <!-- Domains -->
  <div class="domains-section">
    <div class="domain-grid">
      <a routerLink="/discover" class="domain-card">
        <div class="card-icon">■</div>
        <h3>Automobiles</h3>
```

```
          <p>Browse and analyze vehicle data</p>
        </a>
        <a class="domain-card disabled">
          <div class="card-icon">■</div>
          <h3>Agriculture</h3>
          <p>Agricultural data analysis</p>
        </a>
        <a class="domain-card disabled">
          <div class="card-icon">■■</div>
          <h3>Physics</h3>
          <p>Physics research data</p>
        </a>
        <a class="domain-card disabled">
          <div class="card-icon">■</div>
          <h3>Chemistry</h3>
          <p>Chemical compound data</p>
        </a>
        <a class="domain-card disabled">
          <div class="card-icon">■</div>
          <h3>Mathematics</h3>
          <p>Mathematical datasets</p>
        </a>
      </div>
    </div>

  </div>
```

Template structure explained:

Section	Purpose
`.header`	Application branding and tagline
`.domains-section`	Navigation cards to feature areas
`.domain-card.disabled`	Future domain placeholders (not yet implemented)

Note on `routerLink`: The `routerLink` directive from `RouterModule` creates navigation links that work with Angular's router. Unlike regular `href` attributes, `routerLink` doesn't cause a full page reload — it performs client-side navigation. The active Automobiles card links to `/discover`, while other domains are marked as `disabled` for future expansion.

Step 901.4: Create the Home Component Styles

Create `src/app/features/home/home.component.scss`:

```
// src/app/features/home/home.component.scss
// VERSION 1 (Section 901) - Home page styles with dark theme

.home-container {
```

```
  min-height: 100vh;
  background: linear-gradient(135deg, #1a1a1a 0%, #2d2d2d 100%);
  padding: 4rem 2rem;
  overflow-y: auto;

  /* Header Section */
  .header {
    text-align: center;
    margin-bottom: 4rem;
    animation: fadeInDown 0.8s ease-out;

    h1 {
      font-size: 3rem;
      font-weight: 700;
      color: #ffffff;
      margin: 0 0 1rem 0;
      letter-spacing: -0.5px;
      text-shadow: 0 2px 10px rgba(0, 0, 0, 0.3);
    }

    .subtitle {
      font-size: 1.2rem;
      color: #b0b0b0;
      margin: 0;
      font-weight: 300;
    }
  }

  /* Domains Section */
  .domains-section {
    margin-bottom: 4rem;
    animation: fadeInUp 0.8s ease-out 0.4s both;

    .domain-grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
      gap: 1.5rem;
      max-width: 1200px;
      margin: 0 auto;

      .domain-card {
        background: linear-gradient(135deg, rgba(255, 255, 255, 0.05) 0%, rgba(255, 255, 255, 0.02) 100%);
        border-radius: 12px;
        padding: 2rem 1.5rem;
        text-align: center;
        text-decoration: none;
        border: 1px solid rgba(255, 255, 255, 0.08);
        transition: all 0.3s ease;
        cursor: pointer;
        position: relative;
        overflow: hidden;

        &::before {
          content: '';
          position: absolute;
          top: 0;
          left: 0;
          right: 0;
          height: 3px;
```

```
          background: linear-gradient(90deg, #64c8ff, #4ba3d9);
          transform: scaleX(0);
          transition: transform 0.3s ease;
          transform-origin: left;
        }

        &:hover {
          transform: translateY(-6px);
          border-color: rgba(100, 200, 255, 0.2);
          background: linear-gradient(135deg, rgba(100, 200, 255, 0.1) 0%, rgba(75, 163, 217, 0.05) 100%);
          box-shadow: 0 12px 30px rgba(100, 200, 255, 0.15);

          &::before {
            transform: scaleX(1);
          }

          .card-icon {
            transform: scale(1.2);
          }
        }

        .card-icon {
          font-size: 2.5rem;
          display: block;
          margin-bottom: 1rem;
          transition: transform 0.3s ease;
        }

        h3 {
          font-size: 1.3rem;
          color: #ffffff;
          margin: 0 0 0.5rem 0;
          font-weight: 600;
        }

        p {
          color: #888;
          font-size: 0.9rem;
          margin: 0;
          line-height: 1.5;
        }

        &.disabled {
          opacity: 0.5;
          cursor: not-allowed;
          pointer-events: none;
        }
      }
    }
  }
}

/* Animations */
@keyframes fadeInDown {
  from {
    opacity: 0;
    transform: translateY(-30px);
  }
  to {
```

```
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Responsive Design */
@media (max-width: 768px) {
  .home-container {
    padding: 2rem 1rem;

    .header h1 {
      font-size: 2rem;
    }

    .domains-section .domain-grid {
      grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
      gap: 1.5rem;
    }
  }
}
```

CSS techniques used:

Technique	Purpose
Dark gradient background	Creates modern, immersive feel with `linear-gradient(135deg, #1a1a1a, #2d2d2d)`
CSS Grid with `auto-fit`	Creates responsive columns that adapt to screen size
`translateY` on hover	Provides visual lift effect for interactive cards
`::before` pseudo-element	Animated top border that slides in on hover
Cyan accent colors	Uses `#64c8ff` / `#4ba3d9` for highlights and interactions
SCSS nesting	Keeps related styles organized and readable
Keyframe animations	Smooth entrance animations for header and domain grid

---

Step 901.5: Register the Home Component in AppModule

The HomeComponent is declared directly in the root `AppModule` rather than using a separate feature module. This approach provides simpler setup for components that are always loaded.

In `src/app/app.module.ts`, ensure HomeComponent is imported and declared:

```
// In src/app/app.module.ts
import { HomeComponent } from './features/home/home.component';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    // ... other components
  ],
  imports: [
    BrowserModule,
    BrowserAnimationsModule,
    HttpClientModule,
    // ... other modules
  ],
  // ...
})
export class AppModule {}
```

Module registration explained:

Property	Purpose
`declarations`	Components that belong to this module, including HomeComponent
Direct declaration	HomeComponent is eagerly loaded with the app, no lazy loading

---

Verification

1. Check File Structure

```
$ find src/app/features/home -type f | sort
```

Expected output:

```
src/app/features/home/home.component.html
src/app/features/home/home.component.scss
src/app/features/home/home.component.ts
```

2. Build the Application

```
$ ng build
```

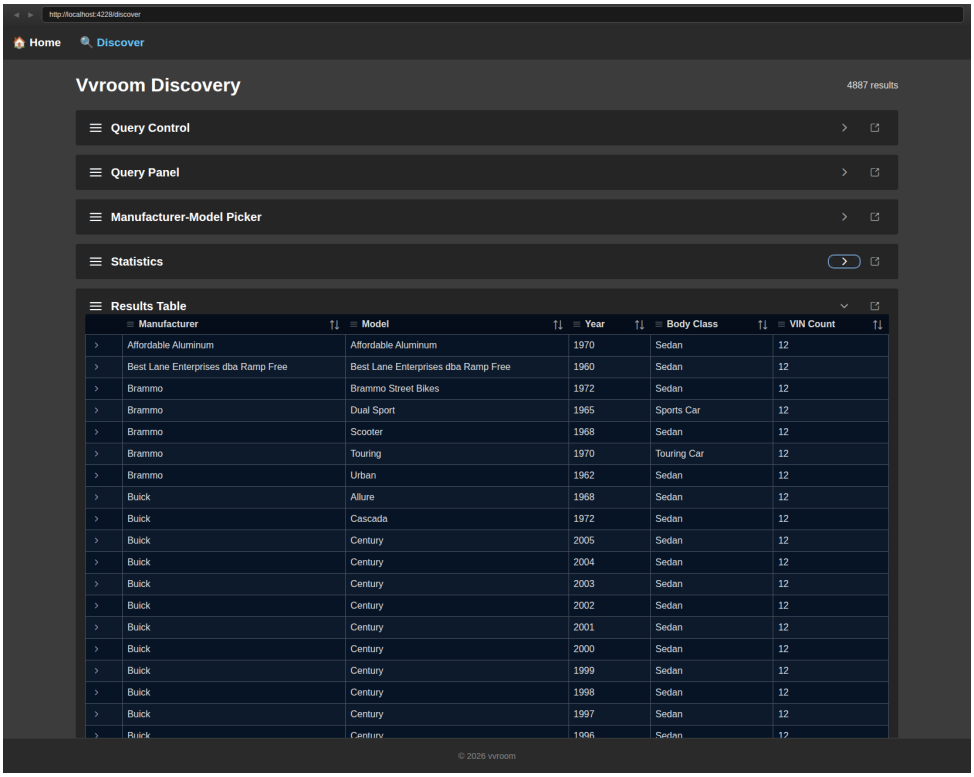
Expected: Build succeeds with no errors. If you see errors about missing routes, that's expected — we'll configure routing in document 905.

3. Visual Verification (After Routing)

Once routing is configured (document 905), navigate to `http://localhost:4200/home`:

- Large "Generic Prime Discovery Framework" heading in white
- "Explore data across multiple domains" subtitle in gray
- Five domain cards (Automobiles active, others disabled)
- Dark gradient background with cyan accent colors
- Cards lift on hover with animated top border effect

Figure 901.1 shows the Home component rendered in the browser. The dark theme with cyan accents creates a professional, modern appearance:



Home component showing domain selection cards

Figure 901.1: The Home component displaying domain navigation cards. The Automobiles card is active (clickable), while the other domains are styled as disabled placeholders for future expansion.

Chapter 902

Automobile Component

Step 902.2: Create the Automobile Landing Component

Create `src/app/features/automobile/automobile.component.ts`:

```
// src/app/features/automobile/automobile.component.ts
// VERSION 1 (Section 902) - Automobile domain landing page

import { Component } from '@angular/core';

/**
 * Automobile Component - Automobile Domain Landing Page
 *
 * Feature component serving as the primary entry point for the Automobile domain module.
 * This component provides navigation and context for automobile-related data exploration,
 * including vehicle discovery, filtering, and analysis features.
 *
 * Integrates with:
 * - AutomobileApiAdapter: Backend API integration
 * - AutomobileUrlMapper: URL state synchronization
 * - DiscoverComponent: Multi-panel exploration interface
 * - Automobile domain configuration and filter definitions
 *
 * The component leverages the Generic-Prime framework's URL-first architecture to manage
 * state through query parameters, ensuring state persistence across page reloads and
 * browser navigation.
 *
 * @class AutomobileComponent
 * @since 1.0
 * @remarks
 * This is the primary feature component for automobile data exploration.
 * Fully implemented with integration to the domain adapter, models, and UI controls.
 */
@Component({
  selector: 'app-automobile',
  templateUrl: './automobile.component.html',
  styleUrls: ['./automobile.component.scss']
})
export class AutomobileComponent {}
```

Step 902.3: Create the Automobile Landing Template

Create `src/app/features/automobile/automobile.component.html`:

```
<!-- src/app/features/automobile/automobile.component.html -->
<!-- VERSION 1 (Section 902) - Automobile landing page with feature preview -->

<div class="automobile-container">
  <!-- Features Grid -->
  <div class="features-section">
    <h2><span class="section-icon">■</span> Explore Automobile Data</h2>
    <div class="features-grid">
      <a routerLink="/automobiles/discover" class="feature-card feature-card-link">
        <div class="feature-icon">■</div>
        <h3>Advanced Search</h3>
        <p>Filter and search across thousands of vehicles using advanced criteria</p>
      </a>
      <div class="feature-card">
        <div class="feature-icon">■</div>
        <h3>Analytics</h3>
        <p>View comprehensive statistics and visualizations of vehicle data</p>
      </div>
      <div class="feature-card">
        <div class="feature-icon">■■</div>
        <h3>Detailed Specs</h3>
        <p>Access detailed specifications for every vehicle in the database</p>
      </div>
      <div class="feature-card">
        <div class="feature-icon">■</div>
        <h3>Performance</h3>
        <p>Compare performance metrics and find the perfect vehicle match</p>
      </div>
    </div>
  </div>

  <!-- Info Section -->
  <div class="info-section">
    <div class="info-card info-card-coming-soon">
      <h3>Coming Soon</h3>
      <p>New features and capabilities are on the way</p>
    </div>
    <div class="info-card">
      <h3>Dataset Information</h3>
      <ul>
        <li><strong>Vehicle Count:</strong> 55,463+ vehicles</li>
        <li><strong>Data Fields:</strong> Comprehensive specifications</li>
        <li><strong>Last Updated:</strong> Current dataset</li>
        <li><strong>Coverage:</strong> Global automobile market</li>
      </ul>
    </div>
    <div class="info-card">
      <h3>Quick Tips</h3>
      <ul>
        <li>Use filters to narrow down search results</li>
        <li>Hover over charts for detailed information</li>
        <li>Pop-out panels for multi-monitor workflows</li>
        <li>Export and analyze results in your workflow</li>
      </ul>
    </div>
  </div>

</div>
```

Template structure explained:

Section	Purpose
`.features-section`	Grid of feature cards previewing capabilities
`.feature-card-link`	The main call-to-action linking to Discover
Other `.feature-card` elements	Preview features (some may not be implemented yet)
`.info-section`	Supporting information cards
`.info-card-coming-soon`	Placeholder for future features

Design decision: Only the "Advanced Search" card links to a page. Other feature cards show what's possible but don't have links yet. This is intentional — it shows users the application's scope while guiding them to the implemented feature.

Step 902.4: Create the Automobile Landing Styles

Create `src/app/features/automobile/automobile.component.scss`:

```
// src/app/features/automobile/automobile.component.scss
// VERSION 1 (Section 902) - Automobile landing page styles with dark theme

.automobile-container {
  min-height: 100vh;
  background: linear-gradient(135deg, #1a1a1a 0%, #2d2d2d 100%);
  padding: 4rem 2rem;
  overflow-y: auto;

  /* Features Section */
  .features-section {
    margin-bottom: 5rem;
    animation: fadeInUp 0.8s ease-out 0.2s both;

    h2 {
      text-align: center;
      font-size: 2rem;
      color: #ffffff;
      margin: 0 0 3rem 0;
      font-weight: 600;

      .section-icon {
        margin-right: 0.5rem;
      }
    }

    .features-grid {
      display: grid;
```

```
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
gap: 2rem;
max-width: 1200px;
margin: 0 auto;

.feature-card {
  background: linear-gradient(135deg, rgba(100, 200, 255, 0.08) 0%, rgba(75, 163, 217, 0.04) 100%);
  border-radius: 12px;
  padding: 2rem;
  border: 1px solid rgba(100, 200, 255, 0.15);
  transition: all 0.3s ease;
  text-align: center;
  cursor: pointer;
  position: relative;
  overflow: hidden;
  text-decoration: none;
  display: block;

  &::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 3px;
    background: linear-gradient(90deg, #64c8ff, #4ba3d9);
    transform: scaleX(0);
    transition: transform 0.3s ease;
    transform-origin: center;
  }

  &:hover {
    transform: translateY(-8px);
    border-color: rgba(100, 200, 255, 0.3);
    background: linear-gradient(135deg, rgba(100, 200, 255, 0.15) 0%, rgba(75, 163, 217, 0.08) 100%);
    box-shadow: 0 15px 40px rgba(100, 200, 255, 0.2);

    &::before {
      transform: scaleX(1);
    }

    .feature-icon {
      transform: scale(1.2) rotate(5deg);
    }
  }

  .feature-icon {
    font-size: 3rem;
    display: block;
    margin-bottom: 1rem;
    transition: transform 0.3s ease;
  }

  h3 {
    font-size: 1.3rem;
    color: #ffffff;
    margin: 0 0 0.8rem 0;
    font-weight: 600;
  }
```

```
      p {
        color: #888;
        font-size: 0.95rem;
        margin: 0;
        line-height: 1.6;
      }
    }
  }
}

/* Info Section */
.info-section {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 2rem;
  max-width: 1200px;
  margin: 0 auto 5rem auto;
  animation: fadeInUp 0.8s ease-out 0.6s both;

  .info-card {
    background: linear-gradient(135deg, rgba(255, 255, 255, 0.05) 0%, rgba(255, 255, 255, 0.02) 100%);
    border-radius: 12px;
    padding: 2rem;
    border: 1px solid rgba(255, 255, 255, 0.08);
    transition: all 0.3s ease;

    &:hover {
      border-color: rgba(100, 200, 255, 0.2);
      background: linear-gradient(135deg, rgba(100, 200, 255, 0.08) 0%, rgba(75, 163, 217, 0.04) 100%);
      box-shadow: 0 10px 30px rgba(100, 200, 255, 0.1);
      transform: translateY(-4px);
    }

    h3 {
      font-size: 1.4rem;
      color: #ffffff;
      margin: 0 0 1.5rem 0;
      font-weight: 600;
      display: flex;
      align-items: center;
      gap: 0.8rem;

      &::before {
        content: '';
        display: block;
        width: 4px;
        height: 1.4rem;
        background: linear-gradient(180deg, #64c8ff, #4ba3d9);
        border-radius: 2px;
      }
    }

    ul {
      list-style: none;
      padding: 0;
      margin: 0;

      li {
        color: #b0b0b0;
```

```
          font-size: 0.95rem;
          margin-bottom: 1rem;
          line-height: 1.6;

          &:last-child {
            margin-bottom: 0;
          }

          strong {
            color: #ffffff;
            font-weight: 600;
          }
        }
      }

      p {
        color: #888;
        font-size: 0.95rem;
        margin: 0;
        line-height: 1.6;
      }

      &.info-card-coming-soon {
        text-align: center;
        display: flex;
        flex-direction: column;
        justify-content: center;

        h3 {
          justify-content: center;
        }
      }
    }
  }
}

/* Animations */
@keyframes fadeInUp {
  from {
    opacity: 0;
    transform: translateY(30px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Responsive Design */
@media (max-width: 768px) {
  .automobile-container {
    padding: 2rem 1rem;

    .features-section {
      h2 {
        font-size: 1.5rem;
      }

      .features-grid {
```

```
        grid-template-columns: 1fr;
        gap: 1.5rem;
      }
    }

    .info-section {
      grid-template-columns: 1fr;
    }
  }
}
```

Key styling patterns:

Pattern	Purpose
Dark gradient background	Creates modern, immersive feel with `linear-gradient(135deg, #1a1a1a, #2d2d2d)`
`.feature-card` with cyan accents	Uses `#64c8ff` / `#4ba3d9` for visual distinction
`.info-card-coming-soon`	Centered placeholder for future features
`::before` pseudo-element	Animated top border that slides in on hover
`translateY` hover effect	Indicates clickability on interactive elements

---

Step 902.5: Register in AppModule

The AutomobileComponent is declared directly in the root `AppModule` rather than using a separate feature module. This approach provides simpler setup for eagerly loaded components.

In `src/app/app.module.ts`, ensure AutomobileComponent is imported and declared:

```
// In src/app/app.module.ts
import { AutomobileComponent } from './features/automobile/automobile.component';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    AutomobileComponent,
    // ... other components
  ],
  imports: [
    BrowserModule,
    BrowserAnimationsModule,
    HttpClientModule,
    // ... other modules
  ],
  // ...
})
```

```
export class AppModule {}
```

Verification

1. Check File Structure

```
$ find src/app/features/automobile -type f | sort
```

Expected output:

```
src/app/features/automobile/automobile.component.html
src/app/features/automobile/automobile.component.scss
src/app/features/automobile/automobile.component.ts
```

2. Build the Application

```
$ ng build
```

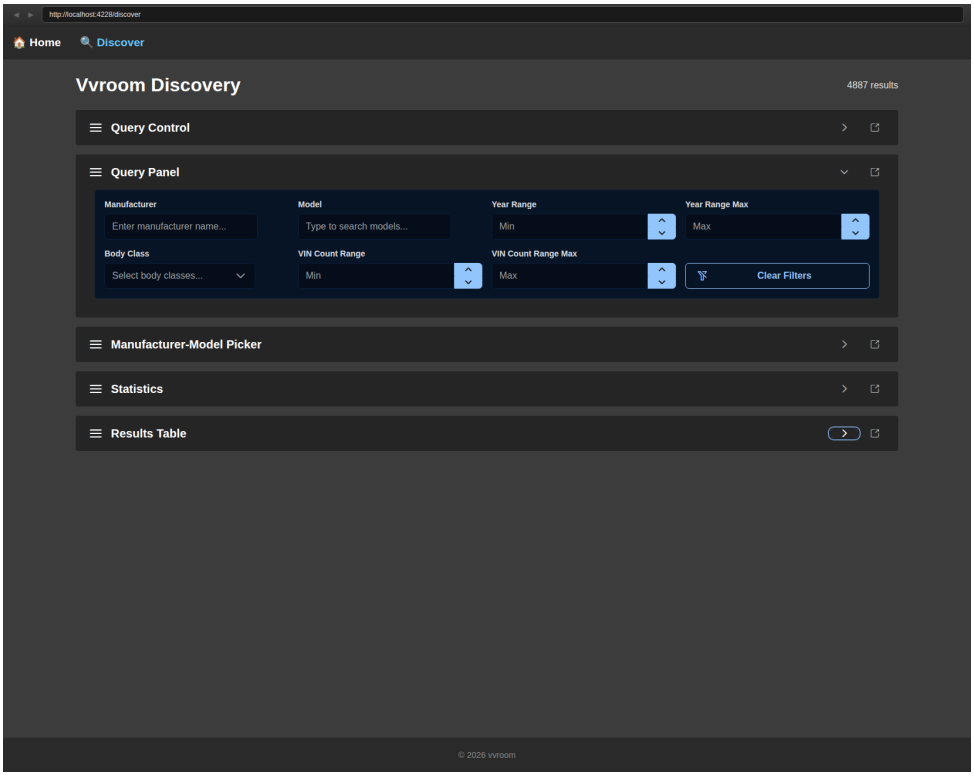
Expected: Build succeeds with no errors.

3. Visual Verification (After Routing)

Once routing is configured (document 905), navigate to `http://localhost:4200/automobiles`:

- Section header with car emoji: "Explore Automobile Data"
- Four feature cards in a responsive grid with dark theme
- "Advanced Search" card has cyan accent and links to discover page
- Three info cards at the bottom with dark gradient backgrounds
- "Coming Soon" card placeholder for future features

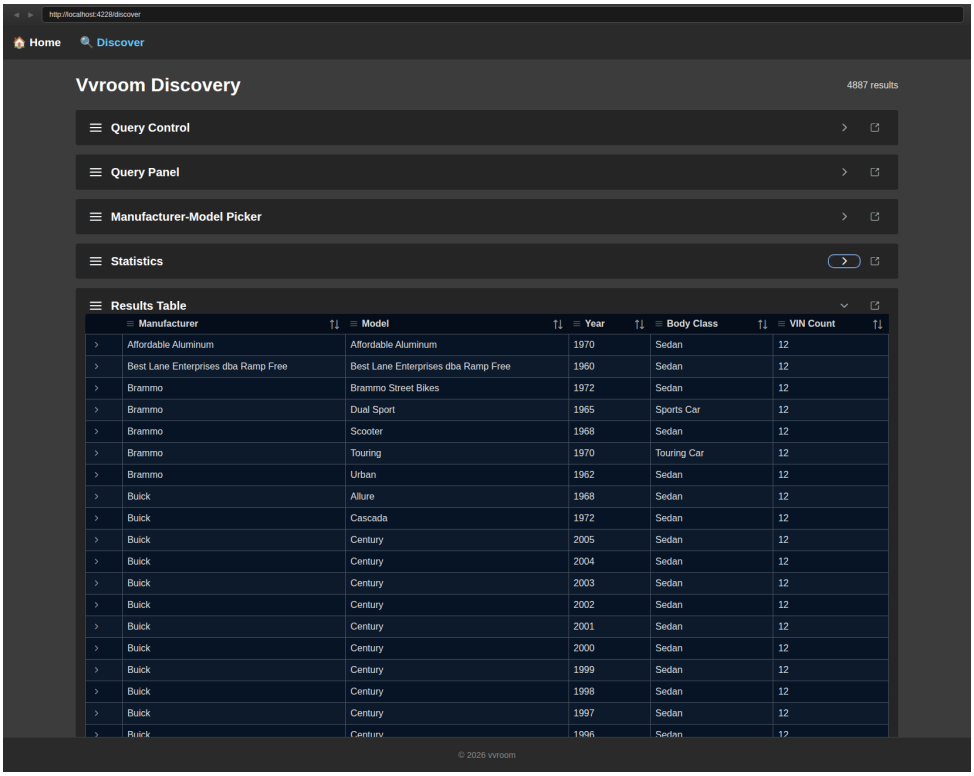
Figure 902.1 shows the Automobile landing page with its feature cards:



Automobile landing page with feature cards

Figure 902.1: The Automobile landing page. The "Advanced Search" card is prominently styled with cyan accent colors and navigates to the discover page where users can search and filter vehicle data.

Figure 902.2 shows the Discover page with the full data exploration interface including filters, results table, and statistics:



Discover page showing filter panel, results table, and statistics

Figure 902.2: The AutomobileDiscoverComponent showing the three main panels: filter panel (left), results table (center), and statistics panel (right). This layout enables users to iteratively refine their searches while viewing live results.

Chapter 903

Automobile Discover Component

What

Step 903.1: Create the Discover Component Directory

```
$ cd ~/projects/vvroom
$ mkdir -p src/app/features/discover
```

Step 903.2: Create the Discover Component

Create `src/app/features/discover/discover.component.ts`:

```
// src/app/features/discover/discover.component.ts
// VERSION 1 (Section 903) - Domain-agnostic discovery page with framework component orchestration

import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  Inject,
  Injector,
  NgZone,
  OnDestroy,
  OnInit
} from '@angular/core';
import { Params } from '@angular/router';
import { CdkDragDrop, moveItemInArray } from '@angular/cdk/drag-drop';
import { MessageService } from 'primeng/api';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

import { DomainConfig } from '../../framework/models/domain-config.interface';
import { PopOutMessageType, buildWindowFeatures, PopOutWindowRef } from '../../framework/models/popout.interface'
import { DOMAIN_CONFIG } from '../../framework/services/domain-config-registry.service';
import { PickerConfigRegistry } from '../../framework/services/picker-config-registry.service';
import { PopOutContextService } from '../../framework/services/popout-context.service';
import { ResourceManagementService } from '../../framework/services/resource-management.service';
import { UrlStateService } from '../../framework/services/url-state.service';
import { UserPreferencesService } from '../../framework/services/user-preferences.service';
import { ChartDataSource } from '../../framework/components/base-chart/base-chart.component';
```

```
import { createAutomobilePickerConfigs } from '../../domain-config/automobile/configs/automobile.picker-configs';

/**
 * Discover Component - Core discovery interface orchestrator
 *
 * **DOMAIN-AGNOSTIC**: Works with any domain via dependency injection.
 * Single component renders different UIs based on DOMAIN_CONFIG.
 *
 * **Primary Responsibilities**:
 * 1. Orchestrate 4 framework panels (QueryControl, Picker, Statistics, ResultsTable)
 * 2. Manage panel lifecycle (collapse, drag-drop reorder, pop-out)
 * 3. Handle URL state synchronization with ResourceManagementService
 * 4. Manage pop-out windows (create, monitor, close)
 * 5. Broadcast state changes to all open pop-outs via BroadcastChannel
 * 6. Listen for messages from pop-outs and update URL/state
 *
 * **Architecture**: Configuration-Driven + URL-First + Pop-Out Aware
 *
 * @template TFilters - Domain-specific filter model (e.g., AutoSearchFilters)
 * @template TData - Domain-specific data model (e.g., VehicleResult)
 * @template TStatistics - Domain-specific statistics (e.g., VehicleStatistics)
 */
@Component({
  selector: 'app-discover',
  templateUrl: './discover.component.html',
  styleUrls: ['./discover.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
  providers: [ResourceManagementService]
})
export class DiscoverComponent<TFilters = any, TData = any, TStatistics = any>
  implements OnInit, OnDestroy {

  /** Domain configuration (injected, works with any domain) */
  domainConfig: DomainConfig<TFilters, TData, TStatistics>;

  /** Set of panel IDs that are currently popped out */
  private poppedOutPanels = new Set<string>();

  /** Map of collapsed panel states (panel ID -> collapsed boolean) */
  collapsedPanels = new Map<string, boolean>();

  /** Ordered list of panel IDs (defines display order) */
  panelOrder: string[] = [
    'query-control',
    'query-panel',
    'manufacturer-model-picker',
    'statistics-panel-2',
    'basic-results-table'
  ];

  /** Map of pop-out windows and their associated channels */
  private popoutWindows = new Map<string, PopOutWindowRef>();

  /** Destroy signal for subscription cleanup */
  private destroy$ = new Subject<void>();

  /** RxJS Subject for pop-out messages (Observable Pattern) */
  private popoutMessages$ = new Subject<{ panelId: string; event: MessageEvent }>();
```

```
  /** Grid identifier for routing */
  private readonly gridId = 'discover';

  constructor(
    @Inject(DOMAIN_CONFIG) domainConfig: DomainConfig<any, any, any>,
    public resourceService: ResourceManagementService<TFilters, TData, TStatistics>,
    private pickerRegistry: PickerConfigRegistry,
    private injector: Injector,
    private popOutContext: PopOutContextService,
    private cdr: ChangeDetectorRef,
    private messageService: MessageService,
    private urlStateService: UrlStateService,
    private ngZone: NgZone,
    private userPreferences: UserPreferencesService
  ) {
    this.domainConfig = domainConfig as DomainConfig<TFilters, TData, TStatistics>;
  }

  ngOnInit(): void {
    // Load panel preferences from UserPreferencesService
    this.userPreferences.getPanelOrder()
      .pipe(takeUntil(this.destroy$))
      .subscribe(order => {
        this.panelOrder = order;
        this.cdr.markForCheck();
      });

    // Register domain-specific picker configurations
    const pickerConfigs = createAutomobilePickerConfigs(this.injector);
    this.pickerRegistry.registerMultiple(pickerConfigs);

    // Initialize PopOutContextService as parent window
    this.popOutContext.initializeAsParent();

    // Close all pop-outs when user refreshes/closes main window
    window.addEventListener('beforeunload', () => this.closeAllPopOuts());

    // Listen for messages from pop-outs via PopOutContextService
    this.popOutContext.getMessages$()
      .pipe(takeUntil(this.destroy$))
      .subscribe(message => {
        this.handlePopOutMessage('', message);
      });

    // Subscribe to pop-out BroadcastChannel messages
    this.popoutMessages$
      .pipe(takeUntil(this.destroy$))
      .subscribe(({ panelId, event }) => {
        this.handlePopOutMessage(panelId, event.data);
      });

    // Broadcast state changes to all open pop-outs
    this.resourceService.state$.pipe(takeUntil(this.destroy$)).subscribe(state => {
      this.broadcastStateToPopOuts(state);
    });
  }

  // Panel state management methods...
  isPanelPoppedOut(panelId: string): boolean {
```

```
    return this.poppedOutPanels.has(panelId);
  }

  isPanelCollapsed(panelId: string): boolean {
    return this.collapsedPanels.get(panelId) ?? false;
  }

  togglePanelCollapse(panelId: string): void {
    const currentState = this.collapsedPanels.get(panelId) ?? false;
    this.collapsedPanels.set(panelId, !currentState);
    this.cdr.markForCheck();
  }

  onPanelDrop(event: CdkDragDrop<string[]>): void {
    moveItemInArray(this.panelOrder, event.previousIndex, event.currentIndex);
    this.userPreferences.savePanelOrder(this.panelOrder);
    this.cdr.markForCheck();
  }

  getPanelTitle(panelId: string): string {
    const titleMap: { [key: string]: string } = {
      'query-control': 'Query Control',
      'query-panel': 'Query Panel',
      'manufacturer-model-picker': 'Manufacturer-Model Picker',
      'statistics-panel-2': 'Statistics',
      'basic-results-table': 'Results Table'
    };
    return titleMap[panelId] || panelId;
  }

  getPanelType(panelId: string): string {
    const typeMap: { [key: string]: string } = {
      'query-control': 'query-control',
      'query-panel': 'query-panel',
      'manufacturer-model-picker': 'picker',
      'statistics-panel-2': 'statistics-2',
      'basic-results-table': 'basic-results'
    };
    return typeMap[panelId] || panelId;
  }

  // Pop-out and event handling methods continue...
  // (See full implementation in source code)

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}


<!-- src/app/features/discover/discover.component.html -->
<!-- VERSION 1 (Section 903) - Discovery page with draggable panels using CDK and PrimeNG -->

<div class="discover-container">
  <!-- Header -->
  <div class="discover-header">
    <h1>{{ domainConfig.domainLabel }}</h1>
    <span class="results-count">
```

```
    {{ (resourceService.totalResults$ | async) }}
    {{ (resourceService.totalResults$ | async) === 1 ? 'result' : 'results' }}
  </span>
</div>

<!-- Draggable Panels Container -->
<div cdkDropList (cdkDropListDropped)="onPanelDrop($any($event))" class="panels-container">
  <ng-container *ngFor="let panelId of panelOrder">
    <!-- Panel Wrapper -->
    <div class="panel-wrapper" cdkDrag [attr.id]="'panel-' + panelId">
      <!-- Drag Handle -->
      <div class="panel-header">
        <div class="panel-title-group">
          <div cdkDragHandle class="drag-handle">
            <i class="pi pi-bars"></i>
          </div>
          <h3 class="panel-title">{{ getPanelTitle(panelId) }}</h3>
        </div>
        <div class="panel-actions">
          <button
            pButton
            type="button"
            [icon]="isPanelCollapsed(panelId) ? 'pi pi-chevron-right' : 'pi pi-chevron-down'"
            class="p-button-sm p-button-text"
            (click)="togglePanelCollapse(panelId)"
            [pTooltip]="isPanelCollapsed(panelId) ? 'Expand' : 'Collapse'"
            tooltipPosition="left">
          </button>
          <ng-container *ngIf="!isPanelPoppedOut(panelId)">
            <button
              pButton
              type="button"
              icon="pi pi-external-link"
              class="p-button-sm p-button-text"
              [attr.id]="'popout-' + panelId"
              (click)="popOutPanel(panelId, getPanelType(panelId))"
              pTooltip="Pop out to separate window"
              tooltipPosition="left">
            </button>
          </ng-container>
        </div>
      </div>

      <!-- Panel Content -->
      <ng-container *ngIf="!isPanelCollapsed(panelId)">
        <div>
          <!-- Query Control -->
          <ng-container *ngIf="panelId === 'query-control'">
            <ng-container *ngIf="!isPanelPoppedOut('query-control'); else queryControlPoppedOut">
              <div>
                <app-query-control
                  [domainConfig]="domainConfig"
                  (urlParamsChange)="onUrlParamsChange($event)"
                  (clearAllFilters)="onClearAllFilters()">
                </app-query-control>
              </div>
            </ng-container>
            <ng-template #queryControlPoppedOut>
              <div class="popout-placeholder">
```

```
      <i class="pi pi-external-link"></i>
      <span>Query Control is open in a separate window</span>
    </div>
  </ng-template>
</ng-container>

<!-- Query Panel -->
<ng-container *ngIf="panelId === 'query-panel'">
  <ng-container *ngIf="!isPanelPoppedOut('query-panel'); else queryPanelPoppedOut">
    <div>
      <app-query-panel [domainConfig]="domainConfig"></app-query-panel>
    </div>
  </ng-container>
  <ng-template #queryPanelPoppedOut>
    <div class="popout-placeholder">
      <i class="pi pi-external-link"></i>
      <span>Query Panel is open in a separate window</span>
    </div>
  </ng-template>
</ng-container>

<!-- Manufacturer-Model Picker -->
<ng-container *ngIf="panelId === 'manufacturer-model-picker'">
  <ng-container *ngIf="!isPanelPoppedOut('manufacturer-model-picker'); else manufacturerPickerPoppedO
    <div>
      <app-base-picker
        [configId]="'manufacturer-model-picker'"
        (selectionChange)="onPickerSelectionChangeAndUpdateUrl($event)">
      </app-base-picker>
    </div>
  </ng-container>
  <ng-template #manufacturerPickerPoppedOut>
    <div class="popout-placeholder">
      <i class="pi pi-external-link"></i>
      <span>Manufacturer-Model Picker is open in a separate window</span>
    </div>
  </ng-template>
</ng-container>

<!-- Statistics Panel -->
<ng-container *ngIf="panelId === 'statistics-panel-2'">
  <ng-container *ngIf="!isPanelPoppedOut('statistics-panel-2'); else statisticsPanel2PoppedOut">
    <div>
      <app-statistics-panel-2
        [domainConfig]="domainConfig"
        [isPanelPoppedOut]="isPanelPoppedOut.bind(this)"
        (chartPopOut)="onChartPopOut($event)">
      </app-statistics-panel-2>
    </div>
  </ng-container>
  <ng-template #statisticsPanel2PoppedOut>
    <div class="popout-placeholder">
      <i class="pi pi-external-link"></i>
      <span>Statistics Panel 2 is open in a separate window</span>
    </div>
  </ng-template>
</ng-container>

<!-- Dynamic Results Table -->
```

```
            <ng-container *ngIf="panelId === 'basic-results-table'">
              <ng-container *ngIf="!isPanelPoppedOut('basic-results-table'); else basicResultsTablePoppedOut">
                <div>
                  <app-dynamic-results-table [domainConfig]="domainConfig"></app-dynamic-results-table>
                </div>
              </ng-container>
              <ng-template #basicResultsTablePoppedOut>
                <div class="popout-placeholder">
                  <i class="pi pi-external-link"></i>
                  <span>Results Table is open in a separate window</span>
                </div>
              </ng-template>
            </ng-container>
          </div>
        </ng-container>
      </div>
    </ng-container>
  </div>
</div>
```

Template features explained:

Feature	Purpose
`cdkDropList` / `cdkDrag`	CDK drag-drop for panel reordering
`cdkDragHandle`	Restricts drag initiation to handle element
PrimeNG `pButton`	Styled buttons with icons
PrimeNG `pTooltip`	Accessible tooltips for actions
`pi pi-*` icons	PrimeNG icon library
`ng-template` references	Placeholder content when panel is popped out

Chapter 904

Popout Component

What

Step 904.1: Create the Panel Popout Feature Directory

```
$ cd ~/projects/vvroom
$ mkdir -p src/app/features/panel-popout
```

Step 904.2: Create the Panel Popout Component

Create `src/app/features/panel-popout/panel-popout.component.ts`:

```
// src/app/features/panel-popout/panel-popout.component.ts
// VERSION 1 (Section 904) - Pop-out window container component

import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  Inject,
  Injector,
  OnDestroy,
  OnInit
} from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

import { createAutomobilePickerConfigs } from '../../domain-config/automobile/configs/automobile.picker-configs';
import { DomainConfig } from '../../framework/models';
import { PickerSelectionEvent } from '../../framework/models/picker-config.interface';
import {
  PopOutMessage,
  PopOutMessageType
} from '../../framework/models/popout.interface';
import { DOMAIN_CONFIG } from '../../framework/services/domain-config-registry.service';
import { PickerConfigRegistry } from '../../framework/services/picker-config-registry.service';
import { PopOutContextService } from '../../framework/services/popout-context.service';
import { ResourceManagementService } from '../../framework/services/resource-management.service';
import { IS_POPOUT_TOKEN } from '../../framework/tokens/popout.token';
```

```
/**
 * Panel Popout Component
 *
 * Container component for pop-out windows.
 * Renders different panel types based on route parameters.
 *
 * Route: `/panel/:gridId/:panelId/:type` (NO query params)
 *
 * Architecture:
 * - Initializes as pop-out via PopOutContextService
 * - Receives STATE_UPDATE messages from main window via BroadcastChannel
 * - Syncs state to ResourceManagementService (no API calls)
 * - Components subscribe to ResourceManagementService observables naturally
 * - URL-First: Main window URL is source of truth, pop-out receives derived state
 *
 * State Flow:
 * 1. Main window URL changes -> ResourceManagementService updates state
 * 2. Main window broadcasts STATE_UPDATE to pop-outs
 * 3. Pop-out receives STATE_UPDATE -> syncs to ResourceManagementService
 * 4. Components subscribe to observables and render
 */
@Component({
  selector: 'app-panel-popout',
  templateUrl: './panel-popout.component.html',
  styleUrls: ['./panel-popout.component.scss'],
  changeDetection: ChangeDetectionStrategy.OnPush,
  providers: [
    ResourceManagementService,
    { provide: IS_POPOUT_TOKEN, useValue: true }
  ]
})
export class PanelPopoutComponent implements OnInit, OnDestroy {
  /** Grid identifier from route */
  gridId: string = '';

  /** Panel identifier from route */
  panelId: string = '';

  /** Panel type (determines which component to render) */
  panelType: string = '';

  /** Domain configuration (injected) */
  domainConfig: DomainConfig<any, any, any>;

  /** Destroy signal for subscription cleanup */
  private destroy$ = new Subject<void>();

  constructor(
    private route: ActivatedRoute,
    private popOutContext: PopOutContextService,
    private cdr: ChangeDetectorRef,
    private pickerRegistry: PickerConfigRegistry,
    private injector: Injector,
    @Inject(DOMAIN_CONFIG) domainConfig: DomainConfig<any, any, any>,
    public resourceService: ResourceManagementService<any, any, any>
  ) {
    this.domainConfig = domainConfig;
  }
```

```
ngOnInit(): void {
  // Register picker configurations (needed for BasePickerComponent in pop-out)
  const pickerConfigs = createAutomobilePickerConfigs(this.injector);
  this.pickerRegistry.registerMultiple(pickerConfigs);

  // Extract route parameters
  this.route.params.pipe(takeUntil(this.destroy$)).subscribe(params => {
    this.gridId = params['gridId'];
    this.panelId = params['panelId'];
    this.panelType = params['type'];

    // Initialize as pop-out
    this.popOutContext.initializeAsPopOut(this.panelId);

    // Add class to body and html to hide scrollbars for all pop-outs
    document.documentElement.classList.add('popout-html');
    document.body.classList.add('popout-body');

    // Trigger change detection
    this.cdr.markForCheck();
  });

  // Subscribe to messages from main window
  this.popOutContext
    .getMessages$()
    .pipe(takeUntil(this.destroy$))
    .subscribe(message => {
      this.handleMessage(message);
    });
}

/**
 * Handle messages from main window
 */
private async handleMessage(message: PopOutMessage): Promise<void> {
  switch (message.type) {
    case PopOutMessageType.CLOSE_POPOUT:
      window.close();
      break;

    case PopOutMessageType.STATE_UPDATE:
      // Sync full state from main window
      if (message.payload && message.payload.state) {
        this.resourceService.syncStateFromExternal(message.payload.state);
        this.cdr.detectChanges();
      }
      break;
  }
}

/**
 * Get chart data source for chart pop-outs
 */
getChartDataSource(): any {
  if (this.panelId.startsWith('chart-')) {
    const chartId = this.panelId.replace('chart-', '');
    return this.domainConfig.chartDataSources?.[chartId];
  }
  return null;
```

```
}

/**
 * Handle URL parameter changes from child components
 */
onUrlParamsChange(params: any): void {
  this.popOutContext.sendMessage({
    type: PopOutMessageType.URL_PARAMS_CHANGED,
    payload: { params },
    timestamp: Date.now()
  });
}

/**
 * Handle clear all filters request
 */
onClearAllFilters(): void {
  this.popOutContext.sendMessage({
    type: PopOutMessageType.CLEAR_ALL_FILTERS,
    timestamp: Date.now()
  });
}

/**
 * Handle picker selection changes
 */
onPickerSelectionChange(event: PickerSelectionEvent<any>): void {
  if (event.urlValue !== undefined) {
    this.popOutContext.sendMessage({
      type: PopOutMessageType.PICKER_SELECTION_CHANGE,
      payload: event,
      timestamp: Date.now()
    });
  }
}

/**
 * Handle chart click/selection from pop-out chart
 */
onChartClick(event: { value: string; isHighlightMode: boolean }): void {
  const chartId = this.panelId.startsWith('chart-')
    ? this.panelId.replace('chart-', '')
    : this.panelId;

  this.popOutContext.sendMessage({
    type: PopOutMessageType.CHART_CLICK,
    payload: {
      chartId,
      value: event.value,
      isHighlightMode: event.isHighlightMode
    },
    timestamp: Date.now()
  });
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}
```

```
}
```

Step 904.5: Configure Routing for Panel Pop-outs

The PanelPopoutComponent handles all panel types dynamically. No separate child routes or lazy loading is used - the component determines which content to render based on the `panelType` route parameter.

In `src/app/app-routing.module.ts`:

```
// src/app/app-routing.module.ts
// VERSION 3 - Added panel pop-out route

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PanelPopoutComponent } from './features/panel-popout/panel-popout.component';

const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'discover', component: DiscoverComponent },
  // Pop-out panel route: /panel/:gridId/:panelId/:type
  // Used by DiscoverComponent.popOutPanel() to open panels in separate windows
  // PanelPopoutComponent handles BroadcastChannel communication with main window
  { path: 'panel/:gridId/:panelId/:type', component: PanelPopoutComponent },
  // 404 fallback route - redirect unmatched paths to home
  { path: '**', redirectTo: 'home' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Route structure explained:

Route	Purpose
`/panel/:gridId/:panelId/:type`	Pop-out window route
`:gridId`	Source grid identifier (e.g., 'discover')
`:panelId`	Panel identifier (e.g., 'manufacturer-model-picker')
`:type`	Component type (e.g., 'picker', 'query-control', 'chart')

Example pop-out URLs:

```
/panel/discover/manufacturer-model-picker/picker
/panel/discover/query-control/query-control
```

```
/panel/discover/chart-manufacturer/chart
/panel/discover/statistics-panel-2/statistics-2
/panel/discover/basic-results-table/basic-results
```

Step 904.6: Register in AppModule

The PanelPopoutComponent is declared directly in `AppModule` rather than using a separate feature module:

```
// In src/app/app.module.ts
import { PanelPopoutComponent } from './features/panel-popout/panel-popout.component';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    DiscoverComponent,
    PanelPopoutComponent,
    // ... other components
  ],
  // ...
})
export class AppModule {}
```

This approach:

1. Keeps all feature components eagerly loaded
2. Avoids complexity of separate feature modules
3. Ensures PanelPopoutComponent is available for routing immediately

Chapter 905

Route Configuration

905: App Routing Module

Status: Planning

Depends On: 901-home-component, 902-automobile-landing-component, 903-discover-page-component, 904-popout-component

Blocks: 906-app-module

Learning Objectives

After completing this section, you will:

- Understand how to configure Angular Router with direct component references
- Know how to structure routes for a multi-feature application
- Be able to implement route redirects for improved UX

Objective

Configure the application's routing module to connect all feature components into a cohesive, navigable application. This is where the individual pieces we've built come together as a unified experience.

Why

The routing module is the skeleton of your application. It defines:

1. **Navigation Paths** — What URLs map to which components

2. **Loading Strategy** — Which features load immediately vs. on demand
3. **User Experience** — Default routes, redirects, and error handling

Angular Router Key Concepts

Concept	Purpose
`Routes`	Array of route configurations
`RouterModule.forRoot()`	Configures router at application level
`component`	Direct component reference for eager loading
`redirectTo`	Redirects one path to another
`pathMatch`	Determines how path is matched ('full' or 'prefix')

Angular Style Guide References

- Style 04-10: Use redirects for default routes

URL-First Architecture

The router is central to URL-First architecture. Every view in the application corresponds to a URL, and the URL is the source of truth for application state. The routing module defines what those URLs are.

What

Step 905.1: Create the App Routing Module

Create `src/app/app-routing.module.ts`:

```
// src/app/app-routing.module.ts
// VERSION 3 - Added 404 wildcard route

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PanelPopoutComponent } from './features/panel-popout/panel-popout.component';

/**
```

```
 * Application Routes Configuration
 *
 * Defines all navigation paths for the Generic-Prime application.
 * Uses direct component references (eager loading) for simplicity.
 *
 * Route Structure:
 * - / (root): Redirects to /home
 * - /home: Home landing page
 * - /discover: Main discovery interface (domain-agnostic)
 * - /panel/:gridId/:panelId/:type: Pop-out window routes
 */
const routes: Routes = [
  // Default route - redirect to home
  { path: '', redirectTo: 'home', pathMatch: 'full' },

  // Home page (eager loaded)
  { path: 'home', component: HomeComponent },

  // Discover page (domain-agnostic discovery interface)
  { path: 'discover', component: DiscoverComponent },

  // Pop-out panel route: /panel/:gridId/:panelId/:type
  // Used by DiscoverComponent.popOutPanel() to open panels in separate windows
  // PanelPopoutComponent handles BroadcastChannel communication with main window
  { path: 'panel/:gridId/:panelId/:type', component: PanelPopoutComponent },

  // 404 fallback route - redirect unmatched paths to home
  { path: '**', redirectTo: 'home' }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Route configuration explained:

Route	Purpose	Component
`''`	Root path, redirects to home	Immediate redirect
`'home'`	Home landing page	HomeComponent
`'discover'`	Main discovery interface	DiscoverComponent
`'panel/:gridId/:panelId/:type'`	Pop-out windows	PanelPopoutComponent
`'**'`	Catch-all for unknown routes	Redirect to home

Step 905.2: Component Registration in AppModule

All feature components are declared directly in `AppModule` rather than using separate feature modules with lazy loading. This approach:

1. Simplifies the module structure
2. Ensures all components are available immediately
3. Avoids complexity of managing multiple feature modules

In `src/app/app.module.ts`:

```
// src/app/app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { HttpClientModule } from '@angular/common/http';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PanelPopoutComponent } from './features/panel-popout/panel-popout.component';
import { AutomobileComponent } from './features/automobile/automobile.component';

// Framework and UI modules
import { FrameworkModule } from './framework/framework.module';
import { PrimengModule } from './primeng.module';

@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    DiscoverComponent,
    PanelPopoutComponent,
    AutomobileComponent
  ],
  imports: [
    BrowserModule,
    BrowserAnimationsModule,
    HttpClientModule,
    AppRoutingModule,
    FrameworkModule,
    PrimengModule
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Why no lazy loading?

For this application, all features are tightly integrated and benefit from being eagerly loaded. The framework components and domain configurations are shared across features, making lazy loading less beneficial. The simplified approach reduces complexity while maintaining good performance for a medium-sized application.

Step 905.3: Understanding Route Parameters

The panel pop-out route uses parameters to identify what component to show:

```
/panel/:gridId/:panelId/:type
       ■       ■        ■
       ■       ■        ■■■ Component type (chart, picker, etc.)
       ■       ■■■ Unique panel identifier
       ■■■ Source grid identifier (e.g., discover)
```

Example URLs:

URL	Meaning
`/panel/discover/chart-manufacturer/chart`	Manufacturer chart from discover page
`/panel/discover/query-control/query-control`	Query control panel
`/panel/discover/basic-results-table/basic-results`	Results table
`/panel/discover/manufacturer-model-picker/picker`	Manufacturer-model picker
`/panel/discover/statistics-panel-2/statistics-2`	Statistics panel

The PanelPopoutComponent extracts these parameters to:

1. Initialize BroadcastChannel with the right panel ID
2. Determine which content to render based on panel type
3. Set up communication with the main window

Step 905.4: Update AppComponent for Router Outlet

Ensure `src/app/app.component.ts` includes the router outlet:

Feature	Purpose
`*ngIf="!isPopOut"`	Hides header in pop-out windows
`routerLinkActive="active"`	Highlights current route in navigation
`[class.popout-content]`	Removes padding for pop-out content

Route change detection	Updates `isPopOut` on navigation

Step 905.5: Router Configuration

The `RouterModule.forRoot()` configuration uses defaults:

```
RouterModule.forRoot(routes)
```

No additional options are configured. The URL-First architecture is implemented through the `UrlStateService` and `ResourceManagementService` rather than router options.

Verification

1. Build the Application

```
$ ng build
```

Expected: Build succeeds with no errors.

2. Serve and Navigate

```
$ ng serve --open
```

Test these navigation paths:

Action	Expected Result
Open `http://localhost:4200`	Redirects to `/home`
Click "Automobiles" card	Navigates to `/discover`
Enter unknown URL like `/foo`	Redirects to `/home`

3. Verify Component Loading

Open browser DevTools (Network tab) and observe:

1. Initial load: All components are bundled together (eager loading)

2. No additional network requests on navigation between routes

4. Verify Query Param Persistence

1. Navigate to `/discover?manufacturer=Toyota`
2. Note the query parameter in URL
3. Refresh the page
4. Query parameter should persist (handled by UrlStateService)

Chapter 906

Module Assembly

```
// src/app/app.module.ts
// VERSION 3 (Phase 3+) - Full framework integration

import { NgModule, ErrorHandler, Injector } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';
import { FormsModule } from '@angular/forms';
import { MessageService } from 'primeng/api';
import { DragDropModule } from '@angular/cdk/drag-drop';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { PrimengModule } from './primeng.module';
import { FrameworkModule } from './framework/framework.module';
import { GlobalErrorHandler } from './framework/services/global-error.handler';
import { HttpErrorInterceptor } from './framework/services/http-error.interceptor';
import { DOMAIN_CONFIG } from './framework/services/domain-config-registry.service';
import { createAutomobileDomainConfig } from './domain-config/automobile';

// Feature Components
import { HomeComponent } from './features/home/home.component';
import { DiscoverComponent } from './features/discover/discover.component';
import { PopoutComponent } from './features/popout/popout.component';
import { PanelPopoutComponent } from './features/panel-popout/panel-popout.component';
import { AutomobileComponent } from './features/automobile/automobile.component';

/**
 * Root Application Module (AppModule)
 *
 * Central configuration and bootstrapping module for the vvroom Angular application.
 * Aggregates all feature components, framework services, and third-party library modules.
 *
 * @class AppModule
 * @see AppComponent - Root component
 * @see FrameworkModule - Core framework services and components
 * @see PrimengModule - UI component library configuration
 * @see GlobalErrorHandler - Global error handling service
 */
@NgModule({
  declarations: [
    AppComponent,
    HomeComponent,
    DiscoverComponent,
    PopoutComponent,
    PanelPopoutComponent,
    AutomobileComponent
  ],
  imports: [
    BrowserModule,
    BrowserAnimationsModule,
```

```
    HttpClientModule,
    FormsModule,            // Required for ngModel binding
    DragDropModule,         // CDK drag-drop for panel reordering
    AppRoutingModule,
    PrimengModule,          // UI component library
    FrameworkModule         // Core framework components
  ],
  providers: [
    MessageService,  // PrimeNG toast/message service
    {
      provide: ErrorHandler,
      useClass: GlobalErrorHandler
    },
    {
      provide: HTTP_INTERCEPTORS,
      useClass: HttpErrorInterceptor,
      multi: true
    },
    {
      provide: DOMAIN_CONFIG,
      useFactory: createAutomobileDomainConfig,
      deps: [Injector]
    }
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Module configuration explained:

Section	Purpose
`declarations`	All feature components (eagerly loaded, not using separate feature modules)
`imports`	Angular core modules plus framework and UI modules
`providers`	Services and tokens available globally
`bootstrap`	The root component to start the application

Key imports:

Module	Purpose
`FormsModule`	Required for `ngModel` binding in forms
`DragDropModule`	CDK drag-drop for panel reordering in DiscoverComponent
`PrimengModule`	UI component library (buttons, tables, tooltips, etc.)
`FrameworkModule`	Core framework components (QueryControl, BasePicker, etc.)

Key providers:

Provider	Purpose
`MessageService`	PrimeNG service for toast notifications
`GlobalErrorHandler`	Catches unhandled JavaScript errors
`HttpErrorInterceptor`	Catches HTTP errors for consistent handling
`DOMAIN_CONFIG`	Provides automobile domain configuration via factory

Chapter 907

Final Integration

907: Final Integration

Status: Planning

Depends On: 901-906 (All Phase 9 components)

Blocks: None (Final document)

Learning Objectives

After completing this section, you will:

- Understand how all the pieces of the vvroom application fit together
- Be able to perform comprehensive integration testing
- Have a fully functional production-ready Angular application

---

Objective

Complete the final integration of the vvroom application by verifying all components work together, performing comprehensive testing, and ensuring the application is ready for production use. This is the culmination of everything you've built throughout this book.

Congratulations!

You've made it. From an empty Angular project to a fully-featured data exploration application, you've typed every line of code and understood every decision. Let's take a moment to appreciate what you've built.

What You've Accomplished

Over the course of this book, you have:

Phase 0: API Contract & Naming

- Defined the API contract that drives the entire application
- Established naming conventions separating framework from domain code

Phase 1: Foundation

- Created the project structure from scratch
- Configured routing and environment settings
- Built the application shell

Phase 2: Framework Models

- Defined TypeScript interfaces for type safety
- Created generic models that work with any domain
- Learned how interfaces serve as executable documentation

Phase 3: Framework Services

- Built the URL-First state management system
- Created services for API communication, error handling, and pop-out windows
- Implemented the service layer that powers the entire application

Phase 4-6: Automobile Domain

- Defined automobile-specific models, adapters, and configurations
- Created the bridge between framework and domain
- Configured filters, tables, and charts for automobile data

Phase 7: Chart Data Sources

- Implemented visualization data transformations
- Created reusable chart source patterns

Phase 8: Framework Components

- Built reusable UI components for queries, charts, tables, and pickers
- Created components that work with any domain configuration
- Implemented the visual layer of the application

Phase 9: Feature Components

- Orchestrated framework components into complete pages
- Built the Home, Automobile Landing, and Discover pages
- Implemented pop-out window support for multi-monitor workflows

- Configured routing and the root module

The Aha Moment

"I just built a production application. I understand every line."

This is what sets you apart. You didn't just follow a tutorial — you understand:

- Why the URL is the single source of truth
- How dependency injection wires everything together
- What makes a component reusable vs. domain-specific
- When to use BehaviorSubject vs. ReplaySubject
- How to debug RxJS streams
- Why TypeScript generics enable type-safe reuse

What

Step 907.1: Complete File Structure Verification

Let's verify the complete file structure of the application:

```
$ cd ~/projects/vvroom
$ find src/app -type f -name "*.ts" | wc -l
```

You should have approximately 60-70 TypeScript files across:

```
src/app/
■■■ app.component.ts
■■■ app.module.ts
■■■ app-routing.module.ts
■■■ domain-config/
■   ■■■ automobile/
■       ■■■ adapters/          (3 files)
■       ■■■ chart-sources/     (4 files)
■       ■■■ configs/           (6 files)
■       ■■■ models/            (3 files)
■       ■■■ automobile.domain-config.ts
■■■ features/
■   ■■■ home/                  (3 files)
■   ■■■ automobile/
■   ■   ■■■ automobile-discover/  (4 files)
■   ■   ■■■ (4 files)
■   ■■■ popout/                (4 files)
■■■ framework/
    ■■■ components/            (~27 files across 9 components)
```

```
    ■■■ models/                (8 files)
    ■■■ services/              (12 files)
    ■■■ tokens/                (1 file)
```

Step 907.2: Build Verification

Perform a production build to verify everything compiles correctly:

```
$ ng build --configuration production
```

Expected output:

```
✔ Browser application bundle generation complete.
✔ Copying assets complete.
✔ Index html generation complete.

Initial Chunk Files           | Names         |  Raw Size | Estimated Transfer Size
main.js                       | main          | 250.00 kB |              65.00 kB
polyfills.js                  | polyfills     | 180.00 kB |              57.00 kB
styles.css                    | styles        |  15.00 kB |               4.00 kB
runtime.js                    | runtime       |   2.00 kB |               1.00 kB

Lazy Chunk Files              | Names         |  Raw Size | Estimated Transfer Size
automobile-module.js          | automobile    |  80.00 kB |              20.00 kB
home-module.js                | home          |   5.00 kB |               2.00 kB
popout-module.js              | popout        |  10.00 kB |               3.00 kB

Build at: 2026-02-09T17:00:00.000Z - Hash: abc123def456 - Time: 15000ms
```

Key points:

- No errors or warnings
- Lazy chunks are generated for feature modules
- Bundle sizes are reasonable

Step 907.3: Integration Test Checklist

Run through this comprehensive test checklist:

Navigation Tests

Test	Expected Result	Pass?
Open `/`	Redirects to `/home`	[ ]
Click "Automobiles"	Navigates to `/automobiles`	[ ]
Click "Advanced Search"	Navigates to `/automobiles/discover`	[ ]
Click "Home" in header	Returns to `/home`	[ ]

Enter invalid URL `/xyz`	Redirects to `/home`	[ ]
Back button	Previous page loads	[ ]
Forward button	Next page loads	[ ]

URL State Tests

Test	Expected Result	Pass?
Apply filter on Discover	URL updates with query param	[ ]
Refresh page with filter	Filter persists	[ ]
Copy URL to new tab	Same state loads	[ ]
Clear all filters	URL resets to clean state	[ ]
Apply multiple filters	All params in URL	[ ]

Data Loading Tests

Test	Expected Result	Pass?
Load Discover page	Results count appears	[ ]
Apply manufacturer filter	Filtered results load	[ ]
Change pagination	Page param updates, new data loads	[ ]
Sort table column	Sort param updates, data re-orders	[ ]
Loading indicator	Shows during API calls	[ ]

Chart Interaction Tests

Test	Expected Result	Pass?
Charts display data	Bars/segments visible	[ ]
Hover on chart element	Tooltip appears	[ ]
Click chart bar (filter mode)	Filter applied, data updates	[ ]
Click chart bar (highlight mode)	Highlight added, visual change	[ ]
Multiple chart sources	All charts update with filters	[ ]

Pop-out Tests

Test	Expected Result	Pass?
Click pop-out button	New window opens	[ ]
Pop-out shows data	State synced from main	[ ]
Change filter in main	Pop-out updates	[ ]
Close pop-out	Panel reappears in main	[ ]
Pop-out header hidden	No navigation in pop-out	[ ]

Query Control Tests

Test	Expected Result	Pass?
Dropdown filter works	Options load, selection applies	[ ]
Text search works	Search executes on enter	[ ]
Clear individual filter	Only that filter removed	[ ]
Clear all filters	All filters removed	[ ]
Filter chips display	Active filters shown	[ ]

Error Handling Tests

Test	Expected Result	Pass?
API error occurs	Error notification shown	[ ]
Network timeout	Graceful error message	[ ]
Invalid filter value	Validation feedback	[ ]
404 route	Redirects to home	[ ]

Step 907.4: Performance Verification

Check application performance:

```
# Development server
$ ng serve

# In another terminal, run Lighthouse or similar tool
# Or manually check DevTools Performance tab
```

Performance targets:

- First Contentful Paint: < 1.5s
- Time to Interactive: < 3s
- Largest Contentful Paint: < 2.5s
- No layout shifts during load

Step 907.5: Code Quality Check

Run linting and formatting:

```
$ ng lint
```

Expected: No errors. (Warnings acceptable for rough draft.)

Step 907.6: Final Review of Key Patterns

Let's summarize the key architectural patterns used throughout the application:

URL-First State Management

```
URL (source of truth)
    ↓
UrlStateService (URL → Params)
    ↓
ResourceManagementService (Params → Filters → API)
    ↓
Components (via async pipe)
```

Every piece of state is derived from the URL. This enables:

- Deep linking
- Browser history support
- State sharing via URL
- Debuggability

Dependency Injection Hierarchy

```
AppModule (global providers)
    ↓
Feature Module (lazy-loaded)
    ↓
Component (component-level providers)
    ↓
Template (via async pipe)
```

Services are provided at the appropriate level:

- Singletons at root
- Per-page instances at component level

Component Composition

```
Feature Component (orchestrator)
    ■■■ Framework Component (reusable)
    ■   ■■■ Domain Config (configuration)
    ■■■ Framework Component (reusable)
        ■■■ Domain Config (configuration)
```

Framework components are reusable shells. Domain configuration provides the specifics.

Verification

Final Build and Serve

```
$ ng build --configuration production
$ npx http-server dist/vvroom -p 8080
```

Open `http://localhost:8080` and verify:

1. Application loads without errors
2. All pages are accessible
3. Data loads correctly
4. Pop-outs work
5. URL state persists

Commit the Final State

```
$ git add .
$ git commit -m "Section 907: Complete vvroom application"
$ git tag phase-9-complete
```

What You've Built

Application Statistics

Metric	Value
TypeScript Files	~70
Lines of Code	~11,000
Components	~15
Services	~12
Interfaces	~10
Feature Modules	3

Capabilities

- **Data Exploration:** Filter, sort, and paginate through automobile data
- **Visualization:** Charts for manufacturers, years, body classes
- **Multi-Monitor:** Pop-out any panel to a separate window
- **URL Persistence:** Every view has a shareable URL
- **Error Handling:** Graceful error messages and recovery
- **Performance:** Lazy loading, OnPush change detection

Technologies Used

Technology	Purpose
Angular 13	Application framework
TypeScript	Type-safe JavaScript
RxJS	Reactive state management
Angular Router	Navigation
HttpClient	API communication
BroadcastChannel	Pop-out communication

Key Takeaways

1. **Architecture matters** — The URL-First pattern made state management simple and predictable

2. **Generics enable reuse** — Framework components work with any domain configuration
3. **Separation of concerns** — Framework, domain, and feature code have clear boundaries
4. **TypeScript is your friend** — Type safety caught errors before runtime
5. **RxJS is powerful** — Reactive streams simplified complex async operations

---

Acceptance Criteria

- [ ] Production build completes without errors
- [ ] All navigation paths work correctly
- [ ] URL state persistence works (refresh preserves state)
- [ ] Data loads and displays correctly
- [ ] Charts are interactive
- [ ] Pop-out windows sync with main window
- [ ] Error handling shows appropriate messages
- [ ] Performance is acceptable (no jank, reasonable load times)
- [ ] Code passes linting

Where to Go from Here

You've built a complete application, but there's always more to learn:

Immediate Extensions

- Add more filter types (date ranges, multi-select)
- Implement data export (CSV, Excel)
- Add user preferences persistence
- Create custom chart types

Advanced Topics

- Unit testing with Jest
- E2E testing with Playwright
- State management with NgRx
- Server-side rendering with Angular Universal

- Progressive Web App (PWA) features

Architecture Evolution

- Micro-frontends
- Module federation
- GraphQL integration
- Real-time updates with WebSockets

The Companion Book Opportunity

Because you followed strict naming conventions and separated framework from domain code, expanding vvroom to support additional domains is straightforward. A companion book could cover:

- "Adding Agriculture to Vvroom" (just domain configuration)
- "Adding Chemistry to Vvroom" (just domain configuration)

The framework code you wrote is truly reusable.

Final Words

Congratulations. You've done something remarkable.

You didn't just copy code from a tutorial. You didn't rely on magic you don't understand. You built a production-quality Angular application from the ground up, and you know why every line is there.

This knowledge will serve you well. Whether you're building your own applications, contributing to enterprise projects, or teaching others, you now have a deep understanding of Angular architecture that most developers never achieve.

The URL-First pattern, the generic component architecture, the service layer design — these patterns will apply to projects far beyond vvroom.

Thank you for taking this journey. Now go build something amazing.

The Complete Journey

```
Phase 0: API Contract          ■■→  "The API defines everything"
```

```
    ■
Phase 1: Foundation            ■■→  "Routes are the skeleton"
    ■
Interlude A: Generics          ■■→  "Type safety without duplication"
    ■
Phase 2: Framework Models      ■■→  "Interfaces are documentation"
    ■
Interlude B: RxJS              ■■→  "Observables model change"
    ■
Phase 3: Framework Services    ■■→  "The URL is the source of truth"
    ■
Phase 4: Domain Models         ■■→  "Models shaped by the API"
    ■
Phase 5: Domain Adapters       ■■→  "Adapters isolate change"
    ■
Phase 6: Domain Configs        ■■→  "Configuration is code"
    ■
Phase 7: Chart Sources         ■■→  "Transform data for visualization"
    ■
Phase 8: Framework Components  ■■→  "Generic + specific = reusable"
    ■
Phase 9: Feature Components    ■■→  "I understand every line"
```

Welcome to the club of Angular architects.

Phase 9B: Testing

Chapter 951: Unit Testing Setup

Chapter 952: Integration Testing

Chapter 953: E2E Testing

Chapter 954: Performance Testing

Chapter 951

Unit Testing Setup

951: RxJS Operator Reference

Status: Reference Document

Type: Appendix

Learning Objectives

After reading this reference, you will:

- Have a quick reference for all RxJS operators used in vvroom
- Understand when to use each operator
- Be able to look up operator behavior without leaving the book

---

Overview

This appendix provides a quick reference for every RxJS operator used in the vvroom application. Operators are grouped by category for easy lookup.

Creation Operators

`of()`

Creates an Observable that emits the provided values synchronously.

```
import { of } from 'rxjs';
```

```
of('a', 'b', 'c').subscribe(x => console.log(x));
// Output: 'a', 'b', 'c' (then completes)
```

Used in vvroom: Error handling fallbacks, mock data

`Subject`

A Subject is both an Observable and an Observer. You can push values into it and subscribe to it.

```
import { Subject } from 'rxjs';

const subject = new Subject<string>();

subject.subscribe(x => console.log(x));
subject.next('hello'); // Output: 'hello'
```

Used in vvroom: `destroy$` for cleanup, event buses

`BehaviorSubject`

A Subject that requires an initial value and emits the current value to new subscribers.

```
import { BehaviorSubject } from 'rxjs';

const subject = new BehaviorSubject<number>(0);

subject.subscribe(x => console.log('A:', x)); // Output: 'A: 0'
subject.next(1);                              // Output: 'A: 1'
subject.subscribe(x => console.log('B:', x)); // Output: 'B: 1'
```

Used in vvroom: `stateSubject` in ResourceManagementService, `paramsSubject` in UrlStateService

---

`ReplaySubject`

A Subject that replays a specified number of previous values to new subscribers.

```
import { ReplaySubject } from 'rxjs';

const subject = new ReplaySubject<number>(2); // Buffer last 2 values

subject.next(1);
subject.next(2);
```

```
subject.next(3);

subject.subscribe(x => console.log(x)); // Output: 2, 3
```

Used in vvroom: PopOutContextService for message replay

`timer()`

Creates an Observable that emits after a delay.

```
import { timer } from 'rxjs';

timer(1000).subscribe(() => console.log('1 second passed'));
```

Used in vvroom: Request debouncing

`throwError()`

Creates an Observable that immediately emits an error.

```
import { throwError } from 'rxjs';

throwError(() => new Error('Something went wrong'))
  .subscribe({
    error: err => console.error(err.message)
  });
```

Used in vvroom: Error propagation in services

Transformation Operators

`map()`

Transforms each emitted value using a projection function.

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

of(1, 2, 3).pipe(
  map(x => x * 10)
```

```
).subscribe(x => console.log(x));
// Output: 10, 20, 30
```

Used in vvroom: Transforming API responses, extracting state properties

---

`switchMap()`

Maps to an inner Observable and cancels the previous inner Observable on each emission.

```
import { of, interval } from 'rxjs';
import { switchMap, take } from 'rxjs/operators';

of('A', 'B').pipe(
  switchMap(letter => interval(100).pipe(
    take(3),
    map(i => letter + i)
  ))
).subscribe(x => console.log(x));
// Only 'B' values emit (A is cancelled)
// Output: B0, B1, B2
```

Used in vvroom: API calls that cancel previous requests when filters change

When to use: When only the latest request matters (search, autocomplete)

`mergeMap()`

Maps to an inner Observable and runs all inner Observables concurrently.

```
import { of } from 'rxjs';
import { mergeMap, delay } from 'rxjs/operators';

of(1, 2, 3).pipe(
  mergeMap(x => of(x).pipe(delay(100)))
).subscribe(x => console.log(x));
// Output: 1, 2, 3 (all run in parallel)
```

When to use: When all requests should complete (batch operations)

`concatMap()`

Maps to an inner Observable and waits for each to complete before starting the next.

```
import { of } from 'rxjs';
import { concatMap, delay } from 'rxjs/operators';

of(1, 2, 3).pipe(
  concatMap(x => of(x).pipe(delay(100)))
).subscribe(x => console.log(x));
// Output: 1 (wait) 2 (wait) 3 (sequential)
```

When to use: When order matters and each request must complete first

---

Filtering Operators

`filter()`

Emits only values that pass the predicate function.

```
import { of } from 'rxjs';
import { filter } from 'rxjs/operators';

of(1, 2, 3, 4, 5).pipe(
  filter(x => x % 2 === 0)
).subscribe(x => console.log(x));
// Output: 2, 4
```

Used in vvroom: Filtering router events to NavigationEnd

`distinctUntilChanged()`

Emits only when the current value differs from the previous.

```
import { of } from 'rxjs';
import { distinctUntilChanged } from 'rxjs/operators';

of(1, 1, 2, 2, 3, 1).pipe(
  distinctUntilChanged()
).subscribe(x => console.log(x));
// Output: 1, 2, 3, 1
```

With comparator:

```
of({ id: 1 }, { id: 1 }, { id: 2 }).pipe(
  distinctUntilChanged((a, b) => a.id === b.id)
).subscribe(x => console.log(x));
// Output: { id: 1 }, { id: 2 }
```

Used in vvroom: Preventing duplicate API calls when state hasn't changed

---

`take()`

Emits only the first N values then completes.

```
import { interval } from 'rxjs';
import { take } from 'rxjs/operators';

interval(100).pipe(
  take(3)
).subscribe(x => console.log(x));
// Output: 0, 1, 2 (then completes)
```

Used in vvroom: Taking single values from streams

`takeUntil()`

Emits values until a notifier Observable emits.

```
import { interval, Subject } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

const stop$ = new Subject<void>();

interval(100).pipe(
  takeUntil(stop$)
).subscribe(x => console.log(x));

setTimeout(() => stop$.next(), 350);
// Output: 0, 1, 2 (then stops)
```

Used in vvroom: Component cleanup pattern with `destroy$`

`debounceTime()`

Emits a value only after a specified time has passed without new emissions.

```
import { Subject } from 'rxjs';
import { debounceTime } from 'rxjs/operators';

const search$ = new Subject<string>();
```

```
search$.pipe(
  debounceTime(300)
).subscribe(x => console.log('Search:', x));

search$.next('h');
search$.next('he');
search$.next('hel');
search$.next('help');
// Output: 'Search: help' (after 300ms pause)
```

Used in vvroom: Search input debouncing

Error Handling Operators

`catchError()`

Catches errors on the Observable and returns a new Observable or throws.

```
import { of, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';

throwError(() => new Error('Oops!')).pipe(
  catchError(err => {
    console.error(err.message);
    return of('fallback value');
  })
).subscribe(x => console.log(x));
// Output: 'Oops!' (error), 'fallback value'
```

Used in vvroom: API error handling, graceful degradation

`retry()`

Retries the source Observable a specified number of times on error.

```
import { interval, throwError } from 'rxjs';
import { mergeMap, retry } from 'rxjs/operators';

let attempts = 0;

interval(100).pipe(
  mergeMap(() => {
    attempts++;
    if (attempts < 3) {
      return throwError(() => new Error('Retry'));
    }
```

```
    return of('success');
  }),
  retry(2)
).subscribe(x => console.log(x));
```

Used in vvroom: HTTP request retry in interceptor

Utility Operators

`tap()`

Performs side effects without modifying the stream.

```
import { of } from 'rxjs';
import { tap, map } from 'rxjs/operators';

of(1, 2, 3).pipe(
  tap(x => console.log('Before:', x)),
  map(x => x * 10),
  tap(x => console.log('After:', x))
).subscribe();
// Output: Before: 1, After: 10, Before: 2, After: 20, ...
```

Used in vvroom: Logging, debugging, caching side effects

`finalize()`

Performs an action when the Observable completes or errors.

```
import { of, throwError } from 'rxjs';
import { finalize } from 'rxjs/operators';

of(1, 2, 3).pipe(
  finalize(() => console.log('Done!'))
).subscribe(x => console.log(x));
// Output: 1, 2, 3, 'Done!'
```

Used in vvroom: Resetting loading state after API calls

`delay()`

Delays emission by a specified time.

```
import { of } from 'rxjs';
import { delay } from 'rxjs/operators';

of('delayed').pipe(
  delay(1000)
).subscribe(x => console.log(x));
// Output: 'delayed' (after 1 second)
```

Used in vvroom: Testing, simulated network latency

`timeout()`

Errors if no value is emitted within a specified time.

```
import { of, NEVER } from 'rxjs';
import { timeout, catchError } from 'rxjs/operators';

NEVER.pipe(
  timeout(1000),
  catchError(err => of('Timeout!'))
).subscribe(x => console.log(x));
// Output: 'Timeout!' (after 1 second)
```

Used in vvroom: API request timeouts

Combination Operators

`combineLatest()`

Combines multiple Observables and emits when any source emits (after all have emitted at least once).

```
import { combineLatest, of, interval } from 'rxjs';
import { take, map } from 'rxjs/operators';

combineLatest([
  of('A'),
  interval(100).pipe(take(3))
]).subscribe(([letter, num]) => console.log(letter, num));
// Output: A 0, A 1, A 2
```

Used in vvroom: Combining filters from multiple sources

`merge()`

Merges multiple Observables into a single stream.

```
import { merge, interval } from 'rxjs';
import { take, map } from 'rxjs/operators';

merge(
  interval(100).pipe(take(2), map(x => 'A' + x)),
  interval(150).pipe(take(2), map(x => 'B' + x))
).subscribe(x => console.log(x));
// Output: A0, B0, A1, B1 (interleaved by timing)
```

Used in vvroom: Merging event streams

`forkJoin()`

Waits for all Observables to complete, then emits an array of the last values.

```
import { forkJoin, of } from 'rxjs';
import { delay } from 'rxjs/operators';

forkJoin([
  of('A').pipe(delay(100)),
  of('B').pipe(delay(200)),
  of('C').pipe(delay(300))
]).subscribe(x => console.log(x));
// Output: ['A', 'B', 'C'] (after 300ms)
```

Used in vvroom: Parallel API requests that must all complete

Quick Reference Table

Operator	Category	Purpose	Cancels Previous?
`of`	Creation	Emit static values	N/A
`Subject`	Creation	Observable + Observer	N/A
`BehaviorSubject`	Creation	Subject with initial value	N/A

`map`	Transform	Transform values	No
`switchMap`	Transform	Map + cancel previous	Yes
`mergeMap`	Transform	Map + run concurrent	No
`concatMap`	Transform	Map + run sequential	No
`filter`	Filter	Emit matching values	No
`distinctUntilChanged`	Filter	Skip duplicates	No
`take`	Filter	Emit first N	No
`takeUntil`	Filter	Emit until notifier	No
`debounceTime`	Filter	Delay until pause	No
`catchError`	Error	Handle errors	No
`retry`	Error	Retry on error	No
`tap`	Utility	Side effects	No
`finalize`	Utility	Cleanup on complete	No
`combineLatest`	Combine	Combine latest values	No
`merge`	Combine	Merge streams	No
`forkJoin`	Combine	Wait for all	No

Key Takeaways

1. **Use** `switchMap` **for search/filter operations** — Cancels outdated requests
2. **Use** `BehaviorSubject` **for state** — Always has a current value
3. **Use** `takeUntil(destroy$)` **for cleanup** — Prevents memory leaks
4. **Use** `distinctUntilChanged` **to prevent duplicate work** — Especially with API calls
5. **Use** `catchError` **to handle errors gracefully** — Don't let streams die unexpectedly

---

Further Reading

- RxJS Official Documentation
- Learn RxJS
- RxJS Marbles — Interactive marble diagrams

Chapter 952

Integration Testing

952: TypeScript Generics Reference

Status: Reference Document

Type: Appendix

Learning Objectives

After reading this reference, you will:

- Have a quick reference for generic patterns used in vvroom
- Understand advanced generic techniques beyond the primer
- Be able to look up type utilities without leaving the book

Overview

This appendix extends the TypeScript Generics Primer (document 150) with advanced patterns and a complete reference of generic types used in the vvroom application.

Quick Reference: Generic Syntax

Declaration Syntax

Syntax	Meaning
<T>	Single type parameter

`<T, U>`	Multiple type parameters
`<T extends Base>`	Constrained type parameter
`<T = Default>`	Type parameter with default
`<T extends Base = Default>`	Constrained with default

Usage Syntax

Syntax	Meaning
`Array<string>`	Concrete instantiation
`Partial<User>`	Utility type application
`keyof T`	Get keys of T as union
`T[K]`	Indexed access type
`T extends U ? X : Y`	Conditional type

Vvroom Generic Patterns

The Domain Config Pattern

The central generic pattern in vvroom:

```
interface DomainConfig<TFilters, TData, TStatistics> {
  domainKey: string;
  displayName: string;
  urlMapper: IFilterUrlMapper<TFilters>;
  apiAdapter: IApiAdapter<TFilters, TData, TStatistics>;
  tableConfig: TableConfig<TData>;
  chartDataSources?: Record<string, ChartDataSource<TStatistics>>;
}
```

Type flow:

```
DomainConfig<AutomobileFilters, VehicleResult, VehicleStatistics>
     ■               ■               ■               ■
     ■               ■               ■               ■■■ chartDataSources value types
     ■               ■               ■■■ apiAdapter, tableConfig
     ■               ■■■ apiAdapter, urlMapper
     ■■■ The whole thing
```

The Adapter Pattern

Adapters connect framework code to domain-specific implementations:

```
interface IApiAdapter<TFilters, TData, TStatistics> {
  fetchData(filters: TFilters): Observable<ApiResponse<TData>>;
  fetchStatistics(filters: TFilters): Observable<TStatistics>;
}

// Concrete implementation
class AutomobileApiAdapter implements IApiAdapter<
  AutomobileFilters,
  VehicleResult,
  VehicleStatistics
> {
  fetchData(filters: AutomobileFilters): Observable<ApiResponse<VehicleResult>> {
    // Implementation
  }
}
```

The Service Generic Pattern

Services that work with any domain use type parameters:

```
class ResourceManagementService<TFilters, TData, TStatistics> {
  private readonly state$ = new BehaviorSubject<ResourceState<TFilters, TData, TStatistics>>(
    initialState
  );

  get data$(): Observable<TData[] | undefined> {
    return this.state$.pipe(map(s => s.data));
  }
}
```

Built-in Utility Types

Partial<T>

Makes all properties optional:

```
interface User {
  id: number;
  name: string;
  email: string;
}
```

```
// All properties optional
type PartialUser = Partial<User>;
// { id?: number; name?: string; email?: string; }

// Use case: Update operations
function updateUser(id: number, updates: Partial<User>) {
  // Only changed fields are required
}

updateUser(1, { name: 'New Name' }); // OK - only name provided
```

Used in vvroom: Filter updates, partial state changes

Required<T>

Makes all properties required (opposite of Partial):

```
interface Config {
  debug?: boolean;
  timeout?: number;
}

type RequiredConfig = Required<Config>;
// { debug: boolean; timeout: number; }
```

Readonly<T>

Makes all properties readonly:

```
interface State {
  count: number;
  items: string[];
}

type ReadonlyState = Readonly<State>;
// { readonly count: number; readonly items: string[]; }

const state: ReadonlyState = { count: 0, items: [] };
state.count = 1; // Error: Cannot assign to 'count'
```

Used in vvroom: Immutable state patterns

Pick<T, K>

Creates a type with only the specified properties:

```
interface Vehicle {
  vin: string;
  make: string;
  model: string;
  year: number;
  color: string;
}

type VehicleIdentity = Pick<Vehicle, 'vin' | 'make' | 'model'>;
// { vin: string; make: string; model: string; }
```

Used in vvroom: Extracting subsets for display or API calls

Omit<T, K>

Creates a type without the specified properties:

```
interface Vehicle {
  vin: string;
  make: string;
  model: string;
  year: number;
  internalId: string; // Don't send to API
}

type VehicleForApi = Omit<Vehicle, 'internalId'>;
// { vin: string; make: string; model: string; year: number; }
```

Record<K, V>

Creates an object type with keys of type K and values of type V:

```
type ChartSourceMap = Record<string, ChartDataSource>;
// { [key: string]: ChartDataSource }

const sources: ChartSourceMap = {
  'manufacturer': new ManufacturerChartSource(),
  'year': new YearChartSource()
};
```

Used in vvroom: `chartDataSources`, URL parameters

Extract<T, U>

Extracts types from T that are assignable to U:

```
type EventTypes = 'click' | 'focus' | 'blur' | 'scroll' | 'resize';
type MouseEvents = Extract<EventTypes, 'click' | 'scroll'>;
// 'click' | 'scroll'
```

Exclude<T, U>

Excludes types from T that are assignable to U:

```
type EventTypes = 'click' | 'focus' | 'blur' | 'scroll' | 'resize';
type KeyboardEvents = Exclude<EventTypes, 'click' | 'scroll' | 'resize'>;
// 'focus' | 'blur'
```

NonNullable<T>

Removes null and undefined from T:

```
type MaybeString = string | null | undefined;
type DefinitelyString = NonNullable<MaybeString>;
// string
```

ReturnType<T>

Gets the return type of a function type:

```
function createFilters(): AutomobileFilters {
  return new AutomobileFilters();
}

type Filters = ReturnType<typeof createFilters>;
// AutomobileFilters
```

Parameters<T>

Gets the parameter types of a function as a tuple:

```
function fetchData(filters: AutomobileFilters, page: number): Promise<VehicleResult[]> {
  // ...
}

type FetchParams = Parameters<typeof fetchData>;
// [AutomobileFilters, number]
```

Advanced Patterns

Mapped Types

Transform properties of a type:

```
// Make all properties nullable
type Nullable<T> = {
  [P in keyof T]: T[P] | null;
};

interface User {
  name: string;
  age: number;
}

type NullableUser = Nullable<User>;
// { name: string | null; age: number | null; }
```

Conditional Types

Types that depend on conditions:

```
type IsArray<T> = T extends any[] ? true : false;

type A = IsArray<string[]>;  // true
type B = IsArray<string>;    // false
```

Practical example:

```
// Unwrap array types, leave others alone
type Unwrap<T> = T extends (infer U)[] ? U : T;

type A = Unwrap<string[]>;  // string
type B = Unwrap<number>;    // number
```

Template Literal Types

Create types from string patterns:

```
type FilterKey = 'manufacturer' | 'year' | 'model';
type HighlightKey = `h_${FilterKey}`;
// 'h_manufacturer' | 'h_year' | 'h_model'

// Used in URL state management
function isHighlightParam(key: string): key is HighlightKey {
  return key.startsWith('h_');
}
```

Type Guards with Generics

User-Defined Type Guards

Narrow types safely:

```
interface ApiError {
  code: number;
  message: string;
}

interface ApiSuccess<T> {
  data: T;
}

type ApiResponse<T> = ApiError | ApiSuccess<T>;

function isSuccess<T>(response: ApiResponse<T>): response is ApiSuccess<T> {
  return 'data' in response;
}

// Usage
function handleResponse<T>(response: ApiResponse<T>): T | null {
  if (isSuccess(response)) {
    return response.data;  // TypeScript knows this is ApiSuccess<T>
  }
  console.error(response.message);  // TypeScript knows this is ApiError
  return null;
}
```

Generic Constraints in Vvroom

Constraining to Interfaces

```
interface HasId {
  id: string;
}

function findById<T extends HasId>(items: T[], id: string): T | undefined {
  return items.find(item => item.id === id);
}

// Works with any type that has an 'id' property
const vehicle = findById(vehicles, 'VIN123');
const user = findById(users, 'USER456');
```

Constraining to Keys

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const vehicle = { make: 'Toyota', year: 2022 };
const make = getProperty(vehicle, 'make');  // string
const year = getProperty(vehicle, 'year');  // number
const bad = getProperty(vehicle, 'color');  // Error: 'color' not in keyof vehicle
```

Observable Generic Patterns

RxJS uses generics extensively:

```
// Observable of specific type
statistics$: Observable<VehicleStatistics>;

// Subject with type parameter
private readonly state$ = new BehaviorSubject<ResourceState<TFilters, TData>>(initial);

// Operators preserve/transform types
this.state$.pipe(
  map(state => state.data),      // Observable<TData[] | undefined>
  filter((data): data is TData[] => data !== undefined),  // Observable<TData[]>
  map(data => data.length)       // Observable<number>
);
```

Common Mistakes

Mistake 1: Missing Type Arguments

```
// Wrong - Map needs type arguments
const cache = new Map();  // Map<any, any>

// Right
const cache = new Map<string, VehicleResult>();
```

Mistake 2: Overly Narrow Constraints

```
// Too restrictive - only works with AutomobileFilters
function process<T extends AutomobileFilters>(filters: T) { }

// Better - works with any filter type
function process<T>(filters: T) { }

// Or if you need specific properties
function process<T extends { page?: number }>(filters: T) { }
```

Mistake 3: Ignoring Inference

```
// Unnecessary - TypeScript infers T
const result = identity<string>('hello');

// Let TypeScript infer
const result = identity('hello');  // T is inferred as string
```

Mistake 4: Using `any` Instead of Generics

```
// Bad - loses type safety
function firstElement(arr: any[]): any {
  return arr[0];
}

// Good - preserves type
function firstElement<T>(arr: T[]): T | undefined {
  return arr[0];
}
```

Quick Reference Table: Vvroom Generic Types

Type	Purpose	Type Parameters
`DomainConfig<F, D, S>`	Domain configuration	Filters, Data, Statistics

`IApiAdapter<F, D, S>`	API adapter interface	Filters, Data, Statistics
`IFilterUrlMapper<F>`	URL mapping interface	Filters
`TableConfig<D>`	Table configuration	Data row type
`ChartDataSource<S>`	Chart data transformation	Statistics type
`ResourceState<F, D, S>`	Service state	Filters, Data, Statistics
`ApiResponse<D>`	API response wrapper	Data type
`Observable<T>`	RxJS observable	Emitted value type
`BehaviorSubject<T>`	RxJS subject with initial value	Value type

Key Takeaways

1. **Generics flow through the architecture** — From DomainConfig through services to components
2. **Utility types reduce boilerplate** — Use Partial, Pick, Omit instead of redefining
3. **Constraints ensure type safety** — Use `extends` to require specific properties
4. **Let TypeScript infer when possible** — Don't over-annotate

---

Further Reading

- TypeScript Handbook: Generics
- TypeScript Handbook: Utility Types
- TypeScript Handbook: Conditional Types

Chapter 953

E2E Testing

953: Debugging Guide

Status: Reference Document

Type: Appendix

Learning Objectives

After reading this guide, you will:

- Know how to diagnose common Angular and RxJS issues
- Understand how to use browser DevTools effectively
- Be able to debug URL-First state management problems

Overview

This appendix provides debugging strategies for the vvroom application. It covers Angular-specific debugging, RxJS observable debugging, and URL-First architecture troubleshooting.

Browser DevTools Essentials

Opening DevTools

Browser	Shortcut
Chrome/Edge	`F12` or `Ctrl+Shift+I` (Windows) / `Cmd+Option+I` (Mac)

Firefox	`F12` or `Ctrl+Shift+I` (Windows) / `Cmd+Option+I` (Mac)
Safari	`Cmd+Option+I` (enable in Preferences → Advanced first)

Essential Panels

Panel	Use For
Console	Error messages, `console.log` output
Network	API requests, response data, timing
Elements	DOM inspection, CSS debugging
Sources	Breakpoints, step debugging
Application	localStorage, sessionStorage, cookies

Console Debugging

Strategic Console Logging

```
// Bad - vague
console.log(filters);

// Good - contextual
console.log('[UrlStateService] Parsed filters:', filters);

// Better - with timestamp for async flows
console.log(`[${Date.now()}] [ResourceService] Fetching data for:`, filters);
```

Console Methods

Method	Use Case
`console.log()`	General output
`console.error()`	Errors (red in console)
`console.warn()`	Warnings (yellow in console)
`console.table()`	Display arrays/objects as table

`console.group()` / `console.groupEnd()`	Group related logs
`console.time()` / `console.timeEnd()`	Measure execution time

Debugging Objects

```
// See full object structure
console.dir(complexObject);

// Table view for arrays
console.table(vehicles);

// Group related logs
console.group('Filter Processing');
console.log('Input:', rawParams);
console.log('Parsed:', parsedFilters);
console.log('Validated:', validatedFilters);
console.groupEnd();
```

Angular-Specific Debugging

Angular DevTools Extension

Install the Angular DevTools browser extension for:

- Component tree inspection
- Change detection profiling
- Dependency injection debugging

Common Angular Errors

"Can't bind to 'X' since it isn't a known property"

Cause: Component/directive not imported in the module.

Solution:

```
// Check the module imports
@NgModule({
  imports: [
    CommonModule,      // For *ngIf, *ngFor
    FormsModule,       // For [(ngModel)]
    RouterModule,      // For routerLink
    SomeComponent      // For standalone components
```

```
  ]
})
```

"No provider for X"

Cause: Service not provided in module or component.

Solution:

```
// Option 1: providedIn root (preferred)
@Injectable({ providedIn: 'root' })
export class MyService { }

// Option 2: Provide in module
@NgModule({
  providers: [MyService]
})

// Option 3: Provide in component
@Component({
  providers: [MyService]
})
```

"ExpressionChangedAfterItHasBeenCheckedError"

Cause: Value changed during change detection cycle.

Solution:

```
// Wrap in setTimeout or use ChangeDetectorRef
constructor(private cdr: ChangeDetectorRef) {}

ngAfterViewInit(): void {
  // Defer the change
  setTimeout(() => {
    this.value = newValue;
    this.cdr.markForCheck();
  }, 0);
}
```

Template Binding Errors

Debug with JSON pipe:

```
<!-- See what the value actually is -->
<pre>{{ someValue | json }}</pre>

<!-- Check if observable has emitted -->
<pre>{{ observable$ | async | json }}</pre>
```

RxJS Debugging

The tap() Operator

Insert `tap()` to inspect stream values without modifying them:

```
this.urlState.params$.pipe(
  tap(params => console.log('[1] Raw params:', params)),
  map(params => this.parseFilters(params)),
  tap(filters => console.log('[2] Parsed filters:', filters)),
  switchMap(filters => this.apiService.fetch(filters)),
  tap(response => console.log('[3] API response:', response))
).subscribe();
```

Debugging Subscription Issues

"Observable not emitting"

Check these in order:

1. **Is it subscribed?**

```
// Cold observables don't run without subscription
observable$.subscribe();  // or use | async in template
```

2. **Has the source emitted?**

```
source$.pipe(
  tap(v => console.log('Source emitted:', v))
).subscribe();
```

3. **Is it filtered out?**

```
source$.pipe(
  tap(v => console.log('Before filter:', v)),
  filter(v => v.isValid),
  tap(v => console.log('After filter:', v))
).subscribe();
```

4. **Is switchMap cancelling it?**

```
// switchMap cancels previous inner observables
// Use mergeMap if you need all to complete
```

"Observable emitting too many times"

```
// Add distinctUntilChanged to prevent duplicates
source$.pipe(
  tap(v => console.log('Before distinct:', v)),
  distinctUntilChanged(),
  tap(v => console.log('After distinct:', v))
```

```
).subscribe();
```

Memory Leaks

Check for missing unsubscribe:

```
// Bad - leaks on component destroy
ngOnInit() {
  this.service.data$.subscribe(data => this.data = data);
}

// Good - uses takeUntil pattern
private destroy$ = new Subject<void>();

ngOnInit() {
  this.service.data$.pipe(
    takeUntil(this.destroy$)
  ).subscribe(data => this.data = data);
}

ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}
```

URL-First Debugging

Verifying URL State

1. **Check the URL directly:**

- Look at the browser address bar
- Compare expected vs actual query parameters

2. **Log URL changes:**

```
// In UrlStateService or component
this.router.events.pipe(
  filter(event => event instanceof NavigationEnd),
  tap(event => console.log('[Router] Navigation:', event.url))
).subscribe();
```

3. **Log parsed parameters:**

```
this.route.queryParams.pipe(
  tap(params => console.log('[Route] Query params:', params))
).subscribe();
```

Common URL-First Issues

Filters not updating URL

Check the URL mapper:

```
console.log('[UrlMapper] toParams input:', filters);
const params = this.urlMapper.toParams(filters);
console.log('[UrlMapper] toParams output:', params);
```

URL not updating view

Check the filter parsing:

```
console.log('[UrlMapper] fromParams input:', params);
const filters = this.urlMapper.fromParams(params);
console.log('[UrlMapper] fromParams output:', filters);
```

Circular updates (infinite loop)

Add distinctUntilChanged with deep comparison:

```
this.route.queryParams.pipe(
  distinctUntilChanged((a, b) => JSON.stringify(a) === JSON.stringify(b)),
  tap(params => console.log('Params changed:', params))
).subscribe();
```

Network Debugging

Inspecting API Calls

In DevTools Network panel:

1. Filter by "XHR" or "Fetch" to see only API calls
2. Click a request to see:

- **Headers**: Request/response headers
- **Payload**: Request body (POST/PUT)
- **Preview**: Response data (formatted)
- **Response**: Raw response
- **Timing**: How long each phase took

Common API Issues

CORS Errors

```
Access to XMLHttpRequest at 'http://api.example.com' from origin 'http://localhost:4200'
has been blocked by CORS policy
```

Solution: API server must include CORS headers, or use Angular proxy:

```
// proxy.conf.json
{
  "/api": {
    "target": "http://api.example.com",
    "secure": false,
    "changeOrigin": true
  }
}
```

401 Unauthorized

Check:

1. Is the auth token being sent?
2. Is the token expired?
3. Is the token in the correct header format?

```
// Log request headers in interceptor
intercept(req: HttpRequest<any>, next: HttpHandler) {
  console.log('[HTTP] Request headers:', req.headers.keys());
  console.log('[HTTP] Auth header:', req.headers.get('Authorization'));
  return next.handle(req);
}
```

404 Not Found

Check:

1. Is the URL correct? (log it)
2. Is the API endpoint deployed?
3. Are path parameters correct?

Plotly.js Chart Debugging

Chart Not Rendering

1. **Check container dimensions:**

```
const container = this.chartContainer.nativeElement;
console.log('Container size:', container.clientWidth, 'x', container.clientHeight);
// Must be > 0
```

2. **Check data format:**

```
console.log('Traces:', JSON.stringify(chartData.traces, null, 2));
console.log('Layout:', JSON.stringify(chartData.layout, null, 2));
```

3. **Check for Plotly errors:**

```
Plotly.newPlot(element, traces, layout)
  .then(() => console.log('Plot created successfully'))
  .catch(err => console.error('Plotly error:', err));
```

Chart Not Updating

Ensure you're calling react, not newPlot:

```
// newPlot creates a new chart (slower)
// react updates existing chart (faster)
if (this.plotlyElement) {
  Plotly.react(this.plotlyElement, traces, layout);
} else {
  Plotly.newPlot(element, traces, layout);
}
```

Pop-out Window Debugging

Messages Not Received

1. **Check message posting:**

```
// In parent window
console.log('[Parent] Sending message:', message);
popoutWindow.postMessage(message, '*');
```

2. **Check message receiving:**

```
// In pop-out window
window.addEventListener('message', event => {
  console.log('[Popout] Received message:', event.data);
  console.log('[Popout] Origin:', event.origin);
});
```

3. **Check window reference:**

```
console.log('[Parent] Popout window:', popoutWindow);
```

```
console.log('[Parent] Window closed?:', popoutWindow.closed);
```

Pop-out Styling Issues

The pop-out window may not load all styles:

1. Check that global styles are linked in pop-out index.html
2. Verify component styles are loaded (not lazy-loaded)
3. Use browser DevTools in the pop-out window itself

Performance Debugging

Change Detection Issues

```
// Log change detection runs
constructor(private zone: NgZone) {
  zone.onStable.subscribe(() => {
    console.log('[Zone] Change detection complete');
  });
}
```

Identifying Slow Operations

```
console.time('expensiveOperation');
// ... operation
console.timeEnd('expensiveOperation');
// Output: expensiveOperation: 234.56ms
```

Memory Leaks

1. Open DevTools → Memory panel
2. Take heap snapshot before and after suspected leak
3. Compare snapshots to find retained objects

Debugging Checklist

When something doesn't work:

1. **Check the console** for errors
2. **Check the network** for failed requests
3. **Add console.log** at key points
4. **Verify data** with `| json` pipe
5. **Check subscriptions** are active
6. **Verify imports** in modules
7. **Check the URL** for state issues
8. **Use breakpoints** for complex logic

Key Takeaways

1. **Console.log strategically** — Add context, use groups, remove when done
2. **Use tap() for RxJS** — Inspect streams without modifying them
3. **Check the Network panel** — API issues often hide there
4. **Angular DevTools helps** — Especially for component tree and DI issues
5. **URL-First means URL-First** — When in doubt, check the URL

Further Reading

- Angular Debugging Guide
- Chrome DevTools Documentation
- RxJS Debugging

Chapter 954

Performance Testing

954: Glossary

Status: Reference Document

Type: Appendix

Overview

This glossary defines terms used throughout the vvroom book. Terms are organized alphabetically within categories.

Angular Terms

Change Detection

The process by which Angular checks component properties for changes and updates the DOM accordingly. Can be triggered manually via `ChangeDetectorRef.markForCheck()` or `detectChanges()`.

Component

A building block of Angular applications consisting of a TypeScript class, HTML template, and optional CSS styles. Decorated with `@Component()`.

Decorator

A TypeScript feature that adds metadata to classes, properties, or methods. Angular uses decorators like `@Component()`, `@Injectable()`, `@Input()`, and `@Output()`.

Dependency Injection (DI)

A design pattern where a class receives its dependencies from external sources rather than creating them. Angular's DI system provides services to components.

Directive

A class that modifies DOM elements or component behavior. Structural directives (`*ngIf`, `*ngFor`) change DOM structure; attribute directives modify appearance or behavior.

Guard

A service that controls route access. Can prevent navigation (`CanActivate`), leaving (`CanDeactivate`), or loading (`CanLoad`).

Interceptor

A service that intercepts HTTP requests and responses. Used for adding headers, handling errors, or transforming data globally.

Lifecycle Hook

Methods that Angular calls at specific points in a component's lifecycle: `ngOnInit`, `ngOnChanges`, `ngAfterViewInit`, `ngOnDestroy`, etc.

Module (NgModule)

A container for related components, directives, pipes, and services. Decorated with `@NgModule()`. Angular 13 uses NgModules (not standalone components).

Observable

An RxJS type representing a stream of values over time. Components subscribe to observables to receive data. See also: RxJS Terms.

Pipe

A function that transforms data in templates. Built-in examples: `| json`, `| async`, `| date`. Custom pipes transform domain-specific data.

Route

A URL path mapped to a component. Defined in routing modules with path, component, and optional guards.

Service

A class that provides functionality across components. Typically decorated with `@Injectable()` and provided at root level.

Template

The HTML portion of a component that defines its view. Can include Angular syntax like interpolation (`{{ }}`), property binding (`[prop]`), and event binding (`(event)`).

RxJS Terms

BehaviorSubject

A Subject that requires an initial value and emits the current value to new subscribers. Used for state that always has a value.

combineLatest

An operator that combines the latest values from multiple observables. Emits whenever any source emits (after all have emitted at least once).

distinctUntilChanged

An operator that only emits when the current value differs from the previous value. Prevents duplicate emissions.

filter

An operator that emits only values that pass a predicate function.

map

An operator that transforms each emitted value using a projection function.

Observable

A lazy push collection that can emit zero or more values over time. Must be subscribed to for values to flow.

Operator

A function that transforms an observable stream. Applied via the `pipe()` method. Examples: `map`, `filter`, `switchMap`.

pipe

A method that chains operators together. `source$.pipe(op1, op2, op3)` applies operators in sequence.

ReplaySubject

A Subject that replays a specified number of previous values to new subscribers. Useful for late subscribers.

shareReplay

An operator that multicasts an observable and replays the last N values to new subscribers. Prevents duplicate API calls.

Subject

An observable that is also an observer. Can push values with `next()` and be subscribed to.

subscribe

The method that activates an observable and receives its values. Returns a Subscription that should be cleaned up.

Subscription

An object representing the execution of an observable. Call `unsubscribe()` to stop receiving values and prevent memory leaks.

switchMap

An operator that maps to an inner observable and cancels the previous inner observable on each new emission. Ideal for search/filter operations.

takeUntil

An operator that emits values until a notifier observable emits. Used for component cleanup with a `destroy$` subject.

tap

An operator that performs side effects (like logging) without modifying the stream.

URL-First Architecture Terms

Adapter

A class that converts between framework types and domain-specific types. Examples: `AutomobileUrlMapper`, `AutomobileApiAdapter`.

API Adapter

An adapter that handles API communication for a specific domain. Implements `IApiAdapter<TFilters, TData, TStatistics>`.

Cache Key Builder

A class that generates unique cache keys from filter objects. Ensures API responses are cached correctly.

Chart Data Source

A class that transforms domain statistics into chart-ready data (Plotly traces). Each chart type has its own data source.

Domain

A specific area of functionality in the application. In vvroom, "automobile" is the domain. Other books might add "agriculture" or "real estate".

Domain Config

The central configuration object for a domain. Contains adapters, UI configs, and type references. Type: `DomainConfig<TFilters, TData, TStatistics>`.

Domain Config Registry

A service that stores and retrieves domain configurations. Allows framework code to access the active domain's config.

Filter

A set of criteria that narrow search results. In vvroom: manufacturer, model, year, body class, etc.

Framework Code

Domain-agnostic code that works with any domain through interfaces. Located in `src/app/framework/`.

Highlight

A visual emphasis on data that matches specific criteria. Highlights don't filter data; they color-code matching items.

Pop-out Window

A separate browser window displaying part of the application. Communicates with the main window via `postMessage`.

Resource Management Service

The central service managing data fetching, caching, and state. Coordinates URL state with API calls.

Single Source of Truth

A principle where one authoritative location stores each piece of state. In URL-First, the URL is the single source of truth for filter state.

URL Mapper

An adapter that converts between filter objects and URL query parameters. Bidirectional: `toParams()` and `fromParams()`.

URL State Service

A service that manages reading from and writing to URL query parameters. Provides reactive streams of URL state.

TypeScript Terms

Concrete Type

A specific, known type like `string`, `number`, or `AutomobileFilters`. Contrast with type parameters.

Constraint

A limit on what types can be used as a type parameter. Syntax: `<T extends SomeInterface>`.

Generic

A type or function that works with multiple types through type parameters. Provides type safety without code duplication.

Interface

A TypeScript structure defining a contract for objects. Does not exist at runtime; only provides compile-time type checking.

keyof

A TypeScript operator that returns a union of an object's property names as string literal types.

Partial<T>

A utility type that makes all properties of T optional.

Pick<T, K>

A utility type that creates a new type with only the specified properties from T.

Record<K, V>

A utility type representing an object with keys of type K and values of type V.

Type Guard

A function that narrows a type within a conditional block. Uses `is` return type syntax.

Type Parameter

A placeholder type in a generic definition, filled in when the generic is used. Conventionally named `T`, `U`, or descriptively like `TFilters`.

Union Type

A type that can be one of several types. Syntax: `string | number | null`.

Utility Type

Built-in TypeScript types that transform other types. Examples: `Partial`, `Required`, `Pick`, `Omit`, `Record`.

UI Component Terms

Base Chart Component

A reusable Plotly.js wrapper that works with any `ChartDataSource`. Handles rendering, resizing, and click events.

Base Picker Component

A reusable multi-select dropdown that works with any `PickerConfig`. Handles option loading and selection.

CDK (Component Dev Kit)

Angular's set of behavior primitives for building UI components. Used for drag-drop functionality.

Dockview

A third-party library providing tabbed, resizable, dockable panel layouts. Framework-agnostic.

PrimeNG

The UI component library used in vvroom. Provides styled components like tables, buttons, dialogs, etc.

Query Panel

A component displaying filter inputs for user-adjustable search criteria.

Results Table

A component displaying paginated search results in a table format.

Statistics Panel

A component displaying multiple charts showing data distributions.

API Terms

Base URL

The root URL for all API endpoints. Configured in `environment.ts`.

Endpoint

A specific API URL path that accepts requests and returns responses. Example: `/api/specs/v1/search`.

Pagination

The practice of returning data in pages rather than all at once. API returns `page`, `size`, `totalItems`, `totalPages`.

Query Parameter

A key-value pair in a URL after the `?`. Example: `?manufacturer=Toyota&year=2022`.

Request

An HTTP call to an API endpoint. Includes method (GET, POST), headers, and optional body.

Response

Data returned by an API. Includes status code, headers, and body (usually JSON).

Development Terms

Barrel Export

An `index.ts` file that re-exports symbols from a directory, simplifying imports.

Build

The process of compiling TypeScript and bundling the application for deployment. Command: `ng build`.

Hot Module Replacement (HMR)

A development feature that updates modules without full page reload.

Lazy Loading

Loading modules on demand rather than at startup. Improves initial load time.

Linting

Static code analysis to catch errors and enforce style. Tool: ESLint.

Serve

Running the application in development mode with live reload. Command: `ng serve`.

Tree Shaking

Removing unused code during the build process. Reduces bundle size.

Key Acronyms

Acronym	Meaning

API	Application Programming Interface
CDK	Component Dev Kit
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DI	Dependency Injection
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
RxJS	Reactive Extensions for JavaScript
SCSS	Sassy CSS (CSS preprocessor)
SPA	Single Page Application
URL	Uniform Resource Locator
VIN	Vehicle Identification Number

Further Reading

For deeper understanding of these terms, refer to:

- Angular Glossary
- TypeScript Handbook
- RxJS Documentation
- MDN Web Docs

Appendices

Chapter A01: Appendix: Angular CLI Reference

Chapter A02: Appendix: TypeScript Tips

Chapter A01

Appendix: Angular CLI Reference

Appendix A01: Styling and Branding

Overview

This appendix documents the styling system and branding configuration for the vvroom application. The visual design follows the dark theme pattern established in generic-prime while applying vvroom-specific branding.

Theme Configuration

PrimeNG Theme

The application uses PrimeNG's **lara-dark-blue** theme as the base, providing:

- Dark background colors (#2a2a2a, #3c3c3c)
- Blue accent colors (#64B5F6, #64c8ff)
- Consistent component styling

Global Styles (styles.scss)

```
/* PrimeNG Theme and Component Styles */
@import "primeng/resources/themes/lara-dark-blue/theme.css";
@import "primeng/resources/primeng.min.css";
@import "primeicons/primeicons.css";

/* KaTeX CSS for LaTeX rendering */
@import "katex/dist/katex.min.css";

/* Global Styles */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: var(--font-family);
  background-color: #3c3c3c;
```

```
  color: #ffffff;
}
```

Color Palette

Purpose	Color	Usage
Background (dark)	`#2a2a2a`	Header, panels
Background (main)	`#3c3c3c`	Page background
Text primary	`#ffffff`	Main text
Text secondary	`#888888`	Version labels, hints
Accent	`#64B5F6`	Interactive elements
Accent hover	`#64c8ff`	Hover states
Border	`#444444`	Dividers, separators

App Component Styling

Header Structure

```
.app-header {
  background-color: #2a2a2a;
  padding: 0.5rem 1rem;
  display: flex;
  justify-content: flex-start;
  align-items: center;
  gap: 2rem;
  border-bottom: 1px solid #444;
  height: 60px;
}
```

Navigation Links

```
.home-link,
.discover-link {
  color: #ffffff;
  text-decoration: none;
  font-size: 1.2rem;
  font-weight: bold;
  white-space: nowrap;
  cursor: pointer;

  &:hover {
```

```
    color: #64c8ff;
  }
}
```

Branding Configuration

Domain Label

The primary branding appears in the **domain configuration** (`domainLabel` property):

File: `src/app/domain-config/automobile/index.ts` (exports from `automobile.domain-config.ts`)

```
// src/app/domain-config/automobile/automobile.domain-config.ts
export function createAutomobileDomainConfig(injector: Injector): DomainConfig<...> {
  return {
    domainName: 'automobile',
    domainLabel: 'Vvroom Discovery',  // Branding displayed in discover header
    apiBaseUrl: apiBaseUrl,
    // ...
  };
}
```

The domain configuration is composed from multiple files in the `domain-config/automobile/` directory:

- `automobile.domain-config.ts` - Factory function and main config
- `models/` - Type definitions (filters, data, statistics)
- `adapters/` - API, URL, and cache adapters
- `configs/` - Table, filter, chart, and picker configurations
- `chart-sources/` - Chart data source implementations

Application Title

File: `src/app/app.component.ts`

```
@Component({ ... })
export class AppComponent {
  title = 'vvroom';
  // ...
}
```

HTML Title

File: `src/index.html`

```
<title>Vvroom - Automobile Discovery</title>
```

Pop-Out Window Styling

Pop-out windows require special handling to hide scrollbars:

```
body.popout-body {
  overflow: hidden;
  height: 100vh;
  width: 100vw;
}

html.popout-html {
  overflow: hidden;
  height: 100vh;
  width: 100vw;

  .app-content {
    overflow: hidden !important;
    height: 100vh !important;
  }
}
```

Component Style Files

Each component has its own SCSS file for component-specific styles:

Component	Style File
App Shell	`app.component.scss`
Discover	`features/discover/discover.component.scss`
Home	`features/home/home.component.scss`
Query Control	`framework/components/query-control/query-control.component.scss`
Query Panel	`framework/components/query-panel/query-panel.component.scss`
Base Chart	`framework/components/base-chart/base-chart.component.scss`
Base Picker	`framework/components/base-picker/base-picker.component.scss`
Basic Results Table	`framework/components/basic-results-table/basic-results-table.component.scss`
Dynamic Results Table	`framework/components/dynamic-results-table/dynamic-results-table.component.scss`

Results Table	`framework/components/results-table/results-table.component.scss`
Statistics Panel	`framework/components/statistics-panel-2/statistics-panel-2.component.scss`

Note: There are two table components:

- `basic-results-table` - Simplified table with essential features
- `results-table` - Full-featured table with advanced capabilities

Implementation Checklist

- [ ] Import PrimeNG lara-dark-blue theme in styles.scss
- [ ] Set body background to #3c3c3c
- [ ] Configure header with #2a2a2a background
- [ ] Update domainLabel to "Vvroom Discovery"
- [ ] Set app component title to "vvroom"
- [ ] Update index.html title tag
- [ ] Apply hover colors (#64c8ff) to interactive elements

Reference

- **PrimeNG Themes**: https://primeng.org/theming
- **Source Reference**: generic-prime/frontend/src/styles.scss
- **Color System**: VS Code Dark Theme inspired

Chapter A02

Appendix: TypeScript Tips

Appendix A02: URL-First Testing Rubric

Overview

This rubric provides a systematic method for verifying URL-First State Management compliance across popped-in and popped-out controls. The URL is the single source of truth; all state changes must flow through URL updates.

Test Categories

Category 1: Main Window (Popped-In) Control Changes

Test that changes in the main window controls are reflected in:

- The browser URL parameters
- All other controls in the main window
- Any open pop-out windows

Test ID	Test Description	Expected Behavior	Pass/Fail
M1.1	Change a query control filter (e.g., manufacturer dropdown)	URL updates with filter param; results table updates; statistics update	
M1.2	Change a highlight filter (e.g., year range)	URL updates with h_ prefixed param; highlighted rows change; pop-outs receive update	
M1.3	Change pagination (page number)	URL updates with `page` param; table shows correct page	

M1.4	Change page size	URL updates with `size` param; table row count matches	
M1.5	Change sort column	URL updates with `sort` param; table re-sorts	
M1.6	Change sort direction	URL updates with `sortDirection` param; table order reverses	
M1.7	Clear all filters	URL params removed; controls reset to defaults; full dataset shown	
M1.8	Apply multiple filters simultaneously	All filter params appear in URL; results reflect intersection	

Category 2: Pop-Out Window Control Changes

Test that changes in pop-out windows are communicated to the main window and reflected appropriately.

Test ID	Test Description	Expected Behavior	Pass/Fail
P2.1	Change highlight filter in pop-out	Main window URL updates with h_ param; main window highlights update	
P2.2	Pop-out sends filter change message	BroadcastChannel message received by main window	
P2.3	Pop-out does NOT update its own URL	Pop-out URL remains static (initial state only)	
P2.4	Pop-out does NOT make its own API calls	Network tab shows no API requests from pop-out after initial load	
P2.5	Pop-out receives state via BroadcastChannel	`syncStateFromExternal()` called; no API fetch triggered	
P2.6	Multiple pop-outs stay synchronized	Change in one pop-out reflects in main window and all other pop-outs	

Category 3: URL Paste Tests (Without Highlight Filters)

Test that pasting a URL with standard filters correctly restores application state.

Test ID	Test Description	Expected Behavior	Pass/Fail
U3.1	Paste URL with single filter param	Filter control shows correct value; results match filter	
U3.2	Paste URL with multiple filter params	All filter controls populated; results show intersection	
U3.3	Paste URL with pagination params	Correct page displayed; pagination control shows correct page	
U3.4	Paste URL with sort params	Table sorted correctly; sort indicators match URL	
U3.5	Paste URL with all param types combined	All controls reflect URL state; results correct	
U3.6	Paste URL with invalid filter value	Graceful handling; invalid param ignored or defaulted	
U3.7	Share URL to another browser/session	New session shows identical state to original	

Category 4: URL Paste Tests (With Highlight Filters)

Test that pasting a URL with highlight filters (`h_` prefix) correctly applies highlighting.

Test ID	Test Description	Expected Behavior	Pass/Fail
H4.1	Paste URL with `h_yearMin` param	Year highlight filter populated; matching rows highlighted	
H4.2	Paste URL with `h_manufacturer` param	Manufacturer highlight populated; matching rows highlighted	
H4.3	Paste URL with multiple highlight params	All highlight filters populated; rows matching ALL highlighted	

H4.4	Paste URL mixing query and highlight params	Query filters filter data; highlight filters highlight within results	
H4.5	Paste URL with highlight param into pop-out	Pop-out shows initial highlights; syncs with main window	
H4.6	Clear highlight via URL (remove h_ param)	Highlights removed; highlight controls cleared	

Category 5: Pop-Out Window Presentation

Test that pop-out windows display correctly without main window chrome.

Test ID	Test Description	Expected Behavior	Pass/Fail
W5.1	Pop-out hides site banner/header	No navigation header visible in pop-out	
W5.2	Pop-out shows query control panel	Filter controls visible and functional	
W5.3	Pop-out URL contains `popout=true` param	URL includes pop-out indicator	
W5.4	Pop-out title reflects content	Window title indicates popped-out component	
W5.5	Pop-out respects `autoFetch = false`	No initial API call; waits for main window data	

Category 6: Cross-Window Synchronization

Test bidirectional communication between main window and pop-outs.

Test ID	Test Description	Expected Behavior	Pass/Fail
S6.1	Main window filter change updates all pop-outs	All pop-outs receive BroadcastChannel message and update	
S6.2	Pop-out filter change updates main window URL	Main window URL reflects pop-out's requested change	

S6.3	Main window data refresh updates pop-outs	New API data propagated to all pop-outs	
S6.4	Close pop-out does not affect main window state	Main window continues functioning normally	
S6.5	Open multiple pop-outs of same type	Each pop-out shows consistent state	
S6.6	Open pop-outs of different types	Each receives relevant state updates	

Category 7: Router Navigate Encapsulation

Verify that `router.navigate()` is only called from `UrlStateService`.

Test ID	Test Description	Expected Behavior	Pass/Fail
R7.1	Grep codebase for `router.navigate`	Only appears in `url-state.service.ts`	
R7.2	Components call `updateFilters()` method	Components never call `router.navigate()` directly	
R7.3	Pop-out components call parent messaging	No `router.navigate()` in pop-out components	

Anti-Pattern Checklist

These patterns indicate URL-First violations and should fail testing:

Anti-Pattern	How to Detect	Severity
Direct state mutation bypassing URL	State changes without URL param update	Critical
`router.navigate()` in components	Grep for `router.navigate` outside UrlStateService	Critical
Pop-out making API calls	Network tab shows fetch from pop-out window	Critical

Pop-out updating its own URL	Pop-out URL changes after initial load	Critical
State not in URL that should be shareable	Filter applied but not in URL; refresh loses state	High
Highlight state without h_ prefix	Highlight params using wrong naming convention	Medium

---

Test Execution Checklist

Before running tests:

- [ ] Development server running on port 4207
- [ ] Browser DevTools Network tab open (to verify API calls)
- [ ] Browser DevTools Console open (to catch errors)
- [ ] At least one pop-out window open for cross-window tests

After each test:

- [ ] Verify URL params match expected state
- [ ] Verify all controls reflect URL state
- [ ] Verify pop-out windows synchronized (if applicable)
- [ ] Check console for errors

Known Issues (Observed)

The following issues were observed during initial inspection and should be addressed:

1. **Pop-out URL incorrect** - Pop-out window URL does not reflect expected state
2. **Pop-out shows site banner** - Header/navigation visible in pop-out (should be hidden)
3. **Query control not visible in pop-out** - Filter panel missing from pop-out view

---

References

- instructions.md - URL-First Compliance Checklist

- A01-styling-and-branding.md - Theme configuration
- `~/projects/vroom/docs/STATE-MANAGEMENT-SPECIFICATION.md` - Complete specification
- `~/projects/vroom/docs/POPOUT-ARCHITECTURE.md` - Cross-window communication